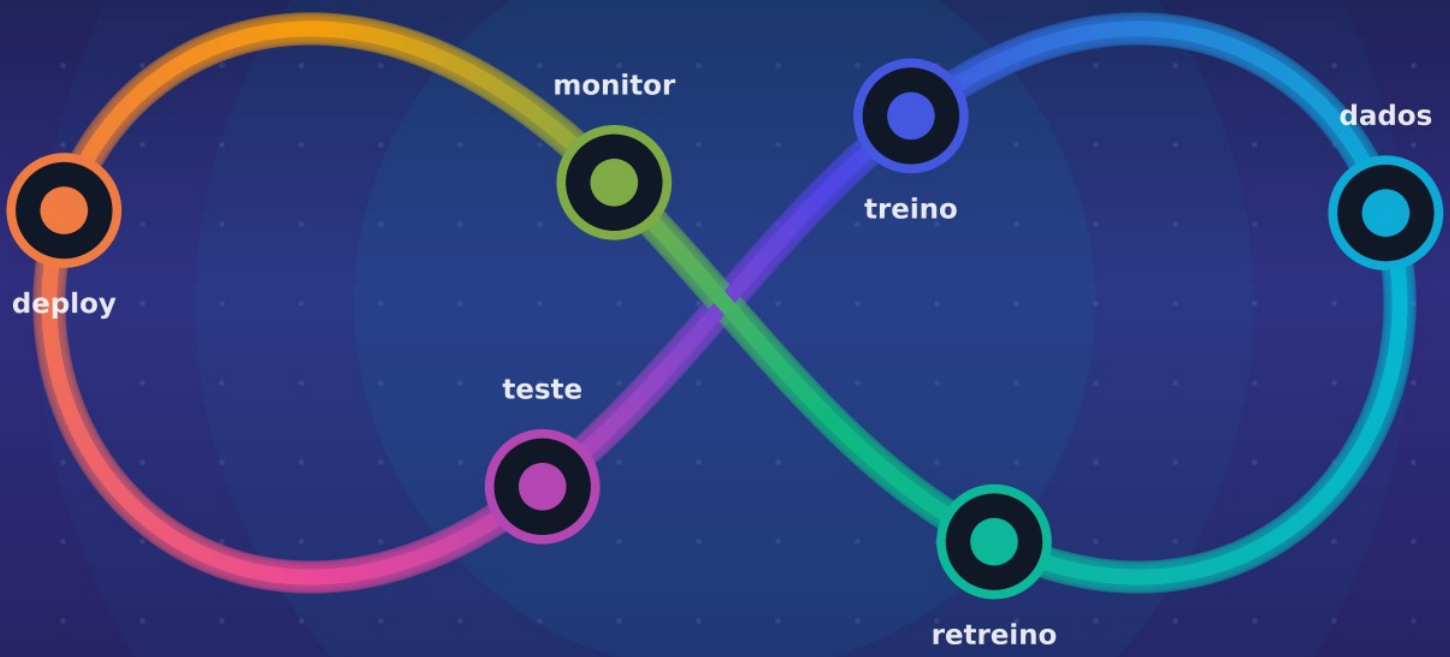
2026



MLOps

Do notebook à produção

com Python

Araken de Medeiros Santos

MLOPS

Do notebook à produção com Python

Araken de Medeiros Santos

2026

Sumário

Apresentação **11**
Para quem é este livro 11
Como o livro está organizado 11
Como estudar com este livro 11
Caixas e símbolos do livro 12
Sobre os códigos 12
Bases públicas da UCI e do Kaggle 13
Leituras recomendadas 13

Parte I — Fundamentos de MLOps **15**

Capítulo 1 — Introdução ao MLOps: do Notebook à Produção **16**
1.1 Uma história de cozinha 16
1.2 Por que tantos modelos não chegam (ou não ficam) em produção 17
1.2.1 Sintomas de um projeto sem MLOps 18
1.3 O que é MLOps 19
1.3.1 Lembrando do DevOps 19
1.3.2 Por que o ML precisa de algo além do DevOps 19
1.4 Os três eixos da mudança: código, dados e modelo 20
1.5 Princípios do MLOps 21
1.5.1 CI, CD e CT 21
1.6 Quem faz o quê: os papéis 21
1.7 Mão na massa: preparando o laboratório 22
1.7.1 Fixando as versões das bibliotecas 22
1.7.2 O primeiro experimento de reprodutibilidade 22
1.7.3 Registrando o "rastro" de uma execução 23
1.8 Maturidade: onde estamos? 23
1.9 Mapa do livro 24
1.10 Resumo do capítulo 24
Exercícios de múltipla escolha 25
Exercícios discursivos 29
Exercícios práticos em Python (scikit-learn) 30
Exercício 1 — O custo de não fixar a semente 30
Exercício 2 — Salvando o modelo com seus metadados 30
Exercício 3 — A ordem dos dados também importa 30
Exercício 4 — As quatro medidas do DevOps aplicadas a um modelo 30
Projeto integrador do capítulo 31
Esqueleto de um projeto reprodutível 31

Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade **32**
2.1 O ciclo de vida de um sistema de ML 32
2.1.1 Três grandes fases 33
2.2 Do problema de negócio à métrica 33
2.2.1 Métrica do modelo não é métrica do negócio 34
2.2.2 Requisitos não funcionais 34
2.3 Dívida técnica em sistemas de ML 34
2.3.1 Principais tipos de dívida 35
2.3.2 Simulando um ciclo de realimentação 36
2.4 ML Test Score: uma lista de verificação para produção 36
2.4.1 Como pontuar 36
2.5 A arquitetura de referência 37

2.6 Níveis de maturidade ... 38
2.7 Reprodutibilidade, rastreabilidade e linhagem ... 39
2.8 Resumo do capítulo ... 40
Exercícios de múltipla escolha ... 41
Exercícios discursivos ... 45
Exercícios práticos em Python (scikit-learn) ... 46
Exercício 1 — Sempre comece pela linha de base ... 46
Exercício 2 — Calculadora de ML Test Score ... 46
Exercício 3 — Quebrando o ciclo de realimentação ... 46
Exercício 4 — "Mudar qualquer coisa muda tudo" (CACE) ... 46
Projeto integrador do capítulo ... 47
Planejando um projeto de retenção de clientes ... 47

Parte II — Dados e Experimentos ... 48

Capítulo 3 — Dados: Qualidade, Validação e Versionamento ... 49
3.1 Lixo entra, lixo sai ... 49
3.2 As dimensões da qualidade dos dados ... 49
3.2.1 Medindo a qualidade com pandas ... 50
3.3 Validação automática de dados ... 51
3.3.1 Um validador simples ... 51
3.4 Vazamento de dados: o inimigo invisível ... 52
3.4.1 Detectando um vazamento ... 53
3.5 Rótulos errados também contam ... 53
3.5.1 Quanto o ruído atrapalha? ... 53
3.6 Como dividir os dados ... 54
3.7 Versionando dados ... 55
3.7.1 A ideia do hash ... 55
3.7.2 Como o DVC funciona ... 56
3.7.3 Dados que mudam com o tempo ... 56
3.8 Documentando os dados ... 57
3.9 Dados pessoais e privacidade desde o início ... 57
3.10 Onde encontrar dados reais ... 58
3.11 Resumo do capítulo ... 58
Exercícios de múltipla escolha ... 59
Exercícios discursivos ... 63
Exercícios práticos em Python (scikit-learn) ... 64
Exercício 1 — Relatório de qualidade de dados ... 64
Exercício 2 — Um validador de esquema que para o pipeline ... 64
Exercício 3 — Caçando um atributo vazado ... 64
Exercício 4 — Divisão aleatória versus divisão por grupo ... 64
Projeto integrador do capítulo ... 65
Portão de dados com quarentena e versionamento ... 65

Capítulo 4 — Pipelines Reprodutíveis com scikit-learn ... 66
4.1 Da receita solta à linha de montagem ... 66
4.2 A linguagem do scikit-learn: fit, transform, predict ... 67
4.3 Construindo um pipeline ... 67
4.4 O experimento do vazamento escondido ... 68
4.5 Colunas diferentes, tratamentos diferentes: ColumnTransformer ... 69
4.6 Transformadores personalizados ... 70
4.7 Configuração separada do código ... 71
4.8 Controlando a aleatoriedade ... 71
4.9 Salvando e carregando o pipeline completo ... 72
4.10 O treino como script ... 73

4.11 Pipelines de ML dentro de pipelines de dados... 74
4.12 Resumo do capítulo... 74
Exercícios de múltipla escolha... 75
Exercícios discursivos... 79
Exercícios práticos em Python (scikit-learn)... 80
Exercício 1 — Padronizar importa para uns, não para outros... 80
Exercício 2 — Dados mistos com ColumnTransformer... 80
Exercício 3 — Um transformador que aprende limites... 80
Exercício 4 — O vazamento da seleção de atributos, em muitas sementes... 80
Projeto integrador do capítulo... 81
Treino como script parametrizado e reprodutível... 81
Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos... 82
5.1 O caderno de laboratório do cientista... 82
5.2 O vocabulário do rastreamento... 83
5.3 Primeiros passos com o MLflow... 83
5.4 Registrando um experimento de verdade... 84
5.4.1 Cuidado com o formato do modelo salvo... 85
5.4.2 O que mais vale registrar... 86
5.4.3 Registro automático (autologging)... 86
5.5 Do experimento ao produto: o registro de modelos... 86
5.5.1 Mover o apelido não atualiza o modelo já carregado... 88
5.6 Promoção automática com portão de qualidade... 89
5.7 Metadados de modelo: o que um "cartão de identidade" deve conter... 90
5.8 Além do MLflow... 90
5.9 Resumo do capítulo... 91
Exercícios de múltipla escolha... 92
Exercícios discursivos... 96
Exercícios práticos em Python (scikit-learn)... 97
Exercício 1 — Três modelos, três execuções, uma consulta... 97
Exercício 2 — Curva de aprendizado como métrica por passo... 97
Exercício 3 — Versões, apelidos e reversão... 97
Exercício 4 — Promoção só com evidência estatística... 97
Projeto integrador do capítulo... 98
Central de experimentos com portão e registro... 98
Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros... 99
6.1 Avaliar é mais difícil do que parece... 99
6.2 Métricas de classificação... 99
6.2.1 O engano da acurácia... 100
6.2.2 Curva ROC e curva de precisão-revocação... 101
6.2.3 AP não é "a" área sob a curva PR... 101
6.3 Escolhendo o limiar pelo custo... 102
6.4 Calibração: as probabilidades dizem a verdade?... 102
6.5 Validação cruzada... 103
6.5.1 Quando os dados têm tempo: não embaralhe!... 104
6.6 Comparando modelos com honestidade... 105
6.7 O viés de seleção e a validação aninhada... 105
6.8 Otimização de hiperparâmetros... 107
6.9 Avaliando por fatias... 108
6.10 Métricas de regressão... 109
6.11 Offline versus online... 109
6.12 Resumo do capítulo... 109
Exercícios de múltipla escolha... 110

Exercícios discursivos ... 114
Exercícios práticos em Python (scikit-learn) ... 115
Exercício 1 — Métricas para classes raras ... 115
Exercício 2 — Escolhendo o limiar pelo custo ... 115
Exercício 3 — Calibração antes e depois ... 115
Exercício 4 — O viés da busca de hiperparâmetros ... 115
Projeto integrador do capítulo ... 116
Relatório de avaliação rigorosa de um modelo de risco ... 116

Parte III — Entrega e Operação ... 117

Capítulo 7 — Empacotamento e Serviço de Modelos ... 118
7.1 O modelo precisa sair do laboratório ... 118
7.2 Quatro formas de entregar previsões ... 118
7.3 O contrato de entrada e saída ... 119
7.4 Uma API de previsão com FastAPI ... 120
7.5 Testando a API sem subir servidor ... 122
7.5.1 Atualizando o modelo de um serviço em execução ... 123
7.6 Latência: não olhe só a média ... 124
7.7 Contêineres: levando o ambiente junto ... 125
7.7.1 Boas práticas com imagens ... 126
7.8 Previsão em lote ... 126
7.9 Estratégias de entrega: um panorama ... 127
7.10 Segurança no serviço de modelos ... 127
7.11 Resumo do capítulo ... 128
Exercícios de múltipla escolha ... 129
Exercícios discursivos ... 133
Exercícios práticos em Python (scikit-learn) ... 134
Exercício 1 — Um contrato gerado a partir dos dados de treino ... 134
Exercício 2 — Uma API mínima com saúde e previsão ... 134
Exercício 3 — Percentis de latência e o ganho do lote ... 134
Exercício 4 — Previsão em lote idempotente com manifesto ... 134
Projeto integrador do capítulo ... 135
API completa com tempo real, lote, limites e testes ... 135

Capítulo 8 — Testes e CI/CD para Machine Learning ... 136
8.1 Testar para dormir tranquilo ... 136
8.2 Testes unitários com pytest ... 137
8.3 Testes de dados ... 138
8.4 Testes de modelo ... 139
8.5 O portão de qualidade ... 141
8.6 Integração contínua (CI) ... 142
8.6.1 Um exemplo de fluxo de CI ... 143
8.6.2 Boas práticas de CI ... 144
8.7 Entrega e implantação contínuas (CD) ... 144
8.8 Treinamento contínuo (CT) ... 144
8.9 Revisão de código e de modelos ... 145
8.10 Resumo do capítulo ... 145
Exercícios de múltipla escolha ... 146
Exercícios discursivos ... 150
Exercícios práticos em Python (scikit-learn) ... 151
Exercício 1 — Testes unitários para uma função de limpeza ... 151
Exercício 2 — Testes de comportamento do modelo ... 151
Exercício 3 — Portão de qualidade que valida as métricas ... 151
Exercício 4 — Os testes realmente protegem? Quebrando o modelo de propósito ... 151

Projeto integrador do capítulo 152
Suíte de testes e CI local de ponta a ponta 152
Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento 153
9.1 O modelo envelhece sozinho 153
9.2 O que monitorar 154
9.3 Os tipos de mudança 155
9.4 Detectando mudanças em um atributo 156
9.5 Mudanças que só aparecem no conjunto 157
9.6 Monitorando as previsões 158
9.7 Desempenho com rótulos atrasados 158
9.7.1 Um detector de deriva em fluxo de erros 159
9.8 Simulação de um modelo que envelhece 159
9.8.1 O DDM em ação 160
9.9 Retreinamento: como e quando 161
9.10 Implantando mudanças com segurança 162
9.10.1 Teste A/B: quanta gente é preciso? 163
9.11 Registrando o que o modelo faz 163
9.12 Alertas, painéis e planos de resposta 164
9.13 Resumo do capítulo 165
Exercícios de múltipla escolha 166
Exercícios discursivos 170
Exercícios práticos em Python (scikit-learn) 171
Exercício 1 — Calculando o PSI à mão 171
Exercício 2 — Deriva em atributo categórico 171
Exercício 3 — O classificador de domínio acha o que os testes simples perdem 171
Exercício 4 — Quanto o DDM demora a perceber? 171
Projeto integrador do capítulo 172
Painel de monitoramento de 12 semanas com três tipos de deriva 172
Parte IV — Confiança e Escala 173
Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas 174
10.1 Do "funciona" ao "merece confiança" 174
10.2 Documentando modelos: o cartão do modelo 175
10.3 Explicabilidade: por que o modelo decidiu isso? 176
10.4 Justiça: o modelo trata os grupos de forma equilibrada? 177
10.5 Privacidade 179
10.5.1 Privacidade diferencial 179
10.5.2 Aprendizado federado 180
10.6 Segurança do ML 180
10.7 Rastreabilidade e auditoria 180
10.8 Armazém de atributos (feature store) 180
10.8.1 A armadilha temporal 181
10.9 Orquestração de fluxos de trabalho 182
10.10 Plataforma, custos e a decisão "construir ou comprar" 183
10.11 LLMOps: operando modelos de linguagem 183
10.12 Olhando adiante 185
10.13 Resumo do capítulo 186
Exercícios de múltipla escolha 187
Exercícios discursivos 191
Exercícios práticos em Python (scikit-learn) 192
Exercício 1 — Cartão do modelo gerado automaticamente 192
Exercício 2 — Explicando o modelo: permutação, PDP e a armadilha da correlação 192
Exercício 3 — Auditoria de justiça e ajuste de limiares 192

Exercício 4 — Privacidade diferencial: o custo do ruído....192
Projeto integrador do capítulo....193
Auditoria de um modelo de crédito: documentação, explicação, justiça e privacidade....193

Parte V — Projeto Final....194

Capítulo 11 — Projeto Final: do Dado à Produção....195
11.1 Chegou a hora de montar a fábrica inteira....195
11.2 O cenário....196
11.3 Visão geral das nove etapas....196
11.4 A arquitetura do projeto....196
11.5 Etapa por etapa....197
11.5.1 Dados e validação....197
11.5.2 Candidatos....197
11.5.3 Rastreamento, registro e portão de qualidade....198
11.5.4 Serviço....198
11.5.5 Testes....198
11.5.6 Produção simulada....198
11.5.7 Monitoramento....198
11.5.8 Treinamento contínuo....199
11.5.9 Governança e relatório....199
11.6 O que esperar dos resultados....199
11.7 O relatório final....199
11.8 Fechamento....200
11.9 Resumo do capítulo....200
Exercícios de múltipla escolha....201
Exercícios discursivos....205
Projeto integrador final....206
Do dado à produção: um sistema de reconhecimento de dígitos com monitoramento e retreino....206

Apêndice A — Gabarito dos exercícios....207

Capítulo 1 — Introdução ao MLOps....208
Múltipla escolha....208
Discursivas (respostas-modelo)....208

Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade....211
Múltipla escolha....211
Discursivas (respostas-modelo)....211

Capítulo 3 — Dados....214
Múltipla escolha....214
Discursivas (respostas-modelo)....214

Capítulo 4 — Pipelines Reprodutíveis com scikit-learn....216
Múltipla escolha....216
Discursivas (respostas-modelo)....216

Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos....218
Múltipla escolha....218
Discursivas (respostas-modelo)....218

Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros....220
Múltipla escolha....220
Discursivas (respostas-modelo)....220

Capítulo 7 — Empacotamento e Serviço de Modelos....222
Múltipla escolha....222
Discursivas (respostas-modelo)....222

Capítulo 8 — Testes e CI/CD para Machine Learning 224
Múltipla escolha 224
Discursivas (respostas-modelo) 224
Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento 226
Múltipla escolha 226
Discursivas (respostas-modelo) 227
Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas 229
Múltipla escolha 229
Discursivas (respostas-modelo) 229
Capítulo 11 — Projeto Final 232
Múltipla escolha 232
Discursivas (respostas-modelo) 232
Apêndice B — Glossário 234
Apêndice C — Guia rápido de decisões, ferramentas e códigos 237
C.1 Mapa de decisão 237
C.2 Classes, funções e comandos principais 238
C.3 Lista de verificação de prontidão para produção 238
C.4 Valores de referência usados no livro 239
Apêndice D — Soluções dos exercícios práticos 240
Capítulo 1 — Introdução ao MLOps 241
Exercício 1 — O custo de não fixar a semente 241
Exercício 2 — Salvando o modelo com seus metadados 241
Exercício 3 — A ordem dos dados também importa 242
Exercício 4 — As quatro medidas do DevOps aplicadas a um modelo 243
Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade 244
Exercício 1 — Sempre comece pela linha de base 244
Exercício 2 — Calculadora de ML Test Score 244
Exercício 3 — Quebrando o ciclo de realimentação 245
Exercício 4 — "Mudar qualquer coisa muda tudo" (CACE) 246
Capítulo 3 — Dados 247
Exercício 1 — Relatório de qualidade de dados 247
Exercício 2 — Um validador de esquema que para o pipeline 247
Exercício 3 — Caçando um atributo vazado 248
Exercício 4 — Divisão aleatória versus divisão por grupo 249
Capítulo 4 — Pipelines Reprodutíveis com scikit-learn 251
Exercício 1 — Padronizar importa para uns, não para outros 251
Exercício 2 — Dados mistos com ColumnTransformer 251
Exercício 3 — Um transformador que aprende limites 252
Exercício 4 — O vazamento da seleção de atributos, em muitas sementes 253
Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos 254
Exercício 1 — Três modelos, três execuções, uma consulta 254
Exercício 2 — Curva de aprendizado como métrica por passo 255
Exercício 3 — Versões, apelidos e reversão 255
Exercício 4 — Promoção só com evidência estatística 257
Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros 258
Exercício 1 — Métricas para classes raras 258
Exercício 2 — Escolhendo o limiar pelo custo 258
Exercício 3 — Calibração antes e depois 259

Exercício 4 — O viés da busca de hiperparâmetros.... 260
Capítulo 7 — Empacotamento e Serviço de Modelos.... 261
Exercício 1 — Um contrato gerado a partir dos dados de treino.... 261
Exercício 2 — Uma API mínima com saúde e previsão.... 261
Exercício 3 — Percentis de latência e o ganho do lote.... 262
Exercício 4 — Previsão em lote idempotente com manifesto.... 263
Capítulo 8 — Testes e CI/CD para Machine Learning.... 265
Exercício 1 — Testes unitários para uma função de limpeza.... 265
Exercício 2 — Testes de comportamento do modelo.... 265
Exercício 3 — Portão de qualidade que valida as métricas.... 267
Exercício 4 — Os testes realmente protegem? Quebrando o modelo de propósito.... 268
Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento.... 270
Exercício 1 — Calculando o PSI à mão.... 270
Exercício 2 — Deriva em atributo categórico.... 270
Exercício 3 — O classificador de domínio acha o que os testes simples perdem.... 271
Exercício 4 — Quanto o DDM demora a perceber?.... 272
Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas.... 273
Exercício 1 — Cartão do modelo gerado automaticamente.... 273
Exercício 2 — Explicando o modelo: permutação, PDP e a armadilha da correlação.... 274
Exercício 3 — Auditoria de justiça e ajuste de limiares.... 275
Exercício 4 — Privacidade diferencial: o custo do ruído.... 276
Apêndice E — Soluções de referência dos projetos integradores.... 277
Capítulo 1 — Introdução ao MLOps.... 278
Esqueleto de um projeto reprodutível.... 278
Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade.... 280
Planejando um projeto de retenção de clientes.... 280
Capítulo 3 — Dados.... 282
Portão de dados com quarentena e versionamento.... 282
Capítulo 4 — Pipelines Reprodutíveis com scikit-learn.... 284
Treino como script parametrizado e reprodutível.... 284
Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos.... 286
Central de experimentos com portão e registro.... 286
Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros.... 289
Relatório de avaliação rigorosa de um modelo de risco.... 289
Capítulo 7 — Empacotamento e Serviço de Modelos.... 291
API completa com tempo real, lote, limites e testes.... 291
Capítulo 8 — Testes e CI/CD para Machine Learning.... 294
Suíte de testes e CI local de ponta a ponta.... 294
Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento.... 297
Painel de monitoramento de 12 semanas com três tipos de deriva.... 297
Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas.... 300
Auditoria de um modelo de crédito: documentação, explicação, justiça e privacidade.... 300
Capítulo 11 — Projeto Final.... 303
Do dado à produção: um sistema de reconhecimento de dígitos com monitoramento e retreino.... 303
Referências.... 309

Apresentação

Para quem é este livro

Este livro foi escrito para um **público misto**: estudantes de graduação e pós-graduação, profissionais de dados e de software que querem colocar modelos de aprendizado de máquina em produção, e curiosos que desejam entender o que acontece **depois** que um modelo é treinado.

Você **não precisa** de experiência prévia em produção. É útil conhecer o básico de Python e a ideia de treinar um modelo, mas cada conceito técnico essencial é explicado no momento em que aparece. Quando uma fórmula surgir, ela virá com uma explicação "em palavras" e, quase sempre, com um exemplo numérico pequeno.

Ao mesmo tempo, o livro avança. Os capítulos começam no nível básico (por que modelos precisam de operação), passam pelo intermediário (pipelines, rastreamento, testes, serviço) e chegam a temas avançados (detecção de deriva, treinamento contínuo, governança, justiça, privacidade, armazém de atributos, orquestração e LLMOps).

Como o livro está organizado

O livro tem **cinco partes** e **onze capítulos**.

- **Parte I — Fundamentos de MLOps:** por que modelos não chegam (ou não ficam) em produção; ciclo de vida, dívida técnica e maturidade.
- **Parte II — Dados e Experimentos:** qualidade, validação e versionamento de dados; pipelines reprodutíveis; rastreamento de experimentos e registro de modelos; avaliação rigorosa e otimização de hiperparâmetros.
- **Parte III — Entrega e Operação:** empacotamento e serviço; testes e CI/CD; monitoramento, deriva e retreinamento.
- **Parte IV — Confiança e Escala:** governança, explicabilidade, justiça, privacidade, armazém de atributos, orquestração e LLMOps.
- **Parte V — Projeto Final:** um sistema completo, do dado à produção, com monitoramento e retreino.

Cada capítulo segue o mesmo roteiro:

1. **Objetivos de aprendizagem**, no início.
2. **Conteúdo** com analogias, figuras, exemplos e códigos em Python.
3. **Resumo** em cartões de memorização.
4. **30 questões de múltipla escolha** e **20 questões discursivas**.
5. **Exercícios práticos em Python** com a biblioteca scikit-learn (e as ferramentas de MLOps do capítulo), em ordem crescente de dificuldade.
6. **Projeto integrador do capítulo**, com contexto, tarefas, entregáveis e critérios de avaliação.
7. Um quadro **"Para ir além"**, com conjuntos de dados públicos da UCI e do Kaggle para praticar.

O **gabarito de todas as questões** está no Apêndice A. As **soluções dos exercícios práticos** estão no Apêndice D, e a **solução de referência dos projetos integradores**, no Apêndice E. Tente resolver sozinho antes de consultá-las.

Como estudar com este livro

- **Estude em blocos curtos.** Sessões de 25 a 40 minutos, com pausas, ajudam a manter a atenção.
- **Pare e explique.** Após cada seção, tente explicar a ideia, em voz alta, como se falasse com um amigo.

- **Rode os códigos.** Altere um parâmetro por vez e observe o que muda.
- **Responda antes de olhar o gabarito.** Tentar lembrar fixa mais do que reler.
- **Volte aos cartões de memorização** dias depois. Revisões espaçadas ajudam a memória de longo prazo.

Caixas e símbolos do livro

> **Dica**
>
> Sugestões práticas para facilitar seu trabalho.

> **Atenção**
>
> Cuidados e erros comuns. Leia com calma.

> **No seu dia a dia**
>
> Exemplos do cotidiano que aproximam o conceito da sua realidade.

> **Saiba mais**
>
> Curiosidades e aprofundamentos para quem quer ir além.

> **Exemplo ou exercício guiado**
>
> Pequenas atividades para você experimentar.

> **Memorize**
>
> O essencial do assunto, em poucas linhas.

Sobre os códigos

Todos os trechos de código **em Python** deste livro foram executados durante a elaboração do material, com as versões de bibliotecas abaixo. Resultados numéricos podem variar um pouco em versões diferentes.

Componente	Versão utilizada
Python	3.13
NumPy	2.5
SciPy	1.18
pandas	3.0
scikit-learn	1.9
matplotlib	3.11
joblib	1.6
MLflow	3.17
FastAPI	0.143

Componente	Versão utilizada
Pydantic	2.14
pytest	9.1

Atenção: os trechos que **não** são Python (comandos de terminal, arquivos `Dockerfile` e fluxos de CI em YAML, comandos do DVC) são **ilustrativos** e **não foram executados** na elaboração do livro, pois o ambiente de escrita não dispunha de Docker, DVC nem de um serviço de CI. Teste-os no seu ambiente e consulte a documentação oficial de cada ferramenta, porque versões e opções mudam com o tempo.

Os exemplos e projetos usam **conjuntos de dados embutidos no scikit-learn** ou **dados simulados** gerados no próprio código, com sementes fixas. Assim, você não precisa baixar arquivos externos e consegue reproduzir os resultados.

Os conjuntos embutidos vêm do repositório **UCI Machine Learning**: *Iris*, *Wine*, *Breast Cancer Wisconsin (Diagnostic)* e *Optical Recognition of Handwritten Digits* (a parte de teste, com 1.797 imagens). Os dados de clientes, de pedidos e de crédito usados em alguns exemplos e projetos são **simulados** e servem apenas para fins didáticos; cada projeto informa a origem dos seus dados. A **produção** e a **deriva** dos projetos também são simuladas.

Bases públicas da UCI e do Kaggle

Ao final de cada capítulo, o quadro **"Para ir além"** sugere conjuntos de dados públicos do repositório **UCI Machine Learning** e do **Kaggle** para você repetir os exercícios com dados reais. Esses conjuntos **não foram usados nem testados nos códigos deste livro**: confira nos sites a descrição, a licença e as colunas antes de usá-los. Em geral, o fluxo é este:

- **UCI:** baixe o arquivo no site do repositório ou use o pacote `ucimlrepo`.
- **Kaggle:** baixe o arquivo no site (é preciso criar uma conta) ou use a API do Kaggle com um token pessoal.

Depois, carregue os dados com `pandas`, inspecione-os (formato, tipos, ausentes) e aplique o roteiro do capítulo.

Leituras recomendadas

Para aprofundar os estudos, alguns livros e artigos de referência são:

- Huyen (2022), *Designing Machine Learning Systems*, visão prática e abrangente de sistemas de ML em produção.
- Sculley et al. (2015), "Hidden technical debt in machine learning systems", o artigo clássico sobre dívida técnica em ML.
- Kreuzberger, Kühl e Hirschl (2023), "Machine learning operations (MLOps): overview, definition, and architecture", uma revisão acadêmica do tema.
- Breck et al. (2017), "The ML test score", uma rubrica de prontidão para produção.
- Forsgren, Humble e Kim (2018), *Accelerate*, sobre medidas de desempenho de entrega de software.
- Beyer et al. (2016), *Site Reliability Engineering*, sobre confiabilidade de sistemas.
- Kohavi, Tang e Xu (2020), *Trustworthy Online Controlled Experiments*, sobre testes A/B.
- Barocas, Hardt e Narayanan (2023), *Fairness and Machine Learning*, sobre justiça em ML.

As referências completas estão ao final do livro. Priorizamos artigos publicados em periódicos de alto impacto, trabalhos apresentados em conferências de referência da área e livros reconhecidos.

Bons estudos!

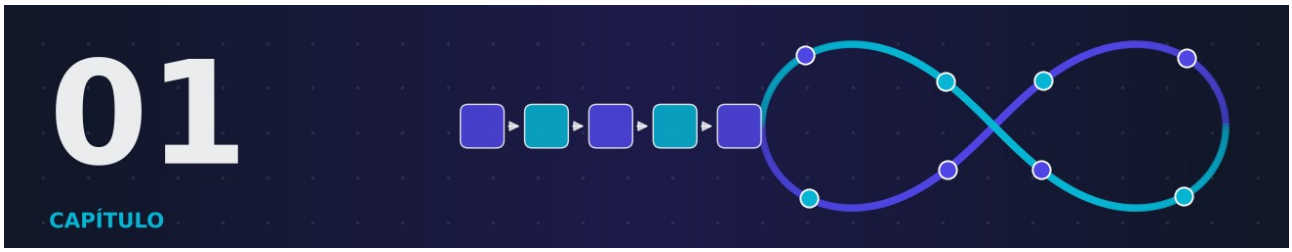
PARTE I

Parte I — Fundamentos de MLOps

Por que a maioria dos modelos não chega à produção, o que é MLOps, como é o ciclo de vida de um sistema de ML e por que a dívida técnica cresce em silêncio.

Nesta parte:

- Cap. 1 — Introdução ao MLOps: do Notebook à Produção
- Cap. 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade



Capítulo 1 — Introdução ao MLOps: do Notebook à Produção

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar, com uma analogia do cotidiano, por que treinar um modelo é só uma pequena parte do trabalho.
- Definir MLOps e distingui-lo de DevOps e de ciência de dados.
- Reconhecer os três eixos que mudam em um sistema de ML: código, dados e modelo.
- Descrever os princípios e os papéis envolvidos em um projeto de MLOps.
- Montar a estrutura básica de um projeto reprodutível em Python.
- Demonstrar, com código, por que a reprodutibilidade não acontece por acaso.

1.1 Uma história de cozinha

Imagine que você fez um **bolo de chocolate** incrível em casa. Sua família adorou. Agora o dono de uma padaria pede: "Preciso de 500 bolos por dia, todos iguais, com entrega às 7 horas, e quero saber se algum sabor mudou".

De repente, o problema é outro:

- Você precisa **anotar a receita exata**, com medidas, e não "um pouco de farinha".
- Precisa de **fornecedores confiáveis**. Se a farinha muda, o bolo muda.
- Precisa **testar** cada lote antes de vender.
- Precisa **vigiar** a qualidade todos os dias.
- Precisa de uma **equipe** com funções claras.

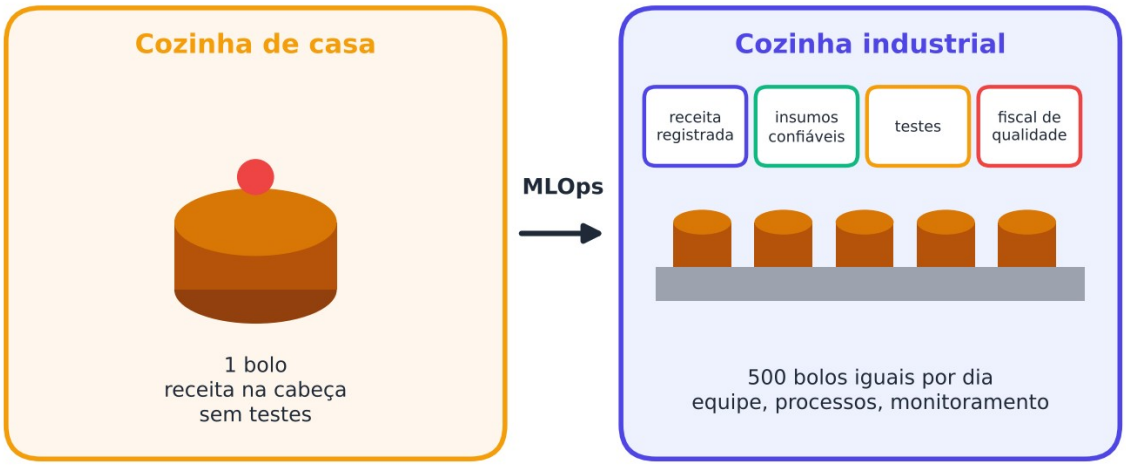


Figura 1.1 — Da cozinha de casa à cozinha industrial: fazer um bolo é diferente de fabricar 500 bolos iguais, todos os dias.

Com modelos de aprendizado de máquina (*machine learning*, ML) é exatamente assim. O **notebook** em que você treinou o modelo é a cozinha de casa. A **produção** é a cozinha industrial. **MLOps** é o conjunto de práticas que leva a receita de uma para a outra, sem perder a qualidade.

> MEMORIZE **Em uma frase**
>
> Treinar um modelo é fazer um bolo. **MLOps é montar a fábrica de bolos** e garantir que ela funcione todos os dias.

1.2 Por que tantos modelos não chegam (ou não ficam) em produção

Quando uma equipe treina um bom modelo, a sensação é de "missão cumprida". Mas a maior parte do esforço começa depois.

Em um artigo muito influente, Sculley e colegas (2015), do Google, observaram que, em sistemas reais de ML, **o código do modelo é uma parte pequena do todo**. Em volta dele há uma infraestrutura enorme: coleta de dados, verificação, extração de atributos, gerenciamento de recursos, ferramentas de análise, serviço e monitoramento.

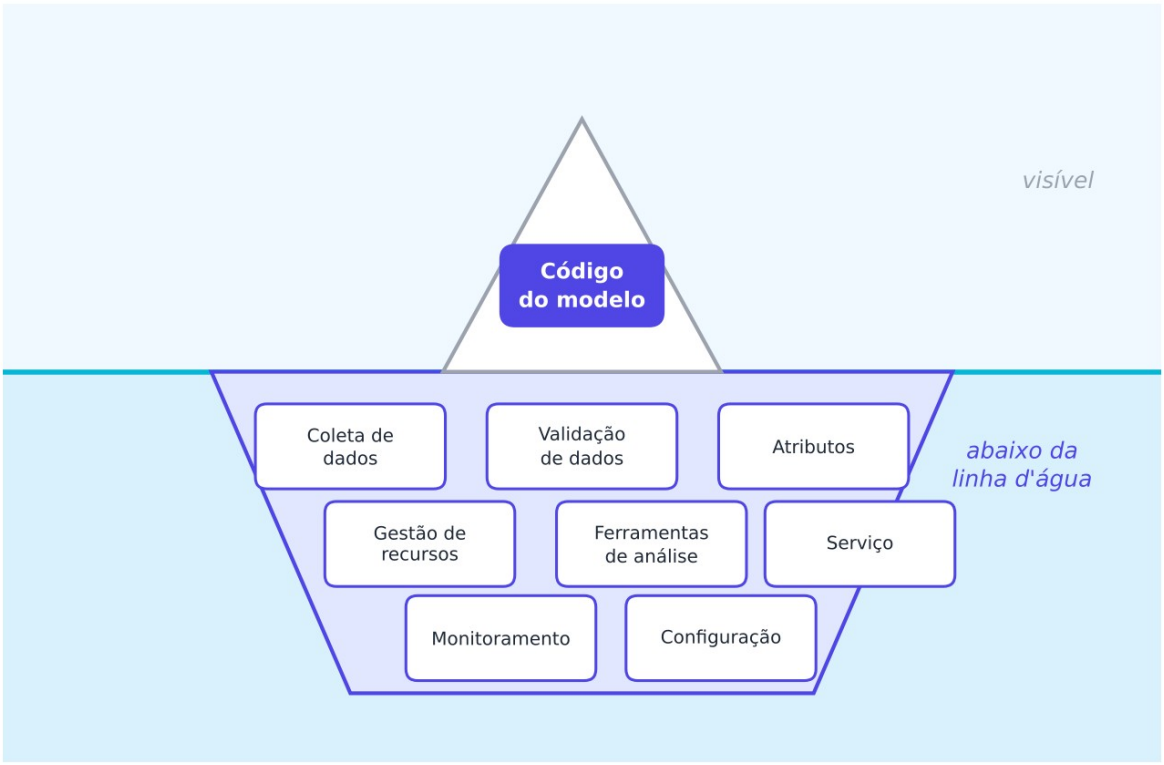


Figura 1.2 — O "iceberg" do ML: o código do modelo é a ponta visível. A maior parte do trabalho está embaixo da linha d'água. Inspirado em Sculley et al. (2015).

Paleyes, Urma e Lawrence (2022) revisaram estudos de caso de implantação de ML e organizaram os desafios ao longo das etapas do ciclo de vida: **coleta e preparação de dados, treinamento, verificação, implantação, monitoramento e atualização**. A mensagem é clara: os problemas aparecem em todas as etapas, e não apenas no treinamento.

Amershi e colegas (2019), da Microsoft, acrescentam que o ML traz dificuldades **diferentes** das do software tradicional: dados difíceis de descobrir e gerenciar, modelos difíceis de reutilizar e de modularizar, e equipes com culturas diferentes (cientistas, engenheiros de software) que precisam trabalhar juntas.

1.2.1 Sintomas de um projeto sem MLOps

Veja se algum destes sintomas soa familiar:

- "Funciona na minha máquina, mas não na do colega."
- "Qual versão dos dados gerou o modelo que está no ar?"
- "O modelo foi ótimo no teste, mas está ruim na vida real."
- "Ninguém sabe quem treinou o modelo nem com quais parâmetros."
- "Levamos três meses para colocar uma pequena melhoria no ar."
- "Só percebemos o problema quando um cliente reclamou."

> **No seu dia a dia**
>
> Pense no **GPS do carro**. O mapa muda: ruas são fechadas, surgem novas avenidas. Se o aplicativo usasse sempre o mapa de 2019, ele levaria você a lugares errados. Alguém precisa **atualizar, testar e vigiar** o sistema continuamente. Isso é MLOps.

1.3 O que é MLOps

A palavra vem da junção de **ML** (*machine learning*) com **Ops** (*operations*, operações). Kreuzberger, Kühl e Hirschl (2023) analisaram a literatura, as ferramentas disponíveis e entrevistaram especialistas. Eles propõem a seguinte ideia: MLOps é um **paradigma**, ou seja, um jeito de trabalhar que reúne boas práticas, conceitos e uma **cultura** de desenvolvimento. Ele cobre o ciclo de ponta a ponta: da concepção à implementação, ao monitoramento, à implantação e à escala dos produtos de ML.

Em palavras simples:

> MEMORIZE **Definição prática**
>
> **MLOps** é o conjunto de práticas, processos e ferramentas para **desenvolver, entregar, operar e melhorar** sistemas de ML de forma **confiável, repetível e contínua**.

1.3.1 Lembrando do DevOps

DevOps nasceu para aproximar quem **desenvolve** o software (Dev) de quem o **opera** (Ops). Ideias centrais: automatizar a construção e os testes, entregar em pequenos passos e monitorar o que está no ar.

Forsgren, Humble e Kim (2018) mostraram, com pesquisa em larga escala, que equipes de alto desempenho se destacam em quatro medidas:

- **Frequência de implantação:** com que frequência colocam mudanças no ar.
- **Tempo de entrega:** quanto tempo uma mudança leva para chegar ao ar.
- **Taxa de falha de mudança:** que fração das mudanças causa problemas.
- **Tempo de recuperação:** quanto tempo levam para consertar um problema.

Essas quatro medidas servem muito bem também para sistemas de ML.

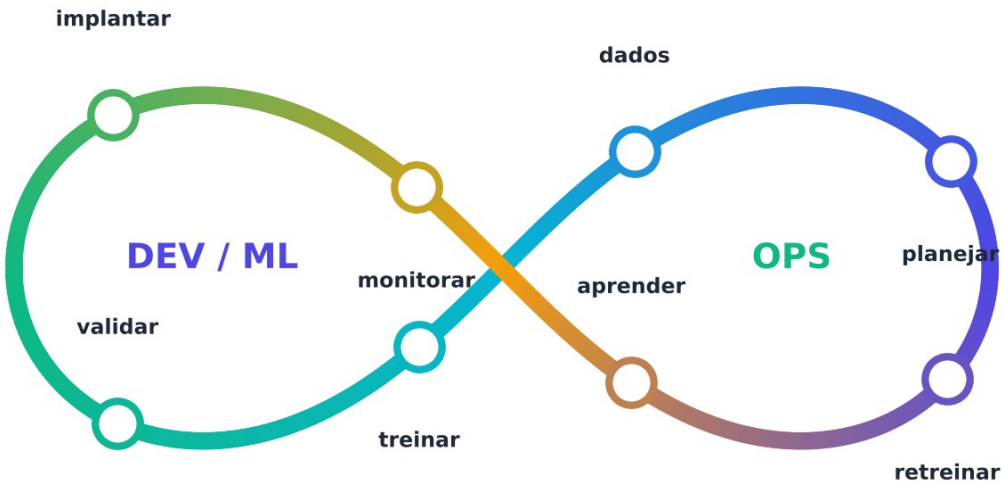


Figura 1.3 — O laço do MLOps: planejar, preparar dados, treinar, validar, implantar, monitorar e voltar ao começo. O lado "Dev" (esquerda) e o lado "Ops" (direita) se alimentam.

1.3.2 Por que o ML precisa de algo além do DevOps

No software tradicional, **o código define o comportamento**. Você escreve regras, e o programa as segue.

No ML, **os dados definem o comportamento**. O código apenas descreve *como aprender*. Mudando os dados, muda o programa que sai do treinamento.

Aspecto	Software tradicional	Sistema de ML
Fonte do comportamento	Regras escritas por pessoas	Padrões aprendidos dos dados
O que se versiona	Código	Código, dados, modelo e configuração
Teste	Entradas e saídas exatas	Desempenho estatístico, limites e comparações
Como falha	Erro, exceção, travamento	Respostas erradas "silenciosas", sem erro algum
Envelhecimento	Só se o código mudar	O mundo muda e o modelo envelhece sozinho
Equipe típica	Engenharia de software	Dados, ciência, engenharia e negócio

A linha mais perigosa é a "falha silenciosa". Um modelo que ficou desatualizado **continua respondendo**, sem nenhuma mensagem de erro. Só que as respostas passam a ser piores.

> **Atenção**
>
> Em ML, "o sistema está no ar" **não significa** "o sistema está certo". É preciso medir a **qualidade das previsões**, e não apenas a disponibilidade do servidor.

1.4 Os três eixos da mudança: código, dados e modelo

Em um projeto de ML, três coisas mudam ao mesmo tempo:

1. **Código:** a lógica de preparação, o treinamento, a API.
2. **Dados:** novos exemplos, novas colunas, novos formatos, novos erros.
3. **Modelo:** o arquivo treinado, com seus parâmetros e hiperparâmetros.

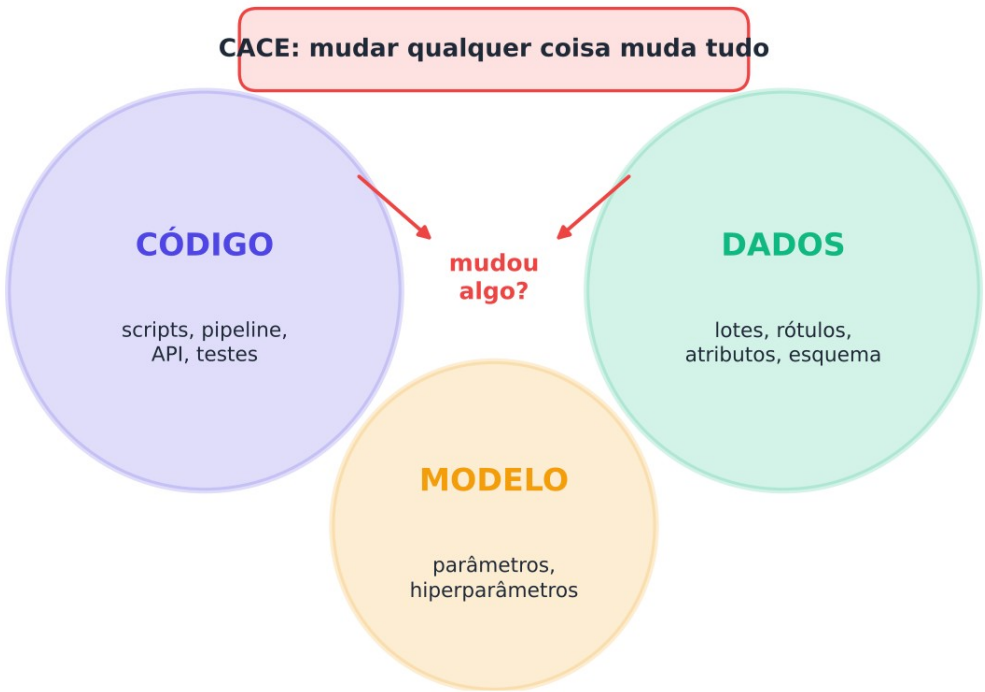


Figura 1.4 — Os três eixos que mudam em um sistema de ML. Para reproduzir um resultado é preciso saber a versão de cada um deles.

Sculley e colegas (2015) resumem essa dificuldade no princípio **CACE**: *Changing Anything Changes Everything* ("mudar qualquer coisa muda tudo"). Se você muda uma coluna de entrada, o modelo inteiro pode se comportar de outro jeito.

> **No seu dia a dia**
>
> Receita de bolo: se você troca o fabricante do chocolate (**dado**), o resultado muda mesmo que a receita (**código**) e o forno (**ambiente**) sejam os mesmos. Para repetir o bolo é preciso anotar *tudo*.

1.5 Princípios do MLOps

Os princípios abaixo aparecem de forma recorrente na literatura (KREUZBERGER; KÜHL; HIRSCHL, 2023; JOHN; OLSSON; BOSCH, 2021). Eles guiam todo o livro.

- **Automação:** o que se repete deve ser feito por máquina (treinar, testar, implantar).
- **Reprodutibilidade:** qualquer pessoa deve conseguir refazer o resultado.
- **Versionamento:** código, dados, modelos e configurações têm histórico.
- **Rastreabilidade:** saber quem fez o quê, quando e com quais dados.
- **Teste contínuo:** testar o código, os dados *e* o modelo.
- **Monitoramento:** acompanhar a qualidade em produção.
- **Colaboração:** equipes diferentes falam a mesma língua.
- **Governança:** responsabilidade, privacidade, justiça e conformidade.

1.5.1 CI, CD e CT

Três siglas aparecem o tempo todo:

- **CI** (*Continuous Integration*, integração contínua): a cada mudança no código, rodam testes automaticamente.
- **CD** (*Continuous Delivery/Deployment*, entrega/implantação contínua): a mudança aprovada chega ao ambiente de produção de forma automática e segura.
- **CT** (*Continuous Training*, treinamento contínuo): o modelo é retreinado automaticamente quando há dados novos ou queda de qualidade. Esta é a peça **própria do ML**.

Humble e Farley (2010) são a referência clássica para CI/CD. O CT é a contribuição específica do MLOps.

1.6 Quem faz o quê: os papéis

Em equipes pequenas, uma pessoa faz tudo. Em equipes maiores, os papéis se separam. Todos precisam entender o conjunto.

Papel	O que faz	Pergunta que responde
Cientista de dados	Explora dados, cria e avalia modelos	"Isso resolve o problema do negócio?"
Engenheiro de dados	Constrói e mantém as fontes e os fluxos de dados	"Os dados chegam certos e no prazo?"
Engenheiro de ML	Transforma o modelo em produto confiável	"Isso roda de forma estável e escalável?"
Engenheiro de plataforma / SRE	Cuida da infraestrutura e da confiabilidade	"O ambiente aguenta a carga?"

Papel	O que faz	Pergunta que responde
Responsável pelo produto	Define objetivos e métricas de negócio	"Isso gera valor?"
Governança / jurídico	Cuida de riscos, privacidade e conformidade	"Isso é seguro, justo e legal?"

1.7 Mão na massa: preparando o laboratório

Todo projeto de MLOps começa por **organização**. Uma estrutura clara de pastas evita bagunça e facilita a automação. Este é o esqueleto que usaremos ao longo do livro:

```bash
meu_projeto_mlops/
├── data/
│   ├── raw/            # dados originais (nunca editar à mão)
│   └── processed/      # dados tratados
├── src/
│   ├── dados.py        # carga e validação
│   ├── treino.py       # pipeline e treinamento
│   └── servir.py       # API de previsão
├── tests/              # testes automatizados
├── models/             # modelos treinados
├── notebooks/          # exploração (não é produção!)
├── requirements.txt    # bibliotecas e versões
└── README.md           # como rodar tudo
```

> **Dica**
>
> O **notebook é ótimo para explorar**, mas ruim para produzir. Ele guarda estado escondido (células executadas fora de ordem) e é difícil de testar. Quando uma ideia amadurece, mova o código para arquivos `.py` em `src/`.

1.7.1 Fixando as versões das bibliotecas

Um arquivo `requirements.txt` registra as versões usadas. Assim, o ambiente pode ser recriado:

```bash
# requirements.txt (exemplo)
numpy==2.5.4
pandas==3.0.5
scikit-learn==1.9.1
joblib==1.6.0
```

Para instalar: `pip install -r requirements.txt`. Use um **ambiente virtual** (`python -m venv .venv`) para isolar cada projeto.

1.7.2 O primeiro experimento de reprodutibilidade

Veja um teste simples. Vamos treinar uma floresta aleatória (*random forest*) duas vezes **sem fixar a semente** aleatória. O que acontece com os resultados?

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
```

```
X, y = load_breast_cancer(return_X_y=True)

def treinar(semente_modelo, semente_divisao):
    Xtr, Xte, ytr, yte = train_test_split(
        X, y, test_size=0.3, random_state=semente_divisao
    )
    modelo = RandomForestClassifier(n_estimators=30, random_state=semente_modelo)
    modelo.fit(Xtr, ytr)
    return modelo.score(Xte, yte)

# Sem fixar sementes (None = aleatório a cada chamada)
sem_semente = [treinar(None, None) for _ in range(5)]
# Fixando as sementes
com_semente = [treinar(42, 42) for _ in range(5)]

print("Sem semente:", np.round(sem_semente, 3))
print("Com semente:", np.round(com_semente, 3))
```

Sem semente, cada execução dá um número ligeiramente diferente. Com a semente fixa, todas dão o mesmo resultado. **Parece pouco, mas é a base da reprodutibilidade.**

> **Atenção**
>
> Fixar a semente é só o começo. Também é preciso fixar as **versões das bibliotecas**, os **dados** e o **ambiente**. Pineau e colegas (2021) discutem, em um periódico de referência, como a falta desses cuidados prejudica a reprodutibilidade da pesquisa em ML.

1.7.3 Registrando o "rastro" de uma execução

Uma prática simples e poderosa: sempre salvar, junto do modelo, **os metadados** que dizem como ele foi feito.

```
PYTHON
import json, platform, sys
from datetime import datetime, timezone
import sklearn

metadados = {
    "data_hora_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    "python": sys.version.split()[0],
    "sistema": platform.system(),
    "scikit_learn": sklearn.__version__,
    "semente": 42,
    "dados": "load_breast_cancer (UCI WDBC)",
}
print(json.dumps(metadados, indent=2, ensure_ascii=False))
```

No Capítulo 5, vamos substituir esta ideia caseira por uma ferramenta de rastreamento de experimentos (MLflow).

1.8 Maturidade: onde estamos?

Nem todo projeto precisa de uma plataforma completa no primeiro dia. John, Olsson e Bosch (2021) propõem um modelo de maturidade para o MLOps, e a literatura costuma descrever uma evolução em **estágios**:

1. **Manual:** tudo em notebooks; implantação manual; sem monitoramento.

2. **Pipeline automatizado de treinamento:** o treino é um fluxo repetível e versionado.
3. **Entrega automatizada (CI/CD):** testes e implantação automáticos.
4. **Operação contínua:** monitoramento, retreinamento e governança integrados.

Voltaremos a esse assunto no Capítulo 2. Por enquanto, guarde: **o objetivo é avançar passo a passo**, resolvendo a dor mais urgente de cada vez.

1.9 Mapa do livro

O livro segue o ciclo de vida do modelo:

- **Parte I:** fundamentos (você está aqui).
- **Parte II:** dados, pipelines, experimentos e avaliação.
- **Parte III:** empacotamento, testes, CI/CD, monitoramento e retreinamento.
- **Parte IV:** governança, explicabilidade e escala (feature stores, orquestração, LLMOps).
- **Parte V:** projeto final de ponta a ponta.

> **Saiba mais**
>
> Para uma visão prática e abrangente de sistemas de ML em produção, consulte Huyen (2022), *Designing Machine Learning Systems*. Para a visão de dívida técnica, leia Sculley et al. (2015). Para uma revisão acadêmica de MLOps, Kreuzberger, Kühl e Hirschl (2023).

1.10 Resumo do capítulo

> MEMORIZE **O que levar deste capítulo**
>
> - O notebook é a cozinha de casa; a produção é a cozinha industrial. **MLOps liga as duas.**
> - O código do modelo é uma **pequena parte** de um sistema de ML (SCULLEY et al., 2015).
> - No ML, **dados definem o comportamento**. Três eixos mudam: **código, dados e modelo**.
> - Falhas de ML costumam ser **silenciosas**: o sistema responde, mas responde pior.
> - Princípios: automação, reprodutibilidade, versionamento, rastreabilidade, teste, monitoramento, colaboração e governança.
> - **CI, CD e CT** são as três engrenagens da automação.
> - Comece organizado: pastas claras, versões fixas, sementes fixas, metadados salvos.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Na analogia do capítulo, treinar um modelo no notebook equivale a:

a) montar uma fábrica de bolos.
b) entregar bolos todos os dias a uma padaria.
c) fiscalizar a qualidade dos bolos vendidos.
d) fazer um bolo na cozinha de casa.

Questão 2. MLOps pode ser definido, de forma prática, como:

a) o conjunto de práticas para desenvolver, entregar, operar e melhorar sistemas de ML de forma confiável e repetível.
b) uma técnica de redução de dimensionalidade.
c) o nome de um algoritmo de aprendizado profundo.
d) uma biblioteca do Python para treinar modelos mais rápido.

Questão 3. Segundo Sculley et al. (2015), em sistemas reais de ML, o código do modelo é:

a) uma pequena parte do sistema, cercada por muita infraestrutura.
b) a parte maior e mais cara do sistema.
c) irrelevante para o funcionamento.
d) a única parte que precisa de testes.

Questão 4. A falha silenciosa em ML significa que:

a) o modelo recusa todas as entradas.
b) o código gera uma exceção que ninguém lê.
c) o sistema continua respondendo, mas as respostas ficam piores sem nenhum erro aparente.
d) o servidor deixa de responder sem aviso.

Questão 5. Qual alternativa descreve melhor a diferença central entre software tradicional e sistema de ML?

a) No ML, os dados ajudam a definir o comportamento; no software tradicional, o código o define.
b) No ML o comportamento nunca muda.
c) No ML não há código.
d) No software tradicional não há testes.

Questão 6. Os três eixos que mudam em um sistema de ML são:

a) precisão, revocação e F1.
b) código, dados e modelo.
c) servidor, rede e banco de dados.
d) treino, teste e validação.

Questão 7. O princípio CACE ("Changing Anything Changes Everything") alerta para o fato de que:

a) mudar um componente pode alterar o comportamento de todo o sistema.
b) cada componente é independente dos demais.
c) mudanças só afetam a interface.
d) apenas mudanças no código são relevantes.

Questão 8. O que significa CI (integração contínua)?

a) Os dados são coletados continuamente de sensores.
b) O modelo é retreinado automaticamente quando os dados mudam.
c) A cada mudança de código, verificações e testes rodam automaticamente.
d) O modelo é implantado manualmente por um engenheiro.

Questão 9. O CT (treinamento contínuo) é a prática que:

a) corrige bugs de código automaticamente.
b) elimina a necessidade de testes.
c) retreina o modelo automaticamente em resposta a dados novos, queda de qualidade ou agendamento.
d) substitui o monitoramento.

Questão 10. Entre as quatro medidas de desempenho de entrega de Forsgren, Humble e Kim (2018) está:

a) a taxa de falha de mudança.
b) a quantidade de parâmetros do modelo.
c) o número de linhas de código.
d) o tamanho do conjunto de teste.

Questão 11. Qual dos papéis é responsável principalmente por transformar o modelo em um produto estável e escalável?

a) Engenheiro de ML.
b) Responsável pelo produto, apenas.
c) Analista de planilhas.
d) Auditor externo, apenas.

Questão 12. Por que o notebook é inadequado como ambiente final de produção?

a) Só funciona em computadores antigos.
b) Guarda estado escondido, executa células fora de ordem e é difícil de testar.
c) Não aceita bibliotecas externas.
d) Não permite usar Python.

Questão 13. Em qual arquivo se registram as versões das bibliotecas para recriar o ambiente?

a) modelo.joblib
b) requirements.txt
c) README.md
d) config.yaml do modelo

Questão 14. Treinar uma floresta aleatória duas vezes sem fixar a semente tende a:

a) impedir o treinamento.
b) aumentar sempre a acurácia.
c) produzir resultados ligeiramente diferentes.
d) produzir sempre resultados idênticos.

Questão 15. Fixar a semente aleatória, por si só, é:

a) suficiente para garantir reprodutibilidade completa.
b) proibido em produção.
c) desnecessário em qualquer situação.
d) apenas o começo: também é preciso fixar versões, dados e ambiente.

Questão 16. Qual é a vantagem de salvar metadados junto do modelo?

a) Permite saber como e com o que o modelo foi feito.
b) Aumenta automaticamente a acurácia.
c) Reduz o tamanho do conjunto de dados.
d) Dispensa o versionamento de código.

Questão 17. Segundo Amershi et al. (2019), uma das dificuldades do ML em relação ao software tradicional é:

a) a inexistência de bibliotecas.
b) a ausência de equipes envolvidas.
c) lidar com dados difíceis de descobrir e gerenciar e com modelos difíceis de reutilizar.
d) o fato de não envolver nenhuma engenharia.

Questão 18. Paleyes, Urma e Lawrence (2022) organizaram os desafios de implantação de ML:

a) somente na etapa de interface com o usuário.
b) ao longo das etapas do ciclo de vida, de dados a monitoramento.
c) apenas na escolha do algoritmo.
d) apenas na etapa de treinamento.

Questão 19. Qual das seguintes é uma prática de governança mencionada entre os princípios do MLOps?

a) Evitar qualquer documentação.
b) Treinar sempre com todos os dados disponíveis.
c) Cuidar de privacidade, justiça e conformidade.
d) Manter o modelo sem versão.

Questão 20. O que o monitoramento em ML precisa medir, além da disponibilidade do servidor?

a) Apenas o tempo de resposta.
b) Somente o número de usuários.
c) A qualidade das previsões.
d) Apenas o consumo de memória.

Questão 21. A estrutura de pastas data/raw e data/processed serve para:

a) armazenar senhas.
b) separar dados originais (nunca editados à mão) de dados tratados.
c) guardar os testes automáticos.
d) guardar apenas modelos.

Questão 22. O objetivo do MLOps em termos de maturidade é:

a) usar sempre a ferramenta mais cara.
b) evitar qualquer automação.
c) avançar passo a passo, resolvendo a dor mais urgente de cada vez.
d) implantar toda a plataforma no primeiro dia.

Questão 23. Em um estágio de maturidade "manual", o que normalmente se observa?

a) Retreino automático e governança integrada.
b) Entrega totalmente automatizada.
c) Testes de dados e de modelo em cada mudança.
d) Notebooks, implantação manual e ausência de monitoramento.

Questão 24. Qual a função de um ambiente virtual (venv) em Python?

a) Isolar as bibliotecas de cada projeto.
b) Criptografar o modelo.
c) Aumentar o desempenho do processador.
d) Armazenar o conjunto de dados.

Questão 25. Uma sentença como "funciona na minha máquina, mas não na do colega" é sintoma típico de:

a) excesso de testes automáticos.
b) ausência de modelo.
c) alta qualidade dos dados.
d) falta de reprodutibilidade do ambiente e das dependências.

Questão 26. Qual das alternativas lista corretamente papéis e perguntas?

a) SRE: "qual a cor do gráfico?"
b) Auditor jurídico: "qual a melhor taxa de aprendizado?"
c) Cientista de dados: "qual a latência do servidor de arquivos?"
d) Engenheiro de dados: "os dados chegam certos e no prazo?"

Questão 27. Qual combinação resume a "automação" no MLOps?

a) SQL, NoSQL e CSV.
b) CPU, GPU e TPU.
c) CI, CD e CT.
d) HTTP, HTML e CSS.

Questão 28. Em ML, falhas costumam ser silenciosas porque:

a) todo erro gera um alerta automático.
b) o modelo continua respondendo, ainda que as respostas pioram sem gerar exceção.
c) os dados nunca mudam.
d) o modelo sempre recusa entradas inválidas.

Questão 29. O que há de diferente no teste de sistemas de ML em comparação ao software tradicional?

a) Testa-se desempenho estatístico e comportamentos, além de entradas e saídas exatas.
b) Não se testa nada.
c) Só se testa a interface gráfica.
d) Testam-se apenas dados, nunca o código.

Questão 30. Qual a recomendação sobre o código que amadurece em um notebook?

a) Mantê-lo para sempre em células soltas.
b) Movê-lo para arquivos .py em src/, onde pode ser testado e automatizado.
c) Convertê-lo em imagem.
d) Apagá-lo após o primeiro uso.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com suas palavras e usando a analogia da cozinha, a diferença entre treinar um modelo e operar um sistema de ML em produção.
2. Descreva o que a figura do "iceberg" de Sculley et al. (2015) quer transmitir sobre sistemas de ML.
3. Por que se diz que, em ML, "os dados definem o comportamento" do sistema? Quais consequências isso traz para versionamento e testes?
4. O que é uma falha silenciosa? Dê um exemplo e explique como detectá-la.
5. Explique o princípio CACE e dê um exemplo de como ele aparece em um projeto de ML.
6. Diferencie CI, CD e CT, indicando qual é próprio do ML.
7. Quais são os quatro indicadores de desempenho de entrega de Forsgren, Humble e Kim (2018)? Como aplicá-los a um projeto de ML?
8. Cite três motivos pelos quais fixar a semente aleatória não basta para garantir reprodutibilidade.
9. Por que é importante salvar metadados (versão das bibliotecas, semente, hash dos dados) ao lado do modelo?
10. Proponha uma estrutura de pastas para um projeto de ML e justifique a função de cada pasta.
11. Descreva três sintomas de um projeto de ML sem práticas de MLOps.
12. Compare brevemente o papel do cientista de dados e do engenheiro de ML em uma equipe.
13. Explique o que são estágios de maturidade em MLOps e por que não é necessário alcançar o mais alto em todo projeto.
14. Por que o monitoramento de ML não pode se limitar à disponibilidade do servidor?
15. Descreva o experimento do capítulo sobre reprodutibilidade e a conclusão que dele se tira.
16. Qual a importância de um ambiente virtual e de um arquivo de requisitos no trabalho em equipe?
17. Por que as equipes de ML e de engenharia de software precisam de uma linguagem comum? Dê um exemplo prático.
18. Por que um modelo precisa ser avaliado também em produção, e não apenas antes da implantação?
19. Explique por que "colocar o modelo no ar" não encerra o projeto de ML. Que atividades continuam depois da implantação?
20. Dê dois exemplos de decisões que podem ser tomadas com previsões em lote e dois que exigem previsões em tempo real, justificando a escolha do padrão em cada caso.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — O custo de não fixar a semente

NÍVEL: BÁSICO

Treine uma `RandomForestClassifier(n_estimators=20)` 15 vezes no conjunto *Breast Cancer Wisconsin* (UCI), usando a mesma divisão treino/teste (`random_state=0`), mas **sem** fixar a semente do modelo. Repita com a semente fixa. Compare a média e o desvio padrão da acurácia nos dois casos.

Dica: a semente do modelo é o parâmetro `random_state` do estimador.

Exercício 2 — Salvando o modelo com seus metadados

NÍVEL: BÁSICO

Treine um pipeline (`StandardScaler` + `LogisticRegression`) no conjunto *Wine* (UCI). Salve em um único arquivo `joblib` um dicionário com o pipeline, a versão do scikit-learn, a semente, a acurácia de teste e o hash SHA-256 dos dados. Recarregue o arquivo e confirme que as previsões são idênticas.

Exercício 3 — A ordem dos dados também importa

NÍVEL: INTERMEDIÁRIO

Alguns algoritmos dependem da ordem em que veem os exemplos. Treine um `SGDClassifier` (com `random_state=0`) no conjunto *Digits* (UCI) em três versões: dados na ordem original, dados embaralhados com a semente 1 e dados embaralhados com a semente 2. Mostre a acurácia de teste e quantas previsões diferem entre as versões. Em seguida, treine uma `LogisticRegression` do mesmo modo e compare.

Exercício 4 — As quatro medidas do DevOps aplicadas a um modelo

NÍVEL: INTERMEDIÁRIO

Crie um `DataFrame` com o histórico de 12 implantações de modelos: data do commit, data da implantação, se causou falha (booleano) e, quando houve falha, o tempo de recuperação em horas. Calcule: frequência de implantação por mês, tempo médio de entrega (commit até implantação, em dias), taxa de falha de mudança e tempo médio de recuperação.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Wine Quality, Heart Disease e Adult (Census Income). Escolha um e repita o projeto, criando a estrutura e testando a reprodutibilidade.
> - **Kaggle:** Titanic e *House Prices*, clássicos para praticar organização de projeto e reprodutibilidade.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Esqueleto de um projeto reprodutível

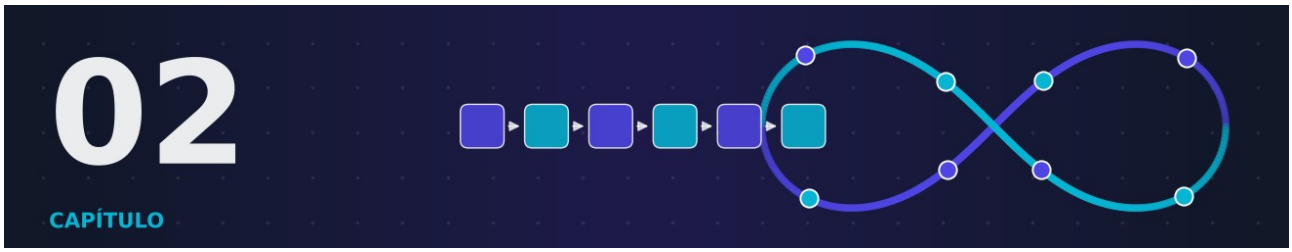
Contexto. Sua equipe vai começar um projeto de classificação com o conjunto *Breast Cancer Wisconsin* (UCI). Antes de qualquer modelagem, o gerente quer ver que o projeto é **reprodutível**.

Tarefas.

1. Crie, em uma pasta temporária, a estrutura `data/`, `src/`, `tests/`, `models/` e um `README.md`.
2. Gere um `requirements.txt` com as versões instaladas de `numpy`, `pandas`, `scikit-learn` e `joblib` (use `importlib.metadata`).
3. Escreva em `src/treino.py` um script que treina um pipeline com semente fixa e imprime um **hash** das previsões de probabilidade para todo o conjunto.
4. Execute o script **duas vezes** em processos separados (`subprocess`) e verifique se os hashes são iguais.
5. Execute uma terceira vez com outra semente e mostre que o hash muda.

Entregáveis. Estrutura criada, `requirements.txt`, saída das três execuções e uma frase concluindo o que garante a reprodutibilidade.

A solução de referência está no Apêndice E.



Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Descrever as etapas do ciclo de vida de um sistema de ML.
- Traduzir um problema de negócio em objetivo, métrica e linha de base.
- Explicar o conceito de dívida técnica em ML e dar exemplos concretos.
- Usar o ML Test Score como lista de verificação de prontidão para produção.
- Reconhecer os componentes de uma arquitetura de referência de MLOps.
- Avaliar o nível de maturidade de um projeto e escolher o próximo passo.

2.1 O ciclo de vida de um sistema de ML

Todo produto de ML passa por etapas. Amershi e colegas (2019) descreveram um fluxo de trabalho com **nove etapas**, observado em equipes da Microsoft:

1. **Requisitos do modelo:** o que o modelo deve fazer?
2. **Coleta de dados:** de onde vêm os dados?
3. **Limpeza de dados:** corrigir erros e lacunas.
4. **Rotulagem de dados:** definir as respostas corretas (quando necessário).
5. **Engenharia de atributos:** criar as entradas do modelo (*features*).
6. **Treinamento do modelo.**
7. **Avaliação do modelo.**
8. **Implantação do modelo.**
9. **Monitoramento do modelo.**

Um **atributo** (*feature*) é uma característica usada como entrada. Em um modelo de crédito, "renda mensal" e "idade" são atributos.



Figura 2.1 — O ciclo de vida de ML é um laço, não uma linha reta. O monitoramento alimenta novos requisitos, novos dados e novos treinos.

O ponto mais importante: **o ciclo é um laço**. O que se aprende no monitoramento volta para o começo. Os próprios autores notam que o processo é bem mais iterativo e cheio de idas e vindas do que o do software tradicional.

2.1.1 Três grandes fases

Para organizar o raciocínio, agrupamos as etapas em três fases:

- **Projeto:** entender o problema, os requisitos e a viabilidade.
- **Desenvolvimento:** dados, atributos, treinamento e avaliação.
- **Operação:** implantação, monitoramento e manutenção.

Este livro dedica uma parte a cada fase, e o MLOps cuida de **costurar** as três.

2.2 Do problema de negócio à métrica

Antes de escrever qualquer código, responda a quatro perguntas:

1. **Qual decisão** o modelo vai apoiar?
2. **Quem** usará a previsão e **quando**?
3. **Como saberemos** que deu certo? (métrica de negócio)
4. **Quanto custa errar**, e que tipo de erro custa mais?

Exemplo: prever cancelamento de clientes

Uma empresa de assinaturas quer reduzir cancelamentos.

- **Decisão:** a quem oferecer um desconto de retenção.
- **Métrica de negócio:** redução do cancelamento líquido em 3 meses.
- **Métrica do modelo:** a precisão entre os 10% de clientes com maior risco (*precision@10%*).
- **Custo do erro:** dar desconto a quem ficaria (perde margem) é mais barato do que perder quem sairia.

2.2.1 Métrica do modelo não é métrica do negócio

Um modelo pode ter ótima **acurácia** (fração de acertos) e ainda assim não gerar valor. Por exemplo, se só 2% dos clientes cancelam, um modelo que sempre responde "não vai cancelar" acerta 98%... e é inútil. Por isso é essencial ter uma **linha de base** (*baseline*): uma solução simples contra a qual o modelo precisa ganhar.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

base = DummyClassifier(strategy="most_frequent").fit(Xtr, ytr)
modelo = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(Xtr, ytr)

print("Linha de base (sempre a classe mais comum):", round(base.score(Xte, yte), 3))
print("Regressão logística:", round(modelo.score(Xte, yte), 3))
```

Dica

Comece sempre pela solução mais simples possível (uma regra, uma média, um modelo linear). Se um modelo sofisticado não supera isso por uma margem que compense o custo, **não vale a pena colocá-lo em produção**.

2.2.2 Requisitos não funcionais

Além da qualidade das previsões, defina:

- **Latência:** quanto tempo uma previsão pode demorar (por exemplo, 100 ms).
- **Vazão** (*throughput*): quantas previsões por segundo.
- **Custo:** quanto custa treinar e servir.
- **Explicabilidade:** é preciso justificar a decisão?
- **Privacidade e conformidade:** há dados pessoais?

2.3 Dívida técnica em sistemas de ML

Imagine que você compra um carro a prestação. Você leva o carro hoje, mas **paga juros** todos os meses. A **dívida técnica** é parecida. Ao tomar atalhos no desenvolvimento, você entrega mais rápido hoje, mas **paga juros** depois, em forma de manutenção cara, bugs e lentidão para mudar.

Sculley e colegas (2015) mostraram que o ML tem todos os problemas de dívida do software comum, **mais** um conjunto próprio de problemas, que existem no nível do sistema e não do código.

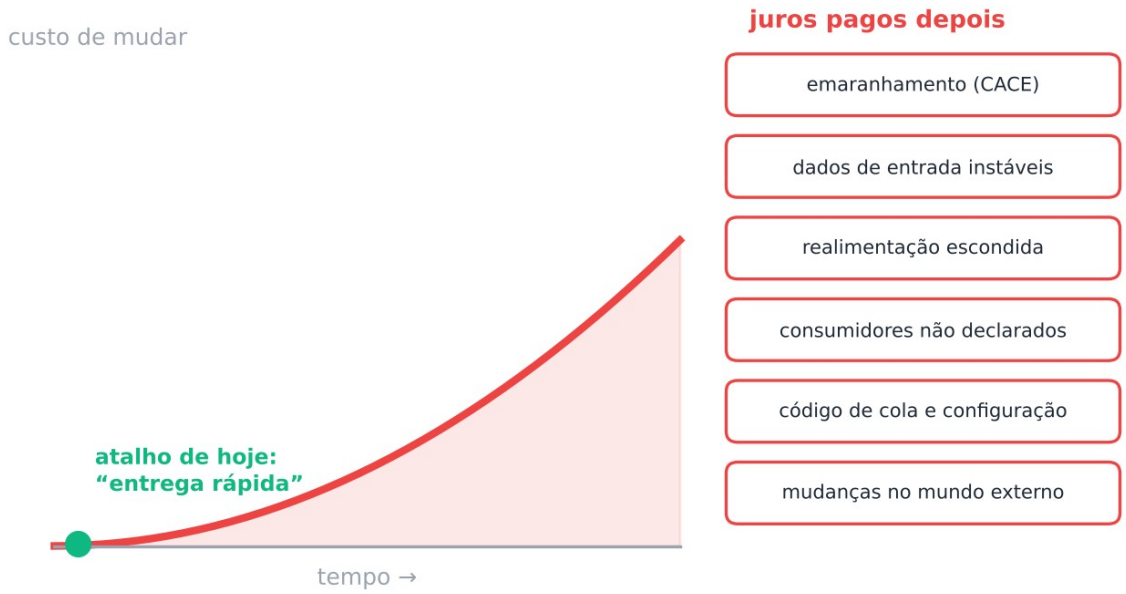


Figura 2.2 — Os "juros" da dívida técnica de ML: atalhos de hoje viram custos de manutenção amanhã. Baseado nos tipos descritos por Sculley et al. (2015).

2.3.1 Principais tipos de dívida

1. Emaranhamento (CACE). Os atributos e componentes se influenciam de modo difícil de prever. Mudar um muda todos.

2. Dependências de dados instáveis. O modelo usa uma entrada que é produzida por outro sistema, que pode mudar sem aviso. É como construir uma casa em terreno que outra pessoa pode mexer.

3. Dependências de dados subutilizadas. Atributos que quase não ajudam, mas que ficam no modelo e criam riscos.

4. Ciclos de realimentação escondidos. Quando o modelo influencia os dados que ele próprio usará no futuro. Exemplo: um recomendador só mostra o que já é popular, e por isso só o popular recebe cliques.

5. Consumidores não declarados. Outros sistemas passam a usar a saída do seu modelo sem você saber. Uma mudança sua quebra o sistema deles.

6. Código de cola e "selva de pipelines". Muito código só para conectar bibliotecas e preparar dados, sem estrutura. Fica difícil de testar e de manter.

7. Código experimental morto. Ramos de experimentos antigos que nunca foram removidos.

8. Dívida de configuração. Muitos parâmetros espalhados em arquivos, linhas de comando e variáveis, sem revisão e sem testes.

9. Mudanças no mundo externo. Limiares, comportamento dos clientes e fontes de dados mudam. O modelo envelhece.

> **No seu dia a dia**
>
> O **ciclo de realimentação** é como um aluno que só estuda o que já sabe. O professor percebe que ele vai bem nesse assunto e dá ainda mais exercícios dele. O aluno nunca aprende o resto, e as notas "provam" que ele é ótimo no que já sabia.

2.3.2 Simulando um ciclo de realimentação

Veja uma simulação pequena. Um sistema mostra os 3 itens mais populares a cada visitante. Todos os itens têm a **mesma qualidade real**. O que acontece com os cliques?

```python
import numpy as np

rng = np.random.default_rng(0)
n_itens, n_visitas = 10, 5000
qualidade = np.full(n_itens, 0.10)   # todos igualmente bons
cliques = np.ones(n_itens)           # contagem inicial (suavização)

for _ in range(n_visitas):
    top3 = np.argsort(-cliques)[:3]  # só mostra os mais populares
    for item in top3:
        if rng.random() < qualidade[item]:
            cliques[item] += 1

print("Cliques por item:", cliques.astype(int))
print("Participação do item mais clicado:", round(cliques.max() / cliques.sum(), 2))
```

Mesmo com qualidade idêntica, **três itens ficam com praticamente todos os cliques** (cerca de 500 cada) e os outros sete nunca voltam a ser mostrados. O sistema "aprendeu" uma desigualdade que ele mesmo criou. Em produção, isso é detectado com monitoramento e com experimentos controlados (Capítulos 9 e 10).

2.4 ML Test Score: uma lista de verificação para produção

Breck e colegas (2017) propuseram uma **rubrica** com 28 testes e monitoramentos, organizados em quatro grupos. Ela responde à pergunta: "meu sistema está pronto para produção?"

Grupo	O que verifica	Exemplos de itens
Dados e atributos	Qualidade e adequação das entradas	Expectativas dos atributos registradas em um esquema; atributos úteis; código de atributos testado
Desenvolvimento do modelo	Boas práticas de modelagem	Hiperparâmetros ajustados; modelo simples comparado; qualidade verificada em subgrupos
Infraestrutura de ML	Confiabilidade do pipeline	Treino reproduzível; pipeline testado de ponta a ponta; reversão (*rollback*) possível
Monitoramento	O que acontece em produção	Invariantes dos dados verificadas; modelo não está obsoleto; qualidade de previsão sem regressões

2.4.1 Como pontuar

Para cada teste:

- **0 ponto:** não é feito.
- **0,5 ponto:** é feito manualmente e está documentado.
- **1 ponto:** é feito de forma **automática** e recorrente.

Soma-se cada grupo. A nota final é o **menor** dos quatro totais, porque a corrente é tão forte quanto o elo mais fraco. Segundo os autores, uma nota baixa indica risco de produção e uma nota alta indica um sistema mais robusto.

Podemos criar uma pequena ferramenta para isso:

```python
def ml_test_score(respostas):
    """respostas: dict grupo -> lista de pontos (0, 0.5 ou 1)."""
    totais = {g: sum(p) for g, p in respostas.items()}
    return min(totais.values()), totais

meu_projeto = {
    "dados":          [1, 0.5, 0, 0, 0.5, 0, 1],
    "modelo":         [0.5, 0, 0.5, 0, 0, 1, 0],
    "infraestrutura": [0.5, 0, 0, 1, 0, 0.5, 0],
    "monitoramento":  [0, 0, 0, 0, 0.5, 0, 0],
}
nota, por_grupo = ml_test_score(meu_projeto)
print("Totais por grupo:", por_grupo)
print("Nota final (mínimo):", nota)
```

> **Atenção**
>
> Esta é uma versão didática, com 7 itens por grupo e exemplos de pontuação inventados por nós. A rubrica original tem os itens exatos e a interpretação das faixas de pontuação. Consulte o artigo de Breck et al. (2017) para usá-la de verdade.

2.5 A arquitetura de referência

Kreuzberger, Kühl e Hirschl (2023) identificaram os componentes técnicos que aparecem em plataformas de MLOps. Aqui está uma versão simplificada:

Componente	Papel	Analogia da cozinha
Fontes de dados e engenharia de dados	Coletar e preparar os dados	Fornecedores e estoque
Armazém de atributos (*feature store*)	Guardar atributos reutilizáveis	Ingredientes já picados
Orquestração de fluxos de trabalho	Executar as etapas na ordem certa	Gerente de produção
Rastreamento de experimentos	Registrar execuções e resultados	Caderno de receitas testadas
Registro de modelos	Catalogar versões dos modelos	Livro oficial de receitas
CI/CD	Testar e implantar automaticamente	Linha de montagem com inspeção
Serviço de modelos	Entregar previsões	Balcão de atendimento
Monitoramento	Vigiar qualidade e saúde	Fiscal de qualidade



Figura 2.3 — Arquitetura de referência simplificada de MLOps, com os componentes que estudaremos ao longo do livro.

Não é preciso ter tudo isso desde o início. A escolha depende do tamanho do problema e da equipe.

2.6 Níveis de maturidade

Uma forma útil de planejar é pensar em **níveis**. Eles ajudam a decidir o próximo passo realista.

Nível	Nome	Como se trabalha	Dor típica	Próximo passo
0	Manual	Notebooks, scripts soltos, implantação manual	Ninguém reproduz o resultado	Versionar e fixar o ambiente
1	Treino automatizado	Pipeline de treino repetível, experimentos rastreados	Implantação ainda manual e arriscada	Testes e CI/CD
2	Entrega automatizada	CI/CD, registro de modelos, testes de dados e modelo	Modelo em produção piora sem ninguém ver	Monitoramento e CT
3	Operação contínua	Monitoramento, alertas, retreino automático, governança	Custos, escala, conformidade	Plataforma e governança

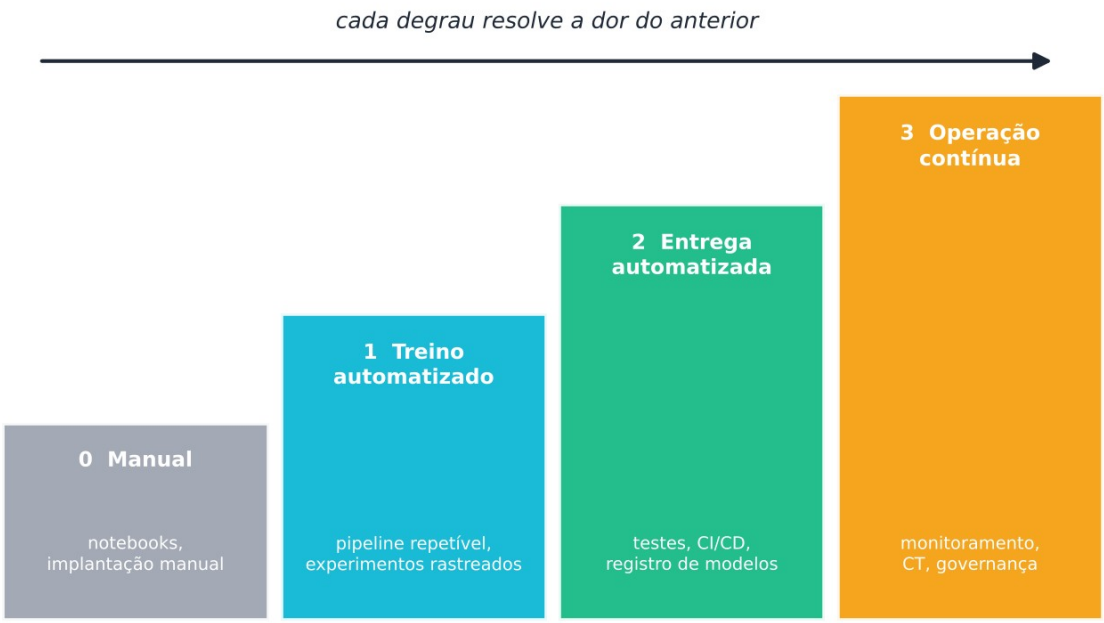


Figura 2.4 — Escada de maturidade do MLOps. Cada degrau resolve a dor do anterior.

John, Olsson e Bosch (2021) estruturam essa evolução em um modelo de maturidade validado com estudos em empresas. Você não precisa subir todos os degraus: **um projeto pequeno pode ser perfeito no nível 1**.

> **Dica**
>
> Pergunte: "qual é a dor mais cara hoje?". Se é reproduzir resultados, comece por versionamento. Se é implantação arriscada, comece por testes e CI/CD. Se é queda de qualidade, comece por monitoramento.

2.7 Reprodutibilidade, rastreabilidade e linhagem

Três palavras parecidas, com significados diferentes:

- **Reprodutibilidade:** obter o mesmo resultado com o mesmo código e os mesmos dados.
- **Rastreabilidade:** saber qual versão de código, dados e configuração gerou cada modelo.
- **Linhagem** (*lineage*): o caminho completo de um dado ou modelo, da origem ao destino.

Quando um cliente pergunta "por que o sistema negou meu crédito?", a rastreabilidade permite voltar até o modelo, os dados e a versão do código daquela decisão. Veremos como implementá-la nos Capítulos 3, 5 e 10.

2.8 Resumo do capítulo

MEMORIZE **O que levar deste capítulo**

- O ciclo de vida de ML tem **nove etapas** (AMERSHI et al., 2019) e funciona como um **laço**.
- Comece pelo **problema de negócio**, pela **métrica** e por uma **linha de base** simples.
- **Dívida técnica** é o custo futuro de atalhos de hoje. Em ML, inclui emaranhamento (CACE), dependências de dados, ciclos de realimentação e consumidores não declarados.
- O **ML Test Score** tem 28 itens em 4 grupos; a nota é o **mínimo** dos grupos.
- Uma arquitetura de referência tem: dados, atributos, orquestração, rastreamento, registro, CI/CD, serviço e monitoramento.
- Avance por **níveis de maturidade**, resolvendo a dor mais cara primeiro.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Quantas etapas tem o fluxo de trabalho de ML descrito por Amershi et al. (2019)?

a) Cinco.
b) Nove.
c) Doze.
d) Três.

Questão 2. Qual destas etapas vem imediatamente antes do treinamento do modelo no fluxo de nove etapas?

a) Engenharia de atributos.
b) Monitoramento.
c) Avaliação do modelo.
d) Implantação.

Questão 3. O ciclo de vida de um sistema de ML é melhor descrito como:

a) uma sequência executada uma só vez.
b) um processo sem relação com o negócio.
c) um laço, em que o monitoramento alimenta novos requisitos, dados e treinos.
d) uma linha reta que termina na implantação.

Questão 4. Um atributo (feature) é:

a) o resultado final da previsão.
b) a métrica de avaliação.
c) o servidor que hospeda o modelo.
d) uma característica usada como entrada do modelo.

Questão 5. Antes de escrever código, qual pergunta ajuda a definir o problema de ML?

a) Qual o nome da biblioteca preferida?
b) Qual linguagem de programação o gerente prefere?
c) Qual o tamanho do monitor?
d) Qual decisão o modelo vai apoiar?

Questão 6. Em um problema com 2% de positivos, um modelo que sempre responde "negativo" tem:

a) AP de 100%.
b) revocação de 100%.
c) acurácia de cerca de 98%, mas é inútil.
d) acurácia de 2%.

Questão 7. A principal função de uma linha de base (baseline) é:

a) dispensar a validação.
b) aumentar a velocidade de treino.
c) fornecer uma solução simples contra a qual o modelo precisa ganhar.
d) substituir o conjunto de teste.

Questão 8. Qual destes é um requisito não funcional de um serviço de previsão?

a) Latência máxima de 100 ms.
b) Escolha do algoritmo de otimização.
c) Número de camadas da rede neural.
d) Cor dos gráficos do relatório.

Questão 9. A dívida técnica pode ser comparada a:

a) um seguro contra erros de previsão.
b) comprar a prestação: entrega-se rápido hoje e pagam-se juros depois.
c) um tipo de imposto sobre dados.
d) um desconto permanente sobre o custo do projeto.

Questão 10. Segundo Sculley et al. (2015), a dívida técnica em ML:

a) ocorre somente em redes neurais.
b) desaparece com o uso de notebooks.
c) existe apenas em projetos sem testes.
d) inclui os problemas do software comum e outros próprios do nível do sistema.

Questão 11. Um ciclo de realimentação escondido acontece quando:

a) os dados de treino são duplicados.
b) o modelo influencia os dados que ele mesmo usará no futuro.
c) o servidor reinicia automaticamente.
d) o código é copiado de outro projeto.

Questão 12. Consumidores não declarados são:

a) outros sistemas que passam a usar a saída do seu modelo sem você saber.
b) clientes que cancelam o serviço.
c) usuários que pagam pelo serviço.
d) analistas que revisam o código.

Questão 13. O que é "código de cola" (glue code)?

a) Muito código só para conectar bibliotecas e preparar dados, sem estrutura.
b) Código gerado automaticamente por um IDE.
c) Código de interface gráfica.
d) Código que une vários modelos em um só.

Questão 14. A "dívida de configuração" refere-se a:

a) muitos parâmetros espalhados, sem revisão ou testes.
b) à lentidão da rede.
c) à falta de memória no servidor.
d) a licenças de software vencidas.

Questão 15. O ML Test Score propõe uma rubrica com quantos testes e monitoramentos?

a) 100, sem grupos.
b) 50, em cinco grupos.
c) 10, em dois grupos.
d) 28, organizados em quatro grupos.

Questão 16. Na pontuação do ML Test Score, um teste feito manualmente e documentado vale:

a) 1 ponto.
b) 0,5 ponto.
c) 0 ponto.
d) 2 pontos.

Questão 17. A nota final do ML Test Score é calculada como:

a) o mínimo dos totais dos quatro grupos.
b) o total do grupo de monitoramento apenas.
c) a média dos quatro grupos.
d) o máximo dos totais.

Questão 18. Qual componente de uma arquitetura de MLOps guarda atributos reutilizáveis para treino e serviço?

a) Registro de modelos.
b) Orquestrador de fluxos.
c) Painel de monitoramento.
d) Armazém de atributos (feature store).

Questão 19. O registro de modelos tem por função:

a) executar o treino em GPUs.
b) catalogar versões dos modelos, como um livro oficial de receitas.
c) distribuir o tráfego entre servidores.
d) validar os dados de entrada.

Questão 20. Qual a função do orquestrador de fluxos de trabalho?

a) Compactar os dados.
b) Gerar novos atributos aleatórios.
c) Escolher o melhor hiperparâmetro.
d) Executar as etapas do pipeline na ordem certa.

Questão 21. No nível de maturidade 0 (manual), a dor típica é:

a) os custos da nuvem estão altos.
b) ninguém consegue reproduzir o resultado.
c) o modelo piora e ninguém vê.
d) a plataforma é complexa demais.

Questão 22. No nível 1 (treino automatizado), qual é o próximo passo recomendado?

a) Voltar aos notebooks.
b) Testes e CI/CD.
c) Desligar o monitoramento.
d) Retreinar manualmente a cada mês.

Questão 23. Qual nível de maturidade adiciona monitoramento, alertas e retreino automático?

a) Manual.
b) Treino automatizado.
c) Entrega automatizada.
d) Operação contínua.

Questão 24. Rastreabilidade significa:

a) ter um modelo com mais parâmetros.
b) saber qual versão de código, dados e configuração gerou cada modelo.
c) ser capaz de treinar o modelo mais rápido.
d) produzir gráficos bonitos.

Questão 25. Reprodutibilidade significa:

a) obter o mesmo resultado com o mesmo código e os mesmos dados.
b) conseguir copiar o modelo de outro projeto.
c) atualizar o modelo toda semana.
d) treinar duas vezes o mesmo modelo com dados diferentes.

Questão 26. Linhagem (lineage) é:

a) a quantidade de memória usada.
b) a métrica mais importante de um projeto.
c) o caminho completo de um dado ou modelo, da origem ao destino.
d) a velocidade de treino de um modelo.

Questão 27. Na simulação do ciclo de realimentação, com exploração zero, o que aconteceu?

a) Três itens concentraram praticamente todos os cliques.
b) O item mais popular perdeu sempre.
c) Todos os itens receberam cliques iguais.
d) Nenhum item recebeu cliques.

Questão 28. Uma pergunta útil para decidir o próximo passo de maturidade é:

a) Qual é a dor mais cara hoje?
b) Qual a ferramenta mais famosa?
c) Qual a tecnologia mais nova do mercado?
d) Quantas pessoas existem na empresa?

Questão 29. Por que a métrica de negócio difere da métrica do modelo?

a) Porque um modelo com boa métrica técnica pode não gerar valor para o negócio.
b) Porque a do modelo é irrelevante.
c) Porque são sempre iguais.
d) Porque a de negócio é sempre mais fácil de medir.

Questão 30. Quais são as três grandes fases em que se agrupam as etapas do ciclo de vida de ML?

a) Projeto, desenvolvimento e operação.
b) Planejamento, compra e descarte.
c) Treino, validação e teste.
d) Entrada, processamento e saída.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Descreva as nove etapas do fluxo de trabalho de ML segundo Amershi et al. (2019) e explique por que o ciclo é um laço.
2. Por que se deve construir uma linha de base antes de um modelo complexo? Dê um exemplo.
3. Explique a dívida técnica em ML e dê três exemplos de tipos de dívida.
4. Explique o ciclo de realimentação escondido com um exemplo do cotidiano e diga como mitigá-lo.
5. Descreva como funciona a pontuação do ML Test Score e por que a nota final é o mínimo dos grupos.
6. Cite quatro componentes de uma arquitetura de referência de MLOps e explique a função de cada um.
7. Descreva os níveis de maturidade do MLOps e dê um exemplo de dor típica em cada um.
8. Diferencie reprodutibilidade, rastreabilidade e linhagem.
9. Quais perguntas devem ser respondidas ao transformar um problema de negócio em um problema de ML?
10. Explique por que um modelo com ótima acurácia pode ser inútil para o negócio.
11. O que são requisitos não funcionais de um serviço de ML? Dê quatro exemplos.
12. Por que é perigoso depender de dados produzidos por outro sistema sem contrato nem monitoramento?
13. Por que devemos pensar em evoluir por níveis de maturidade em vez de implantar uma plataforma completa de uma vez?
14. Como o conceito de "consumidor não declarado" se relaciona com versionamento e comunicação entre equipes?
15. Dê um exemplo de como os requisitos de privacidade podem influenciar a arquitetura de um sistema de ML.
16. Por que os testes e o monitoramento fazem parte do mesmo objetivo de prontidão para produção?
17. Explique a diferença entre métricas offline e a avaliação em produção, e por que ambas são necessárias.
18. Dê um exemplo de decisão de projeto que reduza a dívida técnica e explique por quê.
19. Por que um projeto pequeno pode ser perfeitamente adequado no nível 1 de maturidade? Quando seria hora de subir?
20. Explique a diferença entre "o sistema está no ar" e "o sistema está certo", e como o ML Test Score trata isso.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Sempre comece pela linha de base

NÍVEL: BÁSICO

Gere um problema de cancelamento com 3% de positivos (`make_classification`, 8.000 exemplos, `weights=[0.97]`, `class_sep=1.0`, `random_state=0`). Compare: (a) `DummyClassifier(strategy="most_frequent")`, (b) `DummyClassifier(strategy="stratified")` e (c) uma regressão logística. Mostre acurácia, revocação e AP no teste.

Exercício 2 — Calculadora de ML Test Score

NÍVEL: BÁSICO

Implemente uma função que receba, para cada um dos quatro grupos (dados, modelo, infraestrutura, monitoramento), a lista de pontos (0, 0,5 ou 1) e devolva a nota final (o mínimo dos grupos) e o grupo mais fraco. Aplique a dois projetos fictícios: um "notebook solto" e um "pipeline automatizado", e compare.

Exercício 3 — Quebrando o ciclo de realimentação

NÍVEL: INTERMEDIÁRIO

Reutilize a simulação do ciclo de realimentação (10 itens de qualidade igual, 5.000 visitas, 3 itens exibidos por visita), mas, a cada exibição, com probabilidade `epsilon` troque um dos três itens por um item sorteado ao acaso (exploração). Meça a participação do item mais clicado e quantos itens receberam pelo menos 50 cliques para `epsilon` em 0, 0,1 e 0,3.

Exercício 4 — "Mudar qualquer coisa muda tudo" (CACE)

NÍVEL: INTERMEDIÁRIO

No conjunto *Breast Cancer* (UCI), padronize os atributos e treine uma regressão logística com todos os 30 atributos e outra sem o atributo `mean radius`. Mostre como os coeficientes dos atributos correlacionados com ele (`mean perimeter`, `mean area`) mudam e quantas previsões de teste diferem entre os dois modelos.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Bank Marketing* (resposta a campanhas) e *Default of Credit Card Clients* servem para repetir o raciocínio de custo e benefício.
> - **Kaggle:** *Telco Customer Churn* e *Credit Card Fraud Detection* (muito desbalanceado) para praticar linha de base e métricas de negócio.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Planejando um projeto de retenção de clientes

Contexto. Uma empresa de assinaturas quer reduzir o cancelamento. Uma campanha de retenção custa R$ 20 por cliente contatado e, se o cliente era mesmo um cancelador, o contato evita o cancelamento em 40% dos casos, o que preserva R$ 300 de receita. Você precisa decidir **se vale a pena** usar um modelo e a quem contatar.

Dados simulados. Gere 10.000 clientes com `make_classification` (8 atributos, 8% de cancelamento, `random_state=3`, `class_sep=0.9`).

Tarefas.

1. Defina a métrica de negócio: lucro líquido da campanha = (cancelamentos evitados × R$ 300) − (contatos × R$ 20).
2. Calcule o lucro de três políticas no conjunto de teste: (a) não contatar ninguém; (b) contatar todos; (c) contatar os clientes cujo risco previsto supera um limiar.
3. Para a política (c), use uma regressão logística e escolha o limiar na **validação** (e não no teste) pelo lucro esperado.
4. Compare com uma linha de base: contatar a mesma quantidade de clientes escolhidos ao acaso.
5. Escreva em 3 frases a recomendação para a diretoria, apontando riscos (por exemplo, o efeito de 40% ser incerto).

Entregáveis. Tabela com o lucro de cada política, o limiar escolhido e a recomendação.

A solução de referência está no Apêndice E.

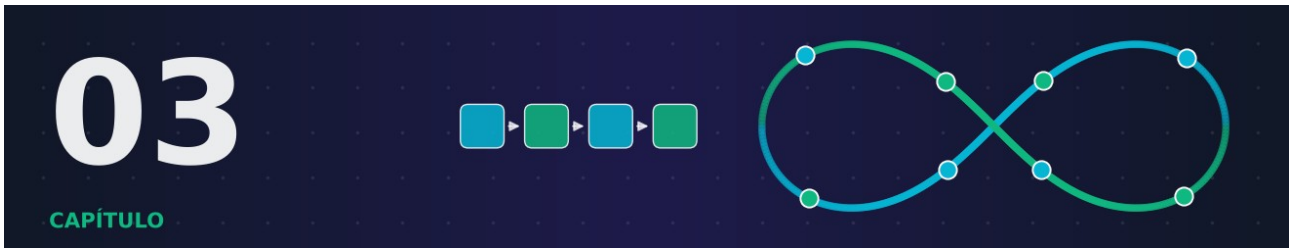
PARTE II

Parte II — Dados e Experimentos

A base de tudo: dados confiáveis e versionados, pipelines reprodutíveis, experimentos rastreados, modelos registrados e avaliações rigorosas.

Nesta parte:

- Cap. 3 — Dados: Qualidade, Validação e Versionamento
- Cap. 4 — Pipelines Reprodutíveis com scikit-learn
- Cap. 5 — Rastreamento de Experimentos e Registro de Modelos
- Cap. 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros



Capítulo 3 — Dados: Qualidade, Validação e Versionamento

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que a qualidade dos dados determina a qualidade do modelo.
- Medir dimensões de qualidade (completude, validade, unicidade, consistência, atualidade) com pandas.
- Escrever validações automáticas de esquema e de intervalo para os dados de entrada.
- Reconhecer e evitar o vazamento de dados (*data leakage*).
- Avaliar o efeito de rótulos errados sobre o desempenho do modelo.
- Versionar dados com identificadores por conteúdo (hash) e entender como o DVC funciona.
- Documentar conjuntos de dados com *datasheets* e usar repositórios públicos (UCI, Kaggle, OpenML).

3.1 Lixo entra, lixo sai

Existe um ditado antigo na computação: ***garbage in, garbage out***. Em português: se entra lixo, sai lixo. Em ML, ele vale em dobro. Um modelo é tão bom quanto os dados com que aprendeu **e** os dados que recebe em produção.

Sambasivan e colegas (2021) estudaram profissionais de IA em projetos de alto risco e chamaram de **cascatas de dados** (*data cascades*) os problemas que nascem cedo nos dados, passam despercebidos e causam danos graves lá na frente. Quase todos os profissionais entrevistados relataram ter sofrido ao menos uma cascata. O estudo conclui que o trabalho com dados é muitas vezes pouco valorizado, embora seja decisivo.

Whang e colegas (2023) discutem os desafios de coleta e qualidade de dados sob a perspectiva de uma "IA centrada em dados", que coloca os dados, e não só o algoritmo, no centro do trabalho.

> **No seu dia a dia**
>
> Pense numa **receita com ingredientes vencidos**. Não importa se o cozinheiro é excelente: o prato sai ruim, e talvez perigoso. Cuidar dos ingredientes (dados) vem antes de cuidar da panela (algoritmo).

3.2 As dimensões da qualidade dos dados

Wang e Strong (1996) perguntaram aos usuários de dados o que "qualidade" significava para eles. A resposta foi além da simples exatidão. Hoje as equipes usam um conjunto de dimensões parecido com este:

Dimensão	Pergunta	Exemplo de problema
Completude	Faltam valores?	Renda vazia em 30% dos clientes

Dimensão	Pergunta	Exemplo de problema
Validade	O valor está no formato e no intervalo permitidos?	Idade = −5 ou 250
Unicidade	Há registros duplicados?	O mesmo cliente aparece duas vezes
Consistência	Os dados concordam entre si?	Data de saída anterior à data de entrada
Atualidade	Os dados estão em dia?	Tabela atualizada pela última vez há 6 meses
Exatidão	O valor corresponde à realidade?	Endereço antigo
Representatividade	Os dados cobrem os casos reais?	Só clientes de uma cidade

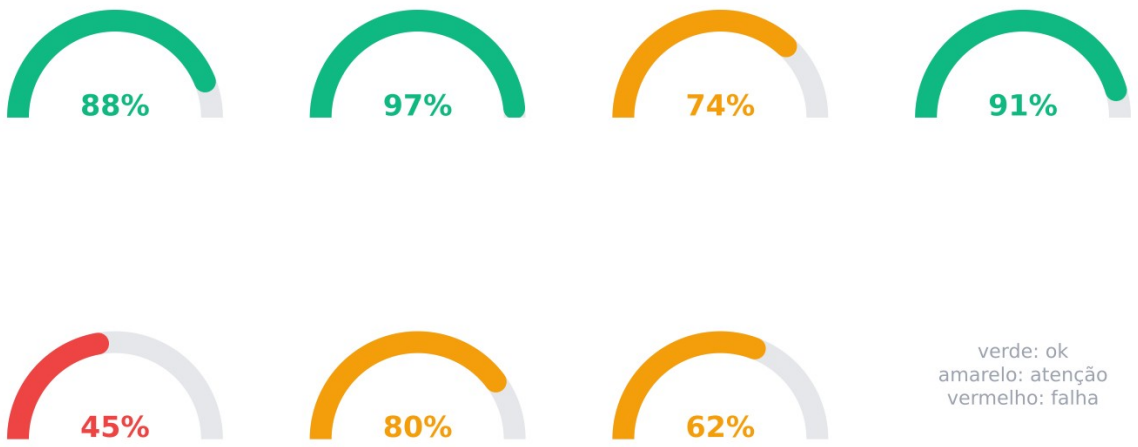


Figura 3.1 — As dimensões da qualidade dos dados, vistas como um painel de instrumentos. Cada medidor deve ser verificado de forma automática.

3.2.1 Medindo a qualidade com pandas

Vamos criar uma tabela de clientes com defeitos conhecidos e medi-los. Usaremos dados simulados, para saber exatamente quais problemas existem.

```python
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 1000
df = pd.DataFrame({
    "id_cliente": np.arange(n),
    "idade": rng.integers(18, 80, n).astype(float),
    "renda": rng.lognormal(8.0, 0.5, n).round(2),
    "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n),
})
# defeitos injetados de propósito
df.loc[rng.choice(n, 120, replace=False), "renda"] = np.nan      # ausentes
df.loc[rng.choice(n, 15, replace=False), "idade"] = -1           # inválidos
df = pd.concat([df, df.sample(25, random_state=1)], ignore_index=True)  # duplicados

relatorio = {
```

```
    "linhas": len(df),
    "% ausentes em renda": round(df["renda"].isna().mean() * 100, 1),
    "idades inválidas (<0 ou >120)": int((~df["idade"].between(0, 120)).sum()),
    "linhas duplicadas": int(df.duplicated().sum()),
    "ids repetidos": int(df["id_cliente"].duplicated().sum()),
}
for k, v in relatorio.items():
    print(f"{k:32s}: {v}")
```

O relatório mostra, em segundos, quatro problemas. Em um projeto real, **essa verificação roda sozinha** toda vez que chegam dados novos.

3.3 Validação automática de dados

Validar dados é como o **controle de passaporte** de um aeroporto: ninguém entra sem documentos válidos. Breck e colegas (2019) descrevem, no contexto do TFX, um sistema de validação de dados com três ideias centrais:

1. **Esquema** (*schema*): descreve o que é esperado de cada coluna (tipo, intervalo, valores permitidos, proporção de ausentes).
2. **Detecção de anomalias nos dados**: compara novos lotes com o esquema e avisa sobre desvios.
3. **Desvio treino–serviço** (*training-serving skew*): detecta se os dados vistos em produção diferem dos dados usados no treino.

Schelter e colegas (2018) trazem uma ideia parecida para dados em larga escala: escrever **"testes unitários para dados"**, ou seja, regras declarativas verificadas automaticamente.

3.3.1 Um validador simples

Ferramentas como Great Expectations, Pandera e o TensorFlow Data Validation fazem isso de forma completa. Para entender a lógica, vamos escrever um validador mínimo:

```python
import numpy as np
import pandas as pd

ESQUEMA = {
    "idade":  {"tipo": "numerico", "min": 18, "max": 100, "max_ausentes": 0.0},
    "renda":  {"tipo": "numerico", "min": 0, "max": 1e6, "max_ausentes": 0.05},
    "cidade": {"tipo": "categorico", "permitidos": {"Natal", "Recife", "Fortaleza"},
               "max_ausentes": 0.0},
}

def validar(df, esquema):
    erros = []
    for col, regra in esquema.items():
        if col not in df.columns:
            erros.append(f"coluna ausente: {col}")
            continue
        serie = df[col]
        frac_nulos = serie.isna().mean()
        if frac_nulos > regra["max_ausentes"]:
            erros.append(f"{col}: {frac_nulos:.1%} de ausentes (máximo {regra['max_ausentes']:.0%})")
        valida = serie.dropna()
        if regra["tipo"] == "numerico":
```

```
            fora = ((valida < regra["min"]) | (valida > regra["max"])).sum()
            if fora:
                erros.append(f"{col}: {fora} valores fora de [{regra['min']}, {regra['max']}]")
        else:
            inesperados = set(valida.unique()) - regra["permitidos"]
            if inesperados:
                erros.append(f"{col}: categorias inesperadas {sorted(inesperados)}")
    return erros

lote = pd.DataFrame({
    "idade": [25, 40, -1, 130, 33],
    "renda": [3000, np.nan, 4500, 2800, np.nan],
    "cidade": ["Natal", "Recife", "Salvador", "Natal", "Fortaleza"],
})
problemas = validar(lote, ESQUEMA)
print("Lote aprovado!" if not problemas else "Lote REPROVADO:")
for p in problemas:
    print(" -", p)
```

Dica

Trate a validação como um **portão** (*gate*). Se o lote reprova, o pipeline **para** e avisa alguém, em vez de treinar com lixo. Isso evita as cascatas de dados.

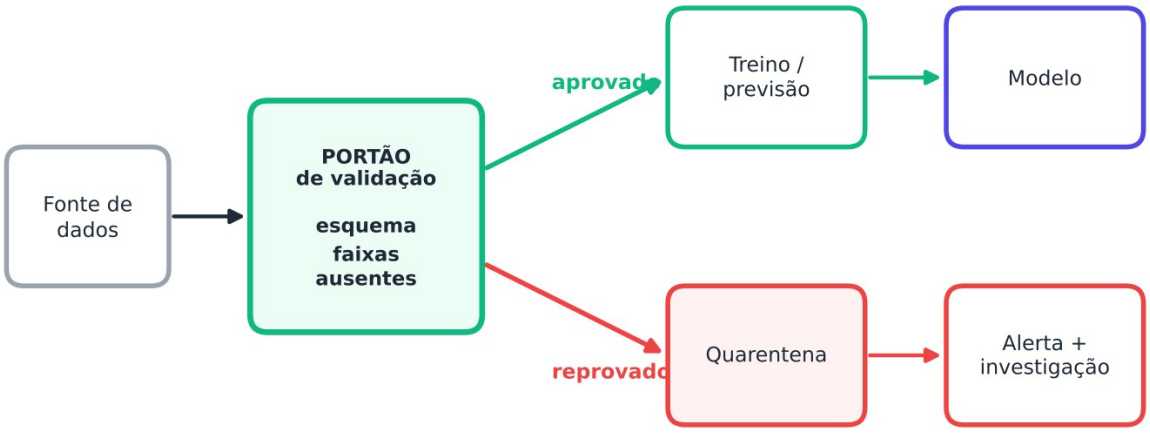


Figura 3.2 — Portões de validação no pipeline de dados: lotes que não passam são desviados para quarentena e geram alerta.

3.4 Vazamento de dados: o inimigo invisível

O **vazamento de dados** (*data leakage*) acontece quando o modelo "vê", durante o treino, uma informação que **não estaria disponível** na hora de prever. Os resultados ficam lindos no teste e péssimos na vida real. Kaufman e colegas (2012) formalizam o problema e mostram como detectá-lo e evitá-lo.

No seu dia a dia

É como um aluno que **vê a prova antes** de fazê-la. A nota é ótima, mas não prova que ele sabe a matéria.

Os tipos mais comuns:

- **Vazamento de alvo:** um atributo é uma consequência do rótulo. Exemplo: prever inadimplência usando "valor cobrado em cobrança judicial".
- **Contaminação treino–teste:** informação do teste entra no treino (por exemplo, normalizar tudo antes de dividir; veja o Capítulo 4).
- **Vazamento temporal:** usar dados do futuro para prever o passado.
- **Duplicatas:** o mesmo registro aparece no treino e no teste.

3.4.1 Detectando um vazamento

Uma pista clássica: **resultado bom demais**. Vamos criar um atributo que vaza o alvo e ver o efeito:

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)

# atributo "vazado": quase uma cópia do alvo, com um pouco de ruído
vazado = (y + rng.normal(0, 0.15, len(y))).reshape(-1, 1)
X_vaza = np.hstack([X, vazado])

def acuracia(entrada):
    Xtr, Xte, ytr, yte = train_test_split(entrada, y, test_size=0.3, random_state=0, stratify=y)
    return RandomForestClassifier(n_estimators=100, random_state=0).fit(Xtr, ytr).score(Xte, yte)

print("Sem vazamento:", round(acuracia(X), 3))
print("Com vazamento:", round(acuracia(X_vaza), 3))
```

A acurácia sobe para praticamente 100%. Em produção, a coluna vazada **não existe** na hora da previsão, e o desempenho desaba. A pergunta de ouro para cada atributo é:

> MEMORIZE **A pergunta do vazamento**
>
> "Na **hora exata** em que o modelo for usado, eu terei **esta informação**?"

3.5 Rótulos errados também contam

Em muitos problemas, os rótulos (as "respostas corretas") vêm de pessoas, e pessoas erram. Northcutt, Athalye e Mueller (2021) mostraram que até conjuntos de dados famosos usados como padrão de comparação têm erros de rotulagem em seus conjuntos de teste, o que pode até mudar a classificação de modelos.

3.5.1 Quanto o ruído atrapalha?

Vamos simular: embaralhar uma fração dos rótulos de **treino** e medir a acurácia em um teste **limpo**.

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
```

```
X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)

for ruido in [0.0, 0.1, 0.2, 0.4]:
    rng = np.random.default_rng(1)
    y_ruim = ytr.copy()
    idx = rng.choice(len(ytr), int(ruido * len(ytr)), replace=False)
    y_ruim[idx] = rng.integers(0, 10, len(idx))
    acc = RandomForestClassifier(n_estimators=100, random_state=0).fit(Xtr, y_ruim).score(Xte, yte)
    print(f"ruído de rótulos = {ruido:4.0%} -> acurácia no teste limpo = {acc:.3f}")
```

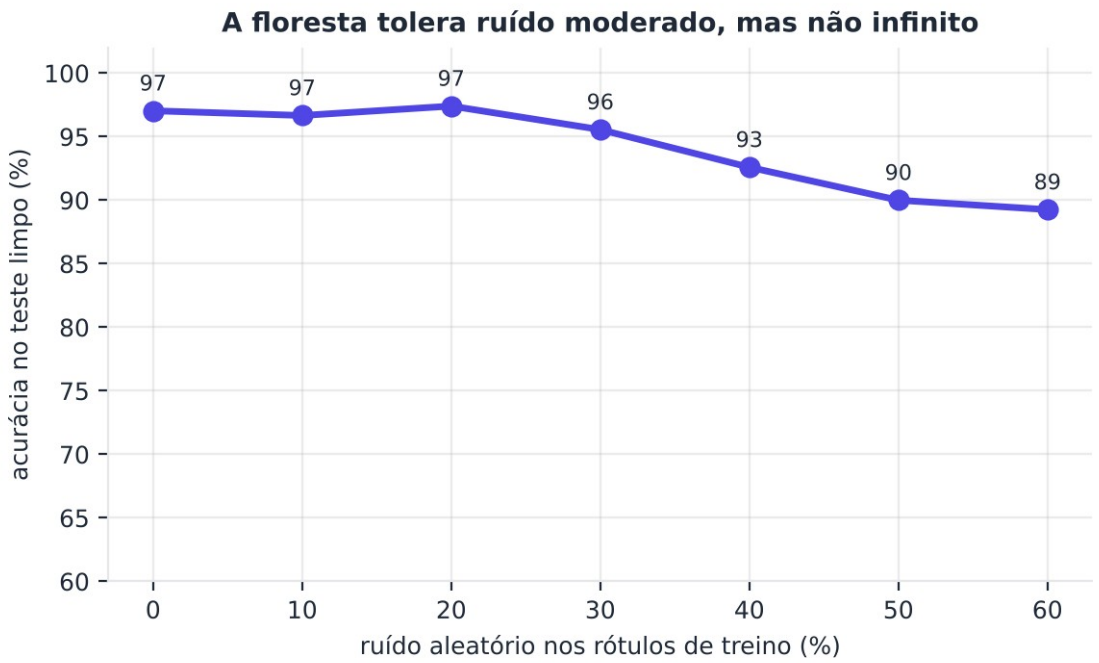


Figura 3.3 — Efeito do ruído aleatório nos rótulos de treino sobre a acurácia em dados de teste limpos (experimento do texto, 1.797 imagens de dígitos).

Observe que a floresta aleatória é bastante **robusta a ruído aleatório moderado**: a acurácia quase não muda com 10% ou 20% de rótulos trocados e só cai de forma visível com 40%. Mas cuidado: erros **sistemáticos** (por exemplo, um rotulador que sempre confunde duas classes) são muito mais danosos do que erros ao acaso. Por isso, em MLOps, **rótulos são tratados como dados de primeira classe**: revisados, amostrados e monitorados.

3.6 Como dividir os dados

Dividir os dados em **treino**, **validação** e **teste** é um ato de honestidade científica.

- **Treino:** onde o modelo aprende.
- **Validação:** onde se comparam modelos e se ajustam hiperparâmetros.
- **Teste:** usado **uma vez**, no final, para estimar o desempenho real.

Estratégia	Quando usar	Cuidado
Aleatória	Dados independentes e sem ordem	Não use com séries temporais
Estratificada	Classes desbalanceadas	Mantém a proporção das classes
Temporal	Dados com tempo (vendas, fraude)	Treina no passado e testa no futuro

Estratégia	Quando usar	Cuidado
Por grupo	Várias linhas por pessoa/cliente	Evita o mesmo indivíduo nos dois lados

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit

# Cada cliente aparece várias vezes (várias compras).
df = pd.DataFrame({
    "cliente": np.repeat(np.arange(50), 6),
    "valor": np.random.default_rng(0).normal(100, 20, 300),
})
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=0)
tr, te = next(gss.split(df, groups=df["cliente"]))
clientes_comuns = set(df.iloc[tr]["cliente"]) & set(df.iloc[te]["cliente"])
print("Clientes presentes nos dois lados:", len(clientes_comuns))
```

Com a divisão por grupo, **nenhum cliente** aparece nos dois conjuntos, evitando que o modelo "decore" pessoas.

3.7 Versionando dados

Você versiona seu código com **Git**: cada mudança deixa um registro. Mas e os dados? Arquivos grandes não cabem bem no Git, e ainda assim precisamos saber **qual versão dos dados** gerou **qual modelo**.

3.7.1 A ideia do hash

Um **hash** é uma "impressão digital" de um conteúdo. Se um único byte mudar, o hash muda por completo. Usamos hashes como identificadores de versão.

```python
import hashlib
import json
import numpy as np
import pandas as pd

def hash_dataframe(df):
    """Impressão digital estável do conteúdo de uma tabela."""
    csv_bytes = df.to_csv(index=False).encode("utf-8")
    return hashlib.sha256(csv_bytes).hexdigest()

df = pd.DataFrame({"x": [1, 2, 3], "y": [10.0, 20.0, 30.0]})
v1 = hash_dataframe(df)

df2 = df.copy()
df2.loc[1, "y"] = 20.1              # mudança mínima
v2 = hash_dataframe(df2)

print("versão 1:", v1[:16], "...")
print("versão 2:", v2[:16], "...")
print("Iguais?", v1 == v2)

manifesto = {"arquivo": "clientes.csv", "sha256": v1, "linhas": len(df),
             "colunas": list(df.columns)}
print(json.dumps(manifesto, indent=2))
```

Guardar esse **manifesto** junto do modelo já resolve o essencial: dá para provar com qual dado o modelo foi treinado.

3.7.2 Como o DVC funciona

O **DVC** (*Data Version Control*) é uma ferramenta que usa essa ideia. O funcionamento, em linhas gerais:

1. Os dados grandes ficam em um **armazenamento remoto** (disco, nuvem).
2. No Git, fica apenas um **arquivo pequeno** com o hash e o caminho do dado.
3. Ao trocar de versão no Git, o DVC traz de volta os dados correspondentes.

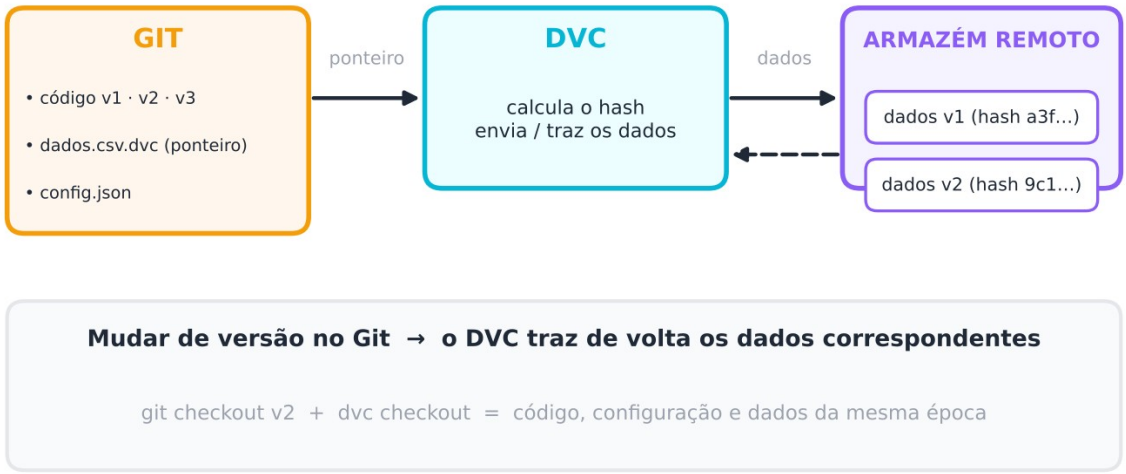


Figura 3.4 — Versionamento de dados no estilo DVC: o Git guarda ponteiros leves; os arquivos pesados ficam em armazenamento remoto.

Comandos típicos (execute no terminal; não são código Python):

```bash
git init && dvc init                        # inicia os dois repositórios
dvc add data/raw/clientes.csv               # cria data/raw/clientes.csv.dvc
git add data/raw/clientes.csv.dvc data/raw/.gitignore
git commit -m "Dados v1"
dvc remote add -d armazem s3://meu-bucket/dados   # define o armazenamento remoto
dvc push                                    # envia os dados para o armazenamento
dvc checkout                                # volta os dados da versão atual do Git
```

> **Atenção**
>
> Os comandos acima são ilustrativos e dependem de instalar o DVC e de configurar o armazenamento. Eles não foram executados na elaboração deste livro. Consulte a documentação oficial da ferramenta para seu ambiente.

3.7.3 Dados que mudam com o tempo

Há três estratégias comuns:

- **Instantâneos** (*snapshots*): guardar uma cópia completa a cada versão. Simples, mas ocupa espaço.
- **Particionamento por data:** guardar `dados/2026-10-01/`, `dados/2026-10-02/` etc.

- **Viagem no tempo** (*time travel*): formatos de tabela que guardam o histórico e permitem consultar o dado "como era" em certa data.

3.8 Documentando os dados

Dados sem documentação são como um remédio sem bula. Gebru e colegas (2021) propuseram as ***datasheets for datasets***: uma "ficha técnica" que acompanha o conjunto de dados e responde perguntas como:

- **Motivação:** por que o conjunto foi criado? Por quem?
- **Composição:** o que há nele? Quantos exemplos? Há dados pessoais?
- **Coleta:** como foi coletado? Houve consentimento?
- **Pré-processamento:** o que foi limpo ou removido?
- **Usos:** para que serve? Para que **não** deve ser usado?
- **Distribuição e manutenção:** quem mantém? Há licença?

Veja um modelo simples em Python, para guardar ao lado do dado:

```python
import json

datasheet = {
    "nome": "Câncer de mama (Diagnóstico de Wisconsin)",
    "origem": "Repositório UCI Machine Learning (embutido no scikit-learn)",
    "exemplos": 569,
    "atributos": 30,
    "alvo": "diagnóstico: maligno (0) ou benigno (1)",
    "uso_indicado": "estudo e ensino de classificação binária",
    "uso_nao_indicado": "apoio real a diagnóstico clínico sem validação médica",
    "limitacoes": ["amostra pequena", "uma única instituição", "sem dados temporais"],
    "licenca": "consultar a página do conjunto de dados no repositório UCI",
}
print(json.dumps(datasheet, indent=2, ensure_ascii=False))
```

3.9 Dados pessoais e privacidade desde o início

Se os dados descrevem pessoas, há obrigações legais (no Brasil, a LGPD, Lei n.º 13.709/2018) e éticas. Duas práticas básicas:

- **Minimização:** colete e guarde apenas o que for necessário.
- **Pseudonimização:** troque identificadores diretos (CPF, e-mail) por códigos que não permitem identificar a pessoa sem uma informação adicional guardada separadamente.

```python
import hashlib
import hmac

SEGREDO = b"guarde-esta-chave-fora-do-codigo"   # em produção: gerenciador de segredos

def pseudonimizar(identificador: str) -> str:
    return hmac.new(SEGREDO, identificador.encode(), hashlib.sha256).hexdigest()[:16]
```

```
print(pseudonimizar("maria@exemplo.com"))
print(pseudonimizar("maria@exemplo.com") == pseudonimizar("maria@exemplo.com"))
```

Note que o mesmo e-mail sempre gera o mesmo código (permite juntar tabelas), mas ninguém consegue reverter o código sem a chave. O Capítulo 10 trata de privacidade com mais profundidade, incluindo a privacidade diferencial (DWORK; ROTH, 2014).

3.10 Onde encontrar dados reais

Para praticar, use repositórios públicos:

- **UCI Machine Learning Repository** (DUA; GRAFF, 2019): centenas de conjuntos clássicos. O pacote `ucimlrepo` facilita o download.
- **Kaggle:** competições e conjuntos de dados da comunidade; exige conta.
- **OpenML** (VANSCHOREN et al., 2013): plataforma aberta de conjuntos e experimentos; o scikit-learn acessa via `fetch_openml`.

> **Saiba mais**
>
> Os conjuntos *Iris*, *Wine*, *Breast Cancer Wisconsin* e *Digits* já vêm embutidos no scikit-learn, e são originários do repositório UCI. Por isso, usamo-los ao longo do livro: não exigem download e garantem os mesmos resultados para todos.

3.11 Resumo do capítulo

> MEMORIZE **O que levar deste capítulo**
>
> - **Garbage in, garbage out**: cuide dos dados antes do algoritmo.
> - Dimensões de qualidade: **completude, validade, unicidade, consistência, atualidade, exatidão e representatividade**.
> - **Valide automaticamente**: esquema, intervalos e categorias. Reprove o lote ruim antes de treinar.
> - **Vazamento de dados** dá resultado bom demais. Pergunte: "terei essa informação na hora de prever?"
> - **Rótulos errados** prejudicam o modelo; trate-os como dados importantes.
> - Divida os dados com cuidado: estratificada, temporal ou por grupo, conforme o caso.
> - **Hash** identifica versões de dados; o **DVC** guarda ponteiros no Git e os dados em armazenamento remoto.
> - Documente com **datasheets** e proteja dados pessoais desde o início.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. A expressão "garbage in, garbage out" significa que:

a) o modelo corrige sozinho os dados ruins.
b) lixo eletrônico deve ser reciclado.
c) dados ruins na entrada geram resultados ruins na saída.
d) todo dado deve ser descartado após o uso.

Questão 2. O que são "cascatas de dados" (data cascades) segundo Sambasivan et al. (2021)?

a) Problemas que nascem cedo nos dados, passam despercebidos e causam danos graves depois.
b) Fluxos de dados em tempo real.
c) Tabelas organizadas em cascata no banco de dados.
d) Backups em camadas.

Questão 3. Qual dimensão da qualidade verifica se há registros duplicados?

a) Unicidade.
b) Validade.
c) Atualidade.
d) Completude.

Questão 4. Idade = −5 é um problema de:

a) atualidade.
b) unicidade.
c) representatividade.
d) validade.

Questão 5. Uma tabela atualizada pela última vez há seis meses, num processo que exige dados do dia, tem problema de:

a) validade de formato.
b) consistência de nomes.
c) atualidade.
d) unicidade.

Questão 6. Data de saída anterior à data de entrada é exemplo de falha de:

a) completude.
b) unicidade.
c) consistência.
d) atualidade.

Questão 7. Dados de uma única cidade usados para prever o comportamento nacional têm problema de:

a) unicidade.
b) formatação.
c) completude.
d) representatividade.

Questão 8. Em um validador de dados, o esquema (schema) descreve:

a) o que é esperado de cada coluna: tipo, intervalo, categorias e ausentes.
b) os hiperparâmetros do modelo.
c) a arquitetura da rede neural.
d) a estrutura de pastas do projeto.

Questão 9. O que é desvio treino–serviço (training-serving skew)?

a) O desequilíbrio entre classes.
b) A diferença de velocidade entre treino e teste.
c) Diferença entre os dados vistos em produção e os usados no treino, ou entre seus processamentos.
d) A diferença de preço entre servidores.

Questão 10. Quando um lote de dados reprova na validação, o pipeline deve:

a) parar ou desviar o lote para quarentena e avisar alguém.
b) treinar mesmo assim.
c) ignorar o erro.
d) apagar todos os dados.

Questão 11. Vazamento de dados (data leakage) ocorre quando:

a) arquivos são apagados sem querer.
b) dados pessoais são divulgados por engano.
c) o servidor perde conexão.
d) o modelo vê, no treino, informação que não estaria disponível na hora de prever.

Questão 12. Qual a pergunta de ouro para detectar vazamento em um atributo?

a) Este atributo é fácil de calcular?
b) Na hora exata do uso do modelo, eu terei esta informação?
c) Este atributo tem muitos valores diferentes?
d) Este atributo é numérico?

Questão 13. Normalizar todos os dados antes de dividir em treino e teste é um exemplo de:

a) validação cruzada aninhada.
b) contaminação treino–teste.
c) vazamento de alvo.
d) boa prática recomendada.

Questão 14. Usar dados do futuro para prever o passado é:

a) amostragem estratificada.
b) calibração.
c) vazamento temporal.
d) validação cruzada.

Questão 15. Uma pista clássica de vazamento é:

a) um arquivo muito grande.
b) um resultado ligeiramente abaixo do esperado.
c) um resultado bom demais para ser verdade.
d) um modelo que demora a treinar.

Questão 16. No experimento com ruído nos rótulos de treino, a floresta aleatória:

a) caiu a zero com 10% de ruído.
b) melhorou sempre que se acrescentava ruído.
c) manteve a acurácia quase igual com 10% a 20% de ruído aleatório e só caiu visivelmente com 40%.
d) deixou de funcionar com qualquer ruído.

Questão 17. Erros sistemáticos de rotulagem costumam ser:

a) menos danosos que erros aleatórios.
b) corrigidos sozinhos pelo modelo.
c) mais danosos que erros aleatórios.
d) irrelevantes.

Questão 18. Northcutt, Athalye e Mueller (2021) mostraram que:

a) rótulos errados nunca afetam benchmarks.
b) só dados de treino têm erros.
c) mesmo conjuntos de teste famosos têm erros de rotulagem que podem alterar a classificação de modelos.
d) conjuntos públicos são sempre perfeitos.

Questão 19. Qual conjunto é usado uma única vez, no final, para estimar o desempenho real?

a) Teste.
b) Reserva de hiperparâmetros.
c) Validação.
d) Treino.

Questão 20. Para dados em que cada cliente aparece em várias linhas, a divisão adequada é:

a) pela ordem alfabética.
b) aleatória das linhas.
c) por grupo (cliente).
d) apenas pelo tamanho.

Questão 21. Para uma série temporal, a divisão adequada é:

a) por ordem alfabética.
b) invertida: treina no futuro e testa no passado.
c) temporal: treina no passado e testa no futuro.
d) aleatória.

Questão 22. Um hash (como SHA-256) de um arquivo de dados é útil porque:

a) criptografa os dados para sempre.
b) muda por completo se um único byte do conteúdo mudar.
c) comprime o arquivo sem perda.
d) acelera a leitura do disco.

Questão 23. No DVC, o que fica no Git?

a) O arquivo de dados completo.
b) Apenas o modelo treinado.
c) Somente as senhas do armazenamento.
d) Um arquivo pequeno de ponteiro com o hash e o caminho do dado.

Questão 24. O que faz o comando dvc push?

a) Treina o modelo.
b) Cria uma branch no Git.
c) Envia os dados versionados para o armazenamento remoto.
d) Apaga os dados locais.

Questão 25. A estratégia de "instantâneos" (snapshots) para versionar dados consiste em:

a) guardar apenas as diferenças.
b) nunca guardar versões.
c) guardar só os rótulos.
d) guardar uma cópia completa a cada versão.

Questão 26. As datasheets for datasets (Gebru et al., 2021) servem para:

a) criptografar os dados.
b) acelerar o treinamento.
c) documentar o conjunto de dados: motivação, composição, coleta, usos e manutenção.
d) substituir testes estatísticos.

Questão 27. A pseudonimização de um identificador pessoal consiste em:

a) publicar o identificador.
b) trocá-lo por um código que só permite identificar a pessoa com uma informação adicional guardada à parte.
c) apagar todas as informações da pessoa.
d) misturar os dados de todas as pessoas.

Questão 28. O princípio da minimização na proteção de dados diz que se deve:

a) coletar e guardar apenas o necessário.
b) compartilhar com todos os parceiros.
c) guardar dados indefinidamente.
d) coletar o máximo possível para uso futuro.

Questão 29. Quais destes repositórios públicos de dados são citados no capítulo?

a) UCI Machine Learning Repository, Kaggle e OpenML.
b) Apenas bancos de dados privados.
c) Somente repositórios de código.
d) Apenas sistemas de arquivos locais.

Questão 30. Qual a utilidade de medir a fração de ausentes por coluna em cada lote novo?

a) Escolher o hiperparâmetro.
b) Reduzir o tamanho do modelo.
c) Calcular a acurácia.
d) Detectar mudanças bruscas na origem dos dados.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Por que se diz que o trabalho com dados determina a qualidade do modelo? Cite o conceito de cascatas de dados.
2. Descreva cinco dimensões da qualidade dos dados e dê um exemplo de problema em cada uma.
3. Explique o funcionamento de um validador de dados baseado em esquema e onde ele se encaixa no pipeline.
4. O que é vazamento de dados? Descreva dois tipos e como evitá-los.
5. Como você investigaria se um atributo suspeito está vazando o alvo?
6. Explique o efeito do ruído nos rótulos de treino e por que erros sistemáticos preocupam mais.
7. Compare as estratégias de divisão aleatória, estratificada, temporal e por grupo, indicando quando usar cada uma.
8. Por que o conjunto de teste deve ser usado uma única vez?
9. Explique o que é um hash de dados e como ele ajuda na rastreabilidade de modelos.
10. Descreva como o DVC versiona dados grandes sem colocá-los no Git.
11. O que deve conter uma datasheet de um conjunto de dados? Dê cinco itens.
12. Explique minimização e pseudonimização, e como se relacionam com a LGPD.
13. Por que a divisão por grupo é importante quando um mesmo paciente tem várias medições?
14. Por que o código de validação deve rodar também em produção, e não só no desenvolvimento?
15. Quais problemas podem surgir ao juntar tabelas sem cuidado com o tempo? Relacione com o vazamento.
16. Explique o que é uma quarentena de dados e como decidir o que fazer com o lote nela.
17. Por que o mesmo código de pré-processamento deve ser usado no treino e no serviço?
18. Mencione três cuidados éticos e legais no uso de dados pessoais para treinar modelos.
19. Por que a representatividade dos dados é uma dimensão de qualidade importante para modelos em produção?
20. Dê dois exemplos de validações que você incluiria em um esquema de dados de clientes (idade, renda, cidade) e explique o que cada uma evita.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Relatório de qualidade de dados

NÍVEL: BÁSICO

Crie um `DataFrame` de 500 pacientes simulados com `idade`, `peso_kg`, `pressao`, `cidade`. Injete: 10% de `peso_kg` ausente, 5 idades negativas, 20 linhas duplicadas e uma cidade escrita de duas formas ("Natal" e "natal "). Escreva uma função que devolva um relatório com a fração de ausentes por coluna, valores fora de intervalo, linhas duplicadas e categorias que só diferem por caixa ou espaços.

Exercício 2 — Um validador de esquema que para o pipeline

NÍVEL: INTERMEDIÁRIO

Escreva um esquema (tipo, mínimo, máximo, fração máxima de ausentes, categorias permitidas) para os atributos do conjunto *Wine* (UCI), baseado nos valores observados no treino com uma folga de 20%. Valide: (a) um lote saudável (amostra do próprio conjunto) e (b) um lote corrompido (álcool multiplicado por 10 e 30% de ausentes em `proline`). O programa deve lançar uma exceção para o lote reprovado.

Exercício 3 — Caçando um atributo vazado

NÍVEL: INTERMEDIÁRIO

No conjunto *Breast Cancer* (UCI), crie um atributo `vazado` que seja uma versão ruidosa do alvo. Para **cada atributo**, calcule a AUC que ele obtém sozinho (como se fosse uma pontuação) e liste os cinco maiores. Mostre que o atributo vazado se destaca de forma suspeita. Depois, treine um modelo com e sem ele e compare as acurácias.

Exercício 4 — Divisão aleatória versus divisão por grupo

NÍVEL: AVANÇADO

Simule 100 pacientes com 10 medições cada. Cada paciente tem um "efeito individual" fixo que influencia o alvo, e as medições do mesmo paciente são parecidas entre si. Treine uma floresta usando (a) divisão aleatória das linhas e (b) divisão por paciente (`GroupShuffleSplit`). Compare as acurácias e explique a diferença.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult (Census Income)* e *Bank Marketing* têm colunas categóricas, ausentes e valores atípicos, ótimos para exercitar validação de dados.
> - **Kaggle:** *Titanic* (muitos ausentes) e *House Prices* (muitas colunas categóricas) para praticar relatórios de qualidade.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Portão de dados com quarentena e versionamento

Contexto. Todo dia chega um lote de clientes de uma loja. Sua equipe precisa impedir que lotes ruins cheguem ao treino e manter um registro de versões.

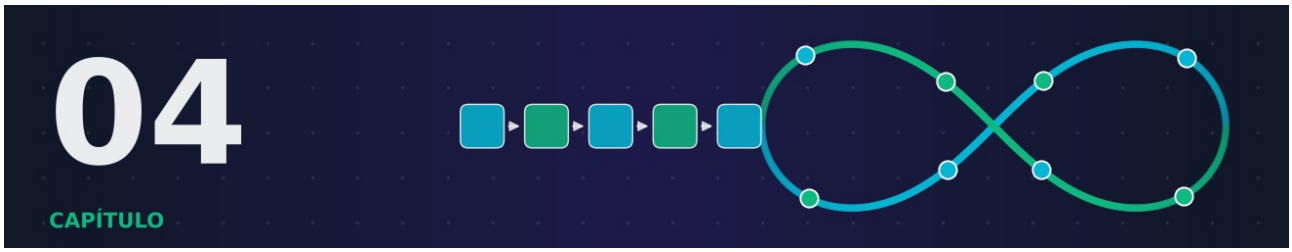
Dados simulados. Gere 6 lotes diários de 300 clientes (`idade`, `renda`, `cidade`, `compras_30d`). Injete defeitos em alguns lotes: no lote 2, 20% de ausentes em `renda`; no lote 4, idades negativas; no lote 5, uma cidade nova ("Salvador"); no lote 6, valores de `renda` 100 vezes maiores (erro de unidade).

Tarefas.

1. Defina um esquema com tipo, intervalos, ausentes máximos e categorias permitidas, a partir do lote 1 (que é saudável).
2. Valide cada lote. Se reprovar, envie-o para uma lista de "quarentena" com os motivos; se aprovar, acrescente-o ao conjunto acumulado.
3. Para cada lote aprovado, registre em um manifesto: número do lote, hash SHA-256, linhas e data.
4. Mostre uma tabela com o resultado de cada lote (aprovado/quarentena, motivos).
5. Verifique que o hash do conjunto acumulado muda a cada lote aprovado, e que reaprovar o mesmo lote duas vezes produz o mesmo hash do lote.

Entregáveis. Tabela por lote, manifesto de versões e uma frase sobre o que faria com os lotes em quarentena.

A solução de referência está no Apêndice E.



Capítulo 4 — Pipelines Reprodutíveis com scikit-learn

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o papel de um pipeline como "linha de montagem" do modelo.
- Usar `Pipeline` e `ColumnTransformer` para unir pré-processamento e modelo.
- Demonstrar, com um experimento, como o pré-processamento fora do pipeline causa vazamento.
- Criar transformadores personalizados compatíveis com o scikit-learn.
- Separar configuração e código e fixar fontes de aleatoriedade.
- Salvar e recarregar um pipeline completo com segurança, verificando versões.
- Organizar o treino como um script reprodutível, executável por linha de comando.

4.1 Da receita solta à linha de montagem

No Capítulo 3 cuidamos dos ingredientes. Agora vamos montar a **linha de montagem**: uma sequência fixa de etapas que transforma dados brutos em previsões.

Pense em uma fábrica de sucos. Cada laranja passa por lavar, descascar, espremer, filtrar e engarrafar. Se alguém pula uma etapa ou muda a ordem, o suco sai diferente. No ML, as etapas são: preencher valores ausentes, converter categorias em números, padronizar escalas, selecionar atributos e treinar o modelo.

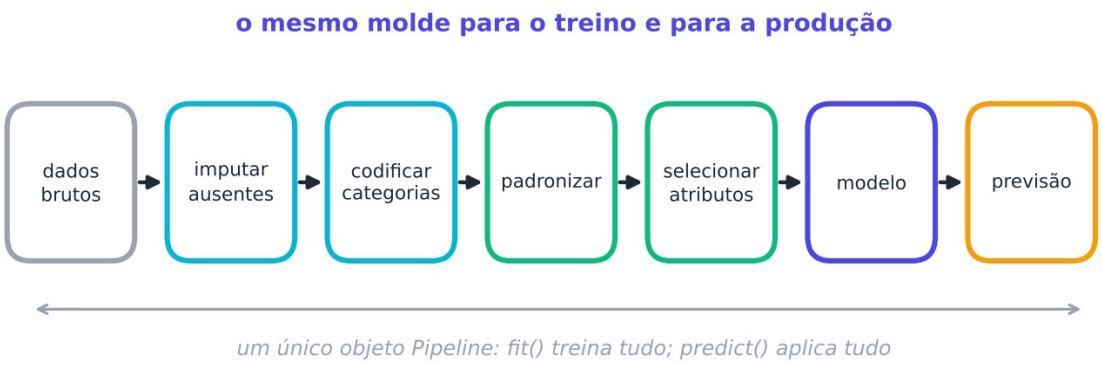


Figura 4.1 — Um pipeline é uma linha de montagem: cada etapa recebe a saída da anterior. O mesmo "molde" vale para o treino e para a produção.

Há dois grandes ganhos ao colocar tudo em um objeto único:

1. **Reprodutibilidade:** qualquer pessoa executa a mesma sequência.

2. **Prevenção de vazamento no pré-processamento:** as etapas aprendem somente com os dados de treino (cada dobra da validação cruzada refaz o ajuste).

> MEMORIZE **Em uma frase**
>
> **Pipeline** = pré-processamento + modelo empacotados em **um único objeto**, que se treina e se usa do mesmo jeito.

4.2 A linguagem do scikit-learn: fit, transform, predict

O scikit-learn (PEDREGOSA et al., 2011) é a biblioteca de ML mais usada em Python para dados tabulares. Buitinck e colegas (2013) descrevem o desenho de sua API, que gira em torno de poucas ideias simples:

- **Estimador** (*estimator*): qualquer objeto que **aprende** dos dados com `fit(X, y)`.
- **Transformador** (*transformer*): estimador que também **transforma** os dados com `transform(X)`. Exemplo: `StandardScaler` aprende a média e o desvio e depois padroniza.
- **Preditor** (*predictor*): estimador que **prevê** com `predict(X)`. Exemplo: `LogisticRegression`.

Em um transformador, `fit` **calcula e guarda** parâmetros (por exemplo, a média de cada coluna). `transform` **usa** esses parâmetros. A regra de ouro:

> ATENÇÃO **Regra de ouro**
>
> `fit` só pode ver os dados de **treino**. No teste e na produção, só se chama `transform` e `predict`.

4.3 Construindo um pipeline

PYTHON
```python
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

pipe = Pipeline([
    ("padronizar", StandardScaler()),
    ("modelo", LogisticRegression(max_iter=1000)),
])
pipe.fit(Xtr, ytr)            # aprende a média/desvio E os coeficientes
print("Acurácia no teste:", round(pipe.score(Xte, yte), 3))
print("Etapas:", list(pipe.named_steps))
```

Repare: chamamos `fit` **uma vez** no pipeline inteiro. Ele treina cada etapa em ordem, passando a saída para a seguinte. Para prever, `pipe.predict(X_novo)` aplica a padronização aprendida no treino e depois o modelo.

> **Dica**
>
> `make_pipeline(StandardScaler(), LogisticRegression())` faz o mesmo, dando nomes automáticos às etapas. Os nomes manuais ajudam depois, quando se ajustam hiperparâmetros (`modelo__C`).

4.4 O experimento do vazamento escondido

Um erro muito comum é aplicar o pré-processamento em **todos** os dados antes de separar treino e teste (ou antes de validar de forma cruzada). A informação do teste "contamina" o treino. Hastie, Tibshirani e Friedman (2009) alertam, em seu livro clássico, para o caso da **seleção de atributos feita fora da validação cruzada**.

Vamos reproduzir o efeito em um caso extremo. Criamos dados **puramente aleatórios**: 100 exemplos, 2.000 atributos, rótulos sorteados. Não existe nada a aprender. A acurácia honesta deveria ser cerca de 50%.

```python
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline

rng = np.random.default_rng(0)
X = rng.normal(size=(100, 2000))
y = rng.integers(0, 2, 100)
cv = StratifiedKFold(5, shuffle=True, random_state=0)

# ERRADO: seleciona os 20 "melhores" atributos usando TODOS os dados
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
errado = cross_val_score(LogisticRegression(max_iter=1000), X_sel, y, cv=cv).mean()

# CERTO: a seleção acontece dentro de cada dobra, via pipeline
pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression(max_iter=1000))
certo = cross_val_score(pipe, X, y, cv=cv).mean()

print(f"Seleção fora do pipeline (vaza): acurácia = {errado:.3f}")
print(f"Seleção dentro do pipeline    : acurácia = {certo:.3f}")
```

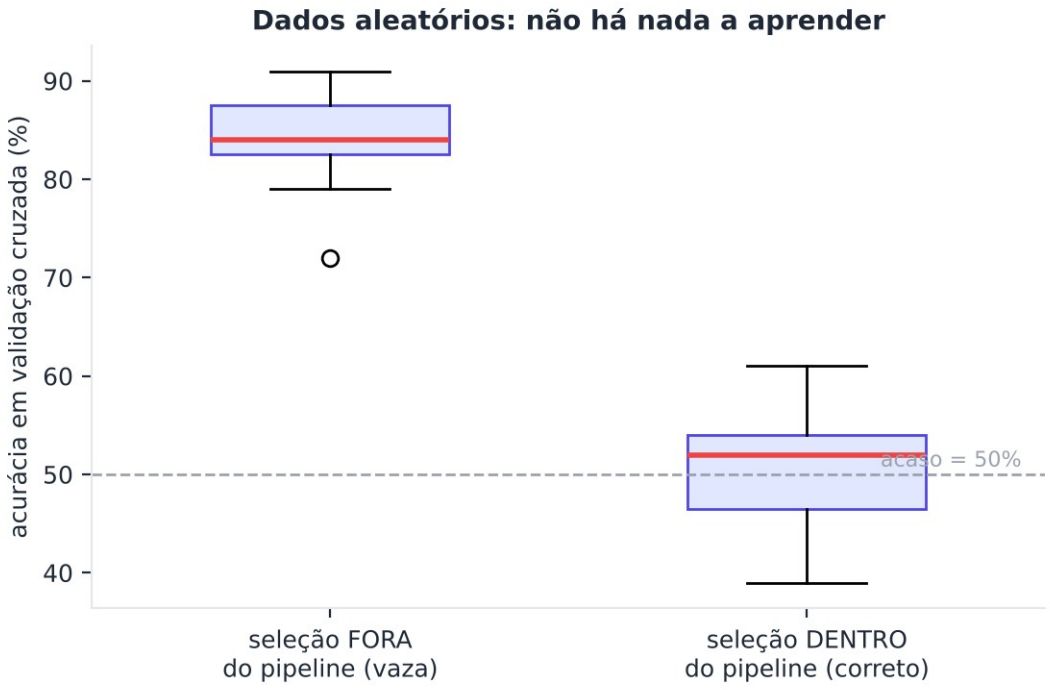


Figura 4.2 — Dados aleatórios não têm padrão. A seleção de atributos feita fora da validação cruzada "descobre" um desempenho falso; dentro do pipeline, o resultado volta a ser ao acaso (cerca de 50%).

O erro faz um modelo sem nenhum poder preditivo parecer excelente. Em um projeto real, isso leva a promessas que a produção não cumpre.

> **No seu dia a dia**
>
> É como escolher, depois da prova, as questões "que o aluno acertou" e dizer que ele estudou aquilo. A seleção olhou o gabarito.

4.5 Colunas diferentes, tratamentos diferentes: ColumnTransformer

Dados reais misturam tipos: números, categorias, datas, textos. Cada tipo pede um tratamento:

- **Numéricas:** preencher ausentes (mediana) e padronizar.
- **Categóricas:** preencher ausentes (valor mais frequente) e converter em colunas 0/1 (***one-hot encoding***).

O **one-hot encoding** transforma uma coluna "cidade" com valores Natal/Recife/Fortaleza em três colunas: "é Natal?", "é Recife?", "é Fortaleza?", cada uma com 0 ou 1.

O `ColumnTransformer` aplica o tratamento certo a cada grupo de colunas e junta o resultado.

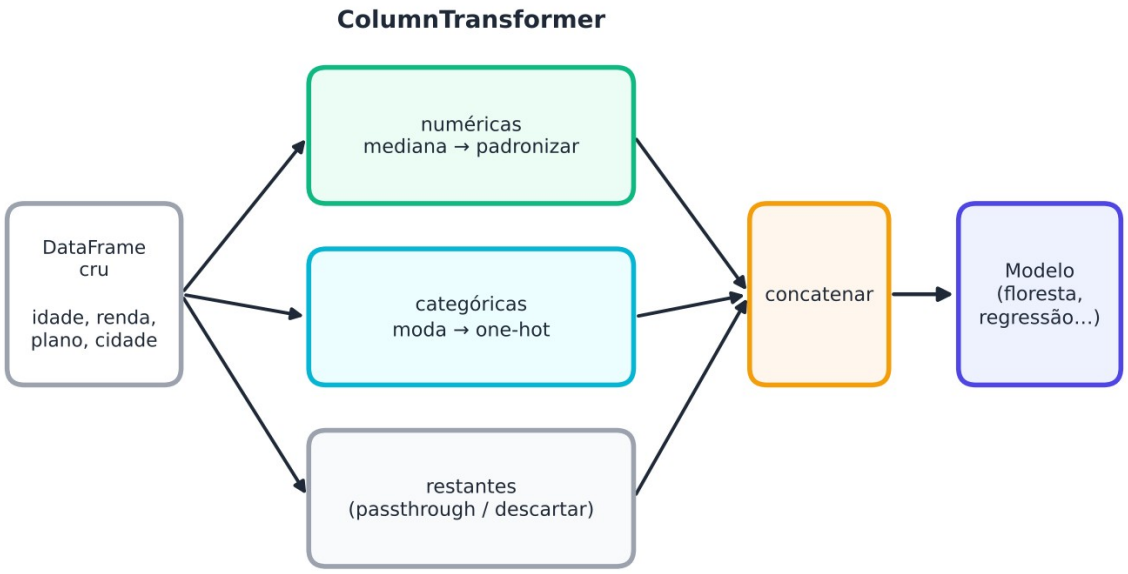


Figura 4.3 — `ColumnTransformer`: cada grupo de colunas segue seu próprio caminho de tratamento, e os resultados são concatenados antes do modelo.

```python
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

rng = np.random.default_rng(0)
n = 2000
df = pd.DataFrame({
    "idade": rng.integers(18, 75, n).astype(float),
```

```
    "renda": rng.lognormal(8, 0.5, n),
    "tempo_cliente": rng.integers(1, 120, n).astype(float),
    "plano": rng.choice(["basico", "padrao", "premium"], n, p=[0.5, 0.35, 0.15]),
    "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n),
})
risco = 0.03 * (50 - df["idade"]) - 0.02 * df["tempo_cliente"] + (df["plano"] == "basico") * 0.8
df["cancelou"] = (risco + rng.normal(0, 0.8, n) > 0.3).astype(int)
for col in ["renda", "plano"]:                      # introduz ausentes
    df.loc[rng.choice(n, 150, replace=False), col] = np.nan

num = ["idade", "renda", "tempo_cliente"]
cat = ["plano", "cidade"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("esc", StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("ohe", OneHotEncoder(handle_unknown="ignore"))]), cat),
])
pipe = Pipeline([("prep", prep), ("modelo", RandomForestClassifier(n_estimators=200, random_state=0))])

Xtr, Xte, ytr, yte = train_test_split(df[num + cat], df["cancelou"], test_size=0.25,
                                      stratify=df["cancelou"], random_state=0)
pipe.fit(Xtr, ytr)
print("Acurácia no teste:", round(pipe.score(Xte, yte), 3))

# o pipeline aceita uma linha crua, com ausentes e categoria nunca vista
novo = pd.DataFrame([{"idade": 33, "renda": np.nan, "tempo_cliente": 8,
                      "plano": "ultra", "cidade": "Natal"}])
print("Probabilidade de cancelar:", round(pipe.predict_proba(novo)[0, 1], 3))
```

Duas escolhas importantes:

- `handle_unknown="ignore"`: se na produção aparecer uma categoria que não existia no treino (como "ultra"), o encoder não quebra. Ele gera zeros para aquela coluna.
- Os imputadores aprendem a **mediana e o valor mais frequente do treino**, e reutilizam esses valores em produção.

> **Dica**
>
> Receba sempre a **linha crua** (como vem do sistema de origem). O pipeline cuida do resto. Isso elimina uma classe inteira de erros: o **desvio treino–serviço** (*training-serving skew*), em que o código de produção faz o pré-processamento de modo diferente do treino.

4.6 Transformadores personalizados

Às vezes é preciso criar um atributo novo, como a razão entre duas colunas. Para que isso entre no pipeline, a etapa precisa obedecer à API (`fit` e `transform`).

```python
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
class RazaoRendaIdade(BaseEstimator, TransformerMixin):
    """Acrescenta a coluna renda/idade, com proteção contra divisão por zero."""
    def fit(self, X, y=None):
        return self                      # nada a aprender

    def transform(self, X):
        X = X.copy()
        X["renda_por_idade"] = X["renda"] / X["idade"].clip(lower=1)
        return X

df = pd.DataFrame({"idade": [20, 40, 0], "renda": [2000.0, 8000.0, 1000.0]})
etapa = Pipeline([("razao", RazaoRendaIdade()), ])
print(etapa.fit_transform(df))
```

> **Atenção**
>
> Um transformador personalizado **não deve** conter regras escondidas que dependam de dados fora do pipeline. Se ele precisa de algum número aprendido (como uma média), esse número deve ser calculado em `fit` e guardado em um atributo terminado em `_`, como `media_`.

4.7 Configuração separada do código

Evite "números mágicos" espalhados pelo código. Reúna parâmetros em um **único lugar**: um dicionário, um arquivo YAML ou JSON. Vantagens:

- Mudar um experimento não exige mexer no código.
- Cada execução pode guardar sua configuração (rastreabilidade).
- Revisões ficam mais fáceis.

```
PYTHON
import json

CONFIG = {
    "semente": 42,
    "tamanho_teste": 0.25,
    "modelo": {"tipo": "random_forest", "n_estimators": 200, "max_depth": None},
    "colunas_numericas": ["idade", "renda", "tempo_cliente"],
    "colunas_categoricas": ["plano", "cidade"],
}
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(CONFIG, f, indent=2, ensure_ascii=False)

with open("config.json", encoding="utf-8") as f:
    lida = json.load(f)
print("Configuração idêntica?", lida == CONFIG)
```

4.8 Controlando a aleatoriedade

Reprodutibilidade exige controlar todas as fontes de acaso:

Fonte de acaso	Como controlar
Divisão treino/teste	`random_state` em `train_test_split`
Algoritmos aleatórios (florestas, K-means…)	`random_state` no estimador
Validação cruzada com embaralhamento	`random_state` no `KFold`/`StratifiedKFold`
NumPy	`np.random.default_rng(semente)`
Paralelismo (`n_jobs`)	Resultados costumam ser iguais; confira se o algoritmo é determinístico
Hardware/bibliotecas	Fixe versões (Capítulo 1)

Um bom hábito é **testar** a reprodutibilidade: treinar duas vezes e comparar as previsões.

```python
import hashlib
import numpy as np
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)

def treinar_e_assinar():
    pipe = make_pipeline(StandardScaler(), RandomForestClassifier(n_estimators=100, random_state=7,
n_jobs=2))
    pipe.fit(X, y)
    previsoes = pipe.predict_proba(X)
    return hashlib.sha256(previsoes.tobytes()).hexdigest()[:12]

a, b = treinar_e_assinar(), treinar_e_assinar()
print("Assinatura 1:", a)
print("Assinatura 2:", b)
print("Reprodutível?", a == b)
```

4.9 Salvando e carregando o pipeline completo

Depois de treinado, o pipeline vira um **artefato** (um arquivo) que será entregue à produção. A forma mais comum em Python é usar `joblib`.

```python
import joblib
import numpy as np
import sklearn
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X, y)

artefato = {"pipeline": pipe, "versao_sklearn": sklearn.__version__, "colunas": 13}
joblib.dump(artefato, "modelo_v1.joblib")
```

```
carregado = joblib.load("modelo_v1.joblib")
assert carregado["versao_sklearn"] == sklearn.__version__, "versão diferente da usada no treino!"
iguais = np.array_equal(pipe.predict(X), carregado["pipeline"].predict(X))
print("Previsões idênticas após recarregar:", iguais)
```

Atenção: segurança e compatibilidade

- **Nunca carregue** um arquivo `pickle`/`joblib` de origem desconhecida: ele pode executar código arbitrário ao ser aberto.
- Um modelo salvo com uma versão do scikit-learn pode **não funcionar** (ou funcionar mal) em outra. Registre a versão junto do arquivo, como fizemos acima.
- Formatos alternativos, como ONNX, ou bibliotecas como o `skops`, ajudam em cenários com mais exigências de segurança e portabilidade.

4.10 O treino como script

O notebook é ótimo para explorar. Para produção, o treino deve ser um **script** que roda de ponta a ponta, sem cliques, com parâmetros por linha de comando. Veja um esqueleto completo e executável, que usa tudo o que aprendemos:

PYTHON

```python
import argparse
import json
import joblib
import sklearn
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler


def treinar(semente=42, n_arvores=100, saida="modelo.joblib"):
    X, y = load_breast_cancer(return_X_y=True)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=semente)
    pipe = make_pipeline(StandardScaler(), RandomForestClassifier(n_estimators=n_arvores,
random_state=semente))
    pipe.fit(Xtr, ytr)
    metricas = {"f1": round(f1_score(yte, pipe.predict(Xte)), 4)}
    joblib.dump({"pipeline": pipe, "versao_sklearn": sklearn.__version__, "metricas": metricas}, saida)
    return metricas


def main(argv=None):
    ap = argparse.ArgumentParser(description="Treina o modelo e salva o pipeline.")
    ap.add_argument("--semente", type=int, default=42)
    ap.add_argument("--n-arvores", type=int, default=100)
    ap.add_argument("--saida", default="modelo.joblib")
    args = ap.parse_args(argv)
    print(json.dumps(treinar(args.semente, args.n_arvores, args.saida)))
```

```
if __name__ == "__main__":
    main([])    # em um arquivo real, use main() para ler a linha de comando
```

Em um arquivo `src/treino.py` real, você chamaria `main()` sem argumentos e executaria no terminal: `python src/treino.py --semente 7 --n-arvores 300`.

4.11 Pipelines de ML dentro de pipelines de dados

Há dois sentidos para a palavra "pipeline", e é bom não confundir:

- **Pipeline do scikit-learn:** a sequência *dentro* do código Python, de pré-processamento + modelo.
- **Pipeline de MLOps:** o fluxo *maior*, que une etapas como **ingestão → validação → treino → avaliação → registro → implantação**, cada uma muitas vezes um programa separado.

O segundo é organizado como um **grafo acíclico dirigido** (DAG, do inglês *directed acyclic graph*): uma lista de tarefas com setas indicando quem depende de quem, sem ciclos. Veremos orquestração no Capítulo 10, mas já dá para treinar o olhar: **cada etapa deve ter entradas e saídas claras**, para poder ser executada, testada e repetida sozinha.

4.12 Resumo do capítulo

MEMORIZE **O que levar deste capítulo**

- **Pipeline** junta pré-processamento e modelo em um só objeto: reprodutível e ajuda a **prevenir o vazamento no pré-processamento** (não impede atributos vindos do futuro, divisões inadequadas ou escolhas feitas olhando o teste).
- `fit` só vê o treino; `transform` e `predict` são usados em teste e produção.
- Seleção ou padronização **fora** da validação cruzada vaza informação e infla resultados.
- `ColumnTransformer` trata cada tipo de coluna de um jeito; `handle_unknown="ignore"` protege contra categorias novas.
- Transformadores personalizados obedecem à API (`fit`/`transform`).
- **Configuração fora do código**, sementes fixas e **teste de reprodutibilidade**.
- Salve o pipeline **inteiro** junto da versão das bibliotecas; nunca abra arquivos `pickle` desconhecidos.
- O treino de produção é um **script** com parâmetros, não um notebook.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Na analogia do capítulo, um pipeline é como:

a) uma receita escrita à mão, sem ordem.
b) um depósito de ingredientes.
c) um painel de alertas.
d) uma linha de montagem em que cada etapa recebe a saída da anterior.

Questão 2. Um estimador no scikit-learn é um objeto que:

a) só salva arquivos.
b) aprende dos dados com fit(X, y).
c) apenas desenha gráficos.
d) só transforma colunas, sem aprender.

Questão 3. O que um transformador faz em transform(X)?

a) Calcula a acurácia.
b) Usa os parâmetros aprendidos em fit para transformar os dados.
c) Treina novamente o modelo do zero.
d) Descarta os dados de treino.

Questão 4. Qual a regra de ouro do fit?

a) fit deve ser chamado em produção a cada previsão.
b) fit deve ver treino e teste.
c) fit só pode ver os dados de treino.
d) fit é opcional em transformadores.

Questão 5. Ao chamar fit em um Pipeline, o que acontece?

a) Somente a primeira etapa é treinada.
b) Somente a última etapa é treinada.
c) Nada é treinado.
d) Cada etapa é treinada em ordem, passando a saída para a seguinte.

Questão 6. Qual é a vantagem de dar nomes às etapas de um Pipeline?

a) Dispensa o uso de sementes.
b) Acelera o treino.
c) Impede que o pipeline seja salvo.
d) Permite ajustar hiperparâmetros por nome, como modelo__C.

Questão 7. No experimento com dados aleatórios, a seleção de atributos feita fora da validação cruzada produziu:

a) erro de execução.
b) acurácia de 0%.
c) acurácia falsamente alta, perto de 80% em vez de cerca de 50%.
d) acurácia exatamente igual ao pipeline correto.

Questão 8. A seleção de atributos feita dentro do Pipeline:

a) é ignorada pela validação cruzada.
b) acontece separadamente em cada dobra, usando só o treino da dobra.
c) usa os rótulos do teste.
d) acontece uma vez, com todos os dados.

Questão 9. O que é one-hot encoding?

a) Converter uma coluna categórica em várias colunas de 0 e 1.
b) Padronizar colunas numéricas.
c) Remover colunas categóricas.
d) Ordenar as categorias por frequência.

Questão 10. O ColumnTransformer serve para:

a) transformar linhas em colunas.
b) aplicar tratamentos diferentes a grupos diferentes de colunas e juntar o resultado.
c) salvar o modelo em disco.
d) escolher o melhor modelo.

Questão 11. handle_unknown="ignore" no OneHotEncoder é útil porque:

a) elimina a necessidade de treinar.
b) reduz o número de linhas.
c) categorias novas em produção não quebram o pipeline.
d) impede qualquer erro de dados.

Questão 12. Os imputadores de um pipeline aprendem a mediana e a moda:

a) com os dados de teste.
b) com os dados futuros.
c) com os dados de treino e as reutilizam em produção.
d) a cada nova previsão.

Questão 13. O que é o desvio treino–serviço?

a) O modelo ser rápido demais.
b) O número de linhas do teste ser pequeno.
c) A semente ser fixa.
d) O pré-processamento de produção ser diferente do usado no treino.

Questão 14. Para criar um transformador personalizado compatível com scikit-learn, a classe deve:

a) implementar fit e transform (por exemplo, herdando de BaseEstimator e TransformerMixin).
b) ter apenas um método print.
c) usar somente variáveis globais.
d) ser escrita em outra linguagem.

Questão 15. Um número aprendido em fit, como uma média, deve ser guardado:

a) no código do usuário.
b) em um atributo terminado em _, como media_.
c) em uma variável global.
d) em um arquivo texto separado.

Questão 16. Por que é recomendável separar a configuração do código?

a) Permite mudar experimentos sem mexer no código e registrar cada configuração.
b) Impede revisões.
c) Evita o uso de sementes.
d) Aumenta a acurácia.

Questão 17. Para a reprodutibilidade, quais fontes de aleatoriedade devem ser controladas?

a) Nenhuma, se o código for o mesmo.
b) Apenas a divisão dos dados.
c) Todas: divisão dos dados, algoritmos aleatórios, validação cruzada embaralhada e geradores do NumPy.
d) Apenas o algoritmo de treino.

Questão 18. Para testar a reprodutibilidade do treino, uma boa prática é:

a) treinar duas vezes e comparar as previsões ou um hash delas.
b) treinar uma vez e confiar.
c) mudar a semente a cada execução.
d) apagar os resultados antigos.

Questão 19. Qual formato é mais usado para salvar um pipeline do scikit-learn em Python?

a) joblib.
b) CSV.
c) MP3.
d) PNG.

Questão 20. O principal risco de carregar um arquivo pickle ou joblib de origem desconhecida é:

a) aumento da latência do disco.
b) mudança da hora do sistema.
c) perda de resolução das imagens.
d) execução de código arbitrário ao abrir o arquivo.

Questão 21. Por que registrar a versão do scikit-learn junto do modelo salvo?

a) Porque o joblib exige.
b) Um modelo salvo em uma versão pode não funcionar corretamente em outra.
c) Para impedir o reuso do modelo.
d) Para aumentar o tamanho do arquivo.

Questão 22. O que se espera de um script de treino de produção?

a) Não salvar nenhum resultado.
b) Depender de células executadas manualmente.
c) Ser executado apenas em notebooks.
d) Rodar de ponta a ponta, sem cliques, com parâmetros por linha de comando.

Questão 23. Qual biblioteca padrão do Python é usada no capítulo para ler argumentos de linha de comando?

a) wave.
b) tkinter.
c) argparse.
d) turtle.

Questão 24. A palavra "pipeline" em MLOps, no sentido mais amplo, designa:

a) o fluxo que une ingestão, validação, treino, avaliação, registro e implantação.
b) um cano físico de dados.
c) apenas o objeto Pipeline do scikit-learn.
d) o hardware do servidor.

Questão 25. Em um DAG, as setas indicam:

a) a ordem alfabética das tarefas.
b) dependências: quem precisa rodar antes de quem.
c) a quantidade de memória usada.
d) a prioridade das tarefas no orçamento.

Questão 26. Qual a utilidade de FunctionTransformer?

a) Criar gráficos.
b) Treinar redes neurais.
c) Escolher hiperparâmetros.
d) Transformar uma função comum em uma etapa de pipeline.

Questão 27. Qual destas práticas ajuda a evitar vazamento na validação cruzada?

a) Colocar o pré-processamento dentro do Pipeline avaliado.
b) Selecionar atributos com todos os rótulos.
c) Usar o teste para ajustar o pré-processamento.
d) Padronizar todos os dados antes de validar.

Questão 28. Em qual situação padronizar os atributos afeta pouco o resultado?

a) Em SVC com kernel RBF.
b) Em florestas aleatórias, que usam cortes em um atributo por vez.
c) Em KNN, que usa distâncias.
d) Em regressão logística regularizada.

Questão 29. Qual o risco de ajustar um imputador de mediana usando todos os dados antes da divisão?

a) Os dados ficam criptografados.
b) A mediana inclui informação do teste, o que contamina a avaliação.
c) O treino fica mais lento sem ganho algum.
d) O imputador deixa de funcionar.

Questão 30. Em um Pipeline, o que o método predict faz?

a) Treina novamente cada etapa.
b) Aplica as transformações aprendidas e depois usa o modelo final.
c) Gera um relatório de qualidade.
d) Salva o pipeline em disco.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a regra "fit só vê o treino" e dê um exemplo de erro por violá-la.
2. Descreva o experimento de dados aleatórios com seleção de atributos e a conclusão sobre vazamento.
3. Como o ColumnTransformer ajuda a evitar o desvio treino–serviço?
4. Por que handle_unknown="ignore" é importante em produção? Qual a sua limitação?
5. Explique o que é um transformador personalizado e quando criá-lo, dando um exemplo.
6. Quais cuidados devem ser tomados com transformadores que aprendem parâmetros?
7. Liste cinco fontes de aleatoriedade que devem ser controladas para a reprodutibilidade de um treino.
8. Por que não se deve usar pickle com arquivos de origem desconhecida? Que alternativas existem?
9. Qual a vantagem de manter a configuração em um arquivo JSON ou YAML?
10. Descreva a estrutura de um script de treino de produção e o que ele deve salvar.
11. Diferencie o pipeline do scikit-learn do pipeline de MLOps.
12. Por que ordenar as etapas do pré-processamento importa? Dê um exemplo.
13. O que significa dizer que um pipeline "recebe a linha crua"?
14. Quais testes você faria para um transformador que corta valores extremos nos percentis aprendidos no treino?
15. Explique por que fixar a semente do modelo e da divisão resolve apenas parte do problema da reprodutibilidade.
16. Como o hash das previsões ajuda a testar a reprodutibilidade?
17. Dê um exemplo em que a padronização é essencial e outro em que é dispensável. Justifique.
18. Por que o código do treino deve ser executável por linha de comando em ambientes de CI?
19. Explique por que um pipeline treinado deve ser tratado como um artefato versionado.
20. Resuma três erros comuns na construção de pipelines e como evitá-los.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Padronizar importa para uns, não para outros

NÍVEL: BÁSICO

No conjunto *Wine* (UCI), compare, por validação cruzada de 5 dobras, a acurácia de KNN (`KNeighborsClassifier`), SVC e floresta aleatória **com** e **sem** `StandardScaler`, sempre dentro de um `Pipeline`. Explique quais algoritmos dependem da escala.

Exercício 2 — Dados mistos com ColumnTransformer

NÍVEL: INTERMEDIÁRIO

Crie 1.500 clientes simulados com `idade`, `renda` (com 8% de ausentes), `plano` (com 5% de ausentes) e `cidade`. O alvo é a compra (0/1), dependente de `idade` e `plano`. Monte um `ColumnTransformer` (mediana + padronização para numéricas; moda + one-hot para categóricas) dentro de um `Pipeline` com regressão logística. Depois, preveja para uma linha com plano e cidade nunca vistos no treino e confirme que não ocorre erro.

Exercício 3 — Um transformador que aprende limites

NÍVEL: INTERMEDIÁRIO

Crie o transformador `Winsorizador`, que, em `fit`, aprende os percentis 1% e 99% de cada coluna e, em `transform`, limita (clip) os valores a esses limites. Verifique em um conjunto simulado com valores extremos que os limites vêm **apenas do treino** e que um valor extremo no teste é cortado pelo limite do treino.

Exercício 4 — O vazamento da seleção de atributos, em muitas sementes

NÍVEL: AVANÇADO

Repita o experimento de dados aleatórios (100 exemplos, 1.000 atributos, rótulos sorteados) para 20 sementes diferentes. Em cada uma, compare a acurácia da validação cruzada com a seleção `SelectKBest(k=10)` feita **fora** versus **dentro** do `Pipeline`. Mostre a média e o desvio de cada abordagem e conte em quantas sementes a abordagem errada passa de 60%.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult (Census Income)* e *Bank Marketing* têm variáveis numéricas e categóricas, ideais para `ColumnTransformer`.
> - **Kaggle:** *Titanic* e *House Prices* são clássicos para praticar pipelines com dados mistos e ausentes.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Treino como script parametrizado e reprodutível

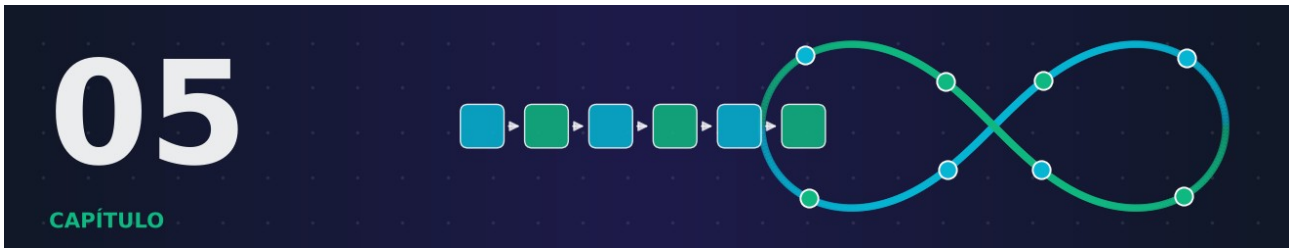
Contexto. O treino do modelo de diagnóstico (conjunto *Breast Cancer*, UCI) hoje vive em um notebook. Você deve transformá-lo em um script configurável.

Tarefas.

1. Escreva um `config.json` com semente, tamanho do teste, número de árvores e profundidade máxima.
2. Escreva `treino.py`, que lê a configuração, monta um `Pipeline` (padronização + floresta), treina, calcula F1 e acurácia no teste e salva em um único arquivo `joblib` o pipeline, a configuração, as métricas, a versão do scikit-learn e um hash dos dados.
3. Escreva `prever.py`, que carrega o arquivo, **verifica a versão do scikit-learn** e prevê para um CSV de entrada, gravando o resultado em outro CSV.
4. Execute `treino.py` duas vezes em processos separados e mostre que as métricas e o hash das previsões são iguais.
5. Altere a configuração (`n_estimators`) e mostre que o artefato muda (hash diferente).

Entregáveis. Os três arquivos, a saída das execuções e uma frase sobre o que ainda faltaria para reprodutibilidade completa.

A solução de referência está no Apêndice E.



Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que experimentos precisam ser registrados de forma sistemática.
- Distinguir parâmetros, métricas, artefatos, tags e execuções (*runs*).
- Registrar experimentos com o MLflow Tracking e comparar execuções.
- Salvar modelos com assinatura (*signature*) e carregá-los de forma uniforme.
- Usar o registro de modelos (*model registry*) com versões e apelidos (*aliases*).
- Aplicar uma regra automática de promoção de modelos baseada em métricas.

5.1 O caderno de laboratório do cientista

Imagine um cozinheiro que testa 40 versões de uma receita, mas **não anota nada**. Na semana seguinte, o chefe pergunta: "Qual versão ficou melhor? Quanto de açúcar usamos?". Ninguém sabe.

É o que acontece quando se treinam modelos sem registro. Um dia alguém diz: "aquele modelo de terça-feira era melhor, volte para ele". Mas qual era? Com quais parâmetros? Com quais dados?

O **rastreamento de experimentos** (*experiment tracking*) é o **caderno de laboratório** do ML. Cada tentativa deixa um registro completo.

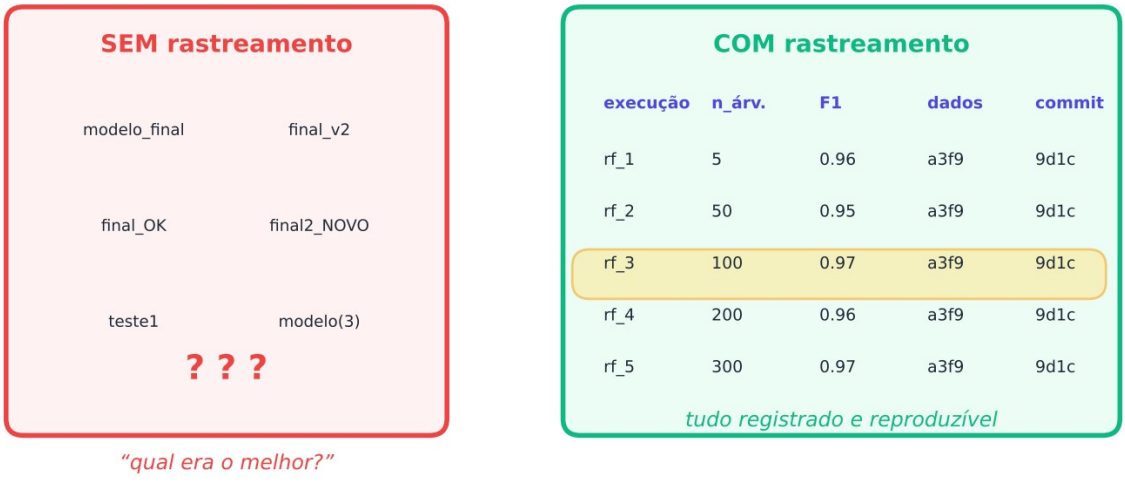


Figura 5.1 — Sem rastreamento, as tentativas se perdem. Com rastreamento, cada execução guarda parâmetros, métricas, artefatos e a origem do código e dos dados.

Zaharia e colegas (2018) apresentaram o **MLflow**, uma plataforma de código aberto criada justamente para organizar o ciclo de vida de ML. No artigo, eles motivam a ferramenta com dificuldades comuns: acompanhar experimentos, reproduzir código e compartilhar e implantar modelos.

5.2 O vocabulário do rastreamento

Termo	O que é	Exemplo
Experimento	Grupo de execuções para um mesmo objetivo	"detecção de cancelamento"
Execução (*run*)	Uma tentativa de treino, do começo ao fim	`rf_200_arvores`
Parâmetro	Valor de entrada, fixo durante a execução	`n_estimators = 200`
Métrica	Número que mede o resultado, pode variar no tempo	`f1 = 0,96`
Artefato	Arquivo produzido: modelo, gráfico, relatório	`matriz_confusao.png`
Tag	Rótulo livre para organizar	`autor = Ana`
Modelo registrado	Modelo catalogado, com versões	`detector_cancer v3`

5.3 Primeiros passos com o MLflow

O MLflow tem alguns componentes principais:

- **Tracking:** registra e consulta execuções.
- **Models:** um formato padrão para empacotar modelos.
- **Model Registry:** catálogo de modelos, com versões e apelidos.
- **Projects:** formato para empacotar código reprodutível (não usaremos aqui).

Para registrar informações, precisamos dizer *onde* guardar. O jeito mais simples é um banco de dados local SQLite.

```python
import math
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")   # banco local, um único arquivo
mlflow.set_experiment("primeiros_passos")

with mlflow.start_run(run_name="minha_primeira_execucao"):
    mlflow.log_param("taxa_aprendizado", 0.1)
    mlflow.log_param("origem_dos_dados", "simulado")
    for epoca in range(1, 6):
        perda = math.exp(-0.5 * epoca)                # perda caindo com o tempo
        mlflow.log_metric("perda", perda, step=epoca)  # métrica com passo
    mlflow.set_tag("autor", "estudante")

execucoes = mlflow.search_runs()
print(execucoes[["run_id", "params.taxa_aprendizado", "metrics.perda"]].to_string(index=False))
```

O trecho `with mlflow.start_run()` abre uma execução, e tudo o que for registrado dentro dele fica associado a ela. Ao sair do bloco, a execução é encerrada.

> **Dica**
>
> Para **ver** os registros em uma interface web, rode no terminal, na pasta do projeto: `mlflow ui --backend-store-uri sqlite:///mlflow.db`. Depois abra o endereço que o programa mostrar (normalmente `http://127.0.0.1:5000`). Esse comando só funciona com o MLflow instalado.

5.4 Registrando um experimento de verdade

Vamos comparar três configurações de uma floresta aleatória. Para cada uma, registramos:

- os **parâmetros** (hiperparâmetros e semente);
- as **métricas** (F1 e acurácia);
- a **impressão digital dos dados** (hash do Capítulo 3), para saber com qual dado foi treinado;
- um **artefato** gráfico (matriz de confusão);
- o **modelo**, com sua **assinatura**.

A **assinatura** (*signature*) descreve os nomes e tipos de entrada e saída esperados. É um contrato: se alguém enviar dados no formato errado, o erro aparece cedo.

```python
import hashlib
import warnings
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import mlflow
import mlflow.sklearn
import pandas as pd
from mlflow.models import infer_signature
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score, f1_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("cancer_rf")

dados = load_breast_cancer(as_frame=True)
X, y = dados.data, dados.target
hash_dados = hashlib.sha256(pd.concat([X, y], axis=1).to_csv(index=False).encode()).hexdigest()[:12]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

for n_arvores in [5, 50, 300]:
    with mlflow.start_run(run_name=f"rf_{n_arvores}"):
        pipe = make_pipeline(StandardScaler(),
                             RandomForestClassifier(n_estimators=n_arvores, random_state=0))
        pipe.fit(Xtr, ytr)
        previsto = pipe.predict(Xte)

        mlflow.log_params({"n_estimators": n_arvores, "semente": 0, "tamanho_teste": 0.25})
        mlflow.log_metrics({"f1": f1_score(yte, previsto), "acuracia": accuracy_score(yte, previsto)})
```

```
        mlflow.set_tags({"hash_dados": hash_dados, "base": "UCI WDBC"})

        fig, ax = plt.subplots(figsize=(3.6, 3.2))
        ConfusionMatrixDisplay.from_predictions(yte, previsto, ax=ax)
        mlflow.log_figure(fig, "matriz_confusao.png")
        plt.close(fig)

        assinatura = infer_signature(Xte, previsto)
        mlflow.sklearn.log_model(pipe, name="modelo", signature=assinatura,
                                 skops_trusted_types=["sklearn.tree._tree.Tree"])

res = mlflow.search_runs(order_by=["metrics.f1 DESC"])
print(res[["tags.mlflow.runName", "params.n_estimators", "metrics.f1", "metrics.acuracia"]]
      .round(4).to_string(index=False))
```

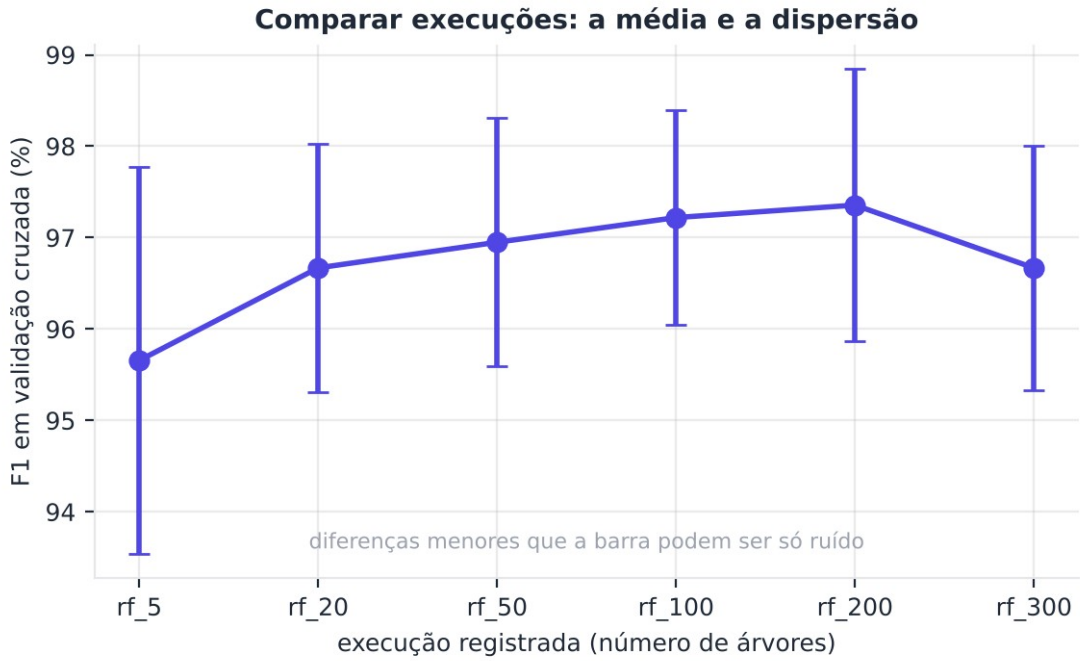


Figura 5.2 — Comparação das execuções do experimento. Com tudo registrado, escolher o melhor modelo vira uma consulta, e não uma memória.

> **Atenção: diferença pequena não é prova**
>
> Neste exemplo, a floresta com **5 árvores** teve F1 ligeiramente maior que as de 50 e 300 árvores. Com apenas 143 exemplos de teste, diferenças dessa ordem podem ser **ruído**. Registrar tudo permite ver isso; decidir com rigor exige as técnicas estatísticas do Capítulo 6.

5.4.1 Cuidado com o formato do modelo salvo

O MLflow salva modelos do scikit-learn, por padrão, em um formato seguro chamado **skops**. Ao carregar, ele exige que você declare quais tipos de objeto são confiáveis (como `sklearn.tree._tree.Tree`, usado nas árvores). Isso evita executar código malicioso escondido em um arquivo de modelo.

> **Atenção**
>
> Formatos como `pickle` e `cloudpickle` podem executar código arbitrário ao serem abertos. Prefira formatos seguros e **só carregue modelos de fontes que você controla**. Versões diferentes do MLflow podem ter padrões diferentes: confira a documentação da versão que usar.

5.4.2 O que mais vale registrar

Quanto mais contexto, melhor a rastreabilidade. Além do que fizemos, considere registrar:

- o **commit do Git** do código (o MLflow costuma capturar automaticamente quando há repositório);
- o arquivo **requirements.txt** ou a lista de pacotes;
- a **duração** do treino e o **consumo de memória**;
- métricas por **subgrupo** de dados (para justiça e robustez, Capítulos 6 e 10);
- **comentários** sobre a motivação do experimento.

5.4.3 Registro automático (autologging)

O MLflow consegue registrar parâmetros, métricas e o modelo **automaticamente** para as bibliotecas mais usadas. É prático para começar:

```python
import warnings
import mlflow
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("autolog_demo")
mlflow.sklearn.autolog(log_models=False)     # registra parâmetros e métricas de treino

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
with mlflow.start_run():
    LogisticRegression(max_iter=5000, C=0.5).fit(Xtr, ytr)

execucao = mlflow.search_runs().iloc[0]
print("Parâmetro C registrado sozinho:", execucao["params.C"])
```

O *autologging* é conveniente, mas **não substitui** o registro consciente das decisões importantes (hash dos dados, semente, métricas de negócio).

5.5 Do experimento ao produto: o registro de modelos

Escolher o melhor modelo não basta. É preciso um **catálogo oficial** de modelos aprovados, com versões. É o **registro de modelos** (*model registry*).

Pense em uma **biblioteca**. Cada livro tem edições. Há uma "edição vigente" que todos devem consultar. O registro funciona assim:

- **Modelo registrado:** o "título" (por exemplo, `detector_cancer`).

- **Versão:** a "edição" (v1, v2, v3...).
- **Apelido** (*alias*): um nome móvel que aponta para uma versão, como `champion` (a que está em produção) e `challenger` (a candidata).

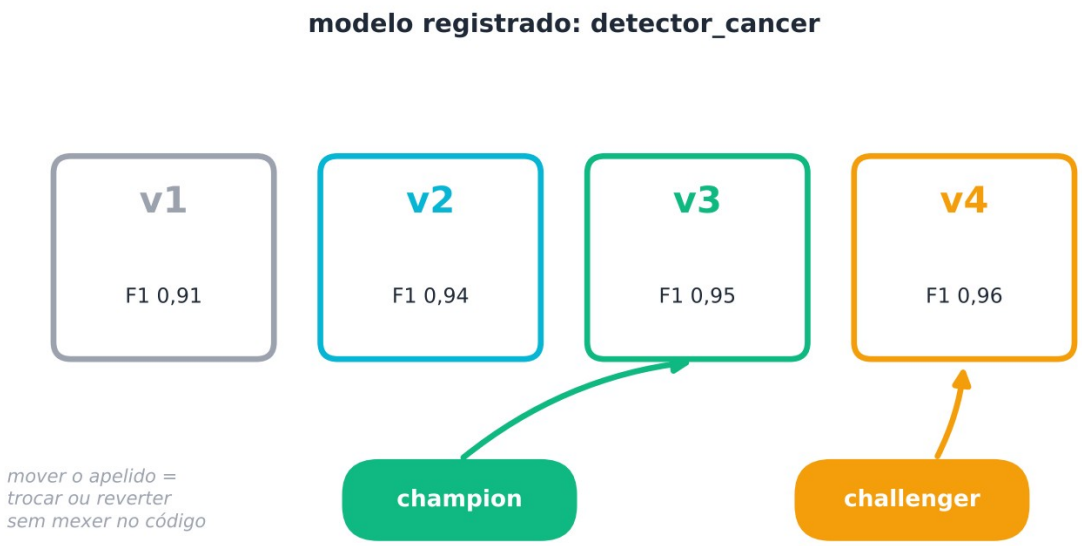


Figura 5.3 — Registro de modelos: versões numeradas e apelidos que apontam para elas. Para trocar o modelo, move-se o apelido `champion`; o serviço precisa então recarregar a versão apontada.

A grande vantagem dos apelidos: o código de produção carrega sempre `models:/detector_cancer@champion`. Para atualizar ou reverter o modelo, **ninguém precisa editar o código de produção**: basta mover o apelido.

Mas atenção a uma distinção importante: **mover o apelido muda a referência no registro, e não o objeto que já está carregado na memória do serviço**. Um serviço que carregou o modelo na partida continua usando a versão antiga até que **recarregue** o modelo (ou seja reimplantado). Veremos isso na demonstração abaixo e, com mais detalhes, no Capítulo 7.

```python
import warnings
import mlflow
import mlflow.sklearn
from mlflow import MlflowClient
from mlflow.models import infer_signature
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("registro_demo")
NOME = "detector_cancer"

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

def treinar_e_registrar(C):
```

```
    with mlflow.start_run(run_name=f"logreg_C{C}"):
        pipe = make_pipeline(StandardScaler(), LogisticRegression(C=C, max_iter=2000)).fit(Xtr, ytr)
        f1 = f1_score(yte, pipe.predict(Xte))
        mlflow.log_params({"C": C})
        mlflow.log_metric("f1", f1)
        info = mlflow.sklearn.log_model(pipe, name="modelo",
                                        signature=infer_signature(Xte, pipe.predict(Xte)))
    versao = mlflow.register_model(info.model_uri, NOME)    # cria v1, v2, ...
    return versao.version, f1

cliente = MlflowClient()
v1, f1_v1 = treinar_e_registrar(0.01)
cliente.set_registered_model_alias(NOME, "champion", v1)     # v1 vai para produção
v2, f1_v2 = treinar_e_registrar(1.0)
cliente.set_registered_model_alias(NOME, "challenger", v2)   # v2 é a candidata

print(f"v{v1}: F1 = {f1_v1:.4f} | v{v2}: F1 = {f1_v2:.4f}")
producao = mlflow.sklearn.load_model(f"models:/{NOME}@champion")
print("Previsões do champion:", producao.predict(Xte[:5]))
print("Champion é a versão:", cliente.get_model_version_by_alias(NOME, "champion").version)
```

5.5.1 Mover o apelido não atualiza o modelo já carregado

Vamos continuar o exemplo: o objeto `producao` foi carregado quando o `champion` era a v1. Agora movemos o apelido para a v2 e comparamos o objeto antigo com um carregamento novo.

```
PYTHON
# continuação (usa 'producao', 'cliente', 'NOME', 'v1', 'v2' e 'Xte' do exemplo anterior)
cliente.set_registered_model_alias(NOME, "champion", v2)          # o registro agora aponta para a v2
print("Registro aponta para a versão:", cliente.get_model_version_by_alias(NOME, "champion").version)

recarregado = mlflow.sklearn.load_model(f"models:/{NOME}@champion")   # novo carregamento pelo alias
dif_antigo = (producao.predict_proba(Xte)[:, 1] != recarregado.predict_proba(Xte)[:, 1]).sum()
print("Objeto carregado antes (ainda é a v1) x objeto recarregado (v2): probabilidades diferentes em",
int(dif_antigo), "de", len(Xte))

def versao_servida(modelo_em_memoria, versao_carregada):
    """O serviço deve registrar qual versão ele REALMENTE está usando."""
    apontada = cliente.get_model_version_by_alias(NOME, "champion").version
    return {"carregada": versao_carregada, "apontada_no_registro": apontada, "desatualizado":
versao_carregada != apontada}

print(versao_servida(producao, v1))
print(versao_servida(recarregado, v2))
```

Em produção, portanto, trocar o apelido é apenas **o primeiro passo**. É preciso um mecanismo de **recarga** (por exemplo, verificar periodicamente a versão apontada e recarregar, ou reimplantar o serviço) e **verificar a versão efetivamente servida**, como na função acima. Com vários processos ou réplicas do serviço, a troca deve ser **coordenada**: cada réplica recarrega em momentos diferentes, e há um intervalo em que respondem com versões diferentes.

> **Saiba mais**
>
> Versões antigas do MLflow usavam **estágios** (*Staging*, *Production*, *Archived*). Hoje a recomendação da própria ferramenta é usar **apelidos** e **tags**, mais flexíveis. Se você encontrar tutoriais com estágios, entenda que a ideia é a mesma: marcar qual versão está em cada fase.

5.6 Promoção automática com portão de qualidade

Mover o apelido `champion` à mão é arriscado. Podemos criar uma **regra automática** (um "portão") que só promove a candidata se ela for melhor **e** passar nas verificações mínimas.

A regra tem de ser conservadora. Por exemplo:

1. As métricas devem ser **válidas**: presentes, numéricas, finitas (sem `NaN`) e dentro do intervalo possível.
2. A candidata deve superar a campeã em pelo menos uma **margem mínima** (para evitar trocar por diferenças de ruído).
3. A candidata deve respeitar um **piso** absoluto de qualidade.
4. O resultado deve ser **registrado** (quem promoveu, quando, por quê).

```python
import math

def decidir_promocao(f1_campea, f1_candidata, margem=0.005, piso=0.90):
    """Retorna (promover?, motivo). Primeiro valida as métricas; depois compara."""
    for nome, v in (("campeã", f1_campea), ("candidata", f1_candidata)):
        if isinstance(v, bool) or not isinstance(v, (int, float)) or not math.isfinite(v) or not 0.0 <=
v <= 1.0:
            return False, f"reprovada: F1 da {nome} inválido ({v!r})"
    if f1_candidata < piso:
        return False, f"reprovada: F1 {f1_candidata:.3f} abaixo do piso {piso:.2f}"
    if f1_candidata < f1_campea + margem:
        return False, f"reprovada: ganho {f1_candidata - f1_campea:+.3f} menor que a margem {margem}"
    return True, f"aprovada: ganho de {f1_candidata - f1_campea:+.3f} sobre a campeã"

casos = [(0.950, 0.980), (0.950, 0.952), (0.950, 0.850), (0.920, 0.961), (0.950, float("nan"))]
for campea, candidata in casos:
    ok, motivo = decidir_promocao(campea, candidata)
    print(f"campeã={campea:.3f} candidata={candidata:.3f} -> {'PROMOVER' if ok else 'manter'}
({motivo})")
```

Em um pipeline real, essa função roda ao final do treino. Se aprovar, chama `set_registered_model_alias(..., "champion", nova_versao)` (e dispara a recarga ou a reimplantação do serviço). Se reprovar, mantém o modelo atual e avisa a equipe.

> **Dica**
>
> Uma comparação justa exige **o mesmo conjunto de teste** para as duas versões. Guarde um conjunto de avaliação fixo e versionado (Capítulo 3). Para decisões importantes, compare também as **incertezas** (Capítulo 6), e não só os valores médios.
>
> Cuidado, porém, com o **uso repetido do mesmo conjunto fixo** em muitas promoções: cada comparação "consome" um pouco da independência desse conjunto, e, depois de dezenas de decisões, o modelo campeão pode estar ajustado a ele. Reponha ou renove periodicamente o conjunto de avaliação, mantenha uma amostra de reserva que só é consultada raramente e registre quantas vezes cada conjunto foi usado em decisões.

5.7 Metadados de modelo: o que um "cartão de identidade" deve conter

Cada modelo registrado deve ter uma ficha mínima:

Campo	Exemplo
Nome e versão	`detector_cancer` v2
Objetivo e uso indicado	Triagem de apoio, não diagnóstico
Dados de treino	hash `a3f9...`, período, fonte
Código	commit `9d1c...`
Parâmetros	`C = 1.0`, semente 0
Métricas	F1 = 0,97 (teste), subgrupos
Limitações conhecidas	Dados de uma só instituição
Responsável	Nome da equipe/pessoa
Data e aprovação	2026-10-17, aprovado por...

O Capítulo 10 aprofunda isso com os ***model cards*** (MITCHELL et al., 2019).

5.8 Além do MLflow

O MLflow é uma opção entre várias para rastrear experimentos e registrar modelos. Existem plataformas comerciais e de código aberto com ideias semelhantes (por exemplo, Weights & Biases, Neptune, Comet, DVC Experiments, ClearML e os serviços de nuvem). O que importa é o **princípio**: registrar de forma sistemática, comparável e reprodutível.

Em plataformas industriais, a mesma ideia aparece como "metadados de ML" integrados ao pipeline, como no TFX (BAYLOR et al., 2017), que guarda a linhagem de cada artefato produzido.

5.9 Resumo do capítulo

MEMORIZE **O que levar deste capítulo**

- **Rastrear experimentos** é manter um caderno de laboratório: parâmetros, métricas, artefatos, tags, dados e código.
- No MLflow: **experimento** agrupa **execuções**; cada execução tem **parâmetros**, **métricas**, **artefatos** e **tags**.
- A **assinatura** do modelo descreve as entradas e saídas esperadas.
- O **registro de modelos** guarda **versões** e **apelidos** (`champion`, `challenger`).
- O código de produção carrega o modelo pelo **apelido**; para trocar ou reverter, mova o apelido **e recarregue o serviço**: o objeto já carregado continua sendo a versão antiga até a recarga.
- Use um **portão de promoção** com margem mínima, piso de qualidade e registro da decisão.
- Prefira formatos de modelo **seguros** e carregue apenas modelos de origem confiável.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O rastreamento de experimentos pode ser comparado a:

a) um caderno de laboratório em que cada tentativa deixa registro completo.
b) um servidor de e-mail.
c) um painel de câmeras de segurança.
d) um depósito de modelos sem organização.

Questão 2. No MLflow, uma execução (run) é:

a) um grupo de experimentos.
b) uma tentativa de treino, do começo ao fim.
c) um tipo de métrica.
d) um arquivo de modelo.

Questão 3. Qual destes é um parâmetro, e não uma métrica?

a) AP = 0,78.
b) F1 = 0,96.
c) Perda = 0,12.
d) n_estimators = 200.

Questão 4. Um artefato no MLflow é:

a) uma conta de usuário.
b) um arquivo produzido pela execução, como um modelo ou gráfico.
c) um tipo de dado numérico.
d) um erro de execução.

Questão 5. O comando mlflow.set_tracking_uri("sqlite:///mlflow.db") define:

a) onde os registros serão guardados (um banco SQLite local).
b) a senha do servidor.
c) o endereço do modelo em produção.
d) o tamanho do conjunto de teste.

Questão 6. O que faz o bloco with mlflow.start_run():?

a) Treina o modelo automaticamente.
b) Apaga o experimento.
c) Envia o modelo para produção.
d) Abre uma execução; tudo que for registrado dentro dele fica associado a ela.

Questão 7. A diferença entre log_param e log_metric é que:

a) a métrica pode ser registrada com passos ao longo do tempo (step).
b) log_metric só aceita texto.
c) não há diferença.
d) log_param só aceita arquivos.

Questão 8. Para que serve registrar o hash dos dados como tag da execução?

a) Reduzir o tempo de treino.
b) Evitar o uso de sementes.
c) Saber com qual versão de dados o modelo foi treinado.
d) Aumentar a acurácia.

Questão 9. A assinatura (signature) de um modelo no MLflow descreve:

a) o endereço do servidor.
b) o número de camadas.
c) os nomes e tipos de entrada e saída esperados.
d) quem escreveu o código.

Questão 10. Qual comando abre a interface web do MLflow para um banco SQLite?

a) mlflow delete --ui
b) mlflow ui --backend-store-uri sqlite:///mlflow.db
c) mlflow train --db
d) mlflow deploy --all

Questão 11. Por que o MLflow salva, por padrão, modelos scikit-learn no formato skops?

a) É um formato mais seguro que o pickle, que exige declarar tipos confiáveis.
b) Porque é o formato mais antigo.
c) Porque ocupa mais espaço, o que garante segurança.
d) Porque dispensa assinatura.

Questão 12. O autologging do MLflow:

a) escolhe o melhor modelo sozinho.
b) implanta o modelo em produção.
c) registra automaticamente parâmetros e métricas de bibliotecas suportadas.
d) elimina a necessidade de registrar o hash dos dados.

Questão 13. O registro de modelos (model registry) guarda:

a) apenas logs de acesso.
b) apenas dados brutos.
c) modelos com versões e apelidos.
d) apenas métricas.

Questão 14. Na analogia da biblioteca, a versão de um modelo equivale a:

a) a edição de um livro.
b) o autor do livro.
c) a estante onde ele fica.
d) o título do livro.

Questão 15. O apelido champion indica:

a) a versão mais antiga.
b) a versão apagada.
c) a versão com menos parâmetros.
d) a versão que está em produção.

Questão 16. Qual a vantagem de carregar o modelo por models:/nome@champion?

a) O modelo nunca erra.
b) Elimina a necessidade de registro.
c) O modelo fica mais rápido.
d) Trocar ou reverter exige mover o apelido e recarregar o serviço, sem editar o código.

Questão 17. Versões antigas do MLflow usavam estágios (Staging, Production). Hoje se recomenda:

a) apenas e-mails.
b) apelidos e tags.
c) nenhum controle de versão.
d) apenas nomes de arquivos.

Questão 18. O portão de promoção de modelos deve ser:

a) sempre aprovado quando a métrica é igual.
b) conservador, com margem mínima, piso de qualidade e registro da decisão.
c) manual, sem critérios.
d) inexistente em ambientes críticos.

Questão 19. Por que exigir uma margem mínima de ganho para promover um modelo?

a) Para economizar disco.
b) Para evitar trocas motivadas por diferenças de ruído.
c) Para esconder resultados.
d) Para reduzir a latência.

Questão 20. Qual a condição para uma comparação justa entre campeão e candidato?

a) Avaliar só o candidato.
b) Avaliar o campeão no treino e o candidato no teste.
c) Usar o mesmo conjunto de avaliação para os dois.
d) Usar conjuntos diferentes para cada um.

Questão 21. Qual dos itens abaixo NÃO faz parte do cartão de identidade mínimo de um modelo registrado?

a) As métricas e limitações.
b) O commit do código.
c) A senha do banco de dados de produção.
d) O hash dos dados de treino.

Questão 22. O MLflow Projects destina-se a:

a) empacotar código de forma reprodutível.
b) servir modelos por HTTP, apenas.
c) registrar métricas por passo.
d) catalogar versões de modelos.

Questão 23. Zaharia et al. (2018) apresentaram o MLflow como:

a) um algoritmo de classificação.
b) uma plataforma de código aberto para organizar o ciclo de vida de ML.
c) uma linguagem de programação.
d) um novo tipo de rede neural.

Questão 24. No TFX (Baylor et al., 2017), a ideia de metadados de ML aparece como:

a) a rotulagem manual de dados.
b) a criptografia de modelos.
c) o ajuste automático de hiperparâmetros apenas.
d) o registro da linhagem de cada artefato produzido no pipeline.

Questão 25. Qual é uma boa razão para registrar métricas por subgrupo em cada execução?

a) Evitar o uso do registro de modelos.
b) Permitir avaliar justiça e robustez, e não apenas a média.
c) Reduzir o espaço em disco.
d) Aumentar o número de execuções.

Questão 26. Se alguém diz "aquele modelo de terça-feira era melhor", o rastreamento permite:

a) identificar a execução, seus parâmetros, dados e código e recriá-la.
b) apagar o histórico.
c) apenas adivinhar a melhor versão.
d) mudar a data do modelo.

Questão 27. Qual a função de log_figure no MLflow?

a) Treinar o modelo.
b) Enviar e-mails.
c) Escolher o melhor hiperparâmetro.
d) Salvar uma figura (como a matriz de confusão) como artefato da execução.

Questão 28. Em qual situação o registro de modelos é especialmente útil?

a) Quando várias pessoas precisam saber qual versão está em produção e voltar a uma anterior se preciso.
b) Quando não há dados.
c) Quando há apenas um modelo e uma pessoa, nunca atualizado.
d) Quando não há métricas.

Questão 29. Para reconstruir um modelo campeão a partir do que foi registrado, é preciso ter:

a) apenas a data.
b) apenas a acurácia.
c) apenas o nome do modelo.
d) parâmetros, semente, dados (por hash) e código versionados.

Questão 30. Uma alternativa ao MLflow para rastrear experimentos é:

a) Microsoft Paint.
b) Uma planilha sem controle de acesso.
c) Weights & Biases (entre outras ferramentas).
d) Um editor de vídeo.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que treinar muitos modelos sem rastreamento é um problema e como o rastreamento resolve.
2. Defina experimento, execução, parâmetro, métrica, artefato e tag no vocabulário do MLflow.
3. O que é a assinatura de um modelo e por que ela é útil?
4. Descreva o fluxo do exemplo do capítulo com três florestas aleatórias registradas no MLflow, incluindo o que foi registrado em cada execução.
5. Por que uma diferença pequena de F1 entre modelos pode não significar nada?
6. Explique o papel dos apelidos no registro de modelos e por que facilitam a reversão.
7. Descreva um portão de promoção automática de modelos e três critérios que ele deve ter.
8. Por que o formato skops é considerado mais seguro que o pickle?
9. O que deve constar na ficha mínima de um modelo registrado?
10. Qual a diferença entre o autologging e o registro explícito? Quando preferir cada um?
11. Como o rastreamento de experimentos apoia a rastreabilidade em auditorias?
12. Explique a ideia de regra de promoção que inclui um piso e uma margem sobre uma linha de base.
13. Dê três exemplos do que mais vale registrar além de parâmetros e métricas.
14. Como você garantiria uma comparação justa entre o campeão e o candidato?
15. Por que o código de produção não deve depender do número da versão do modelo, mas sim do apelido?
16. Cite quatro componentes principais do MLflow e a função de cada um.
17. Descreva como o MLflow pode ser usado com um servidor remoto compartilhado e o que muda em relação ao uso local.
18. Por que o registro de modelos deve incluir o responsável e a data de aprovação?
19. Como você decidiria se um modelo registrado deve ser revertido para uma versão anterior?
20. Explique por que guardar o hash dos dados em cada execução ajuda a detectar erros de comparação entre modelos.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Três modelos, três execuções, uma consulta

NÍVEL: BÁSICO

Com o MLflow (banco SQLite temporário), registre três execuções no conjunto *Wine* (UCI): regressão logística, floresta aleatória e SVC. Em cada uma, registre o nome do modelo como parâmetro, a acurácia de teste como métrica e uma tag com o autor. Depois, use `mlflow.search_runs` para listar as execuções em ordem decrescente de acurácia.

Exercício 2 — Curva de aprendizado como métrica por passo

NÍVEL: INTERMEDIÁRIO

Treine um `GradientBoostingClassifier` com 150 árvores no conjunto *Breast Cancer* (UCI). Use `staged_predict_proba` para calcular a perda logarítmica (*log loss*) no conjunto de validação a cada árvore adicionada e registre-a no MLflow como métrica com `step`. Recupere o histórico com `MlflowClient().get_metric_history` e informe o número de árvores com a menor perda.

Exercício 3 — Versões, apelidos e reversão

NÍVEL: INTERMEDIÁRIO

Registre duas versões de um modelo no registro do MLflow (v1: regressão logística com `C=0.001`; v2: com `C=10`) no conjunto *Breast Cancer* (UCI). Dê à v2 o apelido `champion`. Carregue o modelo pelo apelido e meça a acurácia no teste. Em seguida, **reverta** movendo o apelido para a v1 e meça novamente, **sem editar o código de carga**. Mostre também que o objeto carregado **antes** da reversão continua sendo a versão antiga, até que o modelo seja recarregado pelo apelido.

Exercício 4 — Promoção só com evidência estatística

NÍVEL: AVANÇADO

No conjunto *Digits* (UCI), compare um campeão (regressão logística com `C=1`) e um candidato (SVC com `C=10`) no mesmo conjunto de teste. Calcule, por *bootstrap* com 2.000 reamostragens, o intervalo de confiança de 95% da diferença de acurácia (candidato − campeão). Escreva a regra: promover apenas se o limite inferior do intervalo for maior que zero. Teste a regra também com um candidato quase idêntico ao campeão (regressão logística com `C=0.7`).

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Optical Recognition of Handwritten Digits* (versão completa), *Wine Quality* e *Bank Marketing*.
> - **Kaggle:** *Telco Customer Churn* e *Digit Recognizer (MNIST)* para repetir o processo com mais dados e mais configurações.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Central de experimentos com portão e registro

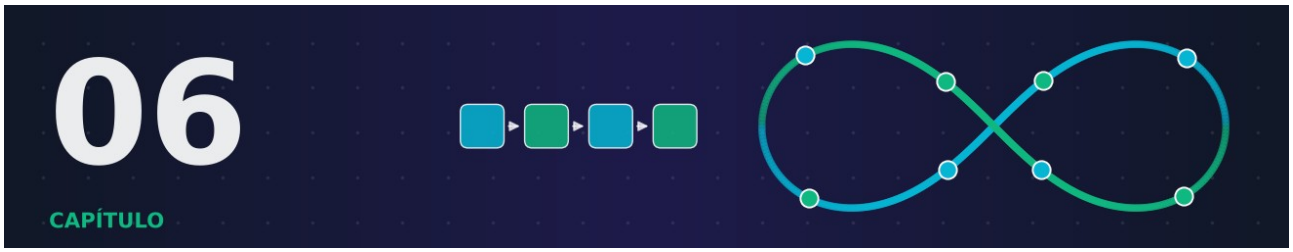
Contexto. Sua equipe vai escolher um modelo para o conjunto *Digits* (UCI) e precisa de um processo auditável.

Tarefas.

1. Defina uma grade com pelo menos 12 configurações de duas famílias de modelos (por exemplo, SVC com 6 combinações de `C` e `gamma` e floresta com 6 combinações de `n_estimators` e `max_depth`).
2. Para cada configuração, abra uma execução do MLflow e registre: parâmetros, F1 macro médio em validação cruzada de 5 dobras (métrica), tempo de treino (métrica), hash dos dados (tag) e o modelo treinado em todo o treino (com assinatura).
3. Consulte as execuções com `search_runs` e escolha a melhor pelo F1 de validação cruzada.
4. Aplique um portão: piso de F1 de 0,95 **e** ganho de pelo menos 0,01 sobre uma linha de base de regressão logística (também registrada).
5. Se aprovado, registre o modelo e dê o apelido `champion`; avalie **uma única vez** no teste.
6. Mostre que é possível reconstruir o modelo campeão a partir dos parâmetros registrados e obter a mesma acurácia.

Entregáveis. Tabela das execuções, decisão do portão, versão registrada e comprovação da reprodução.

A solução de referência está no Apêndice E.



Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Escolher métricas adequadas ao problema, inclusive com classes desbalanceadas.
- Ajustar o limiar de decisão considerando o custo dos erros.
- Verificar e corrigir a calibração das probabilidades.
- Usar validação cruzada, inclusive para séries temporais, e reportar incerteza.
- Comparar modelos com intervalos de confiança por *bootstrap*.
- Evitar o viés de seleção com validação cruzada aninhada.
- Comparar busca em grade e busca aleatória de hiperparâmetros.
- Avaliar o desempenho por subgrupos (fatias) dos dados.

6.1 Avaliar é mais difícil do que parece

Um estudante estuda para a prova com 100 questões. Se a prova tiver **as mesmas 100 questões**, ele tira 10... mas isso não prova que aprendeu a matéria. Só prova que decorou.

O mesmo vale para modelos. O que importa é o desempenho em **dados novos**, e estimar isso de modo honesto exige cuidado. Neste capítulo vamos ver:

1. **Que números** reportar (métricas).
2. **Como medir** sem se enganar (validação).
3. **Como ajustar** o modelo sem inflar o resultado (hiperparâmetros).
4. **Em quem** o modelo funciona bem ou mal (fatias).

D'Amour e colegas (2022) mostraram um fenômeno preocupante, a **subespecificação** (*underspecification*): vários modelos, treinados com o mesmo procedimento, podem ter **o mesmo desempenho no teste** e, ainda assim, **comportar-se de modo muito diferente em produção**. Ou seja: a nota do teste, sozinha, não garante o comportamento real. É um argumento forte a favor de avaliar de várias formas.

6.2 Métricas de classificação

Em uma classificação binária (por exemplo, "fraude" ou "legítima"), cada previsão cai em uma de quatro categorias:

	Previsto: positivo	Previsto: negativo
Real: positivo	Verdadeiro positivo (VP)	Falso negativo (FN)
Real: negativo	Falso positivo (FP)	Verdadeiro negativo (VN)

Esta tabela é a **matriz de confusão**. Dela saem as métricas principais:

$$\text{Precisão} = VP / (VP + FP) \qquad \text{Revocação} = VP / (VP + FN)$$

- **Precisão:** das vezes que o modelo disse "positivo", quantas estavam certas? (Quanto o alarme é confiável.)
- **Revocação** (*recall*, sensibilidade): dos casos positivos reais, quantos o modelo encontrou? (Quanto o modelo "enxerga".)
- **F1:** média harmônica entre precisão e revocação. Só é alta se **as duas** forem altas.

> **No seu dia a dia**
>
> Um **detector de fumaça** muito sensível apita com torrada queimada (baixa precisão, alta revocação). Um muito "frouxo" nunca apita com falsos alarmes, mas pode deixar passar um incêndio real (alta precisão, baixa revocação). O que é pior depende do contexto.

6.2.1 O engano da acurácia

Quando as classes são desbalanceadas, a **acurácia** engana. Em um conjunto com 5% de fraudes, dizer "nunca é fraude" dá 95% de acurácia. Veja:

```python
from sklearn.dummy import DummyClassifier
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, average_precision_score, f1_score,
                             precision_score, recall_score, roc_auc_score)
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=6000, n_features=12, n_informative=5,
                           weights=[0.95], flip_y=0.01, class_sep=0.8, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
print("Fração de positivos no teste:", round(yte.mean(), 3))

def relatorio(nome, modelo):
    modelo.fit(Xtr, ytr)
    prev = modelo.predict(Xte)
    p = modelo.predict_proba(Xte)[:, 1]
    print(f"{nome:14s} acurácia={accuracy_score(yte, prev):.3f} precisão={precision_score(yte, prev,
zero_division=0):.3f} "
          f"revocação={recall_score(yte, prev):.3f} F1={f1_score(yte, prev):.3f} "
          f"AUC-ROC={roc_auc_score(yte, p):.3f} AP={average_precision_score(yte, p):.3f}")

relatorio("Sempre negativo", DummyClassifier(strategy="most_frequent"))
relatorio("Floresta", RandomForestClassifier(n_estimators=200, random_state=0))
```

O modelo "preguiçoso" tem acurácia altíssima e **zero** de revocação: ele não encontra nenhuma fraude.

6.2.2 Curva ROC e curva de precisão-revocação

Muitos modelos produzem uma **pontuação** (um número entre 0 e 1) e não uma resposta direta. A decisão depende de um **limiar**: acima dele, "positivo". Variando o limiar, obtemos curvas:

- **Curva ROC:** taxa de verdadeiros positivos contra taxa de falsos positivos. Resumo: **AUC-ROC**.
- **Curva de precisão-revocação (PR):** precisão contra revocação. Resumo: a **precisão média** (**AP**, de *average precision*).

Davis e Goadrich (2006) estudaram a relação entre as duas curvas, e Saito e Rehmsmeier (2015) mostraram que, com **classes muito desbalanceadas**, a curva PR costuma ser **mais informativa** que a ROC, pois a ROC pode parecer ótima mesmo quando muitos alarmes são falsos.

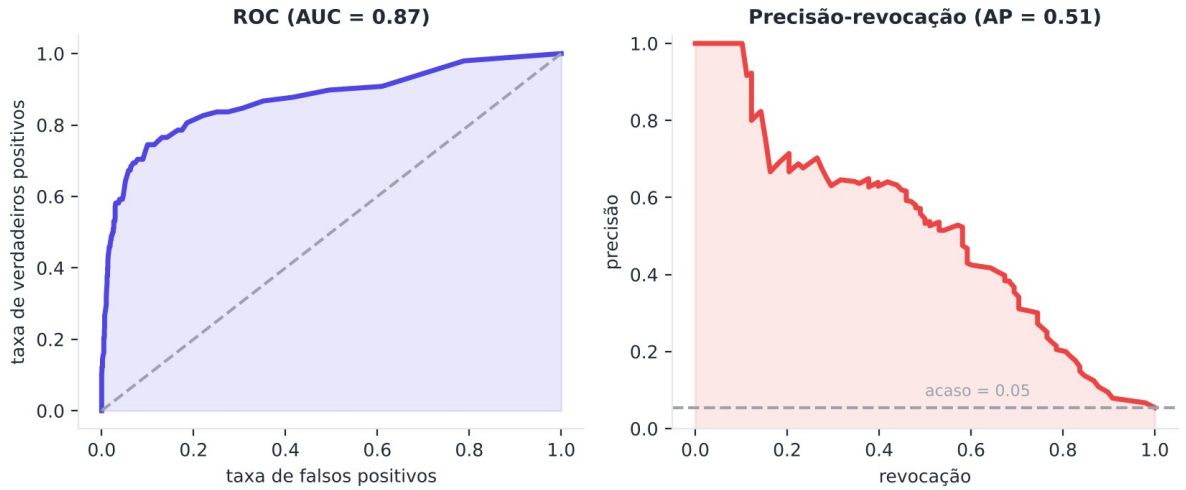


Figura 6.1 — Curva ROC e curva de precisão-revocação para o mesmo modelo, com 5% de positivos. A curva ROC parece mais generosa do que a PR.

6.2.3 AP não é "a" área sob a curva PR

O `average_precision_score` do scikit-learn calcula a **precisão média** (AP): uma **soma ponderada** das precisões em cada limiar, em que o peso é o aumento da revocação. Ela resume a curva PR, mas **não é necessariamente igual à área** obtida integrando a curva pela regra do trapézio; as duas convenções podem dar valores diferentes. Por isso, neste livro chamamos a métrica de **AP** e não de "AUC-PR". Ao comparar números de fontes distintas, confira qual convenção foi usada.

```python
from sklearn.metrics import auc, average_precision_score, precision_recall_curve

y_real = [0, 0, 1, 1]
pontuacao = [0.1, 0.4, 0.35, 0.8]

ap = average_precision_score(y_real, pontuacao)
precisao, revocacao, _ = precision_recall_curve(y_real, pontuacao)
area_trapezio = auc(revocacao, precisao)
print(f"AP (soma ponderada)        : {ap:.4f}")
print(f"Área trapezoidal da curva PR: {area_trapezio:.4f}")
```

6.3 Escolhendo o limiar pelo custo

O limiar padrão de 0,5 raramente é o melhor (no exemplo abaixo, o ótimo fica na borda inferior da faixa testada, o que é típico quando o falso negativo é muito mais caro). Se um falso negativo (fraude não detectada) custa **R$ 500** e um falso positivo (checagem manual desnecessária) custa **R$ 5**, o limiar ideal é muito menor que 0,5.

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=6000, n_features=12, n_informative=5,
                           weights=[0.95], flip_y=0.01, class_sep=0.8, random_state=0)
Xtr, Xval, ytr, yval = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
p = RandomForestClassifier(n_estimators=200, random_state=0).fit(Xtr, ytr).predict_proba(Xval)[:, 1]

CUSTO_FN, CUSTO_FP = 500.0, 5.0
limiares = np.linspace(0.02, 0.98, 49)
custos = []
for t in limiares:
    prev = p >= t
    fn = ((~prev) & (yval == 1)).sum()
    fp = (prev & (yval == 0)).sum()
    custos.append(CUSTO_FN * fn + CUSTO_FP * fp)

melhor = limiares[int(np.argmin(custos))]
print(f"Limiar padrão (0,50): custo total = R$ {custos[np.argmin(np.abs(limiares - 0.5))]:,.0f}")
print(f"Limiar ótimo ({melhor:.2f}):  custo total = R$ {min(custos):,.0f}")
```

> **Atenção**
>
> O limiar deve ser escolhido no conjunto de **validação**, e nunca no de teste. Caso contrário, você estará ajustando o modelo à prova. Além disso, o limiar ótimo **muda** se os custos ou a proporção de positivos mudarem. Ele é parte do sistema e precisa ser versionado e monitorado.

6.4 Calibração: as probabilidades dizem a verdade?

Se o modelo diz "80% de chance de cancelar", será que, entre todos os clientes com 80%, cerca de 80% realmente cancelam? Se sim, o modelo é **calibrado**. Niculescu-Mizil e Caruana (2005) mostraram que diferentes algoritmos produzem probabilidades mais ou menos confiáveis; Guo e colegas (2017) mostraram que redes neurais modernas tendem a ser **superconfiantes**.

Isso importa quando a **probabilidade** entra na decisão: preço de seguro, priorização de atendimento, risco de crédito.

O **escore de Brier** mede o erro quadrático médio das probabilidades (menor é melhor). A calibração pode ser melhorada com `CalibratedClassifierCV`.

```python
import numpy as np
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import brier_score_loss
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=8000, n_features=15, n_informative=6,
                           flip_y=0.05, class_sep=0.7, random_state=1)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, random_state=1, stratify=y)

bruto = RandomForestClassifier(n_estimators=200, random_state=0)
calibrado = CalibratedClassifierCV(RandomForestClassifier(n_estimators=200, random_state=0),
                                   method="isotonic", cv=5)
for nome, m in [("Floresta bruta", bruto), ("Floresta calibrada", calibrado)]:
    p = m.fit(Xtr, ytr).predict_proba(Xte)[:, 1]
    frac, medio = calibration_curve(yte, p, n_bins=8, strategy="quantile")
    erro = np.abs(frac - medio).mean()
    print(f"{nome:20s} Brier = {brier_score_loss(yte, p):.4f} | erro médio de calibração = {erro:.3f}")
```

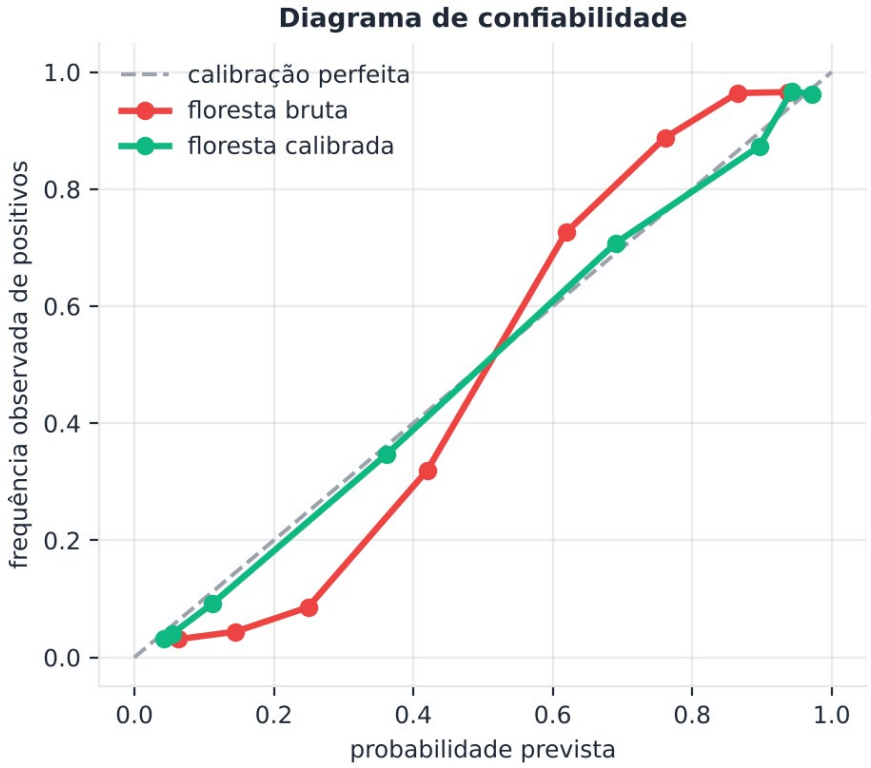


Figura 6.2 — Diagrama de confiabilidade: quanto mais perto da diagonal, mais as probabilidades "dizem a verdade". Floresta bruta versus calibrada.

6.5 Validação cruzada

Reservar um único conjunto de teste desperdiça dados e gera um resultado que depende da sorte da divisão. A **validação cruzada** (*cross-validation*) repete o processo várias vezes. No esquema de *k* dobras (*k-fold*):

1. Divida os dados em *k* partes iguais (por exemplo, 5).
2. Para cada parte: treine nas outras *k* − *1* e teste nela.
3. Resuma os *k* resultados pela **média** e pelo **desvio padrão**.

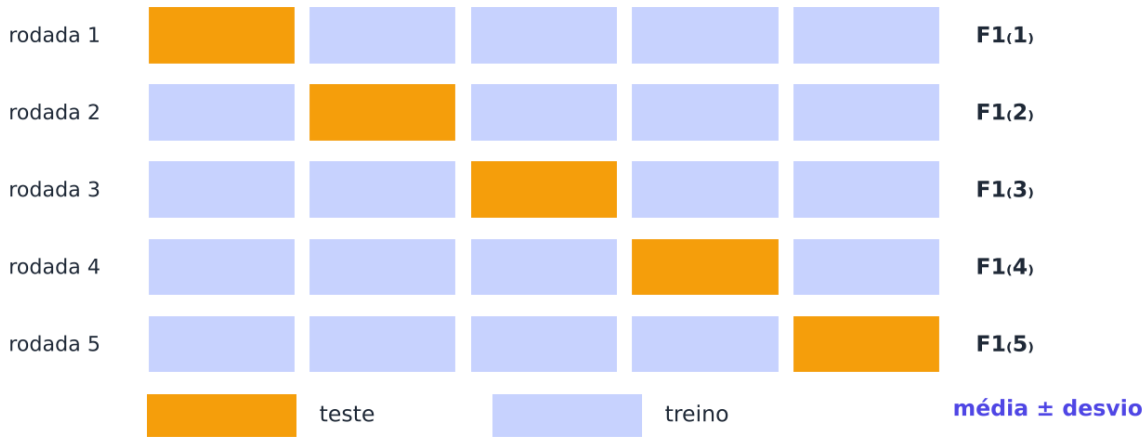


Figura 6.3 — Validação cruzada com 5 dobras. Cada pedaço é usado uma vez como teste (laranja) e quatro vezes como treino (azul).

Kohavi (1995) comparou métodos de estimação de acurácia e recomendou a validação cruzada estratificada de 10 dobras como boa escolha geral. Arlot e Celisse (2010) fazem uma ampla revisão teórica dos procedimentos.

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=0)   # 15 avaliações

modelos = {
    "Regressão logística": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
    "Floresta aleatória": RandomForestClassifier(n_estimators=200, random_state=0),
}
for nome, m in modelos.items():
    s = cross_val_score(m, X, y, cv=cv, scoring="f1")
    print(f"{nome:20s} F1 = {s.mean():.4f} ± {s.std():.4f}   (mín {s.min():.3f}, máx {s.max():.3f})")
```

Sempre reporte a **média e a dispersão**. O desvio padrão entre dobras descreve a variabilidade de **cada** modelo, mas **não é, por si só, um teste da diferença entre dois modelos**: como as dobras são compartilhadas, as estimativas são correlacionadas. Se a diferença entre dois modelos é pequena diante dessa variabilidade, desconfie de que seja ruído e confirme com uma comparação **pareada** (a diferença por dobra, ou o *bootstrap* da próxima seção).

6.5.1 Quando os dados têm tempo: não embaralhe!

Em séries temporais, embaralhar é **trapaça**: o modelo veria o futuro durante o treino. A validação deve respeitar a ordem: sempre treinar no passado e testar no futuro (BERGMEIR; BENÍTEZ, 2012). A classe `TimeSeriesSplit` faz isso.

```python
import numpy as np
```

```
from sklearn.model_selection import TimeSeriesSplit

dias = np.arange(24)    # 24 meses ordenados no tempo
for i, (tr, te) in enumerate(TimeSeriesSplit(n_splits=4, test_size=4).split(dias), 1):
    print(f"Dobra {i}: treino = meses {tr.min()}-{tr.max()} | teste = meses {te.min()}-{te.max()}")
```

6.6 Comparando modelos com honestidade

"O modelo A teve 0,962 e o B teve 0,958. Então A é melhor?" Não necessariamente. Dietterich (1998) discutiu testes estatísticos para comparar algoritmos de classificação, e Demšar (2006) propôs procedimentos para comparar vários classificadores em vários conjuntos de dados. A ideia central: **quantificar a incerteza**.

Uma ferramenta simples e poderosa é o ***bootstrap***: sortear, **com reposição**, exemplos do conjunto de teste muitas vezes, recalcular a métrica a cada vez e olhar a variação.

PYTHON

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
A = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
B = RandomForestClassifier(n_estimators=200, random_state=0).fit(Xtr, ytr)
acerto_A = (A.predict(Xte) == yte).astype(float)
acerto_B = (B.predict(Xte) == yte).astype(float)

rng = np.random.default_rng(0)
dif = []
for _ in range(5000):
    idx = rng.integers(0, len(yte), len(yte))          # reamostragem com reposição
    dif.append(acerto_A[idx].mean() - acerto_B[idx].mean())
lo, hi = np.percentile(dif, [2.5, 97.5])

print(f"Acurácia A = {acerto_A.mean():.3f} | B = {acerto_B.mean():.3f}")
print(f"Diferença A − B = {acerto_A.mean() - acerto_B.mean():+.3f}  (IC 95%: {lo:+.3f} a {hi:+.3f})")
print("O intervalo inclui zero?", lo <= 0 <= hi)
```

Se o intervalo de confiança **inclui o zero**, não há evidência clara de que um modelo seja melhor que o outro. Nesse caso, desempate por critérios práticos: simplicidade, custo, velocidade, explicabilidade.

> **Dica**
>
> Um intervalo largo indica poucos dados de teste. Em vez de dar um veredito apressado, **colete mais dados de avaliação**.

6.7 O viés de seleção e a validação aninhada

Quando você testa muitas configurações e escolhe a melhor **pela mesma validação cruzada**, o resultado da melhor tende a ser **otimista**: parte do seu bom desempenho é sorte. Varma e Simon (2006) mostraram esse viés na

estimação de erro com seleção de modelo, e Cawley e Talbot (2010) discutiram como o superajuste na seleção de modelos compromete a avaliação.

A solução é a **validação cruzada aninhada** (*nested cross-validation*):

- **Laço interno:** escolhe os hiperparâmetros.
- **Laço externo:** estima o desempenho do **procedimento completo** em dados que o laço interno nunca viu.

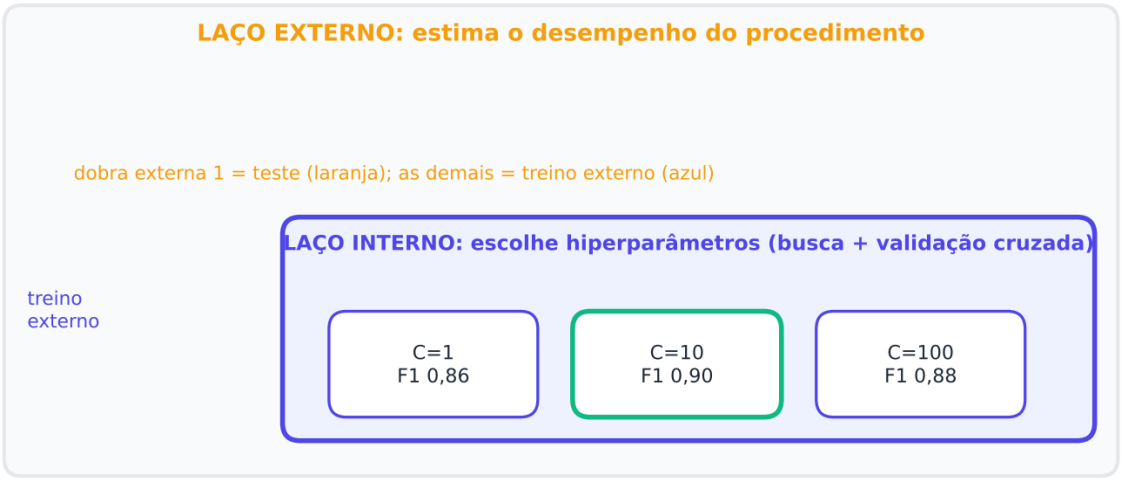


Figura 6.4 — Validação cruzada aninhada: o laço externo avalia; o interno escolhe. O teste do laço externo nunca participa da escolha de hiperparâmetros.

O efeito é mais visível com **poucos dados e muitos atributos**. Vamos simular:

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.svm import SVC

X, y = make_classification(n_samples=80, n_features=60, n_informative=3, n_redundant=0,
                           class_sep=0.4, flip_y=0.2, random_state=2)
grade = {"C": [0.01, 0.1, 1, 10, 100], "gamma": [1e-4, 1e-3, 1e-2, 1e-1, 1]}
interno = StratifiedKFold(5, shuffle=True, random_state=0)
externo = StratifiedKFold(5, shuffle=True, random_state=1)

busca = GridSearchCV(SVC(), grade, cv=interno).fit(X, y)
print(f"Melhor acurácia da busca (otimista) : {busca.best_score_:.3f}")

aninhada = cross_val_score(GridSearchCV(SVC(), grade, cv=interno), X, y, cv=externo)
print(f"Estimativa aninhada (honesta)       : {aninhada.mean():.3f} ± {aninhada.std():.3f}")
```

A primeira nota é a que normalmente aparece em relatórios descuidados. A segunda é mais próxima do que se verá com dados novos.

6.8 Otimização de hiperparâmetros

Parâmetros são aprendidos pelo modelo (como os coeficientes de uma regressão). **Hiperparâmetros** são escolhas feitas **antes** do treino (como o número de árvores ou a profundidade máxima). É como a diferença entre a **receita** (hiperparâmetros: temperatura do forno, tempo) e o **bolo** que sai dela.

Estratégias:

Estratégia	Ideia	Vantagem	Limite
Grade (*grid*)	Testa todas as combinações de uma lista	Simples, completa	Explode com muitos hiperparâmetros
Aleatória (*random*)	Sorteia combinações	Explora melhor o que importa	Pode perder o ótimo
Bayesiana	Usa resultados anteriores para escolher a próxima tentativa	Eficiente com treinos caros	Mais complexa
Multi-fidelidade (Hyperband)	Interrompe cedo as tentativas ruins	Economiza recursos	Exige "orçamento" parcial

Bergstra e Bengio (2012) mostraram que a **busca aleatória** é frequentemente mais eficiente que a grade, porque, na prática, só alguns hiperparâmetros importam muito e a grade desperdiça tentativas repetindo valores dos que não importam.



Figura 6.5 — Grade versus aleatória com 9 tentativas. Se só o eixo horizontal importa, a grade testa apenas 3 valores dele; a busca aleatória testa 9.

Snoek, Larochelle e Adams (2012) popularizaram a otimização bayesiana, Li e colegas (2018) propuseram o Hyperband, e Akiba e colegas (2019) apresentaram o Optuna, uma biblioteca prática que reúne essas ideias. Feurer e colegas (2015) e Hutter, Kotthoff e Vanschoren (2019) tratam do **AutoML**, que automatiza escolhas de modelo e hiperparâmetros.

```python
from scipy.stats import loguniform, randint
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import (GridSearchCV, RandomizedSearchCV, StratifiedKFold,
                                     train_test_split)

X, y = load_breast_cancer(return_X_y=True)
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
modelo = GradientBoostingClassifier(random_state=0)

grade = GridSearchCV(modelo, {"learning_rate": [0.01, 0.1, 0.5],
                              "n_estimators": [50, 150],
                              "max_depth": [2, 3, 4]}, cv=cv, scoring="f1").fit(Xtr, ytr)
aleat = RandomizedSearchCV(modelo, {"learning_rate": loguniform(1e-3, 1),
                                    "n_estimators": randint(30, 300),
                                    "max_depth": randint(1, 6)},
                           n_iter=18, cv=cv, scoring="f1", random_state=0).fit(Xtr, ytr)

for nome, b in [("Grade (18 combinações)", grade), ("Aleatória (18 sorteios)", aleat)]:
    print(f"{nome:24s} F1 (validação) = {b.best_score_:.4f} | F1 (teste) = "
          f"{b.score(Xte, yte):.4f}")
print("Melhores da aleatória:", {k: (round(float(v), 3) if k == "learning_rate" else int(v))
                                 for k, v in aleat.best_params_.items()})
```

> **Atenção**
>
> Nunca escolha hiperparâmetros olhando o conjunto de **teste**. A busca usa só treino e validação (cruzada). O teste serve para **um único** relatório final.

6.9 Avaliando por fatias

Uma média pode esconder desigualdades. Um modelo com 95% de acerto geral pode ter 99% para um grupo e 80% para outro. Por isso, em MLOps, avaliamos por **fatias** (*slices*): por região, faixa de idade, tipo de produto, período do dia etc. Breck e colegas (2017) incluem a verificação de qualidade em fatias importantes entre os itens do ML Test Score.

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import recall_score
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
n = 6000
grupo = rng.choice(["A", "B"], n, p=[0.9, 0.1])            # grupo B é minoria
X = rng.normal(size=(n, 5))
X[:, 0] += np.where(grupo == "B", 1.5, 0.0)                # distribuição diferente
sinal = np.where(grupo == "A", X[:, 1], -X[:, 1])          # a relação com o alvo inverte em B
y = (sinal + rng.normal(0, 0.3, n) > 0).astype(int)

dados = pd.DataFrame(X, columns=[f"x{i}" for i in range(5)]).assign(grupo=grupo, y=y)
tr, te = train_test_split(dados, test_size=0.3, random_state=0, stratify=dados["grupo"])
feats = [f"x{i}" for i in range(5)]
m = RandomForestClassifier(n_estimators=200, random_state=0).fit(tr[feats], tr["y"])
te = te.assign(prev=m.predict(te[feats]))

print(f"Acurácia geral: {(te['prev'] == te['y']).mean():.3f}")
for g, parte in te.groupby("grupo"):
    print(f"  grupo {g}: n = {len(parte):4d} | acurácia = {(parte['prev'] == parte['y']).mean():.3f} "
          f"| revocação = {recall_score(parte['y'], parte['prev']):.3f}")
```

A média geral parece boa, mas o grupo minoritário sofre. Em produção, defina as **fatias críticas** com o negócio e acompanhe-as continuamente (Capítulos 9 e 10).

6.10 Métricas de regressão

Quando a saída é um número (preço, demanda), as métricas mais comuns são:

Métrica	O que mede	Observação
MAE	Erro absoluto médio	Fácil de explicar, na unidade original
RMSE	Raiz do erro quadrático médio	Pune mais os erros grandes
R^2	Fração da variação explicada	Pode enganar fora do treino
MAPE	Erro percentual médio	Instável com valores perto de zero

Escolha a métrica que **espelha o custo real** do erro.

6.11 Offline versus online

Todas as métricas acima são **offline**: calculadas em dados históricos, antes da implantação. No mundo real, o que importa é o efeito **online**: o modelo realmente melhora o negócio? Bernardi e colegas (2019), analisando 150 modelos de ML do Booking.com, relatam que a melhoria de métricas de modelo nem sempre se traduz em ganho de negócio, e que experimentos online são essenciais para saber a verdade. Voltaremos a isso no Capítulo 9.

6.12 Resumo do capítulo

MEMORIZE **O que levar deste capítulo**

- Escolha a métrica pelo **custo do erro**. Com classes raras, acurácia engana: use **precisão, revocação, F1 e AP (precisão média)**.
- O **limiar** de decisão é uma escolha de negócio: ajuste-o na validação, com custos explícitos.
- **Calibre** as probabilidades quando elas entram na decisão; use o escore de Brier.
- **Validação cruzada** reporta média e desvio; em séries temporais, **respeite a ordem**.
- Compare modelos com **intervalos de confiança** (*bootstrap*). Se o intervalo inclui zero, não há vencedor claro.
- A **validação aninhada** evita o viés de seleção.
- **Busca aleatória** costuma superar a grade; métodos bayesianos e Hyperband poupam recursos.
- Avalie **por fatias**: médias escondem desigualdades.
- Métricas **offline** não bastam; confirme com experimentos **online**.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. A subespecificação (underspecification), descrita por D'Amour et al. (2022), significa que:

a) todos os modelos treinados do mesmo jeito são idênticos.
b) a validação cruzada elimina qualquer incerteza.
c) modelos com o mesmo desempenho no teste podem se comportar de modo muito diferente em produção.
d) o conjunto de teste sempre prevê o comportamento real.

Questão 2. A precisão (precision) responde à pergunta:

a) das vezes que o modelo disse "positivo", quantas estavam certas?
b) dos positivos reais, quantos foram encontrados?
c) quantos exemplos há no conjunto?
d) qual a fração de negativos?

Questão 3. A revocação (recall) responde à pergunta:

a) qual a probabilidade média prevista?
b) quantos negativos foram descartados?
c) das previsões positivas, quantas estavam certas?
d) dos casos positivos reais, quantos o modelo encontrou?

Questão 4. O F1 é:

a) a média harmônica entre precisão e revocação.
b) o produto de VP por VN.
c) a soma de FP e FN.
d) a média aritmética entre acurácia e AUC.

Questão 5. Com 5% de fraudes, um modelo que sempre responde "não é fraude" tem:

a) F1 igual a 1.
b) acurácia de 5% e revocação 100%.
c) acurácia e revocação de 50%.
d) acurácia de cerca de 95% e revocação zero.

Questão 6. Segundo Saito e Rehmsmeier (2015), com classes muito desbalanceadas:

a) ambas são idênticas.
b) a curva ROC é sempre melhor.
c) a curva de precisão-revocação costuma ser mais informativa que a ROC.
d) nenhuma curva pode ser usada.

Questão 7. Qual é a AP de um classificador aleatório em um conjunto com 5% de positivos?

a) Exatamente 1,00.
b) Aproximadamente 0,50.
c) Aproximadamente 0,95.
d) Aproximadamente 0,05.

Questão 8. O limiar de decisão padrão de 0,5:

a) raramente é o ótimo quando os custos dos erros são diferentes.
b) nunca deve ser alterado.
c) é sempre o ótimo.
d) deve ser escolhido no conjunto de teste.

Questão 9. Se o falso negativo custa muito mais que o falso positivo, o limiar ótimo tende a ser:

a) maior que 0,5.
b) igual a 1.
c) exatamente 0,5.
d) menor que 0,5.

Questão 10. Em qual conjunto o limiar deve ser escolhido?

a) No conjunto de produção, sem rótulos.
b) Teste.
c) Em qualquer um, depois de ver os resultados.
d) Validação.

Questão 11. Um modelo é bem calibrado quando:

a) todas as probabilidades são 0 ou 1.
b) sua acurácia é a maior possível.
c) tem o menor número de parâmetros.
d) entre os casos com probabilidade prevista de 80%, cerca de 80% são realmente positivos.

Questão 12. O escore de Brier mede:

a) o número de falsos positivos.
b) a quantidade de dados.
c) a latência do modelo.
d) o erro quadrático médio das probabilidades previstas.

Questão 13. Guo et al. (2017) mostraram que redes neurais modernas tendem a ser:

a) superconfiantes (mal calibradas).
b) imunes à mudança de dados.
c) subconfiantes.
d) sempre bem calibradas.

Questão 14. Na validação cruzada de k dobras, cada dobra é:

a) usada uma vez como teste e $k-1$ vezes como treino.
b) usada somente como treino.
c) descartada.
d) usada apenas como validação final.

Questão 15. Por que reportar o desvio padrão da validação cruzada?

a) Para mostrar a incerteza; diferenças pequenas diante da dispersão merecem desconfiança e uma comparação pareada.
b) Porque é obrigatório por lei.
c) Para aumentar a média.
d) Para reduzir o número de dobras.

Questão 16. Em séries temporais, a validação cruzada deve:

a) usar o futuro para treinar.
b) embaralhar todas as linhas.
c) respeitar a ordem do tempo, treinando no passado e testando no futuro.
d) ignorar a data.

Questão 17. O bootstrap consiste em:

a) ajustar hiperparâmetros por grade.
b) remover exemplos atípicos.
c) sortear exemplos com reposição e recalcular a métrica muitas vezes.
d) dividir os dados em k partes.

Questão 18. Se o intervalo de confiança da diferença entre dois modelos inclui zero:

a) os modelos são idênticos.
b) o primeiro é certamente melhor.
c) não há evidência clara de que um seja melhor.
d) o segundo é certamente melhor.

Questão 19. O viés de seleção na busca de hiperparâmetros acontece porque:

a) as classes são equilibradas.
b) a busca nunca encontra bons modelos.
c) o teste é grande demais.
d) a melhor pontuação entre muitas configurações tende a ser otimista.

Questão 20. A validação cruzada aninhada tem:

a) um laço para treino e outro para tradução.
b) apenas um laço.
c) dois laços que escolhem os mesmos dados.
d) um laço interno para escolher hiperparâmetros e um externo para avaliar o procedimento.

Questão 21. Hiperparâmetros são:

a) as previsões do modelo.
b) rótulos dos dados.
c) valores aprendidos pelos dados, como os coeficientes.
d) escolhas feitas antes do treino, como o número de árvores.

Questão 22. Segundo Bergstra e Bengio (2012), a busca aleatória de hiperparâmetros:

a) é sempre pior.
b) exige o conjunto de teste.
c) costuma ser mais eficiente que a busca em grade.
d) só funciona com poucos hiperparâmetros.

Questão 23. O Hyperband (Li et al., 2018) economiza recursos porque:

a) interrompe cedo as tentativas ruins.
b) usa só o conjunto de teste.
c) testa todas as combinações até o fim.
d) dispensa validação.

Questão 24. A otimização bayesiana de hiperparâmetros:

a) testa toda a grade.
b) não usa nenhuma informação.
c) escolhe sempre ao acaso.
d) usa resultados anteriores para escolher a próxima tentativa.

Questão 25. Avaliar por fatias (slices) é importante porque:

a) elimina a necessidade de teste.
b) torna o modelo mais simples.
c) a média geral pode esconder grupos com desempenho muito pior.
d) reduz o tempo de treino.

Questão 26. A métrica RMSE, em regressão:

a) pune mais os erros grandes.
b) é igual à acurácia.
c) só serve para classificação.
d) ignora erros grandes.

Questão 27. A métrica MAPE pode ser instável quando:

a) há poucos atributos.
b) os valores reais estão perto de zero.
c) os valores são grandes.
d) o modelo é linear.

Questão 28. Bernardi et al. (2019), com 150 modelos do Booking.com, relatam que:

a) o negócio não depende de modelos.
b) toda melhoria de métrica gera ganho de negócio.
c) experimentos online são dispensáveis.
d) melhorar métricas do modelo nem sempre se traduz em ganho de negócio.

Questão 29. Qual destes cuidados evita um limiar "ajustado à prova"?

a) Escolher o limiar na validação e medir o custo final no teste.
b) Escolher o limiar sem dados.
c) Escolher o limiar no teste e reportar o mesmo teste.
d) Usar sempre 0,5.

Questão 30. Em CalibratedClassifierCV com method="isotonic":

a) ajusta uma função monotônica que corrige as probabilidades previstas.
b) remove atributos redundantes.
c) escolhe os hiperparâmetros do modelo.
d) treina uma rede neural nova.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com um exemplo, por que a acurácia pode enganar em problemas de classes raras e quais métricas usar em seu lugar.
2. Diferencie precisão e revocação com um exemplo do cotidiano.
3. Por que a curva de precisão-revocação pode ser mais informativa que a ROC em classes muito desbalanceadas?
4. Descreva um procedimento para escolher o limiar de decisão com base em custos.
5. O que é calibração e por que ela importa quando a probabilidade entra na decisão?
6. Explique a validação cruzada de k dobras e as razões para reportar média e dispersão.
7. Por que embaralhar os dados é incorreto na validação de séries temporais?
8. Descreva como construir um intervalo de confiança por bootstrap para a diferença de acurácia entre dois modelos.
9. O que é o viés de seleção na busca de hiperparâmetros e como a validação aninhada o corrige?
10. Compare busca em grade, busca aleatória e otimização bayesiana.
11. Por que avaliar por fatias é parte do ML Test Score e como fazê-lo?
12. Explique o que D'Amour et al. (2022) chamam de subespecificação e uma consequência prática.
13. Qual a diferença entre métricas offline e online, e por que ambas são necessárias?
14. Dê dois exemplos de métricas de regressão e uma limitação de cada.
15. Por que o conjunto de teste não deve ser usado para escolher hiperparâmetros?
16. Quais passos um relatório de avaliação rigorosa deve ter?
17. Por que o desvio treino–teste em desempenho pode indicar sobreajuste? Como investigar?
18. O que são classes desbalanceadas e quais técnicas ajudam a lidar com elas?
19. Explique por que as probabilidades de uma floresta aleatória costumam precisar de calibração.
20. Como você explicaria a um gestor a diferença entre "o modelo é melhor" e "a diferença é estatisticamente clara"?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Métricas para classes raras

NÍVEL: BÁSICO

Gere um problema com 4% de positivos (`make_classification`, 10.000 exemplos, `weights=[0.96]`, `class_sep=0.9`, `random_state=1`). Treine uma floresta aleatória e calcule, no teste: acurácia, precisão, revocação, F1, AUC-ROC e AP. Compare com o modelo "sempre negativo". Explique por que a AP é mais informativa que a AUC-ROC neste caso.

Exercício 2 — Escolhendo o limiar pelo custo

NÍVEL: INTERMEDIÁRIO

No problema do exercício anterior, suponha que um falso negativo custe R$ 400 e um falso positivo custe R$ 8. Divida os dados em treino, validação e teste (50/25/25). Escolha o limiar que minimiza o custo na **validação** e aplique-o no **teste**. Compare o custo do teste com o limiar 0,5 e com o limiar escolhido.

Exercício 3 — Calibração antes e depois

NÍVEL: INTERMEDIÁRIO

Treine uma floresta aleatória e uma floresta calibrada (`CalibratedClassifierCV`, método `isotonic`, `cv=5`) em um problema binário simulado (`make_classification`, 8.000 exemplos, `flip_y=0.05`, `random_state=1`). Calcule o escore de Brier e o erro médio de calibração (diferença absoluta média entre fração observada e probabilidade média por faixa, com 8 faixas por quantil).

Exercício 4 — O viés da busca de hiperparâmetros

NÍVEL: AVANÇADO

Em um problema pequeno e ruidoso (`make_classification`, 80 exemplos, 60 atributos, 3 informativos, `flip_y=0.2`, `class_sep=0.4`), repita 5 vezes (sementes 0 a 4) a comparação entre a melhor pontuação da `GridSearchCV` de um SVC e a estimativa por validação cruzada aninhada. Mostre a média das diferenças.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Default of Credit Card Clients*, *Statlog (German Credit Data)* e *Bank Marketing* para repetir o relatório com dados reais de risco e marketing.
> - **Kaggle:** *Give Me Some Credit* e *Credit Card Fraud Detection* (muito desbalanceado) para praticar AP, limiares e calibração.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Relatório de avaliação rigorosa de um modelo de risco

Contexto. Um banco quer um modelo de risco de inadimplência. Você receberá dados simulados e deve produzir uma avaliação rigorosa, que seja confiável para a diretoria.

Dados simulados. `make_classification` com 12.000 exemplos, 15 atributos (6 informativos), 6% de positivos, `class_sep=0.8`, `random_state=5`. Crie também uma variável de "região" (A: 70%, B: 30%) e faça o atributo 0 ser deslocado em +0,8 para a região B, para criar uma fatia mais difícil.

Tarefas.

1. Divida em treino, validação e teste (60/20/20), estratificando por alvo.
2. Compare regressão logística (com padronização) e floresta aleatória por validação cruzada repetida (5 dobras × 3 repetições) em AP. Reporte média e desvio.
3. Escolha o melhor, calibre-o (isotônica) e reporte o Brier antes e depois, na validação.
4. Escolha o limiar na validação com custos: falso negativo R$ 1.000 e falso positivo R$ 30.
5. No teste, reporte AP, revocação, precisão e custo total, com intervalo de confiança de 95% (*bootstrap*) para a AP.
6. Reporte as mesmas métricas por região. Há desigualdade?
7. Escreva uma recomendação de 4 linhas.

Entregáveis. Tabela de comparação, escolha do limiar, métricas finais com IC, métricas por região e recomendação.

A solução de referência está no Apêndice E.

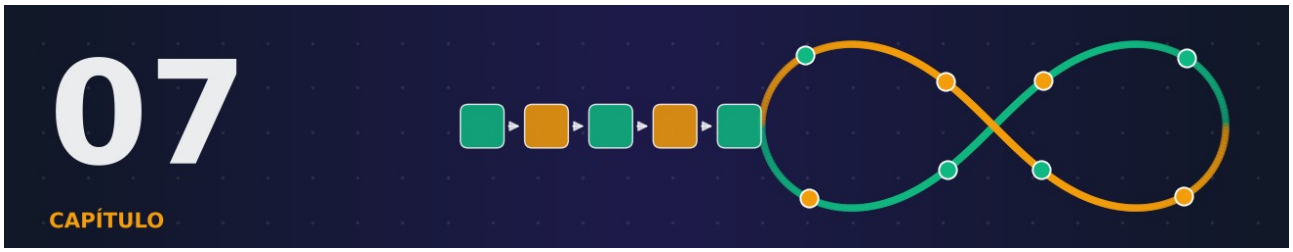
PARTE III

Parte III — Entrega e Operação

Como empacotar e servir modelos, testar e automatizar a entrega (CI/CD) e vigiar o modelo em produção: monitoramento, drift e retreinamento.

Nesta parte:

- Cap. 7 — Empacotamento e Serviço de Modelos
- Cap. 8 — Testes e CI/CD para Machine Learning
- Cap. 9 — Monitoramento, Deriva de Dados e Retreinamento



Capítulo 7 — Empacotamento e Serviço de Modelos

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Escolher entre previsão em lote, em tempo real, em fluxo contínuo e embarcada.
- Definir um contrato de entrada e saída com validação automática.
- Construir e testar uma API de previsão com FastAPI e Pydantic.
- Medir latência com percentis (p50, p95, p99) e interpretá-los.
- Explicar o que são contêineres e ler um Dockerfile.
- Implementar uma previsão em lote idempotente.
- Aplicar boas práticas de segurança no serviço de modelos.

7.1 O modelo precisa sair do laboratório

Até aqui, o modelo vive em um notebook ou em um arquivo. Para gerar valor, alguém precisa **usá-lo**: um aplicativo, um sistema de cobrança, um analista com uma planilha. A etapa de **serviço** (*serving*) é a ponte entre o modelo e quem precisa das previsões.

Pense em um restaurante. A cozinha (treino) pode ser perfeita, mas se não houver **garçom, cardápio e balcão** (serviço), nenhum cliente é atendido. E o cardápio é um **contrato**: o cliente só pode pedir o que está nele.

7.2 Quatro formas de entregar previsões

A escolha da forma de entrega depende de **quando** a previsão é necessária e **quantas** são pedidas.

Padrão	Como funciona	Latência	Exemplo
Lote (*batch*)	Roda em horários marcados sobre muitos registros e grava os resultados	Minutos a horas	Lista diária de clientes com risco de cancelar
Tempo real (*online*)	Um serviço responde a cada pedido individual	Milissegundos	Aprovar uma compra no cartão
Fluxo contínuo (*streaming*)	Processa eventos conforme chegam	Segundos	Detectar fraude em uma sequência de transações
Embarcado (*edge*)	O modelo roda no próprio dispositivo	Imediata, sem rede	Reconhecimento de voz no celular



Figura 7.1 — Quatro padrões de serviço. Começar por lote costuma ser mais simples, barato e seguro; use tempo real quando a decisão realmente não pode esperar.

> **Dica**
>
> Pergunte: "alguém precisa da resposta **agora**, ou amanhã de manhã serve?". Previsões em lote são muito mais fáceis de operar, e muitas decisões de negócio não exigem tempo real (HUYEN, 2022).

7.3 O contrato de entrada e saída

Um serviço de previsão precisa deixar claro:

- **O que recebe:** campos, tipos, unidades, intervalos válidos.
- **O que devolve:** formato da resposta, versão do modelo, probabilidade ou classe.
- **Como falha:** códigos de erro e mensagens.

Em Python, a biblioteca **Pydantic** permite descrever esse contrato como uma classe. Os dados fora das regras são rejeitados **antes** de chegarem ao modelo.

```python
from typing import Literal
from pydantic import BaseModel, Field, ValidationError

class Cliente(BaseModel):
    idade: float = Field(ge=18, le=100)
    renda: float = Field(ge=0, le=1_000_000)
    tempo_cliente: float = Field(ge=0, le=600)
    plano: Literal["basico", "padrao", "premium"]
    cidade: Literal["Natal", "Recife", "Fortaleza"]

valido = Cliente(idade=33, renda=4500, tempo_cliente=8, plano="basico", cidade="Natal")
print("Entrada válida:", valido.model_dump())

for dados in [{"idade": -5, "renda": 4500, "tempo_cliente": 8, "plano": "basico", "cidade": "Natal"},
              {"idade": 30, "renda": 4500, "tempo_cliente": 8, "plano": "ultra", "cidade": "Natal"}]:
    try:
```

```
        Cliente(**dados)
    except ValidationError as e:
        erro = e.errors()[0]
        print("Rejeitada:", erro["loc"][0], "->", erro["msg"])
```

Esse contrato é a **primeira linha de defesa** contra dados ruins em produção, e também a documentação viva da API.

7.4 Uma API de previsão com FastAPI

O **FastAPI** é uma biblioteca moderna para criar serviços web em Python. Uma **API** (*Application Programming Interface*) é uma "porta de entrada" padronizada: outros programas enviam um pedido (por exemplo, os dados de um cliente) e recebem uma resposta (a previsão). A comunicação costuma usar o protocolo **HTTP** e o formato **JSON** (texto estruturado em chaves e valores).

Estes são os endereços (*endpoints*) que criaremos:

- `GET /saude`: responde se o serviço está vivo e qual modelo carregou.
- `POST /previsao`: recebe os dados de um cliente e devolve a probabilidade de cancelamento.

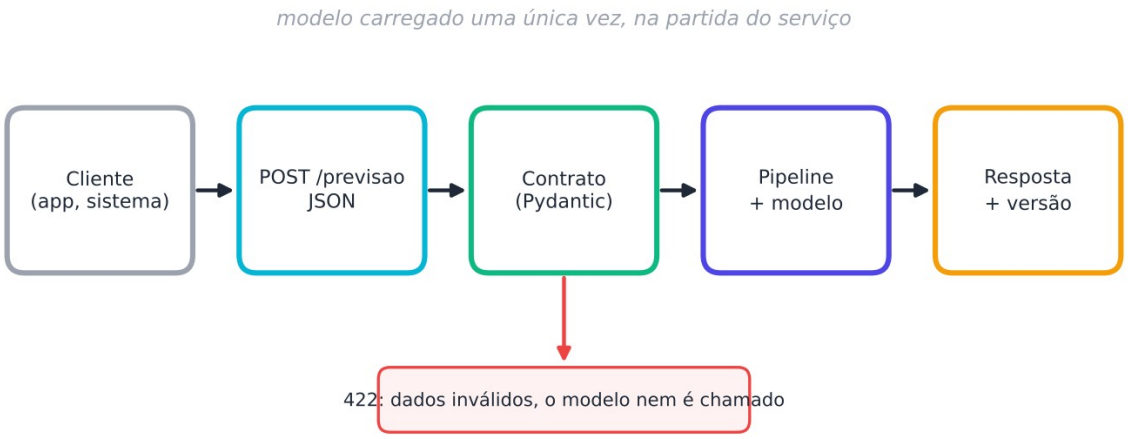


Figura 7.2 — Caminho de um pedido: o cliente envia JSON, o contrato valida, o pipeline prevê, e a resposta volta com a versão do modelo.

Primeiro treinamos e salvamos um pipeline (o mesmo do Capítulo 4, com dados simulados de clientes):

```python
import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

rng = np.random.default_rng(0)
n = 3000
df = pd.DataFrame({
    "idade": rng.integers(18, 75, n).astype(float),
    "renda": rng.lognormal(8, 0.5, n),
```

```
    "tempo_cliente": rng.integers(1, 120, n).astype(float),
    "plano": rng.choice(["basico", "padrao", "premium"], n, p=[0.5, 0.35, 0.15]),
    "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n),
})
risco = 0.03 * (50 - df["idade"]) - 0.02 * df["tempo_cliente"] + (df["plano"] == "basico") * 0.8
y = (risco + rng.normal(0, 0.8, n) > 0.3).astype(int)

num, cat = ["idade", "renda", "tempo_cliente"], ["plano", "cidade"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("esc", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
pipe = Pipeline([("prep", prep),
                 ("modelo", RandomForestClassifier(n_estimators=100, random_state=0))]).fit(df[num +
cat], y)
joblib.dump({"pipeline": pipe, "versao": "1.0.0", "colunas": num + cat}, "modelo.joblib")
print("Modelo salvo.")
```

Agora o serviço. Em um projeto real, este código ficaria em `src/servir.py`:

```python
# continuação (arquivo src/servir.py; no arquivo real, importe também joblib e pandas)
from contextlib import asynccontextmanager
from typing import Literal
from fastapi import FastAPI
from pydantic import BaseModel, Field


class Cliente(BaseModel):
    idade: float = Field(ge=18, le=100)
    renda: float = Field(ge=0, le=1_000_000)
    tempo_cliente: float = Field(ge=0, le=600)
    plano: Literal["basico", "padrao", "premium"]
    cidade: Literal["Natal", "Recife", "Fortaleza"]


class Resposta(BaseModel):
    probabilidade_cancelar: float
    versao_modelo: str


def criar_app(caminho="modelo.joblib"):
    estado = {}

    @asynccontextmanager
    async def ciclo_de_vida(app):
        estado["art"] = joblib.load(caminho)    # carrega UMA vez, na partida do serviço
        yield
        estado.clear()

    app = FastAPI(title="API de previsão de cancelamento", lifespan=ciclo_de_vida)

    @app.get("/saude")
    def saude():
        return {"status": "ok", "versao_modelo": estado["art"]["versao"]}

    @app.post("/previsao", response_model=Resposta)
    def prever(c: Cliente):
        art = estado["art"]
```

```
        linha = pd.DataFrame([c.model_dump()])[art["colunas"]]
        p = float(art["pipeline"].predict_proba(linha)[0, 1])
        return Resposta(probabilidade_cancelar=round(p, 4), versao_modelo=art["versao"])

    return app
```

Pontos importantes:

- O modelo é carregado **uma única vez**, na partida. Carregar a cada pedido seria lentíssimo.
- A resposta traz a **versão do modelo**, o que facilita a rastreabilidade.
- A API recebe a **linha crua**, e o pipeline cuida do pré-processamento, evitando o desvio treino–serviço.

Para executar de verdade, você usaria o servidor `uvicorn` (`uvicorn servir:app --port 8000`) e abriria `http://127.0.0.1:8000/docs`, onde o FastAPI gera uma página de documentação interativa.

7.5 Testando a API sem subir servidor

O FastAPI traz o `TestClient`, que simula pedidos HTTP **dentro do próprio programa**. Assim, a API é testada automaticamente a cada mudança (Capítulo 8).

```python
# continuação (arquivo tests/test_api.py)
from fastapi.testclient import TestClient

ok = {"idade": 33, "renda": 4500, "tempo_cliente": 8, "plano": "basico", "cidade": "Natal"}

with TestClient(criar_app()) as cliente:
    r = cliente.get("/saude")
    print("GET /saude        ->", r.status_code, r.json())

    r = cliente.post("/previsao", json=ok)
    print("POST /previsao    ->", r.status_code, r.json())

    r = cliente.post("/previsao", json=dict(ok, idade=-5))
    print("idade negativa    ->", r.status_code, r.json()["detail"][0]["msg"])

    r = cliente.post("/previsao", json=dict(ok, plano="ultra"))
    print("plano desconhecido->", r.status_code, r.json()["detail"][0]["type"])

    r = cliente.post("/previsao", json={"idade": 30})
    print("campos faltando   ->", r.status_code, f"({len(r.json()['detail'])} erros)")
```

O código **422** significa "entidade não processável": o pedido chegou, mas os dados violam o contrato. Veja que o modelo nem chegou a ser chamado. Esses testes ficam guardados e rodam sempre.

> **Atenção**
>
> A API mostrada é didática. Em produção, acrescente **autenticação**, **limites de taxa**, **registro (log) estruturado** e **tratamento de exceções**, além de um serviço rodando com vários processos (*workers*) para atender pedidos em paralelo.

7.5.1 Atualizando o modelo de um serviço em execução

No Capítulo 5 vimos que mover o apelido `champion` muda a **referência no registro**, não o objeto que já está na memória do serviço. Como a nossa API carrega o modelo **uma vez**, na partida, ela continuaria usando a versão antiga. Há duas saídas comuns:

- **Reimplantar** o serviço (ou reiniciá-lo), para que ele carregue de novo o modelo apontado pelo apelido.
- **Recarregar a quente**: o serviço consulta periodicamente a versão apontada e, se mudou, **carrega o modelo novo antes** e só então troca a referência em uso (assim, nenhum pedido encontra um modelo pela metade).

Em ambos os casos, o serviço deve **informar a versão que realmente carregou** (como já fazem `/saude` e as respostas da API), e o monitoramento deve comparar essa versão com a apontada no registro. Com **vários processos ou réplicas**, cada um recarrega em um momento: durante a transição, **versões diferentes respondem ao mesmo tempo**. Planeje isso (por exemplo, recargas escalonadas ou um canário) e lembre-se de que **caches** de previsões e **dependências** do modelo (como versões de bibliotecas ou de atributos) também precisam ser compatíveis com a versão nova.

A simulação abaixo (sem MLflow, só com dicionários) mostra o fenômeno com três réplicas que verificam o registro em momentos diferentes:

```python
registro = {"champion": "v1"}                          # simula o registro: apelido -> versão
carregar = lambda versao: {"versao": versao}           # simula joblib.load / mlflow.load_model

class Replica:
    def __init__(self, nome):
        self.nome, self.modelo = nome, carregar(registro["champion"])      # carrega na partida

    def verificar_registro(self):
        apontada = registro["champion"]
        if apontada != self.modelo["versao"]:
            novo = carregar(apontada)                  # carrega ANTES de trocar a referência
            self.modelo = novo
            return True
        return False

replicas = [Replica("A"), Replica("B"), Replica("C")]
registro["champion"] = "v2"                            # o apelido é movido
print("Logo após mover o apelido:", {r.nome: r.modelo["versao"] for r in replicas})

for instante, quem in enumerate([["A"], ["B"], ["C"]], start=1):    # cada réplica verifica em um momento
    for r in replicas:
        if r.nome in quem:
            r.verificar_registro()
    print(f"Após a verificação {instante}:", {r.nome: r.modelo["versao"] for r in replicas})
```

Observe que, até a última verificação, o serviço responde com **duas versões ao mesmo tempo**. Registrar a versão em cada resposta (e em cada *log*) permite acompanhar a transição e, se preciso, **reverter**: mover o apelido de volta e repetir a recarga.

7.6 Latência: não olhe só a média

Latência é o tempo entre o pedido e a resposta. Dizer "a latência média é 20 ms" esconde os piores casos. Imagine um restaurante em que a média de espera é 10 minutos, mas 1 em cada 100 clientes espera 1 hora. Esses clientes existem.

Por isso usamos **percentis**:

- **p50** (mediana): metade dos pedidos é mais rápida que isso.
- **p95:** 95% dos pedidos são mais rápidos que isso.
- **p99:** 99% dos pedidos são mais rápidos que isso (a "cauda").

A literatura de confiabilidade de sistemas (BEYER et al., 2016) recomenda acompanhar a distribuição, e não apenas a média. Combinados com o negócio, definem-se **objetivos de nível de serviço** (SLOs), como "p95 abaixo de 100 ms".

```python
import time
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
modelo = RandomForestClassifier(n_estimators=200, random_state=0).fit(X, y)

tempos = []
for i in range(300):
    linha = X[i % len(X)].reshape(1, -1)
    t0 = time.perf_counter()
    modelo.predict_proba(linha)
    tempos.append((time.perf_counter() - t0) * 1000)       # em milissegundos

p50, p95, p99 = np.percentile(tempos, [50, 95, 99])
print(f"p50 = {p50:.1f} ms | p95 = {p95:.1f} ms | p99 = {p99:.1f} ms | média = {np.mean(tempos):.1f}
ms")

# vazão: uma previsão por vez versus um lote de 300
t0 = time.perf_counter(); [modelo.predict_proba(X[i:i+1]) for i in range(300)]; um_a_um =
time.perf_counter() - t0
t0 = time.perf_counter(); modelo.predict_proba(X[:300]); em_lote = time.perf_counter() - t0
print(f"300 previsões uma a uma: {um_a_um*1000:.0f} ms | em um lote só: {em_lote*1000:.0f} ms")
```

Cuidado: este é o tempo isolado de inferência, não a latência do serviço. A latência que o usuário percebe inclui também a rede, a validação do contrato, a montagem da tabela de entrada, a serialização da resposta, as filas e a competição por recursos. Por isso, além de medir o modelo sozinho, faça **testes de carga** da API completa (com ferramentas como Locust, k6 ou wrk, por exemplo), com muitos pedidos simultâneos, e meça também a **disponibilidade** e a taxa de erro sob carga. Os números do projeto do final do capítulo (que usa o cliente de teste) já mostram a diferença entre o modelo e a API completa.

Os números exatos dependem do seu computador, mas o padrão se repete: **processar em lote é muito mais eficiente que um registro por vez**. Por isso, serviços de alto volume agrupam pedidos (*micro-batching*).

7.7 Contêineres: levando o ambiente junto

Lembra do "funciona na minha máquina"? Para o modelo rodar na máquina do servidor, é preciso reproduzir **o mesmo Python, as mesmas bibliotecas e o mesmo sistema**. Os **contêineres** resolvem isso.

Um **contêiner** é uma "marmita lacrada": ele leva dentro **o programa e tudo o que ele precisa para rodar**. Ao contrário de uma máquina virtual completa, compartilha o núcleo do sistema operacional do hospedeiro, sendo leve e rápido para iniciar (MERKEL, 2014). O **Docker** é a ferramenta mais conhecida para isso.

Três palavras-chave:

- **Imagem:** o "molde" da marmita (somente leitura).
- **Contêiner:** uma marmita em uso, criada a partir da imagem.
- **Dockerfile:** a receita que descreve como montar a imagem.

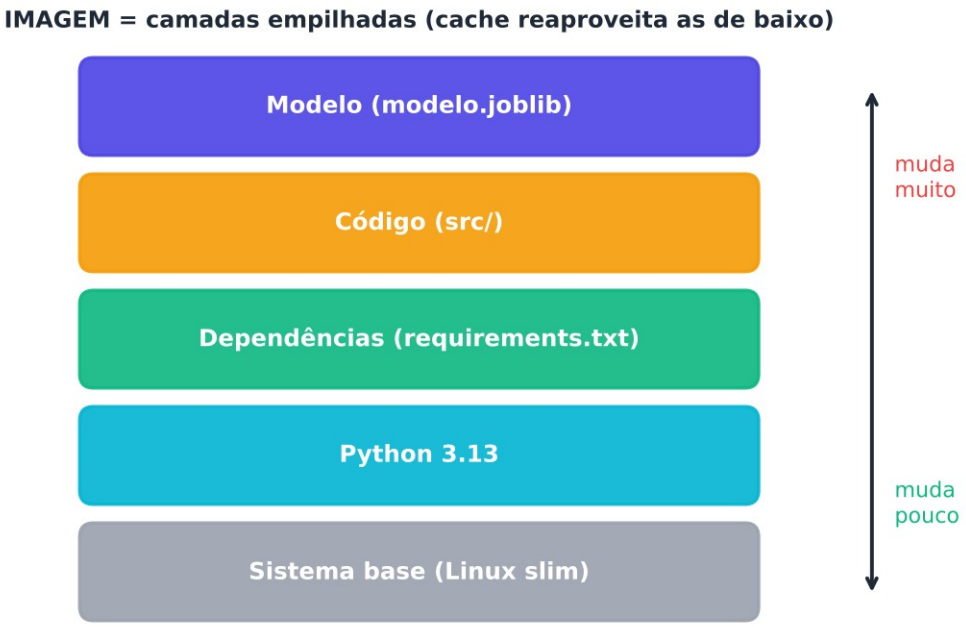


Figura 7.3 — Camadas de uma imagem de contêiner: sistema base, Python, dependências, código e modelo. Camadas que mudam pouco ficam embaixo, para aproveitar o cache.

Este é um `Dockerfile` típico para a nossa API:

```dockerfile
FROM python:3.13-slim

WORKDIR /app

# 1) dependências primeiro: esta camada só muda se o requirements.txt mudar
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 2) código e modelo depois
COPY src/ ./src/
COPY modelo.joblib .

# 3) boa prática: não rodar como administrador
RUN useradd --create-home appuser
```

```
USER appuser

EXPOSE 8000
CMD ["uvicorn", "src.servir:app", "--host", "0.0.0.0", "--port", "8000"]
```

E os comandos para construir e executar:

```bash
docker build -t api-churn:1.0.0 .            # constrói a imagem com um nome e versão
docker run -p 8000:8000 api-churn:1.0.0      # executa, expondo a porta 8000
curl http://127.0.0.1:8000/saude             # testa
```

> **Atenção**
>
> O `Dockerfile` e os comandos acima são **ilustrativos** e não foram executados na elaboração deste livro (o ambiente de escrita não tinha o Docker). Em `src/servir.py` real, você precisaria também criar o objeto `app = criar_app()`. Teste no seu ambiente.

7.7.1 Boas práticas com imagens

- Use uma **imagem base pequena** (`-slim`) e **versões fixas**, nunca `latest`.
- **Ordene as camadas** da que muda menos (dependências) para a que muda mais (código).
- **Não coloque segredos** (senhas, chaves) na imagem.
- Use um **usuário comum**, não o administrador.
- **Etiquete** a imagem com a versão do código e do modelo (rastreabilidade).
- Verifique **vulnerabilidades** das dependências.

7.8 Previsão em lote

Na previsão em lote, o programa lê muitos registros, prevê tudo e grava os resultados. O ponto crítico é a **idempotência**: se o trabalho rodar duas vezes por engano (ou for reexecutado após uma falha), o resultado deve ser **o mesmo**, sem duplicar registros.

Uma forma simples: gravar o resultado em um arquivo cujo nome depende da **data de referência** e **sobrescrevê-lo**, em vez de acrescentar linhas.

```python
import json
import os
import hashlib
import joblib
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# modelo de exemplo salvo
dados = load_wine(as_frame=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(dados.data, dados.target)
```

```
joblib.dump({"pipeline": pipe, "versao": "2.1.0"}, "modelo_lote.joblib")


def prever_em_lote(df_entrada, data_ref, pasta="saidas", tamanho_bloco=50):
    art = joblib.load("modelo_lote.joblib")
    os.makedirs(pasta, exist_ok=True)
    blocos = []
    for ini in range(0, len(df_entrada), tamanho_bloco):                # processa por blocos
        parte = df_entrada.iloc[ini:ini + tamanho_bloco]
        blocos.append(pd.DataFrame({
            "indice": parte.index,
            "classe_prevista": art["pipeline"].predict(parte),
            "versao_modelo": art["versao"],
            "data_ref": data_ref}))
    saida = pd.concat(blocos, ignore_index=True)
    caminho = os.path.join(pasta, f"previsoes_{data_ref}.csv")
    saida.to_csv(caminho, index=False)                                  # sobrescreve: idempotente
    return caminho


c1 = prever_em_lote(dados.data, "2026-10-17")
h1 = hashlib.sha256(open(c1, "rb").read()).hexdigest()[:12]
c2 = prever_em_lote(dados.data, "2026-10-17")                           # reexecução acidental
h2 = hashlib.sha256(open(c2, "rb").read()).hexdigest()[:12]
print("Arquivo:", c1, "| linhas:", len(pd.read_csv(c1)))
print("Reexecução gerou o mesmo resultado?", h1 == h2)
```

> **Dica**
>
> Em previsão em lote, **registre** o que foi previsto (versão do modelo, data de referência, número de linhas, hash da entrada). Será essencial para auditar e para monitorar o modelo (Capítulo 9).

7.9 Estratégias de entrega: um panorama

Ao trocar o modelo em produção, é preciso evitar surpresas. Alguns padrões (aprofundados no Capítulo 9):

Estratégia	Como funciona	Risco
Substituição direta	Troca o modelo de uma vez	Alto
Azul-verde (*blue-green*)	Dois ambientes idênticos; o tráfego passa do antigo (azul) para o novo (verde)	Baixo; reversão rápida
Canário (*canary*)	O modelo novo recebe uma pequena fração do tráfego	Baixo; limita o dano
Sombra (*shadow*)	O modelo novo recebe cópia do tráfego, mas suas respostas **não** são usadas	Muito baixo; compara em silêncio

7.10 Segurança no serviço de modelos

Um modelo exposto na rede é uma porta de entrada. Cuidados mínimos:

- **Valide tudo** na entrada (tipos, intervalos, tamanho máximo do pedido).
- **Autentique** e **autorize** quem pode pedir previsões.

- **Limite a taxa** de pedidos para evitar abuso.
- **Guarde segredos** em um cofre, nunca no código ou na imagem.
- **Atualize dependências** e verifique vulnerabilidades.
- **Não exponha mais do que o necessário**: devolver probabilidades detalhadas, por exemplo, pode facilitar ataques de inferência de associação (SHOKRI et al., 2017), em que um atacante tenta descobrir se uma pessoa estava no conjunto de treino.
- **Registre** os acessos para auditoria, respeitando a privacidade.

7.11 Resumo do capítulo

MEMORIZE **O que levar deste capítulo**

- Escolha o padrão de entrega pelo prazo da decisão: **lote**, **tempo real**, **fluxo contínuo** ou **embarcado**. Lote é o mais simples.
- Defina um **contrato** de entrada e saída (Pydantic) para rejeitar dados ruins cedo.
- Em **FastAPI**: carregue o modelo uma vez, devolva a **versão** na resposta e teste com `TestClient`.
- Meça **percentis de latência** (p50, p95, p99), e não só a média. Lotes são mais eficientes que pedidos únicos.
- **Contêineres** empacotam o ambiente: imagem base pequena, versões fixas, sem segredos, usuário comum.
- Em lote, garanta **idempotência** e registre a execução.
- **Segurança**: valide, autentique, limite a taxa, proteja segredos e atualize dependências.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. A etapa de serviço (serving) é:

a) a etapa de limpeza de dados.
b) a escolha dos hiperparâmetros.
c) a ponte entre o modelo e quem precisa das previsões.
d) a escrita do relatório final.

Questão 2. Qual padrão de entrega de previsões processa muitos registros em horários marcados?

a) Interativo por voz.
b) Embarcado (edge).
c) Lote (batch).
d) Tempo real (online).

Questão 3. Aprovar uma compra no cartão no momento da transação exige:

a) previsão manual.
b) apenas relatório mensal.
c) previsão em lote semanal.
d) previsão em tempo real, com latência de milissegundos.

Questão 4. Um modelo embarcado (edge) é:

a) executado somente em data centers.
b) executado em lote noturno.
c) executado no próprio dispositivo, sem depender da rede.
d) um modelo sem parâmetros.

Questão 5. A recomendação sobre o padrão de entrega é:

a) começar por lote sempre que a decisão puder esperar, por ser mais simples e barato.
b) usar sempre tempo real.
c) usar sempre embarcado.
d) escolher o padrão ao acaso.

Questão 6. Um contrato de entrada e saída de um serviço de previsão define:

a) campos, tipos, intervalos e formato da resposta.
b) apenas a cor da interface.
c) somente os hiperparâmetros.
d) quem pagou pelo servidor.

Questão 7. A biblioteca Pydantic é usada no capítulo para:

a) treinar redes neurais.
b) descrever e validar o contrato de dados por meio de classes.
c) criar gráficos.
d) comprimir arquivos.

Questão 8. O que é uma API?

a) Uma porta de entrada padronizada para que programas se comuniquem.
b) Um tipo de banco de dados.
c) Um formato de imagem.
d) Um algoritmo de ML.

Questão 9. O formato JSON é:

a) uma linguagem de programação.
b) um tipo de imagem.
c) texto estruturado em chaves e valores usado em APIs.
d) um algoritmo de compressão.

Questão 10. Por que carregar o modelo uma vez, na partida do serviço?

a) Por exigência do Docker.
b) Para economizar espaço em disco.
c) Carregá-lo a cada pedido seria muito lento.
d) Porque o modelo muda a cada pedido.

Questão 11. Por que a resposta da API deve incluir a versão do modelo?

a) Substitui os testes.
b) Reduz a latência.
c) Facilita a rastreabilidade de cada previsão.
d) Impede o uso de lote.

Questão 12. O código HTTP 422 significa:

a) o modelo está com sobreajuste.
b) o pedido chegou, mas os dados violam o contrato.
c) o servidor está desligado.
d) a autenticação foi aprovada.

Questão 13. O TestClient do FastAPI permite:

a) simular pedidos HTTP dentro do próprio programa, sem subir um servidor.
b) criar contêineres.
c) gerar dados sintéticos.
d) treinar o modelo.

Questão 14. Latência é:

a) o número de previsões por segundo.
b) o custo mensal do servidor.
c) o tamanho do modelo.
d) o tempo entre o pedido e a resposta.

Questão 15. Por que usar percentis (p95, p99) em vez de apenas a média da latência?

a) Percentis dispensam medições.
b) A média esconde os piores casos, que afetam clientes reais.
c) Percentis são sempre menores que a média.
d) A média é impossível de calcular.

Questão 16. O p95 de latência de 80 ms significa que:

a) a média é 95 ms.
b) 95% dos pedidos são mais rápidos que 80 ms.
c) todos os pedidos duram 80 ms.
d) 5% dos pedidos são mais rápidos que 80 ms.

Questão 17. O ganho de processar previsões em lote, em vez de uma a uma, vem de:

a) eliminar a validação.
b) amortizar custos fixos e usar operações vetorizadas.
c) usar um modelo menor.
d) reduzir os atributos.

Questão 18. Uma imagem de contêiner é:

a) um servidor físico.
b) um contêiner em execução.
c) o "molde" somente leitura a partir do qual os contêineres são criados.
d) um arquivo de dados.

Questão 19. O Dockerfile é:

a) o arquivo do modelo.
b) a receita que descreve como montar a imagem.
c) o arquivo de dados de treino.
d) o log de acesso.

Questão 20. Por que copiar o requirements.txt e instalar as dependências antes de copiar o código no Dockerfile?

a) Para reduzir o tamanho do modelo.
b) Para esconder o código.
c) Porque o Docker exige essa ordem.
d) A camada de dependências só muda quando o requirements.txt muda, aproveitando o cache.

Questão 21. Qual é uma boa prática na construção de imagens?

a) Fixar versões e não colocar segredos na imagem.
b) Usar a etiqueta latest em produção.
c) Rodar sempre como administrador.
d) Gravar senhas no Dockerfile.

Questão 22. Idempotência em previsão em lote significa:

a) cada execução gera um arquivo novo diferente.
b) os resultados são aleatórios.
c) rodar duas vezes produz o mesmo resultado, sem duplicar registros.
d) o job só pode rodar uma vez na vida.

Questão 23. Em implantação azul-verde (blue-green):

a) só 1% do tráfego vai para o novo.
b) o novo recebe cópia do tráfego, sem uso das respostas.
c) todo o tráfego vai para o modelo novo de uma só vez, sem volta.
d) dois ambientes idênticos coexistem e o tráfego passa de um para o outro.

Questão 24. Na implantação canário (canary):

a) o modelo antigo é apagado.
b) nenhum usuário vê o modelo novo.
c) o modelo novo atende uma pequena fração do tráfego.
d) o modelo novo substitui o antigo imediatamente.

Questão 25. No modo sombra (shadow):

a) o modelo antigo é desligado.
b) o modelo novo recebe cópia dos pedidos, mas suas respostas não são usadas.
c) o modelo novo roda apenas à noite.
d) o modelo novo responde a todos os usuários.

Questão 26. Um cuidado de segurança em serviços de modelos é:

a) desligar os logs.
b) gravar senhas no código.
c) validar entradas, autenticar usuários e limitar a taxa de pedidos.
d) expor todos os detalhes do modelo.

Questão 27. Ataques de inferência de associação (Shokri et al., 2017) tentam:

a) quebrar a criptografia do disco.
b) descobrir se uma pessoa estava no conjunto de treino.
c) alterar o código-fonte.
d) aumentar a latência do serviço.

Questão 28. Em uma API, por que receber a linha crua e usar o pipeline treinado?

a) Impede testes.
b) Aumenta a quantidade de código.
c) Obriga o uso de lote.
d) Elimina duplicação do pré-processamento e o desvio treino–serviço.

Questão 29. Qual o papel do endpoint GET /saude?

a) Apagar logs.
b) Receber dados de treino.
c) Treinar o modelo.
d) Informar que o serviço está vivo e qual modelo foi carregado.

Questão 30. O que a vazão (throughput) mede?

a) A quantidade de atributos.
b) A quantidade de previsões atendidas por unidade de tempo.
c) O tempo de uma única previsão.
d) O tamanho do modelo em disco.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Compare os quatro padrões de entrega de previsões (lote, tempo real, fluxo contínuo e embarcado) e dê um exemplo de cada.
2. Por que um contrato de entrada com validação é a primeira linha de defesa de uma API de ML?
3. Explique o papel do lifespan (ciclo de vida) no FastAPI para modelos de ML.
4. Descreva como testar uma API de previsão com TestClient e quais casos incluir.
5. Por que é preciso medir percentis de latência e não apenas a média?
6. Por que processar previsões em lote é mais eficiente por registro que fazer chamadas individuais?
7. Explique o que é um contêiner, em que difere de uma máquina virtual e qual problema ele ajuda a resolver.
8. Liste cinco boas práticas na construção de imagens de contêiner para modelos.
9. O que é idempotência e como garanti-la em uma previsão em lote?
10. Compare as estratégias de implantação substituição direta, azul-verde, canário e sombra quanto ao risco.
11. Cite cinco medidas de segurança para um serviço de modelos e o risco que cada uma reduz.
12. Explique o ataque de inferência de associação e uma forma de reduzi-lo.
13. Por que o desvio treino–serviço pode ocorrer mesmo com o mesmo modelo, e como evitá-lo na API?
14. O que deve ser registrado em uma execução de previsão em lote para auditoria?
15. Qual a importância do endpoint de metadados (/metadados) em uma API de modelo?
16. Descreva três itens que ainda faltariam para tornar a API didática do capítulo pronta para produção.
17. Por que limitar o tamanho do lote em um endpoint de previsão?
18. Como você explicaria a um gestor por que começar por previsões em lote?
19. Qual a relação entre contratos de dados na API e o monitoramento em produção?
20. Por que é recomendável versionar a imagem do contêiner com a versão do modelo e do código?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Um contrato gerado a partir dos dados de treino

NÍVEL: BÁSICO

Para o conjunto *Wine* (UCI), construa um modelo Pydantic dinamicamente (`pydantic.create_model`) com um campo `float` para cada atributo, limitado entre o mínimo e o máximo observados no treino, com folga de 25%. Teste com uma amostra válida e com duas inválidas (valor fora da faixa e campo faltando).

Exercício 2 — Uma API mínima com saúde e previsão

NÍVEL: INTERMEDIÁRIO

Treine uma regressão logística (com padronização) no conjunto *Iris* (UCI) e crie uma API FastAPI com `GET /saude` e `POST /previsao`, que recebe os quatro atributos e devolve a espécie e a probabilidade. Use o `TestClient` para testar: uma chamada válida, uma com comprimento de sépala negativo e uma sem campos.

Exercício 3 — Percentis de latência e o ganho do lote

NÍVEL: INTERMEDIÁRIO

Treine uma floresta aleatória (200 árvores) no conjunto *Digits* (UCI). Meça a latência de 300 previsões individuais e informe p50, p95, p99 e a média. Depois, meça o tempo para prever as mesmas 300 imagens de uma vez e calcule a economia por previsão.

Exercício 4 — Previsão em lote idempotente com manifesto

NÍVEL: INTERMEDIÁRIO

Escreva uma função que preveja, em blocos de 200 linhas, as classes dos 1.797 dígitos do *Digits* (UCI), grave um CSV nomeado com a data de referência e um arquivo `manifesto.json` com a versão do modelo, o número de linhas, o hash da entrada e o hash da saída. Execute duas vezes e prove a idempotência comparando os hashes e conferindo que o número de linhas não dobrou.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Bank Marketing* e *Adult (Census Income)* (para criar APIs com atributos mistos) e *Wine Quality* (regressão ou classificação).
> - **Kaggle:** *Telco Customer Churn* e *Titanic* para treinar um serviço de previsão e escrever seus testes.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

API completa com tempo real, lote, limites e testes

Contexto. Você deve entregar a equipe de operações uma API de previsão de cancelamento de clientes (5 atributos) com endpoints para tempo real e para lote, além de testes.

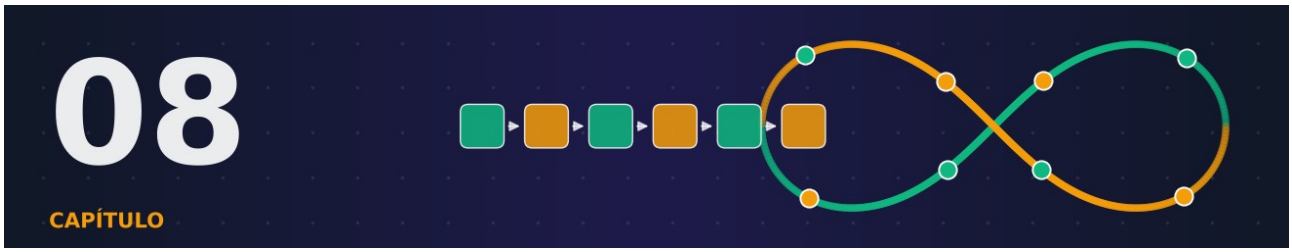
Dados simulados. Use clientes com `idade`, `renda`, `tempo_cliente`, `plano` e `cidade`, como no Capítulo 7 (regra de risco conhecida, `random_state=0`).

Tarefas.

1. Treine e salve um pipeline (`ColumnTransformer` + floresta) com metadados (versão do modelo, colunas, data).
2. Crie uma API com: `GET /saude` (status e versão), `POST /previsao` (um cliente), `POST /previsao_lote` (lista de até 100 clientes) e `GET /metadados`.
3. Defina o contrato com Pydantic (faixas e categorias permitidas). Rejeite lotes vazios ou com mais de 100 itens.
4. Carregue o modelo uma única vez na partida (`lifespan`).
5. Escreva testes com `TestClient` para: caminho feliz, campos inválidos, lote vazio, lote acima do limite e consistência (a previsão individual deve ser igual à do mesmo cliente dentro do lote).
6. Meça a latência (p50 e p95) de 200 chamadas individuais e do lote de 100 clientes por requisição.
7. Liste, em 5 itens, o que ainda faltaria para produção (segurança, observabilidade etc.).

Entregáveis. Código da API e dos testes, resultados dos testes, medidas de latência e a lista de pendências.

A solução de referência está no Apêndice E.



Capítulo 8 — Testes e CI/CD para Machine Learning

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que testar sistemas de ML exige testar código, dados e modelo.
- Escrever testes unitários com pytest para funções e transformadores.
- Criar testes de comportamento do modelo: fumaça, invariância, direção e desempenho mínimo.
- Implementar um portão de qualidade que bloqueia modelos ruins automaticamente.
- Ler e escrever um fluxo de integração contínua (CI) para um projeto de ML.
- Distinguir CI, entrega contínua, implantação contínua e treinamento contínuo (CT).
- Organizar ambientes (desenvolvimento, homologação, produção) e aprovações.

8.1 Testar para dormir tranquilo

Antes de um avião decolar, a tripulação segue uma lista de verificação. Ninguém confia só na memória. No software, essa lista são os **testes automáticos**: pequenos programas que verificam, a cada mudança, se o sistema ainda funciona.

Em ML há um agravante: o sistema pode estar **sem nenhum erro de código** e ainda assim **errado**, porque o dado mudou ou o modelo piorou. Por isso Breck e colegas (2017) organizam os testes em quatro frentes (revise o ML Test Score no Capítulo 2):

1. **Testes de código:** funções, transformadores, API.
2. **Testes de dados:** esquema, intervalos, distribuições.
3. **Testes de modelo:** desempenho, comportamento, fatias.
4. **Testes de infraestrutura:** o pipeline roda de ponta a ponta; a implantação e a reversão funcionam.

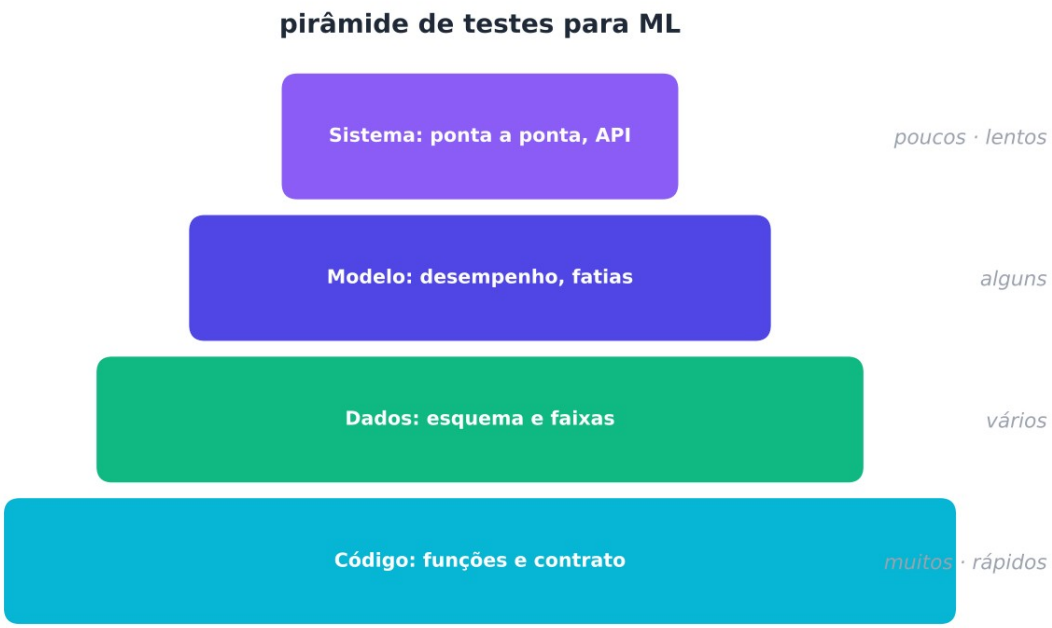


Figura 8.1 — Pirâmide de testes para ML: muitos testes rápidos de código na base; menos testes, mais lentos e mais caros, de modelo e de sistema no topo.

> MEMORIZE **Em uma frase**
>
> Em ML, testa-se o **código**, os **dados** e o **modelo**, e automatiza-se tudo para rodar a cada mudança.

8.2 Testes unitários com pytest

Um **teste unitário** verifica uma pequena peça isolada. O **pytest** é a biblioteca mais usada em Python. Regras simples:

- Cada teste é uma função cujo nome começa com `test_`.
- Dentro dela, usamos `assert` para afirmar o que deve ser verdade.
- Se qualquer `assert` falhar, o teste falha e o problema é apontado.

Vamos testar uma função de limpeza e o transformador do Capítulo 4. Por simplicidade, colocamos "código" e "testes" no mesmo script. Em um projeto real, ficam em `src/` e `tests/`.

PYTHON

```python
import numpy as np
import pandas as pd
import pytest
from sklearn.base import BaseEstimator, TransformerMixin


# ---------- src/preproc.py ----------
def limpar_idade(idade):
    """Converte para número e rejeita valores impossíveis."""
    v = float(idade)
    if not 0 <= v <= 120:
        raise ValueError(f"idade fora do intervalo: {v}")
    return v
```

```
class RazaoRendaIdade(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X["renda_por_idade"] = X["renda"] / X["idade"].clip(lower=1)
        return X
```

Agora os testes:

```
PYTHON
# continuação (arquivo tests/test_preproc.py)
def test_idade_valida():
    assert limpar_idade("35") == 35.0


@pytest.mark.parametrize("ruim", [-1, 121, 500])       # roda o mesmo teste com 3 valores
def test_idade_invalida(ruim):
    with pytest.raises(ValueError):
        limpar_idade(ruim)


def test_razao_sem_divisao_por_zero():
    df = pd.DataFrame({"idade": [0, 20], "renda": [1000.0, 2000.0]})
    saida = RazaoRendaIdade().fit_transform(df)
    assert np.isfinite(saida["renda_por_idade"]).all()
    assert saida["renda_por_idade"].iloc[1] == pytest.approx(100.0)


def test_nao_altera_a_entrada():
    df = pd.DataFrame({"idade": [30], "renda": [3000.0]})
    RazaoRendaIdade().fit_transform(df)
    assert list(df.columns) == ["idade", "renda"]       # a tabela original ficou intacta


if __name__ == "__main__":
    import sys
    sys.exit(pytest.main([__file__, "-q", "-p", "no:cacheprovider"]))
```

No terminal de um projeto real, basta digitar `pytest`. Os testes rodam em milissegundos, então podem rodar a cada alteração do código.

> **Dica**
>
> Teste **comportamentos**, não detalhes internos. E teste os **casos de borda**: valores zero, vazios, negativos, categorias novas e tabelas de uma linha só.

8.3 Testes de dados

No Capítulo 3, construímos um validador de dados. Agora o transformamos em testes. Há dois momentos:

- **Em desenvolvimento:** testes sobre uma amostra fixa de dados versionada, para garantir que o código de preparação funciona.

- **Em operação:** validação de cada lote real que chega, com o pipeline parando se reprovar.

Também vale testar **propriedades estatísticas**. Por exemplo: "a proporção de positivos está entre 2% e 15%" ou "nenhuma coluna tem mais de 5% de ausentes". Mudanças bruscas nessas proporções costumam indicar problema na origem dos dados.

8.4 Testes de modelo

Um modelo não tem "resposta exata". Por isso os testes são **estatísticos** e **comportamentais**. Ribeiro e colegas (2020) propuseram o *CheckList*, uma metodologia de testes comportamentais para modelos de linguagem. Seus tipos de teste servem de inspiração para qualquer modelo:

Tipo	Pergunta	Exemplo
Funcionalidade mínima	O modelo acerta casos simples e óbvios?	Cliente novo, plano premium e longo tempo de casa: risco baixo
Invariância	A previsão se mantém se mudarmos algo irrelevante?	Trocar a cidade (sem efeito no problema) quase não altera o risco
Expectativa direcional	A previsão muda no sentido esperado?	Mais tempo de cliente não deve aumentar o risco de cancelar

Além deles, há os testes mais clássicos:

- **Fumaça** (*smoke test*): o pipeline treina e prevê sem quebrar, em dados minúsculos.
- **Desempenho mínimo:** a métrica no conjunto de validação está acima de um piso.
- **Fatias:** o desempenho em subgrupos críticos também está acima de um piso.
- **Não regressão:** o modelo novo não é pior que o campeão atual além de uma tolerância.
- **Determinismo:** treinar duas vezes com a mesma semente gera o mesmo resultado.

```python
import numpy as np
import pandas as pd
import pytest
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler


def gerar_dados(n=4000, semente=0):
    rng = np.random.default_rng(semente)
    df = pd.DataFrame({
        "idade": rng.integers(18, 75, n).astype(float),
        "tempo_cliente": rng.integers(1, 120, n).astype(float),
        "plano": rng.choice(["basico", "padrao", "premium"], n, p=[0.5, 0.35, 0.15]),
        "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n),
    })
    risco = 0.03 * (50 - df["idade"]) - 0.02 * df["tempo_cliente"] + (df["plano"] == "basico") * 0.8
    y = (risco + rng.normal(0, 0.8, n) > 0.3).astype(int)
    return df, y
```

```
def treinar(df, y, semente=0):
    prep = ColumnTransformer([("num", StandardScaler(), ["idade", "tempo_cliente"]),
                              ("cat", OneHotEncoder(handle_unknown="ignore"), ["plano", "cidade"])])
    return Pipeline([("prep", prep), ("modelo", LogisticRegression(max_iter=1000))]).fit(df, y)


@pytest.fixture(scope="module")
def contexto():
    df, y = gerar_dados()
    Xtr, Xte, ytr, yte = train_test_split(df, y, test_size=0.3, random_state=0, stratify=y)
    return treinar(Xtr, ytr), Xte, yte


def test_desempenho_minimo(contexto):
    modelo, Xte, yte = contexto
    assert f1_score(yte, modelo.predict(Xte)) >= 0.60       # piso combinado com o negócio


def test_invariancia_a_cidade(contexto):
    modelo, Xte, _ = contexto
    base = modelo.predict_proba(Xte)[:, 1]
    trocado = Xte.assign(cidade="Natal")                      # cidade não influencia o risco
    depois = modelo.predict_proba(trocado)[:, 1]
    assert np.abs(base - depois).mean() < 0.03


def test_direcao_tempo_de_cliente(contexto):
    modelo, Xte, _ = contexto
    antes = modelo.predict_proba(Xte)[:, 1]
    depois = modelo.predict_proba(Xte.assign(tempo_cliente=Xte["tempo_cliente"] + 24))[:, 1]
    assert (depois <= antes + 1e-9).mean() >= 0.95            # risco não deve subir


def test_determinismo():
    df, y = gerar_dados(500)
    p1 = treinar(df, y).predict_proba(df)
    p2 = treinar(df, y).predict_proba(df)
    assert np.array_equal(p1, p2)


def test_funcionalidade_minima(contexto):
    modelo, _, _ = contexto
    fiel = pd.DataFrame([{"idade": 60, "tempo_cliente": 110, "plano": "premium", "cidade": "Natal"}])
    novo = pd.DataFrame([{"idade": 20, "tempo_cliente": 1, "plano": "basico", "cidade": "Natal"}])
    assert modelo.predict_proba(fiel)[0, 1] < modelo.predict_proba(novo)[0, 1]


if __name__ == "__main__":
    import sys
    sys.exit(pytest.main([__file__, "-q", "-p", "no:cacheprovider"]))
```

Observe os limites escolhidos (0,60 de F1, 0,03 de diferença média, 95% dos casos). **São decisões de projeto**, combinadas com o negócio. Se forem apertados demais, os testes falham sem razão (testes "frágeis"); se forem frouxos demais, não protegem nada.

> **Atenção: testes com aleatoriedade**
>
> Teste que falha "às vezes" é pior que não ter teste: as pessoas passam a ignorá-lo. Fixe as **sementes**, use **tolerâncias** razoáveis e, se necessário, repita o cálculo com várias sementes e exija desempenho médio acima do limite.

8.5 O portão de qualidade

Os testes de modelo viram um **portão de qualidade** (*quality gate*): um passo do pipeline que **decide se o modelo pode seguir adiante**. Se reprovar, o fluxo para.

O ideal é que o portão seja um programa simples, com **código de saída** (*exit code*): 0 significa "tudo bem" e qualquer outro valor significa "falha". Ferramentas de CI usam esse código para decidir se continuam.

> **Atenção: um portão ingênuo aprova métricas inválidas**
>
> Se o portão apenas testa `metrica < limite`, uma métrica `NaN` (*not a number*, resultado de uma avaliação que falhou) **passa**, porque qualquer comparação com `NaN` é falsa. Em um fluxo automatizado, isso poderia promover um modelo cuja avaliação nem chegou a funcionar. **Primeiro valide as métricas; só depois compare com os pisos.**

Antes de comparar com os pisos, o portão deve verificar:

- a **presença** de todas as métricas obrigatórias;
- o **tipo numérico** válido;
- a ausência de **`NaN` e infinitos**;
- o **intervalo admissível** de cada métrica (F1 e revocação, por exemplo, entre 0 e 1);
- a **quantidade mínima de exemplos avaliados**.

```python
import json
import subprocess
import sys
import textwrap

# arquivo gate.py: valida as métricas e só então compara com os pisos
GATE = textwrap.dedent('''
    import json, math, sys
    PISOS = {"f1": 0.90, "revocacao": 0.85}        # pisos acordados com o negócio
    N_MIN = 200                                    # mínimo de exemplos avaliados

    def validar(m):
        erros = []
        for k in list(PISOS) + ["n_avaliados"]:
            if k not in m:
                erros.append(f"métrica ausente: {k}")
        for k, v in m.items():
            if isinstance(v, bool) or not isinstance(v, (int, float)):
                erros.append(f"{k}: valor não numérico ({v!r})")
            elif not math.isfinite(v):
                erros.append(f"{k}: valor não finito ({v})")
        for k in PISOS:
            v = m.get(k)
            if isinstance(v, (int, float)) and math.isfinite(v) and not 0.0 <= v <= 1.0:
                erros.append(f"{k}: fora do intervalo [0, 1] ({v})")
```

```
        n = m.get("n_avaliados")
        if isinstance(n, (int, float)) and math.isfinite(n) and n < N_MIN:
            erros.append(f"n_avaliados = {n} abaixo do mínimo ({N_MIN})")
        return erros

    metricas = json.load(open("metricas.json"))
    invalidas = validar(metricas)
    if invalidas:
        print("PORTÃO REPROVADO (métricas inválidas):", "; ".join(invalidas)); sys.exit(2)
    falhas = [f"{k}={metricas[k]:.3f} < {v}" for k, v in PISOS.items() if metricas[k] < v]
    if falhas:
        print("PORTÃO REPROVADO:", "; ".join(falhas)); sys.exit(1)
    print("PORTÃO APROVADO")
''')
open("gate.py", "w").write(GATE)

cenarios = [("modelo bom", {"f1": 0.95, "revocacao": 0.93, "n_avaliados": 500}),
            ("modelo ruim", {"f1": 0.95, "revocacao": 0.70, "n_avaliados": 500}),
            ("F1 = NaN", {"f1": float("nan"), "revocacao": 0.93, "n_avaliados": 500}),
            ("métrica ausente", {"f1": 0.95, "n_avaliados": 500}),
            ("revocação 1,7", {"f1": 0.95, "revocacao": 1.7, "n_avaliados": 500}),
            ("poucos exemplos", {"f1": 0.95, "revocacao": 0.93, "n_avaliados": 30})]
for nome, metricas in cenarios:
    json.dump(metricas, open("metricas.json", "w"))
    r = subprocess.run([sys.executable, "gate.py"], capture_output=True, text=True)
    print(f"{nome:16s} -> código {r.returncode} | {r.stdout.strip()}")
```

Os códigos de saída distinguem **reprovado por qualidade** (1) de **reprovado por métrica inválida** (2): o segundo caso indica um problema no processo de avaliação, e não no modelo.

8.6 Integração contínua (CI)

A **integração contínua** (*Continuous Integration*, CI) é a prática de **juntar as mudanças de todos com frequência** e verificar cada junção automaticamente (HUMBLE; FARLEY, 2010). Cada vez que alguém envia código ao repositório, um servidor:

1. baixa o código;
2. instala as dependências em um ambiente limpo;
3. roda verificações de estilo e de tipos;
4. roda os testes;
5. (em ML) treina um modelo pequeno e roda o portão de qualidade;
6. constrói o contêiner;
7. avisa o resultado: ✔ ou ✘.

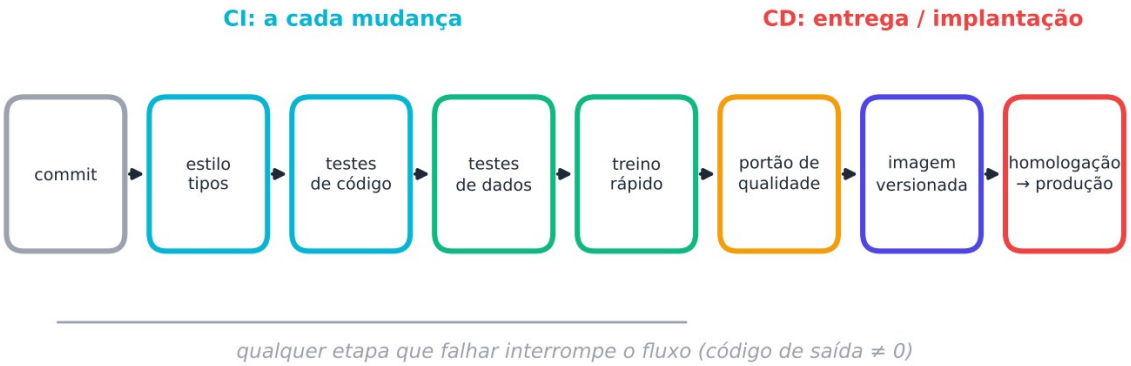


Figura 8.2 — Fluxo de CI/CD para ML: cada mudança passa por verificações, testes, treino, portão de qualidade e construção de imagem antes de chegar à produção.

8.6.1 Um exemplo de fluxo de CI

Existem várias ferramentas de CI (GitHub Actions, GitLab CI, Jenkins, entre outras). Veja um exemplo no formato do GitHub Actions. Ele fica no arquivo `.github/workflows/ci.yml`:

```yaml
name: ci-ml
on:
  pull_request:
  push:
    branches: [main]

jobs:
  testes:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
          cache: pip
      - name: Instalar dependências
        run: pip install -r requirements.txt
      - name: Estilo e tipos
        run: |
          ruff check src tests
          mypy src
      - name: Testes de código e de dados
        run: pytest tests/ -q
      - name: Treino rápido (amostra pequena)
        run: python src/treino.py --amostra 0.1 --saida modelo_ci.joblib
      - name: Portão de qualidade
        run: python src/gate.py metricas.json
      - name: Construir imagem
        run: docker build -t api-churn:${{ github.sha }} .
```

> **Atenção**
>
> O arquivo YAML acima é **ilustrativo**. Ele não foi executado na elaboração do livro e supõe que os arquivos `src/treino.py` e `src/gate.py` existam com esses argumentos. Versões de ações (`@v4`, `@v5`) mudam com o tempo; consulte a documentação do serviço de CI que você usar.

8.6.2 Boas práticas de CI

- **Rápido:** quanto mais rápido o retorno, mais as pessoas confiam e usam. Deixe os testes lentos para etapas posteriores.
- **Ambiente limpo e fixo:** versões fixas das bibliotecas; nada "da máquina de alguém".
- **Falhar cedo:** coloque as verificações baratas antes das caras.
- **Mesmo comando local e na CI:** quem roda `pytest` na própria máquina deve ver o mesmo que a CI.
- **Segredos protegidos:** senhas e chaves ficam no cofre da ferramenta, nunca no repositório.
- **Ramos curtos:** mudanças pequenas e frequentes são mais fáceis de revisar e de reverter.

8.7 Entrega e implantação contínuas (CD)

A sigla **CD** tem dois significados parecidos:

- **Entrega contínua** (*Continuous Delivery*): toda mudança aprovada **fica pronta** para ir à produção, mas alguém aperta o botão.
- **Implantação contínua** (*Continuous Deployment*): toda mudança aprovada **vai sozinha** para a produção.

Em ML, muitas equipes preferem a entrega contínua com **aprovação humana** no ponto de colocar um novo modelo em produção, sobretudo em áreas de alto impacto (crédito, saúde, justiça).

Os **ambientes** costumam ser:

Ambiente	Para que serve	Dados
Desenvolvimento	Experimentar à vontade	Amostras, dados simulados
Homologação (*staging*)	Ensaio geral, o mais parecido possível com produção	Cópia anonimizada ou recente
Produção	Atender usuários reais	Dados reais

Forsgren, Humble e Kim (2018) mostram que equipes que entregam com frequência **e** têm bons testes tendem a ter menos falhas, e não mais. Entregar pequeno e sempre é mais seguro que entregar grande e raramente.

8.8 Treinamento contínuo (CT)

O **treinamento contínuo** (*Continuous Training*, CT) é a parte exclusiva do ML: **retreinar** o modelo de forma automática. Os **gatilhos** mais comuns são:

Gatilho	Exemplo
Agendamento	Todo domingo à noite
Chegada de dados	Mil novos exemplos rotulados
Queda de desempenho	A métrica em produção cai abaixo de um limite
Mudança nos dados (*drift*)	O monitoramento detecta distribuição diferente

Gatilho	Exemplo
Mudança no código	Alguém alterou o pipeline

O pipeline de CT não pode simplesmente colocar o novo modelo no ar: ele **treina, avalia, passa pelo portão de qualidade, compara com o campeão e só então registra** o candidato (Capítulo 5). O Capítulo 9 detalha o retreinamento.

> **No seu dia a dia**
>
> Pense no **aplicativo do celular** que atualiza sozinho. O fabricante testa a nova versão, libera para poucos usuários, observa e só depois libera para todos. Atualizar modelos exige o mesmo cuidado.

8.9 Revisão de código e de modelos

Testes automáticos não substituem a **revisão humana**. Em um **pull request** (pedido de integração), outra pessoa lê as mudanças. Uma boa lista de verificação para projetos de ML:

- O que mudou: código, dados, hiperparâmetros ou métricas?
- Os testes novos cobrem a mudança?
- Os resultados foram **reproduzidos** pelo revisor ou pela CI?
- Há risco de **vazamento** ou de **desvio treino–serviço**?
- O desempenho por **fatias** foi verificado?
- A documentação e o **cartão do modelo** foram atualizados?

8.10 Resumo do capítulo

> MEMORIZE **O que levar deste capítulo**
>
> - Em ML, teste **código, dados e modelo**; e a infraestrutura de ponta a ponta.
> - **pytest**: funções `test_...`, `assert`, `parametrize` e `raises`.
> - Testes de modelo: **fumaça, funcionalidade mínima, invariância, direção, desempenho mínimo, fatias, não regressão e determinismo**.
> - O **portão de qualidade** é um programa com código de saída 0 (aprova) ou diferente de 0 (reprova); ele **valida as métricas** (presença, tipo, `NaN`, intervalo e número de exemplos) antes de compará-las com os pisos.
> - **CI**: a cada mudança, ambiente limpo, estilo, testes, treino rápido, portão e imagem.
> - **CD**: entrega contínua (botão humano) ou implantação contínua (automática); ambientes de desenvolvimento, homologação e produção.
> - **CT**: retreinar por agendamento, dados novos, queda de desempenho ou *drift*, sempre com portão e comparação com o campeão.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Por que testar sistemas de ML exige mais que testar código?

a) O sistema pode estar sem erro de código e ainda assim errado por causa de dados ou do modelo.
b) Porque ML não tem código.
c) Porque testes de código são proibidos em ML.
d) Porque modelos nunca mudam.

Questão 2. As quatro frentes de teste de Breck et al. (2017) incluem:

a) testes de dados, de modelo, de infraestrutura e de monitoramento.
b) apenas testes manuais.
c) apenas testes de interface gráfica.
d) testes de hardware apenas.

Questão 3. Na pirâmide de testes, a base é composta por:

a) testes feitos só uma vez por ano.
b) testes de uso por clientes finais.
c) poucos testes lentos de sistema.
d) muitos testes rápidos de código.

Questão 4. No pytest, uma função de teste é identificada por:

a) um nome que começa com test_.
b) a palavra reservada main.
c) um comentário especial no início.
d) um nome que termina com _final.

Questão 5. O que faz assert em um teste?

a) Reinicia o computador.
b) Imprime o resultado.
c) Cria um arquivo.
d) Afirma uma condição que deve ser verdadeira; se falhar, o teste falha.

Questão 6. Para que serve @pytest.mark.parametrize?

a) Salvar o modelo.
b) Rodar o mesmo teste com vários valores de entrada.
c) Pular os testes.
d) Medir a latência.

Questão 7. pytest.raises é usado para:

a) medir tempo de execução.
b) verificar que um código lança a exceção esperada.
c) criar exceções aleatórias.
d) ignorar erros.

Questão 8. Um caso de borda é:

a) um erro de digitação do desenvolvedor.
b) um valor típico e comum.
c) um valor extremo ou incomum, como zero, vazio, negativo ou lista de um elemento.
d) um teste que sempre passa.

Questão 9. Em teste de dados, é útil verificar propriedades estatísticas como:

a) a cor do gráfico.
b) a proporção de positivos estar dentro de uma faixa esperada.
c) a data do commit.
d) o nome da pasta.

Questão 10. O teste de fumaça (smoke test) verifica se:

a) o servidor aguenta milhões de usuários.
b) o pipeline treina e prevê sem quebrar, em dados minúsculos.
c) o código está bem formatado.
d) o modelo atinge o melhor desempenho possível.

Questão 11. O teste de invariância verifica que:

a) mudar algo irrelevante (como a cidade, sem efeito real) altera pouco a previsão.
b) o modelo é igual em todos os dados.
c) a previsão nunca muda.
d) o código não tem erros.

Questão 12. Um teste de expectativa direcional verifica que:

a) o treino leva menos tempo.
b) as previsões são sempre crescentes.
c) o modelo aponta para o norte.
d) a previsão muda no sentido esperado quando uma entrada muda.

Questão 13. Um teste de funcionalidade mínima verifica que:

a) o modelo acerta casos simples e óbvios.
b) o modelo roda sem bibliotecas.
c) o modelo usa o mínimo de atributos.
d) o modelo tem o menor tamanho possível.

Questão 14. O teste de não regressão compara:

a) o modelo novo com o campeão atual, aceitando uma tolerância.
b) o modelo com um modelo aleatório.
c) a latência com a vazão.
d) o treino com o teste.

Questão 15. Por que testes com aleatoriedade sem controle são perigosos?

a) Passam sempre.
b) Falham às vezes, e as pessoas passam a ignorá-los.
c) Aceleram o treino.
d) Impedem a CI.

Questão 16. Um portão de qualidade (quality gate) é:

a) uma ferramenta de visualização.
b) uma etapa que decide se o modelo pode seguir adiante.
c) uma forma de compactar o modelo.
d) um servidor de arquivos.

Questão 17. Em sistemas de CI, o código de saída 0 significa:

a) falha.
b) tempo esgotado.
c) aviso.
d) sucesso.

Questão 18. A sigla CI significa:

a) instrução cíclica.
b) inteligência computacional.
c) integração contínua.
d) implantação interna.

Questão 19. Qual das etapas costuma vir primeiro em um fluxo de CI?

a) Apagar o histórico.
b) Treinar o modelo completo com todos os dados.
c) Implantar em produção.
d) Baixar o código e instalar dependências em ambiente limpo.

Questão 20. A diferença entre entrega contínua e implantação contínua é que:

a) na entrega, alguém aperta o botão; na implantação, a mudança aprovada vai sozinha para a produção.
b) na entrega, nunca se vai à produção.
c) na implantação, nunca há testes.
d) são idênticas.

Questão 21. O ambiente de homologação (staging) serve para:

a) experimentos livres com dados simulados.
b) atender usuários reais.
c) guardar backups.
d) um ensaio geral o mais parecido possível com a produção.

Questão 22. Forsgren, Humble e Kim (2018) mostram que entregar com frequência e bons testes:

a) tende a produzir menos falhas, e não mais.
b) é incompatível com qualidade.
c) só funciona em empresas pequenas.
d) sempre aumenta as falhas.

Questão 23. Qual destes é um gatilho típico de treinamento contínuo (CT)?

a) Mudança de fuso horário do desenvolvedor.
b) Troca do papel de parede do servidor.
c) Queda do desempenho em produção abaixo de um limite.
d) Mudança no tamanho da tela.

Questão 24. Qual a vantagem de usar o mesmo comando para rodar os testes localmente e na CI?

a) Reduz o número de dependências.
b) Impede revisões de código.
c) Elimina a necessidade de testes.
d) Quem roda localmente vê o mesmo resultado que a CI verá.

Questão 25. Em CI, segredos como senhas devem ser:

a) enviados por e-mail.
b) guardados no cofre da ferramenta, nunca no repositório.
c) escritos no README.
d) incluídos no Dockerfile.

Questão 26. Em uma revisão de código de ML (pull request), deve-se verificar:

a) o que mudou (código, dados, hiperparâmetros), os testes e o risco de vazamento.
b) somente o tamanho do arquivo.
c) apenas o nome do autor.
d) apenas a formatação.

Questão 27. Teste "frágil" é aquele que:

a) nunca falha.
b) cobre todo o código.
c) é muito rápido.
d) falha à toa por limites apertados demais.

Questão 28. A ferramenta de estilo (lint) como ruff, em CI, serve para:

a) treinar modelos.
b) criar APIs.
c) compactar dados.
d) detectar problemas de estilo e erros comuns no código.

Questão 29. Por que testar a reprodutibilidade (determinismo) do treino na CI?

a) Para garantir que mudanças inesperadas nos resultados sejam detectadas.
b) Para aumentar a acurácia.
c) Para reduzir o uso de memória.
d) Para evitar o uso de sementes.

Questão 30. Qual a principal vantagem de um teste de desempenho mínimo no conjunto de validação?

a) Garante que o modelo é perfeito.
b) Torna o treino determinístico.
c) Dispensa o monitoramento.
d) Impede que um modelo abaixo de um piso combinado com o negócio avance.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que um sistema de ML pode estar sem erros de código e ainda assim estar errado, e quais testes ajudam a detectar isso.
2. Descreva a pirâmide de testes para ML e dê exemplos de testes em cada nível.
3. Quais são os tipos de teste de comportamento do modelo inspirados no CheckList? Dê um exemplo de cada.
4. O que é um portão de qualidade e como ele se relaciona com o código de saída em CI?
5. Como evitar testes de modelo que falham "às vezes"?
6. Descreva as etapas de um fluxo de CI para um projeto de ML.
7. Diferencie entrega contínua e implantação contínua e discuta quando preferir cada uma em ML.
8. Qual o papel dos ambientes de desenvolvimento, homologação e produção?
9. O que é treinamento contínuo (CT) e quais cuidados ele exige?
10. Por que revisar código e resultados é importante mesmo havendo testes automáticos?
11. Explique como parametrizar testes ajuda na cobertura de casos de borda.
12. Como testar que um transformador personalizado não altera a tabela de entrada?
13. Por que é importante "quebrar o modelo de propósito" para testar os testes?
14. O que são limites "apertados demais" e "frouxos demais" em testes de modelo?
15. Como a CI pode usar um treino em amostra pequena e ainda assim ser útil?
16. Quais verificações extras além de testes e portão uma CI real costuma ter?
17. Por que o desenvolvimento deve ser feito em ramos curtos e com mudanças pequenas?
18. Explique a relação entre testes de dados em desenvolvimento e validação de dados em produção.
19. Descreva como um defeito introduzido no modelo (por exemplo, regularização exagerada) é detectado pela CI.
20. Por que testes de modelo devem ser versionados junto do código?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Testes unitários para uma função de limpeza

NÍVEL: BÁSICO

Escreva a função `normalizar_cidade(texto)` (remove espaços das pontas, coloca em minúsculas, tira acentos e devolve `None` para vazio) e um conjunto de testes `pytest` com `parametrize` para pelo menos 6 casos, incluindo casos de borda. Execute os testes com `pytest.main` e mostre o resumo.

Exercício 2 — Testes de comportamento do modelo

NÍVEL: INTERMEDIÁRIO

Treine um modelo de cancelamento simulado (idade, tempo de cliente, plano) e escreva, com `pytest`, três testes: (1) F1 mínimo no teste; (2) **invariância**: trocar a coluna `cidade` (sem efeito real) altera pouco as probabilidades; (3) **direção**: aumentar o tempo de cliente não deve aumentar o risco em pelo menos 95% dos casos. Execute-os.

Exercício 3 — Portão de qualidade que valida as métricas

NÍVEL: INTERMEDIÁRIO

Escreva um script `gate.py` que leia `metricas.json` e `limites.json` e, **antes** de comparar com os pisos, valide as métricas: presença, tipo numérico, ausência de `NaN` e infinitos, intervalo [0, 1] e número mínimo de exemplos avaliados (`n_avaliados` ≥ 100). Use o código de saída 1 para reprovação por qualidade e 2 para métricas inválidas. Execute-o com `subprocess` em cinco cenários: aprovado, reprovado por uma métrica, reprovado por duas, com `NaN` e com métrica ausente. Mostre que, sem a validação, o caso do `NaN` seria aprovado.

Exercício 4 — Os testes realmente protegem? Quebrando o modelo de propósito

NÍVEL: AVANÇADO

Treine um modelo no conjunto *Breast Cancer* (UCI) e crie uma bateria de testes (F1 mínimo, revocação mínima da classe maligna e consistência de previsões). Depois, "quebre" o modelo de três maneiras: (a) treine com rótulos embaralhados, (b) esqueça de padronizar a entrada em produção (use o modelo sem o `StandardScaler`) e (c) treine com apenas 10 exemplos. Mostre quais testes detectam cada falha.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Wine Quality*, *Heart Disease* e *Breast Cancer Wisconsin* servem para criar suítes de testes de dados e de modelo.
> - **Kaggle:** *Titanic* e *House Prices*: pratique testes de esquema e de desempenho mínimo em dados reais com ausentes.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Suíte de testes e CI local de ponta a ponta

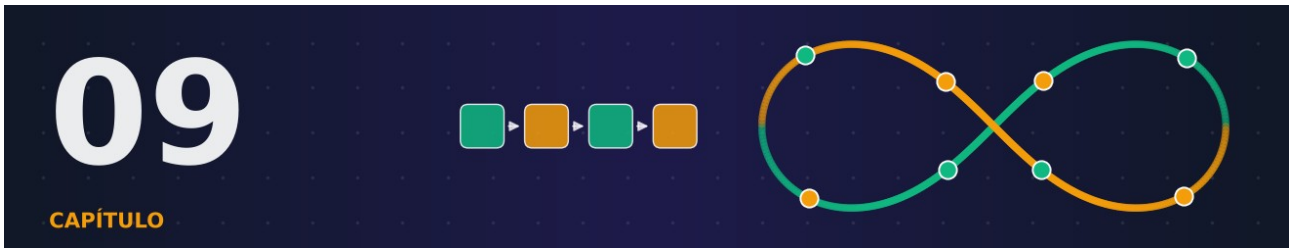
Contexto. Você vai montar, em uma pasta temporária, um mini-projeto com código, dados de teste e testes automáticos, mais um script que simula a CI.

Tarefas.

1. Crie `src/preproc.py` (função `limpar` que remove duplicados e valida faixas) e `src/modelo.py` (função `treinar` que devolve pipeline e métricas, com semente fixa), usando o conjunto *Wine* (UCI).
2. Crie `tests/test_preproc.py` (3 testes), `tests/test_dados.py` (esquema e faixas) e `tests/test_modelo.py` (F1 mínimo, determinismo e invariância a ruído mínimo).
3. Crie `ci.py`, que executa em sequência: verificação de sintaxe (`compileall`), `pytest` e o portão de qualidade; imprime o resultado de cada etapa e retorna código 1 se alguma falhar.
4. Rode `ci.py` no cenário normal (deve passar).
5. Introduza um defeito (por exemplo, remova a semente fixa ou mude um limite) e mostre que a CI falha na etapa correta.
6. Reflita: que outras etapas entrariam numa CI real (estilo, tipos, contêiner)?

Entregáveis. O projeto, a saída da CI no cenário normal e no cenário com defeito, e uma lista de etapas adicionais.

A solução de referência está no Apêndice E.



Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que modelos envelhecem e por que isso é uma falha silenciosa.
- Distinguir mudança de covariáveis, mudança de rótulos e deriva de conceito.
- Calcular KS, PSI e distância de Jensen-Shannon para detectar mudanças em atributos.
- Detectar mudanças multivariadas com um classificador de domínio.
- Monitorar o desempenho com rótulos atrasados e usar um detector de deriva (DDM).
- Comparar estratégias de retreinamento (estático, janela deslizante, incremental).
- Planejar implantação segura (sombra, canário, teste A/B) e registrar previsões.

9.1 O modelo envelhece sozinho

Um pão fresco é delicioso. Dois dias depois, está duro, e **ninguém tocou nele**. Modelos de ML são assim: o código continua igual, mas o **mundo muda**, e o modelo, que aprendeu o mundo de ontem, passa a errar.

Exemplos:

- Um modelo de crédito treinado antes de uma crise econômica.
- Um detector de fraude, enquanto os fraudadores mudam de tática.
- Um recomendador, quando surge uma moda nova.
- Um modelo de demanda, quando um concorrente abre uma loja ao lado.

O pior é que o sistema **continua respondendo**, sem erro, sem alarme. É a falha silenciosa que vimos no Capítulo 1. Gama e colegas (2014) e Lu e colegas (2019) fazem revisões abrangentes sobre como detectar e adaptar-se a essas mudanças, que a literatura chama de **deriva de conceito** (*concept drift*) e, de forma mais geral, **mudança de distribuição** (*dataset shift*).

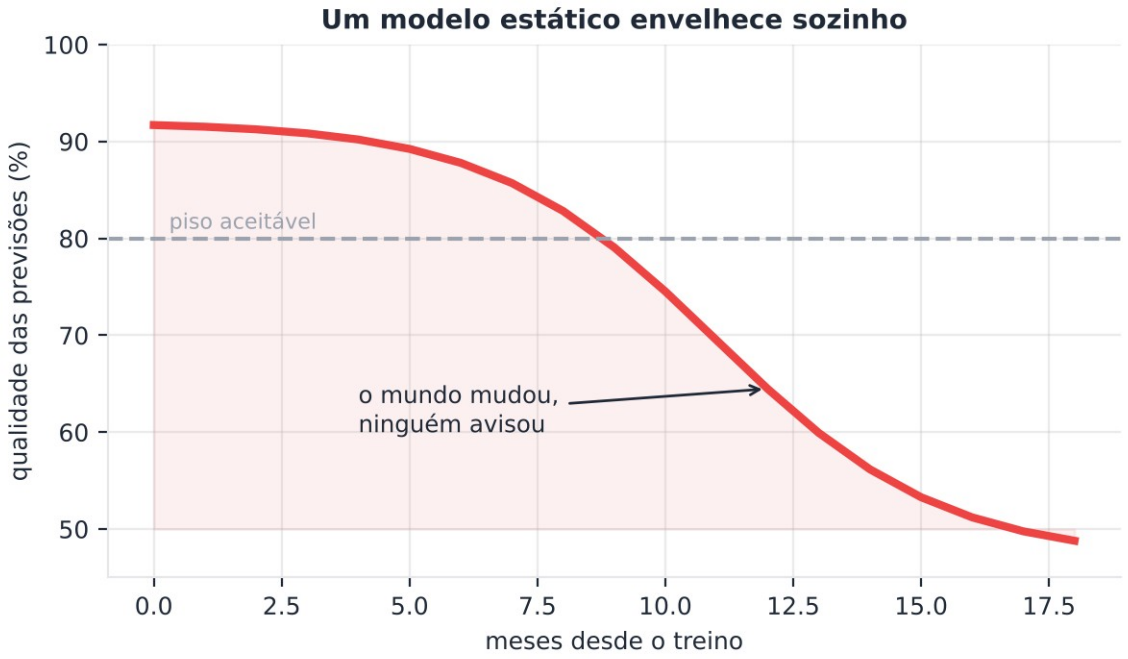


Figura 9.1 — O desempenho de um modelo estático cai com o tempo à medida que o mundo muda. O monitoramento serve para ver a queda antes do prejuízo.

9.2 O que monitorar

Um bom monitoramento observa o sistema em **camadas**. Quanto mais abaixo, mais rápido o sinal; quanto mais acima, mais perto do que importa.

Camada	O que acompanhar	Exemplos de sinais	Rótulo necessário?
Serviço	Saúde do sistema	Latência (p95), taxa de erro, disponibilidade, uso de CPU e memória	Não
Dados de entrada	Qualidade e distribuição das entradas	Ausentes, valores fora do intervalo, PSI, categorias novas	Não
Previsões	Distribuição das saídas	Probabilidade média, fração de positivos, confiança	Não
Desempenho	Acertos do modelo	Acurácia, F1, AP, por fatias	**Sim** (muitas vezes atrasado)
Negócio	Efeito real	Receita, cancelamentos evitados, reclamações	Sim

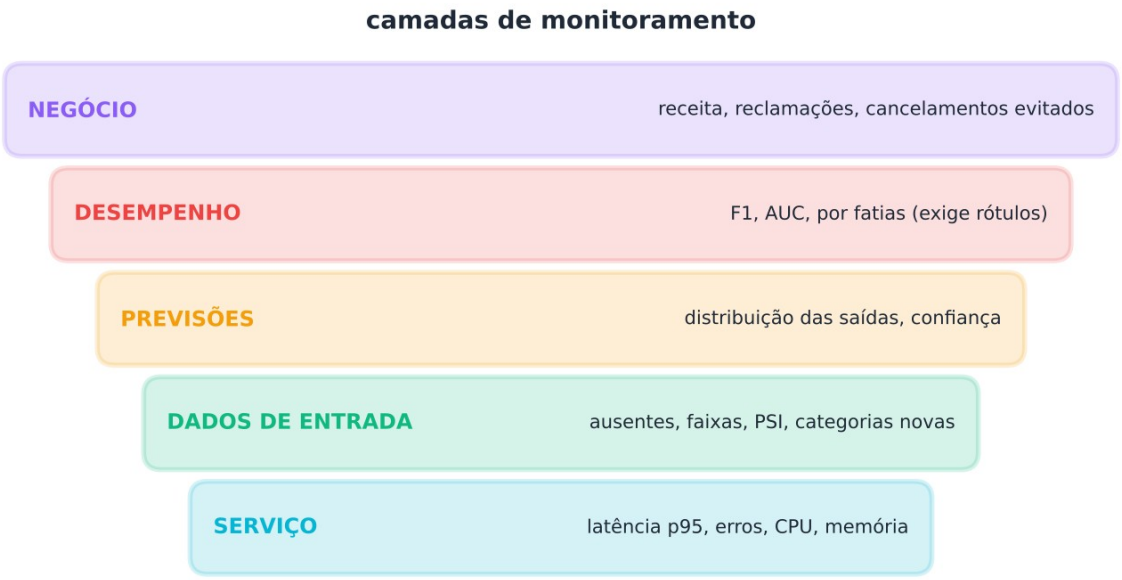


Figura 9.2 — Camadas de monitoramento. As camadas de baixo reagem rápido, mas dizem menos; as de cima dizem mais, mas demoram a chegar.

Breck e colegas (2017) incluem, entre os itens de monitoramento do ML Test Score, verificar as **invariantes dos dados** de entrada, garantir que os atributos de treino e de serviço tenham os mesmos valores, evitar modelos obsoletos e detectar regressões lentas na qualidade das previsões.

9.3 Os tipos de mudança

Seja **X** o conjunto de atributos de entrada e **y** o alvo (aquilo que queremos prever). A literatura distingue três situações (QUIÑONERO-CANDELA et al., 2009; MORENO-TORRES et al., 2012):

Tipo	O que muda	Em palavras	Exemplo
Mudança de covariáveis (*covariate shift*)	A distribuição de **X**	Chegam clientes de perfil diferente, mas a relação entre perfil e resposta é a mesma	Campanha atrai um público mais jovem
Mudança de rótulos (*label shift* ou *prior shift*)	A distribuição de **y**, com **P(X dado y) estável**	As classes mudam de proporção, mas, dentro de cada classe, os atributos se distribuem como antes	Uma doença se torna mais frequente, mas os sintomas de quem a tem continuam os mesmos
Deriva de conceito (*concept drift*)	A **relação** entre X e y	Mesmo perfil, resposta diferente	Hábitos mudam; o mesmo perfil passa a se comportar de outro modo

Atenção a uma condição essencial: o *label shift* **não** é qualquer mudança na frequência dos positivos. Ele supõe que **P(X | y) permanece estável**; só P(y) muda. Essa hipótese é explícita no trabalho de Lipton, Wang e Smola (2018). Se a frequência dos positivos muda porque a relação entre X e y mudou (isto é, P(y | X) mudou), o fenômeno é outro, mais próximo da deriva de conceito, e as correções do *label shift* não se aplicam.

Sugiyama, Krauledat e Müller (2007) estudaram como adaptar a validação à mudança de covariáveis, e Lipton, Wang e Smola (2018) propuseram uma forma de detectar e corrigir a mudança de rótulos usando as previsões do próprio modelo.

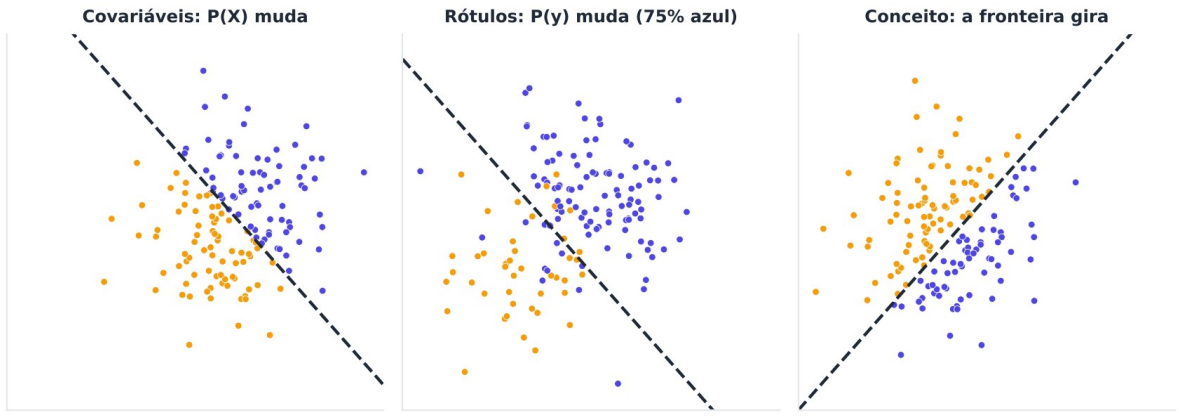


Figura 9.3 — Três tipos de mudança. Na mudança de covariáveis, os pontos se deslocam; na de rótulos, a mistura de classes muda e cada classe mantém sua forma; na deriva de conceito, a fronteira de decisão gira.

Também importa o **jeito** como a mudança acontece (GAMA et al., 2014):

- **Abrupta:** de um dia para o outro (uma lei nova).
- **Gradual:** o conceito antigo e o novo convivem por um tempo.
- **Incremental:** deslizamento lento e contínuo.
- **Recorrente:** volta periodicamente (sazonalidade, como Natal).

> **No seu dia a dia**
>
> Uma **sorveteria**: no verão (covariáveis) chegam muito mais clientes. Isso é esperado e recorrente. Já se uma nova tendência saudável faz o mesmo cliente deixar de pedir sorvete (conceito), é outra coisa. Diagnosticar qual mudança ocorreu determina a ação.

9.4 Detectando mudanças em um atributo

A ideia básica é comparar duas amostras: a de **referência** (o que o modelo viu no treino) e a **atual** (o que chega hoje). Três medidas muito usadas:

1. Teste de Kolmogorov-Smirnov (KS). Compara as distribuições acumuladas de duas amostras e devolve uma estatística (a maior distância entre elas) e um *p-valor* (MASSEY, 1951). Útil para atributos numéricos. Atenção: com amostras enormes, qualquer diferença minúscula vira "significativa"; olhe também o **tamanho** da diferença.

2. PSI (*Population Stability Index*). Divide o atributo em faixas (por exemplo, decis da referência) e compara a fração de cada faixa antes e agora. É popular em crédito. Uma **regra prática** muito usada: abaixo de 0,10 é estável; entre 0,10 e 0,25, mudança moderada; acima de 0,25, mudança importante. São limiares convencionais, não leis: ajuste ao seu contexto.

$$\mathrm{PSI} = \Sigma\,(q_i - p_i) \times \ln(q_i / p_i)$$

Em palavras: some, faixa por faixa, a diferença de proporções multiplicada pelo logaritmo da razão entre elas. Quanto mais diferentes as distribuições, maior o PSI.

3. Distância de Jensen-Shannon. Baseada na divergência de Kullback e Leibler (1951), mas simétrica e limitada (LIN, 1991). A função `scipy.spatial.distance.jensenshannon` devolve a **distância** (a raiz quadrada da divergência de Jensen-Shannon). Com `base=2`, ela vai de 0 (iguais) a 1 (suportes totalmente separados); sem informar a base, o padrão é o logaritmo natural e o máximo é menor que 1. Por isso o código abaixo usa `base=2`.

```python
import numpy as np
from scipy.spatial.distance import jensenshannon
from scipy.stats import ks_2samp

def psi(ref, atual, faixas=10):
    cortes = np.quantile(ref, np.linspace(0, 1, faixas + 1))
    cortes[0], cortes[-1] = -np.inf, np.inf
    p = np.clip(np.histogram(ref, cortes)[0] / len(ref), 1e-6, None)
    q = np.clip(np.histogram(atual, cortes)[0] / len(atual), 1e-6, None)
    return float(np.sum((q - p) * np.log(q / p)))

def jensen_shannon(ref, atual, faixas=20):
    cortes = np.histogram_bin_edges(np.concatenate([ref, atual]), faixas)
    p = np.histogram(ref, cortes)[0] + 1e-9
    q = np.histogram(atual, cortes)[0] + 1e-9
    return float(jensenshannon(p / p.sum(), q / q.sum(), base=2))

rng = np.random.default_rng(0)
ref = rng.normal(50, 10, 5000)                          # distribuição vista no treino
cenarios = {
    "sem mudança": rng.normal(50, 10, 2000),
    "média +3": rng.normal(53, 10, 2000),
    "média +10": rng.normal(60, 10, 2000),
    "desvio padrão ×2": rng.normal(50, 20, 2000),
}
for nome, atual in cenarios.items():
    ks = ks_2samp(ref, atual)
    print(f"{nome:14s} KS={ks.statistic:.3f} (p={ks.pvalue:.2g}) | PSI={psi(ref, atual):.3f} | "
          f"JS={jensen_shannon(ref, atual):.3f}")
```

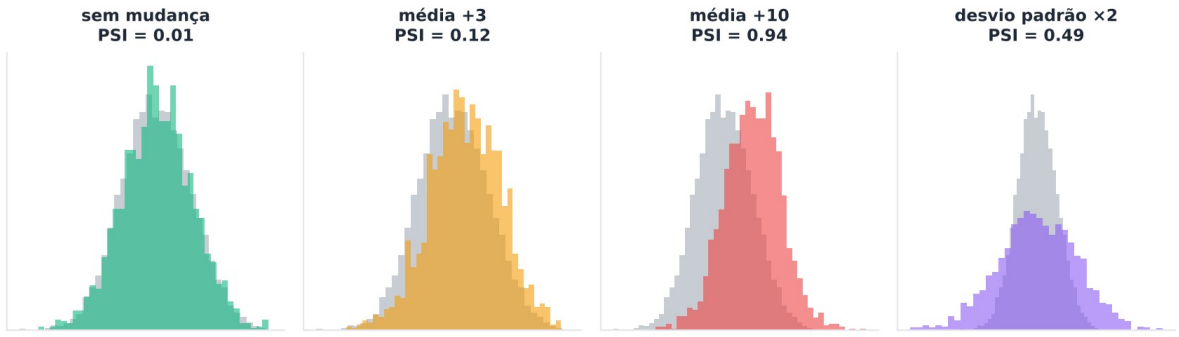


Figura 9.4 — Distribuição de referência (cinza) e distribuições atuais (cor) nos quatro cenários do exemplo. Quanto maior a diferença visual, maiores o PSI e a distância JS.

Para atributos **categóricos**, compare as frequências de cada categoria com o teste do qui-quadrado ou com o PSI, e vigie o surgimento de **categorias novas**.

9.5 Mudanças que só aparecem no conjunto

Às vezes cada atributo, sozinho, parece normal, mas a **relação entre eles** mudou. Por exemplo, renda e idade têm as mesmas médias de antes, porém agora os jovens ganham mais do que os idosos, algo que não acontecia. Testes por atributo não pegam isso.

Uma técnica elegante é o **classificador de domínio**: treine um modelo para distinguir "dado de referência" de "dado atual". Se ele **não consegue** (AUC ≈ 0,5), não há diferença detectável **por esse classificador e com essa**

amostra; isso não demonstra que as distribuições sejam iguais. Se ele **consegue** (AUC alta), há diferença. Rabanser, Günnemann e Lipton (2019) comparam, em um estudo empírico, diversas formas de detectar a mudança de distribuição, incluindo testes de duas amostras; Gretton e colegas (2012) propuseram um teste de duas amostras baseado em kernels (MMD).

```python
import numpy as np
from scipy.stats import ks_2samp
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(0)
ref = rng.normal(size=(3000, 4))
igual = rng.normal(size=(3000, 4))
corr = rng.normal(size=(3000, 4))
corr[:, 1] = 0.8 * corr[:, 0] + 0.6 * corr[:, 1]       # mesmas médias e variâncias, mas colunas 0 e 1 correlacionadas

def auc_de_dominio(A, B):
    X = np.vstack([A, B])
    y = np.r_[np.zeros(len(A)), np.ones(len(B))]
    modelo = RandomForestClassifier(100, min_samples_leaf=5, random_state=0)
    return cross_val_score(modelo, X, y, cv=5, scoring="roc_auc").mean()

print("p-valores do KS por atributo (corr):", [round(float(ks_2samp(ref[:, j], corr[:, j]).pvalue), 2)
for j in range(4)])
print(f"AUC do classificador de domínio, sem mudança : {auc_de_dominio(ref, igual):.3f}")
print(f"AUC do classificador de domínio, com mudança : {auc_de_dominio(ref, corr):.3f}")
```

Os testes por atributo não encontram nada (p-valores altos), mas o classificador de domínio percebe a diferença. Além disso, é possível inspecionar **quais atributos** o classificador mais usa, para investigar a causa.

9.6 Monitorando as previsões

Como não precisamos do rótulo, a distribuição das **saídas** do modelo é um excelente sinal rápido. Se a probabilidade média prevista sobe de 20% para 35% da noite para o dia, algo mudou, no mundo ou no pipeline. Acompanhe também a fração de previsões positivas, a "confiança" e a taxa de recusas.

Rabanser, Günnemann e Lipton (2019) observaram, em seus experimentos, que aplicar testes estatísticos sobre as **saídas do modelo** (em vez de sobre os dados brutos de alta dimensão) é uma maneira prática de detectar mudanças que importam para o modelo.

9.7 Desempenho com rótulos atrasados

O sinal mais direto, o desempenho, esbarra em um problema: o **rótulo demora**. Um cliente só "cancela" meses depois; uma fraude só é confirmada após investigação; um empréstimo só vira inadimplente em semanas. Estratégias:

- **Esperar o rótulo** e calcular o desempenho com atraso (útil, mas lento).
- **Amostrar** casos para rotulagem manual rápida.
- Usar **indicadores adiantados** (*proxies*): cliques, devoluções, reclamações.
- Acompanhar os **sinais sem rótulo** das camadas anteriores (dados e previsões).

9.7.1 Um detector de deriva em fluxo de erros

Quando os rótulos chegam, podemos observar o **fluxo de erros** do modelo. Gama e colegas (2004) propuseram o **DDM** (*Drift Detection Method*): acompanha a taxa de erro p e seu desvio padrão s. Guarda o menor valor já visto de $p + s$ e emite:

- **Alerta**, se $p + s$ passar de (mínimo + 2 desvios);
- **Deriva**, se passar de (mínimo + 3 desvios).

Bifet e Gavaldà (2007) propuseram o ADWIN, que mantém uma janela adaptativa. Aqui implementamos o DDM, mais simples, sobre o mundo simulado da próxima seção.

9.8 Simulação de um modelo que envelhece

Vamos construir um "mundo" simulado de 18 meses em que ocorrem **duas** mudanças ao mesmo tempo, de forma gradual:

1. **Mudança de covariáveis:** o atributo `x0` sobe um pouco a cada mês.
2. **Deriva de conceito:** a fronteira de decisão gira 5 graus por mês.

Treinamos um modelo no mês 0 e o deixamos **estático**. Em paralelo, acompanhamos o PSI de `x0`, a probabilidade média prevista e a acurácia (que, na vida real, só conheceríamos com atraso).

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler


def gerar_mes(m, n=1500):
    r = np.random.default_rng(1000 + m)
    X = r.normal(size=(n, 4))
    X[:, 0] += 0.12 * m                          # mudança de covariáveis
    ang = np.deg2rad(5 * m)                      # deriva de conceito: fronteira gira
    w = np.array([np.cos(ang), np.sin(ang), 0.5, 0.0])
    y = (r.random(n) < 1 / (1 + np.exp(-(X @ w)))).astype(int)
    return X, y


def psi(ref, atual, faixas=10):
    cortes = np.quantile(ref, np.linspace(0, 1, faixas + 1))
    cortes[0], cortes[-1] = -np.inf, np.inf
    p = np.clip(np.histogram(ref, cortes)[0] / len(ref), 1e-6, None)
    q = np.clip(np.histogram(atual, cortes)[0] / len(atual), 1e-6, None)
    return float(np.sum((q - p) * np.log(q / p)))


MESES = 18
dados = [gerar_mes(m) for m in range(MESES)]
Xref, yref = dados[0]
estatico = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(Xref, yref)

linhas = []
for m in range(1, MESES):
```

```
    X, y = dados[m]
    linhas.append({"mes": m,
                   "PSI_x0": psi(Xref[:, 0], X[:, 0]),
                   "prob_media": estatico.predict_proba(X)[:, 1].mean(),
                   "acuracia": (estatico.predict(X) == y).mean()})
print(pd.DataFrame(linhas).round(3).to_string(index=False))
```

Observe a sequência dos sinais: o **PSI** passa de 0,10 no 3.º mês e de 0,25 no 5.º, muito antes de a **acurácia** cair de forma clara (a partir do 10.º ou 12.º mês). Esse é o valor do monitoramento sem rótulos: **um alerta antecipado**.

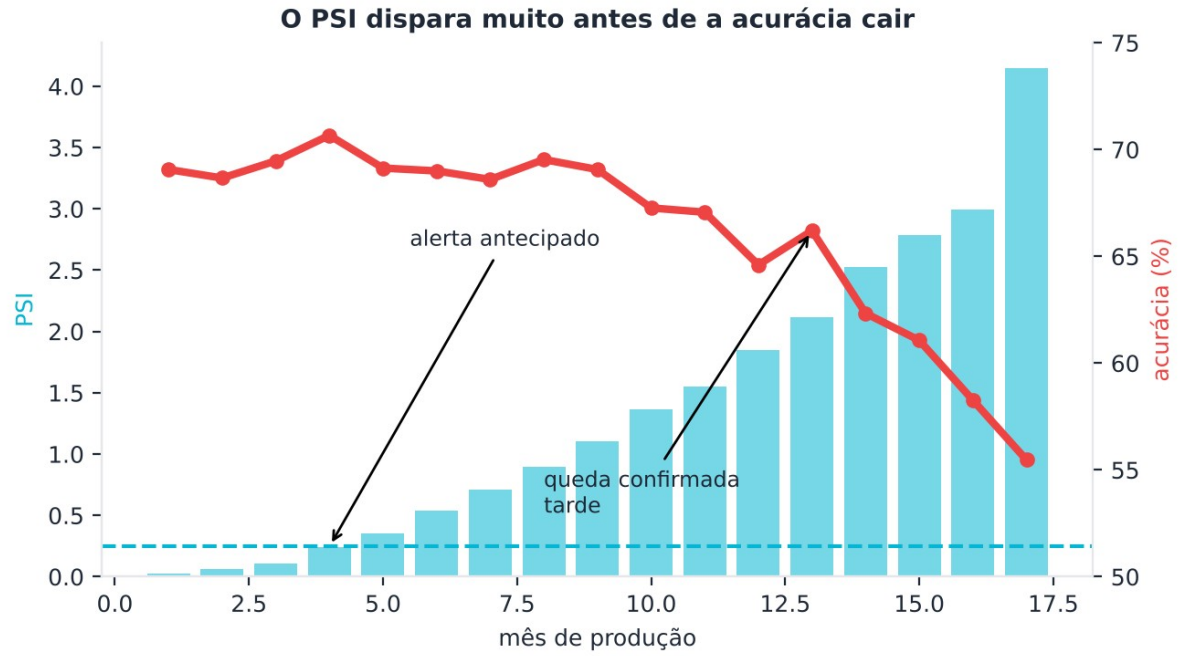


Figura 9.5 — Resultado da simulação: o PSI (barras) dispara cedo; a acurácia do modelo estático (linha vermelha) cai tarde. Estratégias de retreinamento mantêm a acurácia.

9.8.1 O DDM em ação

Agora aplicamos o DDM ao fluxo de erros do modelo estático, supondo que os rótulos chegam sem atraso.

```python
# continuação (usa as funções e os dados da simulação anterior)
class DDM:
    """Drift Detection Method (Gama et al., 2004), versão didática."""
    def __init__(self, min_amostras=500):
        self.n, self.p = 0, 1.0
        self.pmin, self.smin = np.inf, np.inf
        self.min_amostras = min_amostras

    def atualizar(self, erro):
        self.n += 1
        self.p += (erro - self.p) / self.n                # taxa de erro corrente
        s = np.sqrt(self.p * (1 - self.p) / self.n)
        if self.n < self.min_amostras:
            return "ok"
        if self.p + s < self.pmin + self.smin:
            self.pmin, self.smin = self.p, s
        if self.p + s >= self.pmin + 3 * self.smin:
            return "deriva"
        if self.p + s >= self.pmin + 2 * self.smin:
```

```
            return "alerta"
        return "ok"


erros = np.concatenate([(estatico.predict(dados[m][0]) != dados[m][1]).astype(int)
                        for m in range(1, MESES)])
ddm, primeiro = DDM(), {}
for i, e in enumerate(erros):
    estado = ddm.atualizar(e)
    if estado != "ok" and estado not in primeiro:
        primeiro[estado] = i // 1500 + 1              # mês em que ocorreu
print("Primeiro alerta no mês:", primeiro.get("alerta"), "| deriva confirmada no mês:",
primeiro.get("deriva"))
```

9.9 Retreinamento: como e quando

Quando se detecta a deriva, a resposta usual é **retreinar**. Mas há várias formas:

Estratégia	Como funciona	Quando é boa	Cuidado
Estático	Treina uma vez e nunca mais	Mundo estável	Envelhece
Periódico	Retreina a cada X dias com todos os dados	Mudança lenta, dados abundantes	Dados antigos podem pesar demais
Janela deslizante	Retreina só com os últimos N meses	Deriva de conceito	Janela curta = pouco dado; longa = lento
Ponderado por recência	Todos os dados, com mais peso aos recentes	Meio-termo	Escolher o decaimento
Incremental (*online*)	Atualiza o modelo a cada lote (`partial_fit`)	Fluxos contínuos, dados enormes	Sensível a ruído e a dados ruins
Por gatilho	Retreina quando o monitoramento acusa	Economia de recursos	Gatilho mal calibrado

PYTHON

```
# continuação (usa as funções e os dados da simulação anterior)
from sklearn.linear_model import SGDClassifier

fabrica = lambda: make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
sc = StandardScaler().fit(Xref)
online = SGDClassifier(loss="log_loss", learning_rate="constant", eta0=0.05, alpha=1e-4,
random_state=0)
for _ in range(5):
    online.partial_fit(sc.transform(Xref), yref, classes=[0, 1])

linhas = []
for m in range(1, MESES):
    X, y = dados[m]
    ini = max(0, m - 3)                               # janela dos 3 meses anteriores (já rotulados)
    Xj = np.vstack([d[0] for d in dados[ini:m]])
    yj = np.concatenate([d[1] for d in dados[ini:m]])
    janela = fabrica().fit(Xj, yj)
    linhas.append({"mes": m,
                   "estatico": (estatico.predict(X) == y).mean(),
                   "janela_3m": (janela.predict(X) == y).mean(),
```

```
                "incremental": (online.predict(sc.transform(X)) == y).mean()})
    for _ in range(2):
        online.partial_fit(sc.transform(X), y)          # aprende com o mês que acabou
res = pd.DataFrame(linhas).round(3)
print(res.iloc[[0, 5, 10, 12, 14, 16]].to_string(index=False))
print("Média dos últimos 6 meses:", res.tail(6).drop(columns="mes").mean().round(3).to_dict())
```

Compare as colunas: a acurácia do modelo estático **cai**, enquanto as estratégias que reaprendem com dados recentes se mantêm estáveis em torno de 0,70. Nesta simulação, o conceito muda de forma suave; em mudanças abruptas ou ruidosas, a escolha da janela e do ritmo de atualização exige mais cuidado.

> **Atenção: retreinar não é sempre a solução**
>
> Antes de retreinar, **investigue a causa**. Se o PSI disparou porque um sensor quebrou ou uma coluna passou a vir em outra unidade, retreinar com esses dados **ensinaria o erro ao modelo**. Corrija a origem primeiro. E todo modelo retreinado deve passar pelos testes e pelo portão de qualidade (Capítulos 5 e 8) antes de substituir o campeão.

9.10 Implantando mudanças com segurança

Mesmo com bons testes, o mundo real surpreende. Por isso, colocamos modelos novos em produção por etapas:

1. **Modo sombra** (*shadow*): o modelo novo recebe cópia dos pedidos e faz previsões, mas **ninguém as usa**. Comparam-se as respostas com as do modelo atual. Reduz o risco de decisões baseadas no candidato, mas ainda envolve processamento dos dados, consumo de recursos e possíveis efeitos operacionais.
2. **Canário:** o modelo novo atende uma pequena fração (por exemplo, 1% a 5%) dos pedidos. Se tudo estiver bem, aumenta-se gradualmente.
3. **Teste A/B:** divide-se o tráfego **aleatoriamente** entre os modelos e compara-se o efeito no negócio.
4. **Reversão** (*rollback*): se algo der errado, voltar ao modelo anterior em minutos. Com apelidos no registro de modelos (Capítulo 5), basta mover o apelido `champion` e **recarregar o serviço** (um objeto já carregado continua sendo a versão antiga até a recarga); confirme a versão efetivamente servida.

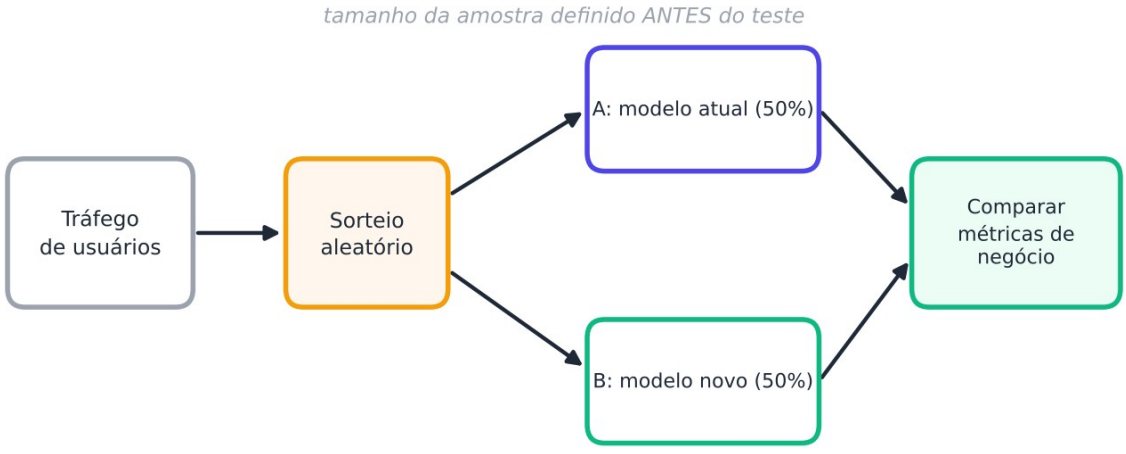


Figura 9.6 — Teste A/B: o tráfego é dividido ao acaso entre o modelo atual (A) e o novo (B), e as métricas de negócio são comparadas.

9.10.1 Teste A/B: quanta gente é preciso?

Kohavi e colegas (2013, 2020) são referências em experimentos controlados online. Dois cuidados centrais: **definir o tamanho da amostra antes** e **não olhar o resultado toda hora para parar quando ficar bonito**. Veja como calcular o tamanho necessário e testar a diferença de duas proporções:

```python
import numpy as np
from scipy.stats import norm

def n_por_grupo(p0, efeito_minimo, alfa=0.05, poder=0.80):
    """Tamanho de amostra por grupo para detectar uma diferença de proporções."""
    p1 = p0 + efeito_minimo
    pm = (p0 + p1) / 2
    za, zb = norm.ppf(1 - alfa / 2), norm.ppf(poder)
    num = (za * np.sqrt(2 * pm * (1 - pm)) + zb * np.sqrt(p0 * (1 - p0) + p1 * (1 - p1))) ** 2
    return int(np.ceil(num / efeito_minimo ** 2))

def teste_duas_proporcoes(x1, n1, x2, n2):
    p1, p2 = x1 / n1, x2 / n2
    p = (x1 + x2) / (n1 + n2)
    erro = np.sqrt(p * (1 - p) * (1 / n1 + 1 / n2))
    z = (p2 - p1) / erro
    return p2 - p1, 2 * (1 - norm.cdf(abs(z)))

print("Taxa atual de 10%. Detectar +1 ponto percentual exige", n_por_grupo(0.10, 0.01), "por grupo.")
print("Detectar +2 pontos percentuais exige", n_por_grupo(0.10, 0.02), "por grupo.")

dif, p_valor = teste_duas_proporcoes(1000, 10000, 1100, 10000)   # A: 10,0% | B: 11,0%
print(f"Resultado: diferença = {dif:+.3f}, p-valor = {p_valor:.3f}")
```

> **Dica**
>
> Um p-valor pequeno indica que os dados são pouco compatíveis com a hipótese de que não há diferença, **sob as condições do teste** (amostras independentes, tamanho fixado de antemão etc.). Ele **não** é a probabilidade de a diferença ser "fruto do acaso" e **não** diz que ela é grande nem importante para o negócio. Sempre olhe o **tamanho do efeito** e o **intervalo de confiança**. E monitore também **métricas de proteção** (por exemplo, latência e reclamações) para garantir que o modelo novo não piora outra coisa.

9.11 Registrando o que o modelo faz

Sem registros (*logs*), não há monitoramento. Para cada previsão, guarde:

- um **identificador** da requisição e o **horário**;
- a **versão** do modelo e do pipeline;
- as **entradas** (ou um resumo/hash, respeitando a privacidade);
- a **saída** (classe e probabilidade);
- a **latência**;
- depois, quando disponível, o **rótulo real** (para calcular desempenho).

O formato mais prático é uma linha de **JSON** por evento, o que facilita buscar e analisar depois.

```python
```

```python
import json
import time
import uuid
from datetime import datetime, timezone

def registrar_previsao(entrada: dict, probabilidade: float, versao: str, inicio: float,
arquivo="previsoes.jsonl"):
    evento = {
        "id": str(uuid.uuid4()),
        "horario_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "versao_modelo": versao,
        "entrada": entrada,
        "probabilidade": round(probabilidade, 4),
        "latencia_ms": round((time.perf_counter() - inicio) * 1000, 2),
        "rotulo_real": None,                              # preenchido depois, quando chegar
    }
    with open(arquivo, "a", encoding="utf-8") as f:
        f.write(json.dumps(evento, ensure_ascii=False) + "\n")
    return evento["id"]

t0 = time.perf_counter()
id_evento = registrar_previsao({"idade": 33, "plano": "basico"}, 0.74, "1.0.0", t0)
print("Evento registrado:", id_evento[:8], "...")
print(open("previsoes.jsonl", encoding="utf-8").read().strip())
```

> **Atenção**
>
> Registros de previsões podem conter **dados pessoais**. Aplique minimização, pseudonimização e prazos de retenção (Capítulo 3). Defina quem pode acessar os logs.

9.12 Alertas, painéis e planos de resposta

Monitorar não é só medir; é **agir**. Boas práticas:

- **Poucos alertas, bem escolhidos.** Alertas demais geram "fadiga de alertas", e as pessoas passam a ignorá-los.
- **Níveis de gravidade.** Informativo, aviso, crítico, com destinatários diferentes.
- **Limiares com base em dados**, revisados periodicamente.
- **Painéis** (*dashboards*) simples: saúde do serviço, qualidade dos dados, previsões, desempenho, fatias e negócio.
- **Plano de resposta** (*runbook*): para cada alerta, o que fazer, quem chama quem, como reverter.
- **Revisão pós-incidente** sem culpados: aprender para evitar a repetição.

Alerta	Gravidade	Primeira ação
Latência p95 acima de 200 ms por 10 min	Aviso	Verificar carga e escalar o serviço
Taxa de ausentes em `renda` acima de 20%	Crítico	Parar o consumo do lote; avisar engenharia de dados
PSI de qualquer atributo-chave acima de 0,25	Aviso	Investigar a origem; avaliar retreino
F1 móvel abaixo do piso por 3 semanas	Crítico	Reverter para o modelo anterior; acionar retreino

9.13 Resumo do capítulo

MEMORIZE **O que levar deste capítulo**

- Modelos **envelhecem sozinhos** e falham em silêncio. Monitore em camadas: serviço, dados, previsões, desempenho e negócio.
- Três mudanças: **covariáveis** (P(X)), **rótulos** (P(y)) e **conceito** (relação X→y). Podem ser abruptas, graduais, incrementais ou recorrentes.
- Para detectar: **KS**, **PSI** (0,10 e 0,25 são regras práticas), **Jensen-Shannon** e o **classificador de domínio** para mudanças multivariadas.
- Sem rótulos, olhe **dados de entrada** e **distribuição das previsões**. Com rótulos atrasados, use **amostragem**, *proxies* e detectores como o **DDM**.
- Retreino: periódico, janela deslizante, ponderado, incremental ou por gatilho. **Investigue a causa antes de retreinar** e passe pelo portão de qualidade.
- Implante com **sombra, canário, teste A/B e reversão rápida**. Defina o tamanho da amostra antes do teste.
- **Registre** previsões (com versão, entrada, saída, latência e rótulo posterior), tenha **alertas poucos e úteis** e **planos de resposta**.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Por que dizemos que o modelo "envelhece sozinho"?

a) Os parâmetros diminuem a cada previsão.
b) O mundo muda e o modelo continua refletindo o passado.
c) O código se desgasta com o uso.
d) O disco apaga arquivos antigos.

Questão 2. A deriva de conceito (concept drift) ocorre quando:

a) apenas o servidor muda de lugar.
b) muda a relação entre os atributos e o alvo.
c) apenas a latência aumenta.
d) apenas o formato do arquivo muda.

Questão 3. A mudança de covariáveis (covariate shift) é uma mudança na distribuição:

a) do hardware.
b) dos atributos de entrada, P(X).
c) dos rótulos, P(y).
d) da relação X→y.

Questão 4. A mudança de rótulos (label shift) é uma mudança:

a) no tamanho do arquivo.
b) na proporção das classes, P(y), com P(X | y) estável.
c) no modelo de regressão apenas.
d) na distribuição de X apenas, com P(y | X) fixo.

Questão 5. Uma mudança que vem e volta periodicamente, como a sazonalidade de Natal, é chamada de:

a) abrupta.
b) inexistente.
c) recorrente.
d) incremental.

Questão 6. Qual camada de monitoramento NÃO exige rótulos?

a) Nenhuma camada exige.
b) Dados de entrada e distribuição das previsões.
c) Desempenho (F1, AUC).
d) Efeito no negócio, sempre.

Questão 7. O teste de Kolmogorov-Smirnov compara:

a) as distribuições acumuladas de duas amostras.
b) a média de três amostras.
c) a correlação entre colunas.
d) o tamanho de dois arquivos.

Questão 8. Em amostras muito grandes, o teste KS tende a:

a) ficar mais lento sem alterar o resultado.
b) apontar como significativas até diferenças minúsculas.
c) nunca rejeitar a hipótese nula.
d) funcionar apenas para categorias.

Questão 9. Uma regra prática para o PSI diz que valores acima de 0,25 indicam:

a) mudança importante.
b) erro de cálculo.
c) mudança moderada.
d) estabilidade.

Questão 10. O PSI compara:

a) a fração de dados em cada faixa antes e agora.
b) o tamanho do modelo em dois momentos.
c) o tempo de treino em duas máquinas.
d) a média de duas colunas.

Questão 11. A distância de Jensen-Shannon é:

a) uma métrica de regressão.
b) uma medida simétrica e limitada, baseada na divergência de Kullback-Leibler.
c) um teste de normalidade.
d) uma medida que não depende de distribuições.

Questão 12. Para atributos categóricos, uma técnica adequada de detecção de mudança é:

a) calcular a média.
b) comparar frequências com o qui-quadrado ou o PSI.
c) ignorar categorias novas.
d) aplicar KS diretamente nas categorias nominais.

Questão 13. O classificador de domínio detecta mudanças porque:

a) tenta distinguir dados de referência dos atuais; se consegue, as distribuições diferem.
b) mede a latência.
c) compara os arquivos byte a byte.
d) prevê o alvo original.

Questão 14. Em qual situação o classificador de domínio é mais útil que testes por atributo?

a) Quando a relação entre atributos mudou e cada marginal parece normal.
b) Quando só há uma coluna.
c) Quando um só atributo mudou muito.
d) Quando não há dados.

Questão 15. Monitorar a distribuição das saídas do modelo é útil porque:

a) elimina a necessidade de dados.
b) substitui qualquer outro monitoramento.
c) não exige rótulos e pode revelar mudanças rapidamente.
d) mede diretamente a acurácia.

Questão 16. Rótulos atrasados significam que:

a) o treino demorou.
b) o valor real só é conhecido algum tempo depois da previsão.
c) os rótulos são inventados.
d) o modelo atrasa para responder.

Questão 17. O DDM (Gama et al., 2004) monitora:

a) o tamanho dos lotes.
b) a memória do servidor.
c) o número de atributos.
d) a taxa de erro e seu desvio, emitindo alerta e deriva.

Questão 18. ADWIN (Bifet e Gavaldà, 2007) é:

a) um método de detecção de mudança com janela adaptativa.
b) um formato de arquivo.
c) um tipo de contêiner.
d) um servidor de modelos.

Questão 19. Na simulação do capítulo, o PSI disparou antes de a acurácia cair porque:

a) a mudança nos atributos aparece antes de prejudicar claramente o desempenho.
b) o PSI usa os rótulos.
c) a acurácia é calculada no treino.
d) o modelo estava calibrado.

Questão 20. Retreinar o modelo imediatamente após qualquer alerta pode ser um erro porque:

a) o retreino é proibido.
b) retreinar sempre piora o modelo.
c) o alerta nunca é verdadeiro.
d) a causa pode ser um problema de dados, e retreinar ensinaria o erro ao modelo.

Questão 21. A estratégia de janela deslizante de retreino:

a) usa dados do futuro.
b) usa só os dados mais recentes.
c) nunca retreina.
d) usa apenas dados antigos.

Questão 22. Aprendizado incremental com partial_fit é indicado quando:

a) há fluxo contínuo de dados e se quer atualizar o modelo por lote.
b) os rótulos são ausentes.
c) nunca há novos dados.
d) o modelo deve permanecer idêntico.

Questão 23. No modo sombra de implantação:

a) o modelo novo responde a todos.
b) o modelo antigo é desligado.
c) só 50% dos clientes veem o novo.
d) o modelo novo faz previsões que não são usadas, para comparação.

Questão 24. No teste A/B, o tráfego é dividido:

a) apenas para clientes novos.
b) aleatoriamente entre modelo atual e modelo novo.
c) de acordo com a preferência dos engenheiros.
d) por ordem alfabética dos clientes.

Questão 25. Por que definir o tamanho da amostra de um teste A/B antes de começar?

a) Para reduzir o custo do servidor.
b) Evitar parar o teste quando o resultado "ficar bonito", o que gera falsos positivos.
c) Porque o servidor exige.
d) Para aumentar o p-valor.

Questão 26. Um p-valor pequeno em um teste A/B indica que:

a) o teste foi inválido.
b) a diferença é enorme.
c) os dados são pouco compatíveis com a hipótese de ausência de diferença, sob as condições do teste, mas isso não diz que a diferença é grande ou importante.
d) o modelo é perfeito.

Questão 27. Qual dos itens abaixo deve constar no registro (log) de cada previsão?

a) O código-fonte inteiro.
b) A senha do usuário.
c) Identificador, horário, versão do modelo, entradas (ou resumo), saída e latência.
d) O conteúdo do disco.

Questão 28. "Fadiga de alertas" ocorre quando:

a) há poucos alertas.
b) há alertas demais e as pessoas passam a ignorá-los.
c) todos os alertas são críticos e corretos.
d) o painel é bonito.

Questão 29. Um runbook (plano de resposta) deve dizer:

a) apenas o nome do desenvolvedor.
b) somente a versão do Python.
c) o que fazer em cada alerta, quem acionar e como reverter.
d) o histórico de preços da nuvem.

Questão 30. Qual sinal sem rótulo é mais rápido, mas indireto, para sugerir problema em produção?

a) O resultado de uma auditoria externa.
b) A queda do lucro anual.
c) O número de linhas de código.
d) A mudança brusca na probabilidade média prevista.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que a falha silenciosa torna o monitoramento indispensável e que camadas de monitoramento você implantaria.
2. Diferencie mudança de covariáveis, mudança de rótulos e deriva de conceito, com um exemplo de cada.
3. Descreva três formas de mudança temporal (abrupta, gradual, incremental, recorrente) com exemplos.
4. Explique o PSI, como é calculado e como interpretá-lo, com a ressalva sobre seus limiares.
5. Por que um teste estatístico com p-valor pequeno pode não indicar mudança relevante para o modelo?
6. Como o classificador de domínio funciona e o que se pode aprender de suas importâncias?
7. Como monitorar o desempenho de um modelo quando os rótulos demoram a chegar?
8. Descreva o DDM e como interpretar seus estados de alerta e deriva.
9. Compare as estratégias de retreino estático, periódico, janela deslizante, incremental e por gatilho.
10. Por que é perigoso retreinar automaticamente sem investigar a causa de um alerta?
11. Descreva o procedimento de uma implantação progressiva com modo sombra, canário e reversão.
12. Como planejar o tamanho de amostra de um teste A/B e por que não olhar o resultado toda hora?
13. O que deve ser registrado em log para cada previsão e quais cuidados de privacidade tomar?
14. Como evitar a fadiga de alertas? Dê três práticas.
15. Por que se deve atualizar a referência do monitoramento depois de um retreino bem-sucedido?
16. Na simulação do capítulo, o que mostra a comparação entre o PSI e a acurácia ao longo do tempo?
17. Explique como uma mudança de covariáveis pode não prejudicar a AUC do modelo, mas ainda assim exigir atenção.
18. Qual a diferença entre monitoramento de dados e monitoramento de desempenho, e por que ambos são necessários?
19. Por que um alerta sem rótulos pode não enxergar a deriva de conceito?
20. Que informações um runbook deve conter para um alerta de queda de F1?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Calculando o PSI à mão

NÍVEL: BÁSICO

Implemente o PSI com 10 faixas (decis da referência). Use uma referência normal (média 0, desvio 1, 5.000 pontos) e compare com amostras de 2.000 pontos em quatro cenários: sem mudança, média +0,3, média +1 e desvio padrão ×2. Classifique cada resultado com a regra prática (< 0,10 estável; 0,10 a 0,25 moderada; > 0,25 importante).

Exercício 2 — Deriva em atributo categórico

NÍVEL: INTERMEDIÁRIO

Crie uma distribuição de referência de `plano` (basico 50%, padrao 35%, premium 15%) e três lotes atuais: (a) mesma distribuição, (b) basico 35%, padrao 40%, premium 25%, (c) igual à referência mas com uma categoria nova "familia" em 8% dos casos. Para cada lote, calcule o teste do qui-quadrado (`scipy.stats.chi2_contingency`) e liste categorias novas.

Exercício 3 — O classificador de domínio acha o que os testes simples perdem

NÍVEL: INTERMEDIÁRIO

Crie uma referência com 5 atributos normais independentes e um lote atual em que os atributos 0 e 1 passaram a ter correlação 0,8, mantendo médias e desvios iguais. Mostre que o teste KS por atributo não acusa nada e que a AUC de um classificador de domínio (floresta, validação cruzada de 4 dobras) sim. Depois, descubra quais atributos o classificador mais usa.

Exercício 4 — Quanto o DDM demora a perceber?

NÍVEL: AVANÇADO

Gere um fluxo de 6.000 erros de um modelo: taxa de erro de 10% nos primeiros 3.000 casos e depois uma mudança para 10% (sem mudança), 15%, 20% ou 30% (quatro cenários). Use o detector DDM (com pelo menos 500 amostras antes de decidir; deriva em mínimo + 3 desvios). Repita com 40 sementes e informe, para cada cenário: em quantas sementes houve falso alarme antes da posição 3.000, em quantas a deriva foi detectada depois da mudança e a mediana do atraso (posição da detecção − 3.000).

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Bike Sharing Dataset*, *Online Retail* e *Electricity (NSW)*, séries com padrões temporais e mudanças ao longo do tempo.
> - **Kaggle:** *Store Sales (Time Series Forecasting)* e *Rossmann Store Sales* para observar sazonalidade e eventos. Meça a deriva entre períodos.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Painel de monitoramento de 12 semanas com três tipos de deriva

Contexto. Um modelo de classificação foi treinado na semana 0. Nas 12 semanas seguintes, o mundo muda de três maneiras: a partir da semana 3, mudança de covariáveis (o atributo 0 se desloca); a partir da semana 6, mudança de rótulos (*label shift*: a proporção de positivos sobe para 35%, mas **P(X | y) permanece igual**, obtida por reamostragem estratificada das classes); e a partir da semana 8, deriva de conceito (a fronteira de decisão gira).

Dados simulados. Gere 12 lotes semanais de 1.200 exemplos, com 4 atributos normais e rótulo gerado por uma função logística da combinação linear dos atributos mais um intercepto que controla a proporção de positivos. Documente suas escolhas.

Tarefas.

1. Treine um modelo (regressão logística) na semana 0 e deixe-o estático.
2. Para cada semana, calcule: PSI do atributo 0, AUC do classificador de domínio, probabilidade média prevista, fração real de positivos e a AUC-ROC do modelo (supondo que o rótulo chegue sem atraso, só para fins de análise).
3. Defina uma regra de alerta sem rótulos (PSI e AUC de domínio) e informe a semana do primeiro alerta e a semana em que a AUC do modelo estático cai mais de 0,10 em relação à da semana 1. O alerta enxerga todas as mudanças?
4. Compare três estratégias: (a) estático; (b) retreino periódico, a cada 3 semanas, com as 3 últimas semanas rotuladas; (c) retreino por gatilho (quando a regra de alerta dispara, espera 2 semanas para que cheguem rótulos do regime novo, retreina com as 3 últimas semanas rotuladas e atualiza a referência do monitoramento).
5. Compare as estratégias pela AUC média e pelo erro médio entre a probabilidade média prevista e a fração real de positivos (calibração global) nas semanas 4 a 12, sem concluir demais de um único indicador.
6. Escreva uma recomendação sobre monitoramento e retreinamento, explicando que tipo de mudança prejudica cada métrica.

Entregáveis. Tabela semanal, semanas dos alertas, comparação das estratégias e recomendação.

A solução de referência está no Apêndice E.

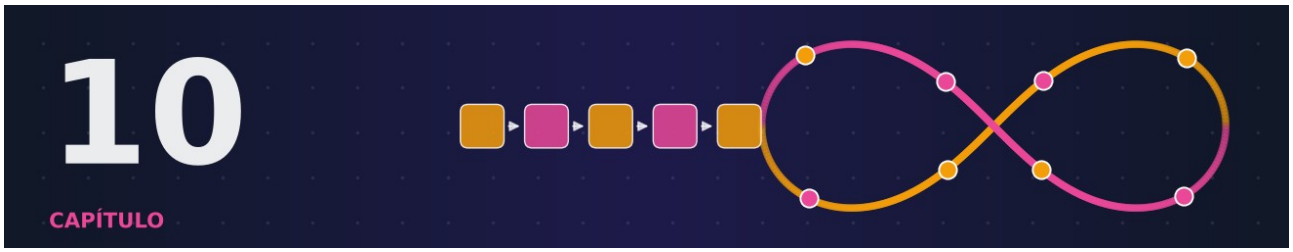
PARTE IV

Parte IV — Confiança e Escala

Governança, explicabilidade, privacidade e ética; feature stores, orquestração, plataformas e a nova fronteira de LLMOps.

Nesta parte:

- Cap. 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas



Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Documentar um modelo com um cartão de modelo (*model card*) e uma ficha de dados.
- Explicar previsões com importância por permutação e dependência parcial.
- Calcular métricas de justiça entre grupos e discutir seus limites.
- Aplicar o mecanismo de Laplace da privacidade diferencial em um exemplo simples.
- Explicar o papel do armazém de atributos e a correção temporal (*point-in-time*).
- Descrever e implementar um orquestrador mínimo de tarefas em grafo (DAG).
- Reconhecer o que muda em LLMOps (prompts, RAG, avaliação e custos).

10.1 Do "funciona" ao "merece confiança"

Até aqui, aprendemos a colocar um modelo no ar e a mantê-lo saudável. Mas existe outra pergunta: **podemos confiar nele**? Quando um modelo decide quem recebe crédito, quem é chamado para uma entrevista ou que tratamento médico sugerir, os erros afetam pessoas.

Governança de ML é o conjunto de políticas, papéis e práticas que garantem que os modelos sejam usados de forma **responsável, transparente, justa, segura e em conformidade com a lei**. Ela responde a perguntas como:

- **Quem** é responsável por este modelo?
- **Para que** ele foi feito, e para que **não** deve ser usado?
- **Com quais dados** foi treinado, e esses dados podiam ser usados?
- **Como decide** e como justificamos uma decisão?
- **Para quem** funciona pior?
- **Como** fiscalizamos e corrigimos?

No Brasil, a **LGPD** (Lei n.º 13.709/2018) trata do uso de dados pessoais, inclusive do direito de solicitar a revisão de decisões tomadas unicamente por tratamento automatizado. Na União Europeia, o **Regulamento de IA** (Regulamento (UE) 2024/1689) classifica sistemas por nível de risco e impõe obrigações de documentação, supervisão humana e gestão de riscos aos de alto risco. Consulte sempre o texto legal vigente e a orientação jurídica da sua organização.

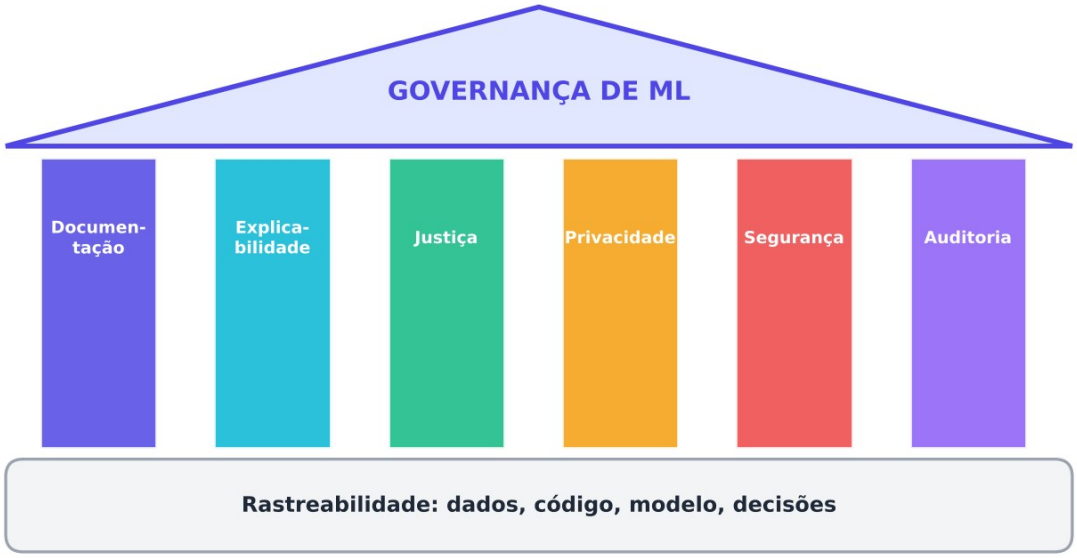


Figura 10.1 — Os pilares da governança de ML: documentação, explicabilidade, justiça, privacidade, segurança e auditoria, todos apoiados por rastreabilidade.

10.2 Documentando modelos: o cartão do modelo

Assim como um remédio tem bula, um modelo deve ter um **cartão de modelo** (*model card*). Mitchell e colegas (2019) propuseram esse documento curto, com informações como:

- **Detalhes do modelo:** quem, quando, versão, tipo, licença.
- **Uso pretendido:** casos de uso indicados e **fora do escopo**.
- **Fatores:** grupos e condições relevantes (idade, região, equipamento).
- **Métricas:** quais e por quê, incluindo **por grupo**.
- **Dados de avaliação e de treino:** origem e características.
- **Análises quantitativas:** resultados desagregados.
- **Considerações éticas:** riscos, limitações.
- **Recomendações:** cuidados e alertas.

Podemos **gerar o cartão automaticamente** a partir dos metadados registrados no pipeline (Capítulo 5), de modo que ele nunca fique desatualizado.

```python
from datetime import date

def gerar_cartao(m: dict) -> str:
    linhas = [f"# Cartão do modelo: {m['nome']} (v{m['versao']})", f"*Gerado em {m['data']}*", ""]
    linhas += ["## Uso pretendido", m["uso"], "", "## Fora do escopo", m["fora_do_escopo"], ""]
    linhas += ["## Dados", f"- Treino: {m['dados_treino']}", f"- Avaliação: {m['dados_avaliacao']}",
""]
    linhas += ["## Métricas"]
    linhas += [f"- {k}: {v:.3f}" for k, v in m["metricas"].items()]
    linhas += ["", "## Desempenho por grupo"]
    linhas += [f"- {g}: F1 = {v:.3f}" for g, v in m["por_grupo"].items()]
    linhas += ["", "## Limitações e riscos"] + [f"- {x}" for x in m["limitacoes"]]
    linhas += ["", f"## Responsável\n{m['responsavel']}"]
```

```
    return "\n".join(linhas)

cartao = gerar_cartao({
    "nome": "detector_cancelamento", "versao": "2.0.1", "data": date(2026, 10, 17).isoformat(),
    "uso": "Priorizar contatos de retenção. A decisão final é sempre humana.",
    "fora_do_escopo": "Decidir encerramento de contratos ou precificação individual.",
    "dados_treino": "Clientes de jan/2024 a jun/2026 (hash a3f9c1...)",
    "dados_avaliacao": "Amostra fixa de jul/2026, não usada no treino",
    "metricas": {"F1": 0.81, "Revocação": 0.84, "AP": 0.77},
    "por_grupo": {"Região Norte": 0.74, "Região Sul": 0.83},
    "limitacoes": ["Pior desempenho para clientes com menos de 3 meses", "Não validado fora do
Brasil"],
    "responsavel": "Equipe de Dados e Retenção",
})
print(cartao)
```

> **Dica**
>
> Inclua o cartão **no registro de modelos** e **exija-o** no portão de promoção: nenhuma versão vai para produção sem cartão preenchido e revisado.

10.3 Explicabilidade: por que o modelo decidiu isso?

Modelos complexos, como florestas e redes neurais, funcionam como **caixas-pretas**: dão a resposta, mas não explicam. Para auditar, depurar e ganhar confiança, usamos técnicas de **explicabilidade** (*explainability*). Guidotti e colegas (2018) e Arrieta e colegas (2020) fazem revisões abrangentes dessas técnicas.

Uma distinção útil:

- **Explicação global:** o que, em geral, importa para o modelo?
- **Explicação local:** por que **esta** previsão específica saiu assim?

Técnica	Tipo	Ideia	Observação
Importância por permutação	Global	Embaralhe um atributo e veja quanto o desempenho cai	Simples; cuidado com atributos correlacionados
Dependência parcial (PDP)	Global	Mostra o efeito médio de um atributo na previsão (FRIEDMAN, 2001)	Pode esconder interações
LIME	Local	Aproxima o modelo, perto de um caso, por um modelo simples (RIBEIRO; SINGH; GUESTRIN, 2016)	Sensível à vizinhança escolhida
SHAP	Local e global	Divide a previsão entre os atributos com base em valores de Shapley (LUNDBERG; LEE, 2017)	Mais pesado; boa fundamentação teórica

Breiman (2001) já havia introduzido a importância por permutação no contexto das florestas aleatórias. O scikit-learn oferece a implementação em `sklearn.inspection`.

```
PYTHON
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
```

```python
from sklearn.inspection import partial_dependence, permutation_importance
from sklearn.model_selection import train_test_split

dados = load_breast_cancer(as_frame=True)
X, y = dados.data, dados.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
modelo = RandomForestClassifier(n_estimators=200, random_state=0).fit(Xtr, ytr)

# importância por permutação, medida no conjunto de TESTE
r = permutation_importance(modelo, Xte, yte, n_repeats=20, random_state=0, scoring="f1")
ordem = np.argsort(-r.importances_mean)[:5]
print("Atributos mais importantes (queda média do F1 ao embaralhar):")
for i in ordem:
    print(f"  {X.columns[i]:22s} {r.importances_mean[i]:.4f} ± {r.importances_std[i]:.4f}")

# dependência parcial do atributo mais importante
melhor = X.columns[ordem[0]]
pd_ = partial_dependence(modelo, Xte, [melhor], grid_resolution=6)
print(f"\nEfeito médio de '{melhor}' na probabilidade da classe 1 (benigno):")
for v, p in zip(pd_["grid_values"][0], pd_["average"][0]):
    print(f"  {melhor} = {v:8.2f} -> prob. média = {p:.3f}")
```

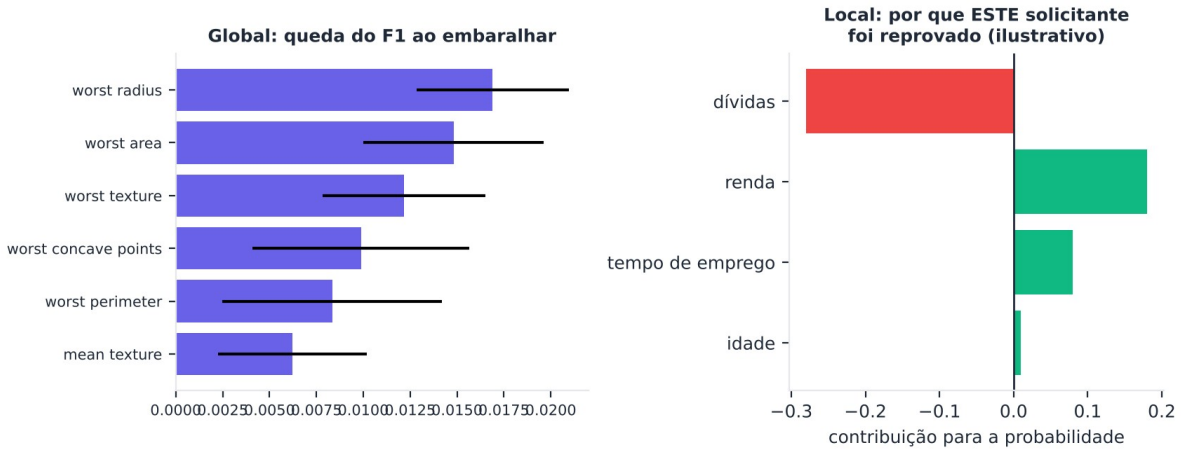


Figura 10.2 — Explicação global (importância por permutação) à esquerda e explicação local (contribuição por atributo em uma previsão) à direita.

> **Atenção: explicações não são causas**
>
> Importância e dependência mostram o que o **modelo** usa, e não o que **causa** o fenômeno no mundo. Se dois atributos são muito correlacionados, a importância se divide entre eles e pode enganar. Use explicações para **investigar e levantar hipóteses**, e não para afirmar relações de causa e efeito.

10.4 Justiça: o modelo trata os grupos de forma equilibrada?

Um modelo pode ser preciso **na média** e ainda assim prejudicar um grupo. As causas incluem dados históricos enviesados, amostras pequenas de certos grupos, rótulos que refletem decisões humanas injustas e *proxies* (atributos que "denunciam" a característica sensível, como CEP para indicar raça ou renda). Mehrabi e colegas (2021) fazem uma revisão de fontes de viés e de definições de justiça; Barocas, Hardt e Narayanan (2023) é um livro de referência sobre o tema.

Algumas métricas comuns, comparando grupos A e B:

Critério	Pergunta	Como medir
Paridade demográfica	A fração de aprovados é parecida entre os grupos?	Taxa de seleção por grupo
Igualdade de oportunidade	Entre os casos positivos segundo o critério definido, a fração de aprovados é parecida?	Revocação (TPR) por grupo (HARDT; PRICE; SREBRO, 2016)
Paridade de precisão	Quando o modelo aprova, acerta com a mesma frequência?	Precisão por grupo

Existe uma **razão de impacto desproporcional**: a taxa de seleção do grupo com menos aprovações dividida pela do grupo com mais. Uma regra prática histórica, a "regra dos quatro quintos", considera suspeito um valor abaixo de 0,8. Ela é uma **referência**, não um veredito.

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
n = 20000
grupo = rng.choice(["A", "B"], n, p=[0.7, 0.3])
habilidade = rng.normal(size=n)                                  # habilidade real, igual nos dois grupos
y = (habilidade + rng.normal(0, 0.8, n) > 0).astype(int)         # resultado positivo segundo o critério definido
# o atributo observado é medido com mais ruído/viés no grupo B (um proxy imperfeito)
x1 = habilidade + np.where(grupo == "B", -0.6, 0.0) + rng.normal(0, 0.5, n)
x2 = habilidade + rng.normal(0, 0.9, n)
dados = pd.DataFrame({"x1": x1, "x2": x2, "grupo": grupo, "y": y})

# treino (50%), validação (25%) e teste (25%), estratificados por grupo
tr, resto = train_test_split(dados, test_size=0.5, random_state=0, stratify=dados["grupo"])
va, te = train_test_split(resto, test_size=0.5, random_state=0, stratify=resto["grupo"])
modelo = LogisticRegression().fit(tr[["x1", "x2"]], tr["y"])    # o grupo NÃO é usado como entrada
va = va.assign(p=modelo.predict_proba(va[["x1", "x2"]])[:, 1])
te = te.assign(p=modelo.predict_proba(te[["x1", "x2"]])[:, 1])

def tabela(te, limiar):
    linhas = {}
    for g, d in te.groupby("grupo"):
        sel = d["p"] >= limiar
        linhas[g] = {"taxa_selecao": sel.mean(), "revocacao": sel[d["y"] == 1].mean(),
                     "precisao": d["y"][sel].mean()}
    return pd.DataFrame(linhas).T

t = tabela(te, 0.5)                                              # métricas no TESTE
print(t.round(3))
print("Razão de impacto (B/A):", round(t.loc["B", "taxa_selecao"] / t.loc["A", "taxa_selecao"], 3))
```

Mesmo **sem usar o grupo como entrada**, o modelo seleciona menos pessoas do grupo B e encontra menos dos casos positivos do grupo B (revocação menor): o viés entrou pelos atributos. Isso mostra que "não usar o atributo sensível" **não basta**.

Há remédios, que operam em momentos diferentes: tratar os **dados** (reamostrar, corrigir rótulos), alterar o **treino** (restrições de justiça) ou ajustar a **saída** (limiares por grupo). Veja o efeito de limiares diferentes por grupo para

igualar a revocação. Atenção ao procedimento: os limiares são **ajustados na validação** e só depois **avaliados no teste**, que não participa da escolha.

```python
# continuação (usa 'va', 'te', 'tabela' e 'np' do exemplo anterior)
def limiar_para_revocacao(d, alvo):
    positivos = d.loc[d["y"] == 1, "p"].to_numpy()
    return float(np.quantile(positivos, 1 - alvo))            # limiar que captura 'alvo' dos positivos

alvo = 0.80
limiares = {g: limiar_para_revocacao(d, alvo) for g, d in va.groupby("grupo")}   # ajuste na VALIDAÇÃO
print("Limiares por grupo (da validação):", {g: round(v, 3) for g, v in limiares.items()})
te2 = te.assign(sel=[p >= limiares[g] for p, g in zip(te["p"], te["grupo"])])
for g, d in te2.groupby("grupo"):
    print(f"grupo {g}: taxa de seleção = {d['sel'].mean():.3f} | revocação = {d.loc[d['y']==1, 'sel'].mean():.3f}"
          f" | precisão = {d.loc[d['sel'], 'y'].mean():.3f}")
```

No teste, as revocações ficam **próximas** (cerca de 0,77 e 0,81), e não idênticas: é o esperado quando os limiares são ajustados em uma amostra e avaliados em outra. Igualar a revocação (aqui, medida em dados de teste que não participaram da escolha dos limiares) melhora a igualdade de oportunidade, mas altera outras métricas. Um rótulo observado representa o critério definido pelo problema (por exemplo, "pagou o empréstimo"), e não um "merecimento" moral; esse critério deve ser questionado e documentado. Há resultados teóricos mostrando que, em geral, **não é possível satisfazer todos os critérios de justiça ao mesmo tempo** (BAROCAS; HARDT; NARAYANAN, 2023). A escolha do critério é uma **decisão social e jurídica**, e não apenas técnica; deve envolver as pessoas afetadas, o jurídico e o negócio.

10.5 Privacidade

Já vimos a minimização e a pseudonimização (Capítulo 3). Mas modelos também podem **vazar** informação sobre os dados de treino. Shokri e colegas (2017) demonstraram **ataques de inferência de associação**: dado um modelo e um registro, o atacante tenta descobrir se aquela pessoa estava no treino.

10.5.1 Privacidade diferencial

A **privacidade diferencial** (DWORK et al., 2006; DWORK; ROTH, 2014) é uma garantia matemática: as **probabilidades dos possíveis resultados** da análise devem mudar pouco (de forma limitada por ε) quando se comparam duas bases que diferem apenas pela presença de uma pessoa, as chamadas **bases vizinhas**. Isso é conseguido adicionando **ruído** calibrado. O parâmetro **ε** (épsilon) controla o equilíbrio: **ε pequeno** = mais privacidade e mais ruído; **ε grande** = menos privacidade e mais precisão.

O **mecanismo de Laplace** para uma contagem: como a presença ou ausência de uma pessoa altera a contagem em, no máximo, 1, somamos ao resultado um ruído da distribuição de Laplace com escala 1/ε.

```python
import numpy as np

rng = np.random.default_rng(0)
contagem_real = 1250        # por exemplo: quantos clientes têm uma doença

for eps in [0.01, 0.1, 1.0, 10.0]:
    ruidos = rng.laplace(loc=0, scale=1 / eps, size=10000)
    erro_medio = np.abs(ruidos).mean()
```

```
    exemplo = contagem_real + rng.laplace(0, 1 / eps)
    print(f"ε = {eps:5.2f} -> erro absoluto médio ≈ {erro_medio:8.1f} | exemplo de resposta divulgada:
{exemplo:9.1f}")
```

Quanto menor o ε, maior o erro: privacidade tem custo em precisão. Para modelos complexos há técnicas de treino com ruído, mas o raciocínio é o mesmo.

10.5.2 Aprendizado federado

No **aprendizado federado** (MCMAHAN et al., 2017), os dados **ficam nos dispositivos** (por exemplo, celulares): cada um treina localmente e envia apenas atualizações do modelo, que são combinadas por um servidor central. É útil quando os dados são sensíveis ou não podem sair de onde estão. Não elimina todos os riscos, e costuma ser combinado com outras proteções.

10.6 Segurança do ML

Além do que vimos no serviço (Capítulo 7), há ameaças específicas:

- **Exemplos adversariais:** entradas ligeiramente alteradas para enganar o modelo.
- **Envenenamento de dados:** inserir dados maliciosos no treino.
- **Roubo de modelo:** reconstruir o modelo por consultas repetidas.
- **Cadeia de suprimentos:** bibliotecas e arquivos de modelo de origem duvidosa (lembre-se do alerta sobre `pickle`).

Mitigações incluem validar dados de entrada e treino, limitar consultas, assinar e verificar artefatos, e testar a robustez.

10.7 Rastreabilidade e auditoria

Para auditar uma decisão, precisamos reconstruir o caminho: **qual versão do modelo, treinada com quais dados e qual código, produziu esta saída com esta entrada**. Isso exige juntar as peças que construímos:

Peça	Onde foi vista
Hash/versão dos dados	Capítulo 3
Pipeline e configuração versionados	Capítulo 4
Execução, parâmetros e métricas	Capítulo 5
Modelo registrado e aprovado	Capítulos 5 e 8
Registro de cada previsão	Capítulo 9
Cartão do modelo e decisão de governança	Este capítulo

10.8 Armazém de atributos (feature store)

Em empresas com muitos modelos, equipes diferentes recriam os mesmos atributos (por exemplo, "gasto médio nos últimos 30 dias") de maneiras ligeiramente diferentes. Isso causa retrabalho e, pior, **desvio treino–serviço**. O **armazém de atributos** resolve guardando atributos **calculados uma única vez e reutilizados**, com duas faces:

- **Armazém offline:** histórico grande, usado para **treinar** e para previsões em lote.

- **Armazém online:** valores mais recentes, acessados em milissegundos para o **serviço em tempo real**.

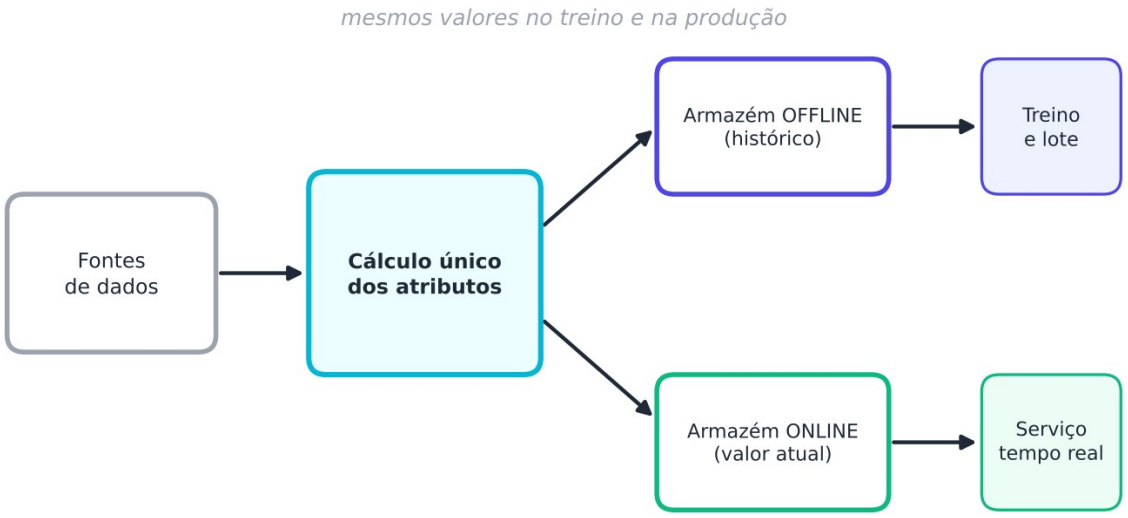


Figura 10.3 — Armazém de atributos: um único cálculo alimenta o treino (offline) e a produção (online), garantindo que ambos usem os mesmos valores.

10.8.1 A armadilha temporal

Ao montar o conjunto de treino, cada exemplo deve usar apenas os valores dos atributos **como eles eram naquele momento**, e nunca valores de depois. É a **correção temporal** (*point-in-time correctness*). Errar isso é um vazamento de dados (Capítulo 3) muito comum. No pandas, a função `merge_asof` faz a junção "pelo valor mais recente até a data do evento":

```python
import pandas as pd

# eventos: cada linha é um cliente em uma data em que precisamos prever
eventos = pd.DataFrame({
    "cliente": [1, 1, 2],
    "data": pd.to_datetime(["2026-03-10", "2026-05-20", "2026-04-15"]),
    "cancelou": [0, 1, 0],
}).sort_values("data")

# atributos calculados em datas diferentes (histórico do armazém)
atributos = pd.DataFrame({
    "cliente": [1, 1, 1, 2, 2],
    "data_calculo": pd.to_datetime(["2026-03-01", "2026-04-01", "2026-05-01", "2026-03-15", "2026-04-10"]),
    "gasto_30d": [100.0, 80.0, 20.0, 300.0, 310.0],
}).sort_values("data_calculo")

correto = pd.merge_asof(eventos, atributos, left_on="data", right_on="data_calculo",
                        by="cliente", direction="backward")
print(correto[["cliente", "data", "data_calculo", "gasto_30d", "cancelou"]])

# ERRADO: usar sempre o valor mais recente do cliente (vaza o futuro)
ultimo = atributos.groupby("cliente")["gasto_30d"].last().rename("gasto_mais_recente")
errado = eventos.merge(ultimo, on="cliente")
print(errado[["cliente", "data", "gasto_mais_recente"]])
```

Veja o cliente 1 em 10/03: o valor correto é o de 01/03 (100), mas a versão errada usaria o de 01/05 (20), um dado que **ainda não existia**.

10.9 Orquestração de fluxos de trabalho

Um pipeline de MLOps tem muitas tarefas: ingerir dados, validar, treinar, avaliar, registrar, implantar. Algumas dependem de outras. O **orquestrador** é o "maestro": executa as tarefas **na ordem certa**, repete as que falharem, agenda execuções e guarda o histórico.

As tarefas e suas dependências formam um **DAG** (grafo acíclico dirigido). Ferramentas conhecidas incluem Apache Airflow, Prefect, Dagster, Kubeflow Pipelines e Argo Workflows. A ideia central cabe em poucas linhas:

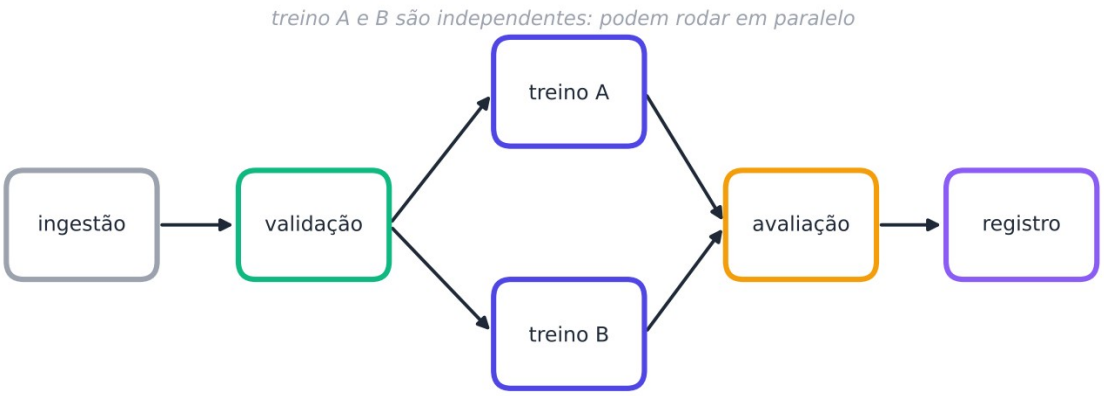


Figura 10.4 — Um DAG de MLOps: setas indicam dependências; tarefas independentes (como treinar dois modelos) podem rodar em paralelo.

```python
from graphlib import TopologicalSorter

def tarefa(nome, falhar_na_tentativa=0):
    contador = {"n": 0}
    def executar():
        contador["n"] += 1
        if contador["n"] <= falhar_na_tentativa:
            raise RuntimeError(f"falha temporária em {nome}")
        return f"{nome}: ok (tentativa {contador['n']})"
    return executar

TAREFAS = {
    "ingestao": tarefa("ingestao"),
    "validacao": tarefa("validacao", falhar_na_tentativa=1),     # falha uma vez, depois passa
    "treino_modelo_A": tarefa("treino_modelo_A"),
    "treino_modelo_B": tarefa("treino_modelo_B"),
    "avaliacao": tarefa("avaliacao"),
    "registro": tarefa("registro"),
}
DEPENDENCIAS = {                        # tarefa -> conjunto de tarefas que precisam rodar antes
    "validacao": {"ingestao"},
    "treino_modelo_A": {"validacao"},
    "treino_modelo_B": {"validacao"},
    "avaliacao": {"treino_modelo_A", "treino_modelo_B"},
    "registro": {"avaliacao"},
```

```
}

def orquestrar(tarefas, dependencias, tentativas=3):
    ordem = list(TopologicalSorter(dependencias).static_order())
    for nome in ordem:
        for t in range(1, tentativas + 1):
            try:
                print(" ", tarefas[nome]())
                break
            except RuntimeError as erro:
                print(f"  ! {erro}; repetindo ({t}/{tentativas})")
        else:
            raise SystemExit(f"tarefa {nome} falhou; pipeline interrompido")

print("Ordem de execução:")
orquestrar(TAREFAS, DEPENDENCIAS)
```

Um orquestrador real faz muito mais (agendamento, paralelismo, alertas, painel, histórico, execução distribuída), mas o princípio é este: **dependências + repetição + registro**.

10.10 Plataforma, custos e a decisão "construir ou comprar"

Em escala, surgem decisões sobre infraestrutura:

- **Nuvem ou local?** Elasticidade e serviços gerenciados *versus* controle e custo previsível.
- **Construir ou comprar?** Construir dá controle; comprar economiza tempo. Evite reinventar o que já existe e é mantido por uma comunidade.
- **Custo.** Treinar e servir consomem CPU, memória, GPU e energia. Monitore **custo por previsão** e **custo por treino**; Hazelwood e colegas (2018) descrevem, para a infraestrutura de ML de uma grande empresa, como a demanda por recursos influencia o projeto dos sistemas.
- **Equipe.** Uma plataforma exige pessoas para operá-la. Muitas vezes, uma solução simples e bem automatizada vence uma plataforma sofisticada que ninguém consegue manter.

> **Dica**
>
> Comece **pequeno**. Escolha a menor solução que resolve a dor atual, automatize o que é repetitivo e só adicione componentes (armazém de atributos, orquestrador distribuído) quando a necessidade for clara. Retome a escada de maturidade do Capítulo 2.

10.11 LLMOps: operando modelos de linguagem

Modelos de linguagem de grande porte (LLMs), baseados na arquitetura *Transformer* (VASWANI et al., 2017) e com capacidade de aprender a partir de poucos exemplos no próprio comando (BROWN et al., 2020), criaram uma variação do MLOps, chamada informalmente de **LLMOps**. Muita coisa continua igual (versionar, testar, monitorar, governar), mas há diferenças:

Aspecto	MLOps clássico	LLMOps
O "programa"	Código + dados + modelo treinado	**Prompts**, modelo base, ferramentas e contexto
Adaptação	Treinar do zero ou *fine-tuning*	Engenharia de prompts, **RAG**, *fine-tuning* leve
Saída	Número ou classe, fácil de comparar	**Texto livre**, difícil de avaliar

Aspecto	MLOps clássico	LLMOps
Avaliação	Métricas objetivas (F1, AUC)	Conjuntos de testes de comportamento, revisão humana, "juízes" automáticos
Custo	Treino e hospedagem	Custo **por token**, latência, tamanho do contexto
Riscos	Deriva, viés	Alucinações, vazamento de dados, injeção de prompt, conteúdo inadequado

O **RAG** (*Retrieval-Augmented Generation*; LEWIS et al., 2020) consiste em **buscar** documentos relevantes em uma base e inseri-los no comando antes de gerar a resposta, reduzindo invenções e permitindo atualizar o conhecimento sem retreinar. Técnicas de ajuste leve, como o **LoRA** (HU et al., 2022), adaptam o modelo treinando poucos parâmetros adicionais, o que diminui custo e tempo.

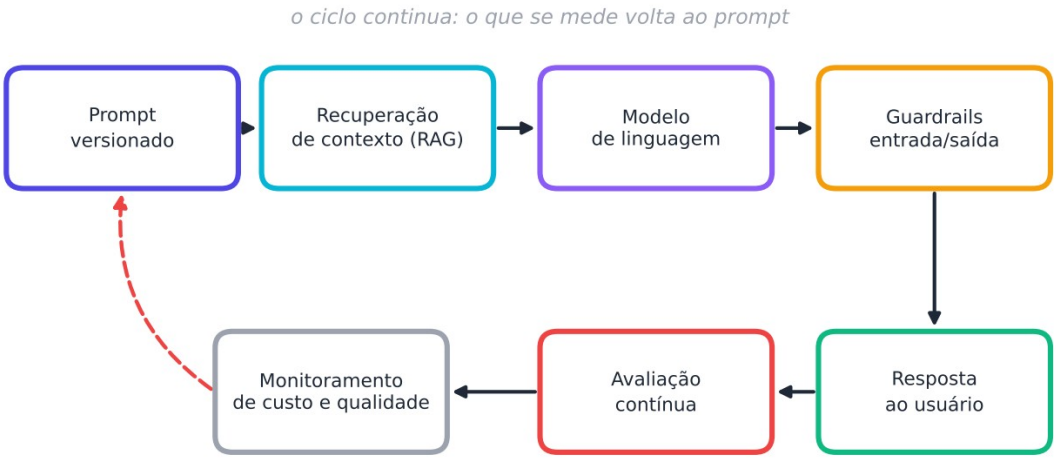


Figura 10.5 — Ciclo de LLMOps: prompt versionado, recuperação de contexto (RAG), modelo, proteções (guardrails), avaliação contínua e monitoramento de custo e qualidade.

O que leva do MLOps ao LLMOps, em prática:

- **Versione os prompts** como código, com identificador e histórico.
- **Teste os prompts** com um conjunto fixo de casos e critérios (regressão de prompt).
- **Monitore** custo, latência, taxa de recusa, reclamações e amostras revisadas por humanos.
- **Proteja:** filtros de entrada e saída, limites de uso, tratamento de dados sensíveis.
- **Registre** os *traces*: prompt, contexto recuperado, resposta e versão do modelo.

O exemplo abaixo usa um **"modelo" simulado** (uma função com regras fixas) apenas para mostrar a mecânica de versionar prompts e rodar testes de regressão. Em um projeto real, a função `chamar_modelo` chamaria um serviço de LLM.

```python
import hashlib

PROMPTS = {}

def registrar_prompt(nome, texto):
    versao = hashlib.sha256(texto.encode()).hexdigest()[:8]
    PROMPTS[(nome, versao)] = texto
```

```
    return versao

def chamar_modelo(prompt, pergunta):
    """MODELO SIMULADO: responde por regras fixas, só para demonstrar o teste."""
    if "formal" in prompt:
        return "Prezado cliente, seu pedido foi enviado." if "pedido" in pergunta else "Não sei
informar."
    return "oi! seu pedido saiu :)" if "pedido" in pergunta else "sei lá"

CASOS = [   # (pergunta, deve conter, não pode conter)
    ("Onde está meu pedido?", "pedido", ":)"),
    ("Qual a cor do céu?", "Não sei", "sei lá"),
]

def avaliar(nome, versao):
    prompt = PROMPTS[(nome, versao)]
    ok = 0
    for pergunta, deve, nao in CASOS:
        resposta = chamar_modelo(prompt, pergunta)
        ok += (deve in resposta) and (nao not in resposta)
    return ok / len(CASOS)

v1 = registrar_prompt("atendimento", "Responda de forma curta e descontraída.")
v2 = registrar_prompt("atendimento", "Responda em tom formal e educado.")
for v in (v1, v2):
    print(f"prompt 'atendimento' versão {v}: {avaliar('atendimento', v):.0%} dos casos de teste")
```

> **Atenção**
>
> Avaliar texto livre é difícil. Usar outro LLM como "juiz" é comum, mas esse juiz também erra e tem vieses. Combine **testes automáticos**, **amostras revisadas por pessoas** e **métricas de negócio**. As práticas de LLMOps ainda evoluem rapidamente: leia a documentação e os artigos mais recentes das ferramentas que usar.

10.12 Olhando adiante

Algumas tendências que acompanham o MLOps:

- **AutoML** e busca automática de modelos (HUTTER; KOTTHOFF; VANSCHOREN, 2019), reduzindo trabalho manual, mas exigindo governança.
- **IA centrada em dados:** investir na qualidade dos dados e dos rótulos (WHANG et al., 2023).
- **Regulação:** exigências crescentes de transparência, avaliação de risco e supervisão humana.
- **Eficiência:** modelos menores, quantização e hardware especializado para reduzir custo e consumo de energia.
- **Plataformas unificadas** de dados e ML, reduzindo a distância entre as equipes.

10.13 Resumo do capítulo

MEMORIZE **O que levar deste capítulo**

- **Governança** reúne documentação, explicabilidade, justiça, privacidade, segurança e auditoria, apoiadas por rastreabilidade.
- O **cartão do modelo** informa uso pretendido, dados, métricas por grupo e limitações; pode ser gerado automaticamente.
- **Explicações** (permutação, PDP, LIME, SHAP) mostram o que o modelo usa, não causas.
- **Justiça** exige medir **por grupo**; não usar o atributo sensível não basta; os critérios podem conflitar e a escolha é social.
- **Privacidade diferencial** troca precisão por privacidade (ε); o **aprendizado federado** mantém os dados nos dispositivos.
- O **armazém de atributos** evita desvio treino–serviço; a **correção temporal** evita vazamento do futuro.
- **Orquestradores** executam DAGs com dependências, repetição e registro.
- **LLMOps** acrescenta prompts versionados, RAG, avaliação de texto livre, custo por token e novos riscos.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Governança de ML é:

a) o conjunto de políticas, papéis e práticas para uso responsável, transparente, justo, seguro e em conformidade.
b) a formatação do código.
c) a compra de servidores.
d) a escolha do algoritmo mais rápido.

Questão 2. O cartão do modelo (model card), proposto por Mitchell et al. (2019), contém:

a) apenas o nome do autor.
b) apenas o preço do projeto.
c) uso pretendido, dados, métricas (inclusive por grupo) e limitações.
d) apenas o código-fonte.

Questão 3. Por que gerar o cartão do modelo automaticamente?

a) Reduz o custo do servidor.
b) Torna o modelo mais preciso.
c) Dispensa a revisão humana.
d) Evita que fique desatualizado em relação aos metadados do pipeline.

Questão 4. Explicação global de um modelo responde a:

a) quem treinou o modelo.
b) o que, em geral, importa para o modelo.
c) por que uma previsão específica saiu assim.
d) qual servidor foi usado.

Questão 5. A importância por permutação mede:

a) o custo de coletar o atributo.
b) a correlação entre o atributo e o tempo.
c) o número de vezes que o atributo aparece no código.
d) a queda de desempenho ao embaralhar um atributo.

Questão 6. Quando dois atributos são muito correlacionados, a importância por permutação:

a) fica sempre maior.
b) torna-se igual a zero para ambos, sempre.
c) tende a se dividir entre eles, subestimando cada um.
d) não é afetada.

Questão 7. A dependência parcial (PDP) mostra:

a) o desempenho em cada grupo.
b) a latência de cada atributo.
c) a correlação entre dois modelos.
d) o efeito médio de um atributo sobre a previsão.

Questão 8. LIME explica previsões:

a) treinando uma rede maior.
b) aproximando o modelo, perto de um caso, por um modelo simples.
c) calculando o hash dos dados.
d) removendo atributos do conjunto.

Questão 9. SHAP (Lundberg e Lee, 2017) distribui a previsão entre os atributos com base em:

a) sorteios aleatórios.
b) valores de Shapley.
c) médias aritméticas simples.
d) ordens alfabéticas.

Questão 10. Explicações de importância e dependência mostram:

a) o que o modelo usa, e não necessariamente causas no mundo real.
b) a verdade absoluta sobre o fenômeno.
c) que o modelo é justo.
d) sempre relações causais.

Questão 11. Uma razão de impacto desproporcional é:

a) a soma das taxas de seleção.
b) o número de atributos sensíveis.
c) o tempo de resposta por grupo.
d) a taxa de seleção do grupo com menos aprovações dividida pela do grupo com mais.

Questão 12. Igualdade de oportunidade (Hardt et al., 2016) significa:

a) revocação (TPR) semelhante entre os grupos.
b) taxa de seleção idêntica.
c) precisão idêntica.
d) mesmo número de exemplos por grupo.

Questão 13. Retirar o atributo sensível (grupo) do modelo:

a) não garante justiça, pois proxies podem carregar o viés.
b) garante justiça.
c) aumenta a precisão.
d) elimina a necessidade de auditoria.

Questão 14. Segundo o capítulo, em geral é impossível:

a) calcular métricas por grupo.
b) auditar modelos.
c) treinar modelos justos.
d) satisfazer simultaneamente todos os critérios de justiça.

Questão 15. A LGPD, no Brasil, trata:

a) da licença do Python.
b) da compra de hardware.
c) da velocidade da internet.
d) do uso de dados pessoais, inclusive da revisão de decisões automatizadas.

Questão 16. O Regulamento de IA da União Europeia classifica sistemas:

a) pelo preço do software.
b) por nível de risco e impõe obrigações aos de alto risco.
c) por tamanho do modelo apenas.
d) pela linguagem de programação.

Questão 17. A privacidade diferencial garante que:

a) o modelo seja perfeito.
b) todos os dados sejam apagados.
c) as probabilidades dos resultados da análise mudem pouco, de forma limitada por ε, entre bases vizinhas (com e sem uma pessoa).
d) o servidor nunca falhe.

Questão 18. No parâmetro ε da privacidade diferencial, um ε pequeno significa:

a) dados idênticos aos originais.
b) menos privacidade e menos ruído.
c) mais privacidade e mais ruído.
d) ausência de ruído.

Questão 19. O mecanismo de Laplace para uma contagem soma ao resultado ruído de escala:

a) ε^2.
b) zero.
c) $100 \cdot \varepsilon$.
d) $1/\varepsilon$.

Questão 20. Repetir a mesma consulta com privacidade diferencial e tirar a média:

a) recupera precisão, mas consome o orçamento de privacidade.
b) é proibido pela LGPD.
c) aumenta o ruído.
d) preserva a privacidade total.

Questão 21. No aprendizado federado (McMahan et al., 2017):

a) a privacidade é garantida por completo.
b) os dados ficam nos dispositivos e só atualizações do modelo são enviadas.
c) não há modelo.
d) todos os dados são enviados a um servidor central.

Questão 22. O armazém de atributos (feature store) tem uma face offline e uma online. A online serve para:

a) rodar testes de CI.
b) guardar histórico gigante para treino.
c) fornecer valores recentes em milissegundos ao serviço em tempo real.
d) armazenar o código-fonte.

Questão 23. Correção temporal (point-in-time) significa:

a) ignorar datas.
b) usar sempre o valor mais recente.
c) usar os valores dos atributos como eram na data do evento, sem informação futura.
d) usar o valor médio histórico.

Questão 24. No pandas, qual função faz a junção pelo valor mais recente disponível até a data do evento?

a) merge_asof.
b) concat.
c) get_dummies.
d) pivot.

Questão 25. Um DAG de MLOps é:

a) um grafo de tarefas com dependências e sem ciclos.
b) um tipo de modelo de regressão.
c) um arquivo de configuração de rede.
d) um painel de gráficos.

Questão 26. Qual é uma função do orquestrador além de executar na ordem?

a) Substituir o monitoramento.
b) Escolher a cor dos gráficos.
c) Repetir tarefas que falharem, agendar execuções e guardar histórico.
d) Treinar o modelo sem dados.

Questão 27. O que é RAG (Lewis et al., 2020)?

a) Buscar documentos relevantes e inseri-los no comando antes de gerar a resposta.
b) Uma técnica de limpeza de dados.
c) Um formato de arquivo.
d) Um algoritmo de compressão de imagens.

Questão 28. No LLMOps, o "programa" é composto principalmente de:

a) apenas código C.
b) somente imagens.
c) apenas planilhas.
d) prompts, modelo base, ferramentas e contexto.

Questão 29. Avaliar texto livre é difícil porque:

a) o texto não pode ser lido por computadores.
b) os modelos de linguagem não geram texto.
c) não existem conjuntos de teste.
d) não há uma resposta única e métricas objetivas simples.

Questão 30. O LoRA (Hu et al., 2022) é uma técnica para:

a) adaptar modelos treinando poucos parâmetros adicionais.
b) validar dados.
c) criar painéis.
d) comprimir imagens.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Descreva os pilares da governança de ML e dê um exemplo de prática em cada um.
2. O que deve constar em um cartão do modelo? Por que incluir desempenho por grupo?
3. Diferencie explicação global e local e cite uma técnica de cada tipo.
4. Por que explicações não devem ser interpretadas como causas? Dê um exemplo.
5. Explique por que remover o atributo sensível não garante justiça e o que se pode fazer em seguida.
6. Compare paridade demográfica, igualdade de oportunidade e paridade de precisão.
7. Descreva o ajuste de limiares por grupo para igualar a revocação e discuta suas consequências.
8. Explique o que é privacidade diferencial e o papel de ε.
9. O que é um ataque de inferência de associação e como mitigá-lo?
10. Explique o aprendizado federado e uma limitação.
11. Cite quatro ameaças de segurança específicas de ML e uma mitigação para cada.
12. Explique como rastreabilidade e auditoria dependem das peças construídas ao longo do livro.
13. Qual o problema que o armazém de atributos resolve e como funciona sua divisão offline/online?
14. Explique a armadilha temporal na montagem do conjunto de treino e como merge_asof ajuda.
15. Descreva a função de um orquestrador e o que significa uma tarefa depender de outra em um DAG.
16. Discuta a decisão "construir ou comprar" uma plataforma de ML.
17. Dê três diferenças entre MLOps clássico e LLMOps.
18. Como versionar e testar prompts em um projeto de LLM?
19. Por que o uso de outro LLM como juiz tem limitações?
20. Quais recomendações finais você daria a uma equipe que quer colocar um modelo de crédito em produção com responsabilidade?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Cartão do modelo gerado automaticamente

NÍVEL: BÁSICO

Treine uma regressão logística no conjunto *Breast Cancer* (UCI) e escreva uma função que gere um cartão do modelo em texto (Markdown) com: nome e versão, uso pretendido e fora do escopo, descrição dos dados, métricas gerais (acurácia, F1, revocação da classe maligna), métricas por faixa de tamanho médio do tumor (`mean radius` abaixo e acima da mediana) e limitações. Salve em arquivo e mostre as primeiras linhas.

Exercício 2 — Explicando o modelo: permutação, PDP e a armadilha da correlação

NÍVEL: INTERMEDIÁRIO

No conjunto *Breast Cancer* (UCI), treine uma floresta aleatória e calcule a importância por permutação dos 5 principais atributos no teste. Depois, acrescente ao conjunto uma cópia quase idêntica do atributo mais importante (com ruído mínimo) e recalcule: o que acontece com a importância dos dois?

Exercício 3 — Auditoria de justiça e ajuste de limiares

NÍVEL: INTERMEDIÁRIO

Simule 20.000 candidatos a crédito de dois grupos (A, 70%; B, 30%) com a mesma habilidade real, mas com um atributo medido com viés no grupo B. Divida em treino, validação e teste (50/25/25). Treine uma regressão logística **sem** o grupo como entrada e calcule, no **teste**, por grupo, taxa de seleção, revocação (TPR) e precisão. Reporte a razão de impacto. Depois, escolha limiares por grupo que levem a revocação a 80% **na validação**, aplique-os no teste e recalcule.

Exercício 4 — Privacidade diferencial: o custo do ruído

NÍVEL: AVANÇADO

Considere 1.000 pessoas com a característica de interesse em 300 delas. Divulgue a contagem com o mecanismo de Laplace (sensibilidade 1) para ε = 0,05; 0,5 e 5. Para cada ε, repita 2.000 vezes e informe o erro absoluto médio. Depois, mostre o efeito de repetir a mesma consulta 100 vezes e fazer a média dos resultados (o que acontece com a privacidade?).

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Statlog (German Credit Data)*, *Adult (Census Income)* (atributos sensíveis, ótimo para estudar justiça) e *Default of Credit Card Clients*.
> - **Kaggle:** *Give Me Some Credit* e *Home Credit Default Risk* para repetir a auditoria. Lembre-se de verificar a licença e as condições de uso antes de publicar resultados.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Auditoria de um modelo de crédito: documentação, explicação, justiça e privacidade

Contexto. Você foi chamado para auditar um modelo de aprovação de crédito antes da implantação. Entregue um dossiê com cartão do modelo, explicações, avaliação de justiça, uma divulgação privada de estatísticas e verificação de correção temporal dos atributos.

Dados simulados. Gere 15.000 solicitantes com `renda`, `idade`, `tempo_emprego`, `dividas` e `grupo` (A: 65%, B: 35%). O alvo "bom pagador" depende de renda, dívidas e tempo de emprego, mas a renda é registrada com erro maior e viés para baixo no grupo B. Defina uma semente e documente.

Tarefas.

1. Treine uma floresta aleatória **sem** `grupo` e avalie no teste: AUC-ROC, F1 e revocação dos bons pagadores, no geral e por grupo.
2. Calcule a razão de impacto (B/A) das aprovações e a diferença de revocação entre os grupos.
3. Explique o modelo: importância por permutação (global) e, para um solicitante do grupo B reprovado, a variação de probabilidade quando cada atributo é trocado pela mediana (explicação local simples).
4. Divulgue de forma privada (Laplace, ε = 0,5) o número de aprovados em cada grupo e compare com o valor real.
5. Faça uma junção temporal correta (`merge_asof`) de um atributo "dívidas_30d" calculado em datas diferentes e compare com a junção ingênua (valor mais recente), mostrando a diferença em pelo menos um caso.
6. Gere o cartão do modelo e escreva a recomendação (aprovar, aprovar com condições ou não aprovar) com justificativas.

Entregáveis. Tabelas de desempenho e justiça, explicações, divulgação privada, demonstração do erro temporal, cartão e recomendação.

A solução de referência está no Apêndice E.

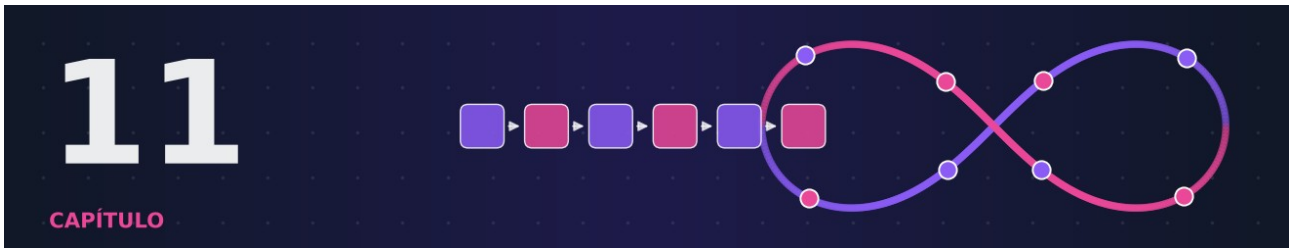
PARTE V

Parte V — Projeto Final

Um sistema completo de ponta a ponta: dados validados, pipeline, rastreamento, serviço, testes, monitoramento e relatório de governança.

Nesta parte:

- Cap. 11 — Projeto Final: do Dado à Produção



Capítulo 11 — Projeto Final: do Dado à Produção

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Planejar um sistema de ML de ponta a ponta, da pergunta de negócio ao relatório de governança.
- Integrar validação de dados, pipeline, rastreamento, registro, API, testes, monitoramento e retreinamento.
- Simular a operação em produção, com deriva e rótulos atrasados.
- Decidir com evidências quando promover, manter ou reverter um modelo.
- Documentar o sistema com cartão de modelo e relatório para diferentes públicos.

11.1 Chegou a hora de montar a fábrica inteira

Ao longo do livro, você aprendeu peças separadas: dados confiáveis, pipelines, rastreamento, registro, API, testes, monitoramento e governança. É como aprender a usar cada máquina de uma cozinha industrial. Agora você vai **abrir a fábrica** e fazê-la funcionar por várias semanas.

Um projeto real raramente é só um modelo. Ele tem **uma pergunta de negócio**, **dados com defeitos**, **decisões de engenharia**, **mudanças no mundo** e **pessoas que precisam confiar no resultado**.



Figura 11.1 — As nove etapas do projeto final e os capítulos que sustentam cada uma.

11.2 O cenário

Uma empresa de logística recebe milhares de formulários por dia, com **dígitos manuscritos** (códigos de pedido). Um sistema de leitura automática classifica cada dígito de 0 a 9. A equipe de dados desenvolveu um primeiro modelo, mas ninguém sabe se ele vai continuar bom quando a câmera da esteira, que hoje é nítida, começar a **perder o foco** e a **desbotar a tinta** das imagens.

> *"Como colocar este modelo em produção de modo que ele seja confiável hoje e continue confiável nos próximos meses?"*

Repare que a pergunta não menciona algoritmos. Ela fala de **confiança ao longo do tempo**.

Para o projeto, usamos as imagens do repositório **UCI Machine Learning** (*Optical Recognition of Handwritten Digits*, a parte de teste, com 1.797 imagens de 8 × 8 pixels, que vem embutida no scikit-learn). A **produção** e a **deriva** são **simuladas** por nós, de forma controlada, para que você consiga repetir os resultados.

> **Memorize**
>
> Primeiro defina a decisão que será apoiada e o que significa "dar certo ao longo do tempo". Só depois escolha as ferramentas.

11.3 Visão geral das nove etapas

Etapa	O que fazer	Capítulos
1. Dados	Validar esquema e faixas, versionar por hash, separar treino/teste/"futuro"	3
2. Candidatos	Linha de base e modelos concorrentes; busca de hiperparâmetros; validação cruzada	4, 6
3. Rastreamento e registro	Registrar cada execução no MLflow; portão de qualidade; apelido `champion`	5
4. Serviço	API com contrato de entrada que carrega o `champion` pelo apelido	7
5. Testes	Testes de desempenho mínimo, robustez, cobertura de classes e determinismo	8
6. Produção simulada	Lotes semanais com degradação progressiva da imagem	9
7. Monitoramento	PSI, classificador de domínio, alertas; desempenho com rótulos atrasados	9
8. Retreinamento	Treinamento contínuo com portão e promoção do novo `champion`	5, 8, 9
9. Governança	Cartão do modelo, desempenho por classe, relatório e recomendações	10

11.4 A arquitetura do projeto

Mantenha o projeto organizado como no Capítulo 1:

```bash
projeto_mlops/
├── data/              # dados originais e lotes simulados de produção
├── src/
│   ├── dados.py       # validação, hash e divisão
```

```
│   ├── treino.py       # candidatos, busca e registro
│   ├── servir.py       # API com contrato
│   ├── monitorar.py    # PSI, classificador de domínio, alertas
│   └── retreinar.py    # treinamento contínuo com portão
├── tests/              # testes de código, dados e modelo
├── mlflow.db           # rastreamento e registro (SQLite)
├── requirements.txt    # versões fixas
├── Dockerfile          # (opcional) empacotamento da API
└── relatorio.md        # relatório final e cartão do modelo
```

O fluxo, em resumo:

1. Os dados passam pelo **portão de validação**.
2. Os candidatos são treinados e **registrados**; o melhor passa pelo **portão de qualidade** e vira `champion`.
3. A **API** carrega o `champion` pelo apelido.
4. A cada semana, o **monitoramento** compara o lote recebido com a referência.
5. Quando o alerta dispara e os rótulos chegam, o **treinamento contínuo** treina um candidato, compara-o com o campeão em dados recentes e, se for melhor por uma margem, **promove** (move o apelido).
6. Depois da **recarga do serviço** (a API recarrega o modelo apontado pelo apelido), ela passa a servir o novo modelo **sem editar o código**, e a versão servida é conferida.

11.5 Etapa por etapa

11.5.1 Dados e validação

Escreva uma função de validação com, no mínimo: número de colunas, ausência de valores nulos, faixa dos pixels (0 a 16) e presença das dez classes. Calcule o **hash** dos dados e registre-o. Separe os dados em três partes:

- **Desenvolvimento:** treino e teste (o teste só será usado uma vez para o relatório).
- **Futuro:** 40% dos dados, reservados para a produção simulada.

11.5.2 Candidatos

Compare, com validação cruzada estratificada:

- uma **linha de base** (classe mais frequente);
- regressão logística;
- floresta aleatória;
- máquina de vetores de suporte (SVC) com **busca aleatória** de hiperparâmetros.

Use como métrica o **F1 macro** (média do F1 de cada classe), que trata as dez classes com igual importância.

> **Dica**
>
> Nem todo pipeline precisa de padronização. Aqui todos os atributos são pixels com a mesma unidade (0 a 16), e padronizar colunas quase constantes (como as bordas da imagem) pode **amplificar ruído**. Decida com base no problema, e não por hábito.

11.5.3 Rastreamento, registro e portão de qualidade

Cada candidato é uma **execução** do MLflow, com parâmetros, métrica e a etiqueta do hash dos dados. O melhor é submetido a um **portão**: F1 macro mínimo (piso) **e** ganho sobre a linha de base. Aprovado, o modelo é registrado e recebe o apelido `champion`.

11.5.4 Serviço

A API recebe uma lista de 64 números (os pixels). O **contrato** rejeita tamanho errado e valores fora de 0 a 16 com código 422. A API carrega `models:/digitos@champion` na partida e devolve o dígito previsto **e a versão do modelo**.

11.5.5 Testes

Escreva testes em pytest sobre o artefato do `champion`:

- **Desempenho mínimo** no teste (F1 macro).
- **Invariância**: ruído muito pequeno não deve mudar a maioria das previsões.
- **Cobertura de classes**: nenhuma classe com revocação abaixo de um piso.
- **Determinismo**: duas previsões iguais para a mesma entrada.

11.5.6 Produção simulada

Divida o "futuro" em **8 lotes semanais**. Para simular a perda de foco da câmera, aplique um **desfoque gaussiano** crescente e um leve desbotamento da tinta. A degradação **cresce por 4 semanas e depois se estabiliza**: o mundo entra em um novo regime. Os **rótulos** de cada semana só chegam na semana seguinte.

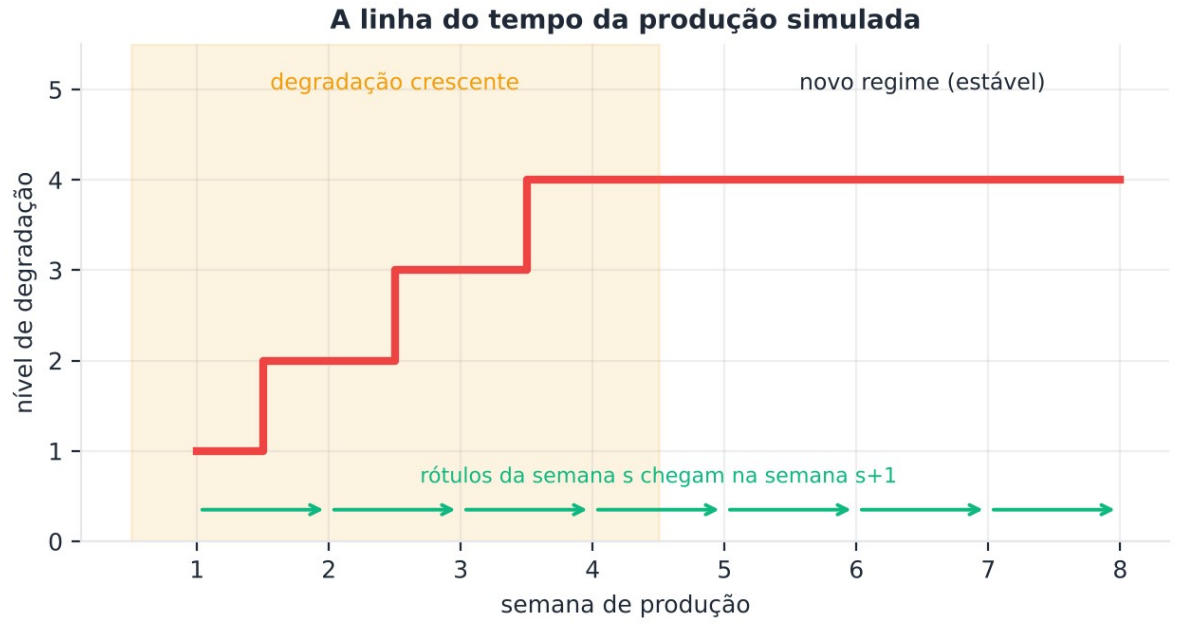


Figura 11.2 — Linha do tempo da produção simulada: a degradação cresce nas semanas 1 a 4 e estabiliza. Os rótulos de uma semana chegam na semana seguinte.

11.5.7 Monitoramento

Para cada semana, calcule:

- o **PSI** do contraste das imagens (desvio padrão dos pixels de cada imagem) contra a referência;
- a **AUC do classificador de domínio** (referência *versus* lote);

- a **acurácia** do modelo, assim que o rótulo chegar.

Defina uma regra de alerta (por exemplo, PSI > 0,25 **ou** AUC > 0,80) e observe **quando** ela dispara em comparação com a queda de acurácia.

11.5.8 Treinamento contínuo

Quando houver alerta **e** rótulos disponíveis, treine um candidato com os dados originais e os lotes rotulados mais antigos (com maior peso para os recentes). Compare-o com o campeão **no lote rotulado mais recente, que o candidato não viu**. Promova apenas se o ganho for maior que uma **margem** (por exemplo, 2 pontos de F1). Depois de promover, **atualize a referência** do monitoramento, para que os alertas passem a comparar com o mundo novo.

11.5.9 Governança e relatório

Gere um **cartão do modelo** com versão, dados, métricas gerais e **por classe**, limitações e uso indicado. Escreva o relatório com os resultados e as recomendações.

11.6 O que esperar dos resultados

Seu número exato dependerá do computador e das versões, mas o **padrão** deve ser semelhante ao da solução de referência (Apêndice E):

- O modelo inicial é muito bom (F1 macro próximo de 0,99 no teste).
- Os **sinais de monitoramento sem rótulo** (PSI e AUC de domínio) disparam **antes** de a acurácia desabar.
- Sem treinamento contínuo, a acurácia cai para menos da metade.
- Com treinamento contínuo, a acurácia **se recupera** depois de duas ou três promoções, e os alertas **se apagam** quando a referência é atualizada.
- O `champion` muda de versão e, depois da recarga do serviço, a API passa a servir o novo modelo sem alteração de código.

> **Atenção: cuidado com as armadilhas**
>
> - **Vazamento temporal:** o candidato do treinamento contínuo **nunca** pode usar rótulos de semanas que ainda não chegaram.
> - **Avaliar no que treinou:** compare campeão e candidato em dados **que nenhum dos dois viu**.
> - **Retreinar sem investigar:** se o PSI dispara porque a imagem veio corrompida, retreinar ensina o erro. Olhe as imagens antes.
> - **Teste de uso único:** o conjunto de teste original serve para o relatório final, e não para escolher o modelo.
> - **Média que esconde:** olhe o desempenho **por classe**.

11.7 O relatório final

Escreva para pessoas diferentes. Uma estrutura sugerida:

1. **Resumo para a diretoria** (5 linhas): decisão recomendada, risco e custo.
2. **Contexto e pergunta de negócio.**
3. **Dados:** origem, validações, versão, limitações.
4. **Modelos:** candidatos, métricas, incerteza e escolha.

5. **Operação:** arquitetura, API, testes e portões.
6. **Monitoramento:** sinais, regras de alerta, o que aconteceu na simulação.
7. **Governança:** cartão do modelo, riscos, justiça entre classes, privacidade, plano de reversão.
8. **Recomendações e próximos passos.**

> **Saiba mais**
>
> Experimente tornar o projeto mais próximo da vida real: empacote a API em um contêiner (Capítulo 7), escreva o fluxo de CI (Capítulo 8), transforme as etapas em um DAG (Capítulo 10) ou repita tudo em outro conjunto de dados, como os sugeridos no quadro "Para ir além".

11.8 Fechamento

Se você chegou até aqui, já conhece o essencial do caminho entre um modelo e um produto confiável. Lembre-se de três ideias:

1. **Modelo bom não é sistema bom.** O sistema inclui dados, código, testes, serviço, monitoramento e pessoas.
2. **O mundo muda.** Projete para a mudança: meça, alerte, retreine com cuidado e esteja pronto para reverter.
3. **Comece simples e evolua.** Automatize a dor mais cara hoje e avance na escada de maturidade.

MLOps é, antes de tudo, uma **cultura de responsabilidade** sobre aquilo que colocamos no mundo. Bom trabalho!

11.9 Resumo do capítulo

> MEMORIZE **O que levar deste capítulo**
>
> - Um projeto de ponta a ponta tem **nove etapas**: dados, candidatos, rastreamento e registro, serviço, testes, produção simulada, monitoramento, treinamento contínuo e governança.
> - Os sinais **sem rótulo** (PSI, AUC de domínio) alertam antes da queda de desempenho; os **rótulos atrasados** confirmam e alimentam o retreino.
> - **Portões** protegem cada passagem: dados, qualidade do modelo e promoção.
> - Compare campeão e candidato em dados **inéditos para ambos** e promova com **margem mínima**.
> - Depois de promover, **atualize a referência** do monitoramento.
> - Documente com **cartão do modelo** e **relatório** claro, incluindo limitações e plano de reversão.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é a pergunta central do projeto final?

a) Como colocar o modelo em produção de modo que seja confiável hoje e continue confiável nos próximos meses?
b) Qual algoritmo tem mais parâmetros?
c) Como treinar o modelo mais rápido possível?
d) Qual a melhor cor para os gráficos do relatório?

Questão 2. No projeto, os dados reais vêm de:

a) um banco de dados de clientes reais.
b) imagens de câmeras da empresa.
c) o conjunto Digits (UCI Optical Recognition of Handwritten Digits), embutido no scikit-learn.
d) uma base paga do Kaggle.

Questão 3. O que se simula para representar a deriva em produção no projeto?

a) A troca do algoritmo por outro.
b) A remoção de classes do conjunto de dados.
c) A queda de energia do servidor.
d) Desfoque da câmera e desbotamento da tinta, crescentes por quatro semanas e depois estáveis.

Questão 4. A primeira etapa do projeto é:

a) escrever o relatório final.
b) validar os dados, calcular o hash e separar desenvolvimento e "futuro".
c) implantar em produção.
d) retreinar o modelo.

Questão 5. Por que reservar 40% dos dados como "futuro"?

a) Para calibrar o limiar.
b) Para treinar o modelo com mais dados.
c) Para simular lotes semanais de produção que o modelo nunca viu.
d) Para escolher os hiperparâmetros.

Questão 6. A métrica principal para comparar candidatos no projeto é:

a) F1 macro em validação cruzada.
b) o número de parâmetros.
c) a latência do servidor.
d) apenas a acurácia no teste.

Questão 7. Quais candidatos o projeto compara?

a) Somente uma rede neural.
b) Linha de base, regressão logística, floresta aleatória e SVC com busca aleatória.
c) Somente a linha de base.
d) Apenas modelos de agrupamento.

Questão 8. Por que o projeto evita padronizar os pixels?

a) Porque aumenta o tamanho do modelo.
b) Padronizar colunas quase constantes pode amplificar ruído, e os pixels já têm a mesma unidade.
c) Porque padronizar é proibido.
d) Porque o SVC não aceita padronização.

Questão 9. O portão de qualidade do projeto exige:

a) que o modelo seja o mais simples possível.
b) aprovação sem critérios.
c) piso de F1 macro e ganho mínimo sobre a linha de base.
d) apenas o menor tempo de treino.

Questão 10. O apelido champion é usado pela API para:

a) guardar a senha.
b) escolher a cor dos gráficos.
c) carregar o modelo vigente pelo apelido, sem editar o código, e recarregar quando a versão muda.
d) desligar o servidor.

Questão 11. O contrato da API do projeto rejeita:

a) tamanho diferente de 64 pixels e valores fora de 0 a 16.
b) apenas imagens em cores.
c) nenhum dado.
d) qualquer imagem.

Questão 12. Os testes em pytest do projeto verificam:

a) desempenho mínimo, invariância a ruído, cobertura de classes e determinismo.
b) somente a latência.
c) apenas a formatação.
d) apenas se o servidor está ligado.

Questão 13. Na simulação, os rótulos de cada semana chegam:

a) com uma semana de atraso.
b) imediatamente.
c) antes dos dados.
d) nunca.

Questão 14. O que o PSI do contraste das imagens mede no projeto?

a) A taxa de erro do modelo.
b) O número de classes.
c) A mudança na distribuição do desvio padrão dos pixels de cada imagem em relação à referência.
d) A latência da API.

Questão 15. Uma regra de alerta sem rótulos do projeto é:

a) PSI do contraste acima de 0,25 ou AUC do classificador de domínio acima de 0,80.
b) latência acima de 1 s.
c) acurácia abaixo de 50%.
d) mais de 10 acessos por minuto.

Questão 16. No treinamento contínuo, o candidato é comparado ao campeão:

a) em dados de treino apenas.
b) no lote rotulado mais recente, que o candidato não viu.
c) no conjunto de teste original, repetidamente.
d) no mesmo lote usado para treiná-lo.

Questão 17. A promoção só ocorre se o candidato superar o campeão por:

a) uma margem mínima de F1 no lote recente.
b) qualquer valor, mesmo zero.
c) um valor negativo.
d) uma margem de 100 pontos.

Questão 18. Depois de promover o novo modelo, o projeto:

a) desliga o monitoramento.
b) apaga o registro de modelos.
c) volta ao modelo inicial.
d) atualiza a referência do monitoramento com os dados recentes.

Questão 19. Qual a lição principal da comparação entre "sem CT" e "com CT"?

a) Não há diferença entre as estratégias.
b) Retreinar sempre piora o modelo.
c) Sem retreino, a acurácia cai muito; com treinamento contínuo, ela se recupera.
d) Só o modelo inicial importa.

Questão 20. Qual cuidado é essencial para evitar vazamento temporal no retreino simulado?

a) Nunca usar rótulos de semanas que ainda não chegaram.
b) Treinar com o teste.
c) Usar todos os dados futuros.
d) Embaralhar as semanas.

Questão 21. O cartão do modelo do projeto inclui:

a) as senhas de acesso.
b) versão, dados, métricas gerais e por classe, limitações e uso indicado.
c) a lista de bibliotecas instaladas apenas.
d) apenas o nome do autor.

Questão 22. Qual a utilidade de olhar o desempenho por classe (dígito)?

a) Evitar o uso de validação cruzada.
b) Descobrir classes que o modelo reconhece mal, apesar de boa média.
c) Reduzir o número de classes.
d) Calcular o hash.

Questão 23. Retreinar quando o PSI dispara por uma imagem corrompida:

a) ensinaria o erro ao modelo; deve-se investigar a causa antes.
b) é proibido pela API.
c) é sempre a melhor solução.
d) não tem consequências.

Questão 24. Qual o papel do conjunto de teste original no projeto?

a) Ser usado uma única vez, para o relatório final.
b) Ser misturado ao treino.
c) Ser usado para escolher hiperparâmetros.
d) Ser apagado.

Questão 25. Por que a produção é simulada de forma controlada?

a) Para que os resultados possam ser repetidos e estudados.
b) Para economizar disco.
c) Porque a produção real é proibida.
d) Para evitar o uso de modelos.

Questão 26. O relatório final deve incluir, entre outros:

a) apenas gráficos.
b) somente a lista de bibliotecas.
c) resumo para a diretoria, dados, modelos, operação, monitoramento, governança e recomendações.
d) apenas o código.

Questão 27. Que extensões o capítulo sugere para tornar o projeto mais próximo da vida real?

a) Trocar o conjunto por dados aleatórios.
b) Remover todos os testes.
c) Desligar o monitoramento.
d) Contêiner para a API, fluxo de CI, DAG de orquestração e repetir em outro conjunto de dados.

Questão 28. Qual das ideias de fechamento resume o livro?

a) Testes são opcionais.
b) Modelo bom não é sistema bom; o mundo muda; comece simples e evolua.
c) A governança só importa em empresas grandes.
d) Basta treinar o modelo uma vez.

Questão 29. Qual alternativa descreve o fluxo resumido do projeto?

a) Treino → fim.
b) API → dados → modelo → relatório.
c) Monitoramento → deleção do modelo.
d) Dados validados → candidatos registrados → portão e champion → API → monitoramento → CT com portão → nova versão servida.

Questão 30. Qual a vantagem de registrar cada candidato do projeto como uma execução do MLflow?

a) Torna os modelos mais rápidos.
b) Dispensa o portão de qualidade.
c) Elimina a necessidade de dados.
d) Permite comparar, auditar e reproduzir as escolhas feitas.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Descreva, em linhas gerais, as nove etapas do projeto final e os capítulos que as sustentam.
2. Explique como funcionam os portões no projeto (dados, qualidade e promoção) e qual risco cada um reduz.
3. Por que o projeto separa desenvolvimento e "futuro" e usa o teste original uma única vez?
4. Descreva o contrato de entrada da API do projeto e as respostas esperadas para entradas válidas e inválidas.
5. Quais testes automáticos o projeto inclui e por que cada um é útil?
6. Como a simulação de produção introduz a deriva e por que a estabilização após quatro semanas é útil para o experimento?
7. Explique por que os sinais sem rótulo (PSI e AUC de domínio) disparam antes da queda de acurácia visível.
8. Descreva a lógica do treinamento contínuo implementado no projeto, incluindo a espera de rótulos e a atualização da referência.
9. Por que o candidato é avaliado no lote mais recente que ele não viu, e não no conjunto de teste original?
10. Que informações devem constar no cartão do modelo final do projeto?
11. Por que as métricas por classe são importantes no projeto, mesmo com boa acurácia geral?
12. Cite três armadilhas do projeto e como evitá-las.
13. Como você explicaria o projeto para a diretoria em cinco linhas?
14. Qual a diferença entre uma simulação como a do projeto e a produção real? Cite três limitações.
15. Como o projeto aplica o princípio de reversão rápida usando o registro de modelos?
16. Que mudanças você faria para adaptar o projeto a um problema de dados tabulares, como churn de clientes?
17. Explique por que o monitoramento da qualidade dos dados de entrada é parte do projeto, mesmo havendo um modelo robusto.
18. Descreva como incluir uma etapa de CI e um contêiner no projeto.
19. Que aprendizados do livro você levaria para um projeto real no seu trabalho? Justifique.
20. Por que o projeto usa um classificador de domínio além do PSI?

SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**

- **UCI:** Optical Recognition of Handwritten Digits (versão completa com 3.823 imagens de treino), Pen-Based Recognition of Handwritten Digits, Wine Quality e Bank Marketing.
- **Kaggle:** MNIST (Digit Recognizer), Fashion-MNIST e *Telco Customer Churn*. Repita o projeto em outro conjunto e compare as conclusões, principalmente o comportamento do monitoramento.

Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador final

Do dado à produção: um sistema de reconhecimento de dígitos com monitoramento e retreino

Contexto. Uma empresa de logística lê dígitos manuscritos em formulários com um modelo de ML. A câmera da esteira vai perder o foco e a tinta vai desbotar aos poucos. Você é responsável por colocar o sistema em produção e garantir que ele continue confiável.

Dados. *Optical Recognition of Handwritten Digits* (UCI), embutido no scikit-learn (`load_digits`): 1.797 imagens de 8 × 8 pixels. A produção e a deriva são **simuladas** por você.

Tarefas.

1. Valide os dados (colunas, ausentes, faixa 0-16, dez classes), calcule o hash e separe desenvolvimento (60%) e "futuro" (40%).
2. Compare uma linha de base, regressão logística, floresta aleatória e SVC com busca aleatória, usando F1 macro em validação cruzada. Registre tudo no MLflow.
3. Aplique um portão de qualidade (piso e ganho sobre a linha de base), registre o melhor modelo e dê a ele o apelido `champion`.
4. Crie uma API (FastAPI) com contrato de entrada que carrega o `champion` pelo apelido. Teste entradas válidas e inválidas.
5. Escreva testes em pytest: desempenho mínimo, invariância a ruído, cobertura de classes e determinismo.
6. Simule 8 semanas de produção com desfoque e desbotamento crescentes por 4 semanas, e estáveis depois. Os rótulos chegam com uma semana de atraso.
7. Monitore cada semana: PSI do contraste, AUC do classificador de domínio e acurácia. Defina uma regra de alerta.
8. Implemente o treinamento contínuo: ao haver alerta e rótulos, treine um candidato, compare-o com o campeão no lote rotulado mais recente (que o candidato não viu) e promova com margem mínima. Depois de promover, atualize a referência do monitoramento.
9. Gere um cartão do modelo com desempenho por classe e escreva uma recomendação final.

Entregáveis. Código organizado, tabela semanal de monitoramento, cartão do modelo e relatório de uma página com a recomendação.

Dica. Evite padronizar colunas quase constantes: neste problema, os pixels já têm a mesma unidade.

A solução de referência está no Apêndice E.

Apêndice A — Gabarito dos exercícios

Esta seção traz a resposta de todas as questões. Nas questões de múltipla escolha, aparecem a alternativa correta e uma breve justificativa. Nas discursivas, aparece uma **resposta-modelo**: a sua resposta pode ser redigida de outro modo, desde que contemple as mesmas ideias.

Capítulo 1 — Introdução ao MLOps

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	O notebook é a cozinha de casa; a produção é a cozinha industrial, e o MLOps liga as duas.
2	a)	MLOps é um paradigma de práticas, processos, ferramentas e cultura para todo o ciclo de vida do sistema de ML.
3	a)	A figura do "iceberg" mostra que o código do modelo é a ponta visível de um sistema muito maior.
4	c)	Diferente de um travamento, o modelo envelhecido continua respondendo normalmente, só que pior.
5	a)	No ML, o comportamento é aprendido dos dados, por isso dados e modelos também precisam ser versionados e testados.
6	b)	Para reproduzir um resultado é preciso conhecer a versão de código, de dados e de modelo.
7	a)	Em sistemas de ML os componentes são emaranhados; alterar uma entrada pode mudar o modelo inteiro.
8	c)	CI integra as mudanças com frequência e verifica cada uma de modo automático.
9	c)	O CT é a peça própria do ML, ao lado de CI e CD.
10	a)	As quatro são frequência de implantação, tempo de entrega, taxa de falha de mudança e tempo de recuperação.
11	a)	O engenheiro de ML cuida da passagem do modelo para a produção de modo confiável.
12	b)	Para produção, o código maduro deve ir para arquivos .py organizados e testáveis.
13	b)	O requirements.txt registra as versões; instala-se com pip install -r requirements.txt.
14	c)	Algoritmos aleatórios dependem da semente; sem fixá-la, a execução varia.
15	d)	A reprodutibilidade completa exige controlar semente, bibliotecas, dados e ambiente.
16	a)	Metadados (versões, semente, dados, data) dão rastreabilidade ao artefato.
17	c)	O estudo da Microsoft destaca dados, modularização e culturas distintas das equipes.
18	b)	Os problemas aparecem em todas as etapas do ciclo de vida, e não apenas no treino.
19	c)	A governança envolve responsabilidade, privacidade, justiça e conformidade.
20	c)	"Estar no ar" não significa "estar certo"; é preciso acompanhar a qualidade das previsões.
21	b)	Manter os dados brutos intocados garante reprodutibilidade do tratamento.
22	c)	Nem todo projeto precisa de uma plataforma completa desde o início.
23	d)	Esse é o ponto de partida; os estágios seguintes automatizam o treino, a entrega e a operação.
24	a)	Ambientes virtuais evitam conflitos de versões entre projetos.
25	d)	Versões diferentes, dados diferentes ou configurações escondidas causam esse problema.
26	d)	O engenheiro de dados cuida das fontes e dos fluxos de dados.
27	c)	Integração, entrega e treinamento contínuos são as três engrenagens da automação.
28	b)	A degradação de qualidade não gera erro de execução, por isso exige monitoramento específico.
29	a)	Além dos testes de código, há testes de dados e de modelo, que são estatísticos.
30	b)	A modularização em src/ facilita testes, versionamento e automação.

Discursivas (respostas-modelo)

1. Treinar é como fazer um bolo em casa: basta a receita certa. Operar é como abastecer uma padaria todos os dias: exige receita registrada, fornecedores confiáveis (dados), testes de cada lote, vigilância da qualidade e equipe com funções claras. MLOps é o conjunto de práticas que leva da cozinha de casa à cozinha industrial.

2. Que o código do modelo é só a parte visível e pequena de um sistema maior. Abaixo estão coleta e verificação de dados, atributos, infraestrutura, ferramentas, serviço e monitoramento. A maior parte do esforço de engenharia está nesse conjunto.
3. Porque o modelo aprende padrões dos dados; mudando os dados, muda o programa resultante. Por isso é preciso versionar dados além do código e do modelo, e testar dados e comportamento do modelo, e não apenas as funções do código.
4. É a piora das respostas sem erro de execução. Exemplo: um modelo de crédito treinado antes de uma crise continua aprovando como antes e passa a errar mais. Detecta-se monitorando a qualidade das previsões, a distribuição das entradas e das saídas e os indicadores de negócio.
5. "Mudar qualquer coisa muda tudo": componentes emaranhados fazem uma alteração local afetar o todo. Exemplo: remover ou reescalar um atributo altera os coeficientes ou as árvores e, assim, as importâncias e as previsões dos demais atributos.
6. CI integra e testa cada mudança de código automaticamente. CD entrega ou implanta automaticamente a mudança aprovada. CT retreina o modelo automaticamente por agenda, dados novos ou queda de qualidade. O CT é o específico do ML.
7. Frequência de implantação, tempo de entrega, taxa de falha de mudança e tempo de recuperação. Em ML, medem-se com que frequência novos modelos vão ao ar, quanto tempo da mudança ao ar, que fração causa problemas e quanto leva para reverter ou corrigir.
8. Versões diferentes das bibliotecas podem mudar os resultados; os dados podem ter mudado; e o ambiente (sistema, hardware, configurações) pode diferir. Também a ordem dos dados e o paralelismo podem influenciar.
9. Porque permitem reconstruir e auditar o modelo: saber com quais dados, código e ambiente foi produzido, detectar incompatibilidades de versão ao carregá-lo e reproduzir o resultado.
10. data/raw e data/processed (dados originais e tratados), src/ (código reutilizável: dados, treino, serviço), tests/ (testes automáticos), models/ (artefatos), notebooks/ (exploração), requirements.txt (versões) e README.md (como rodar). A separação facilita testes, automação e colaboração.
11. Resultados não reproduzíveis ("funciona na minha máquina"); ninguém sabe qual versão de dados ou código gerou o modelo em produção; demora meses para pôr melhorias no ar; problemas só são percebidos quando o cliente reclama.
12. O cientista explora os dados e cria e avalia modelos, perguntando se resolvem o problema do negócio. O engenheiro de ML transforma o modelo em produto confiável, escalável e automatizado, perguntando se roda de forma estável. Eles colaboram, e em equipes pequenas podem ser a mesma pessoa.
13. São níveis que vão do trabalho manual a uma operação contínua com monitoramento e governança. Nem todo projeto precisa do nível mais alto: o custo e a complexidade devem ser proporcionais ao risco e ao valor, e o ideal é resolver a dor mais cara de cada vez.
14. Porque o servidor pode estar no ar e as previsões erradas. É preciso acompanhar a qualidade dos dados de entrada, a distribuição das previsões, o desempenho (quando há rótulos) e o efeito no negócio.
15. Treinou-se uma floresta aleatória várias vezes com e sem semente fixa. Sem semente, as acurácias variam ligeiramente a cada execução; com semente fixa, são idênticas. Conclui-se que a reprodutibilidade exige controlar as fontes de aleatoriedade, além de dados e ambiente.
16. Isolam as dependências do projeto e permitem recriar o mesmo ambiente em qualquer máquina, evitando conflitos de versão e o problema do "funciona na minha máquina".
17. Porque suas culturas e vocabulários diferem, e o sistema depende da colaboração de ambas. Exemplo: o cientista entrega um notebook e o engenheiro precisa de código testável com contrato de entrada e saída; um acordo sobre estrutura de projeto, testes e métricas evita retrabalho.
18. Porque o mundo muda e os dados de produção podem diferir dos de treino; o modelo pode piorar sem avisar. A avaliação contínua, por monitoramento e experimentos, revela o desempenho real e orienta retreinos e reversões.

19. Porque o mundo e os dados mudam e o modelo envelhece. Continuam o monitoramento da qualidade e dos dados, a manutenção do pipeline, o retreinamento, a atualização de versões, a governança e o atendimento a incidentes.

20. Em lote: lista diária de clientes com risco de cancelar; reposição semanal de estoque, pois a decisão pode esperar. Em tempo real: aprovar uma compra no cartão; recomendar um item enquanto o usuário navega, pois a decisão não pode esperar.

Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	Do levantamento de requisitos ao monitoramento, são nove etapas.
2	a)	A ordem é: requisitos, coleta, limpeza, rotulagem, atributos, treino, avaliação, implantação, monitoramento.
3	c)	O que se aprende em produção volta para o início do ciclo.
4	d)	Por exemplo, "renda mensal" e "idade" em um modelo de crédito.
5	d)	Começa-se pela decisão, por quem usa a previsão, pelo critério de sucesso e pelo custo do erro.
6	c)	Alta acurácia pode ocultar um modelo que não encontra nenhum caso positivo.
7	c)	Se o modelo sofisticado não supera a linha de base com folga, não vale a pena produzi-lo.
8	a)	Latência, vazão, custo, explicabilidade e privacidade são requisitos não funcionais.
9	b)	Atalhos de hoje viram custo de manutenção, bugs e lentidão amanhã.
10	d)	Há problemas de emaranhamento, dependências de dados e realimentação, entre outros.
11	b)	Exemplo: um recomendador que só mostra o popular faz apenas o popular receber cliques.
12	a)	Uma mudança no modelo pode quebrar sistemas dependentes que ninguém mapeou.
13	a)	Ele dificulta testes e manutenção e é uma fonte típica de dívida técnica.
14	a)	Configurações em arquivos, linhas de comando e variáveis sem controle geram erros difíceis de achar.
15	d)	Dados e atributos, desenvolvimento do modelo, infraestrutura e monitoramento.
16	b)	Não feito = 0; manual e documentado = 0,5; automático e recorrente = 1.
17	a)	A corrente é tão forte quanto o elo mais fraco.
18	d)	Ele evita recalcular os atributos de formas diferentes e reduz o desvio treino–serviço.
19	b)	Cada modelo registrado tem versões e metadados.
20	d)	Ele é como o gerente de produção que coordena as tarefas e suas dependências.
21	b)	O próximo passo é versionar e fixar o ambiente.
22	b)	A implantação ainda é manual e arriscada, e a automação com testes é o passo seguinte.
23	d)	É o nível mais alto da escada descrita no capítulo.
24	b)	É essencial para auditoria e para responder "por que o sistema decidiu isso?".
25	a)	Difere de rastreabilidade (saber a origem) e de linhagem (o caminho completo).
26	c)	Permite voltar da decisão ao modelo, aos dados e ao código.
27	a)	Mesmo com qualidade idêntica, o sistema ensinou a si mesmo uma desigualdade.
28	a)	Resolve-se primeiro o que mais custa: reprodutibilidade, implantação arriscada ou queda de qualidade.
29	a)	É preciso conectar a precisão do modelo ao efeito na decisão e no resultado financeiro.
30	a)	O MLOps costura as três fases em um ciclo contínuo.

Discursivas (respostas-modelo)

1. Requisitos, coleta, limpeza, rotulagem, engenharia de atributos, treino, avaliação, implantação e monitoramento. É um laço porque o que se observa no monitoramento (queda de qualidade, novos casos) gera novos requisitos, novos dados e novos treinos.

2. Para saber se o modelo agrega valor e evitar complexidade desnecessária. Exemplo: em um problema com 3% de positivos, "sempre negativo" tem 97% de acurácia; o modelo só vale se superar essa referência em métricas adequadas, como revocação e AP.
3. É o custo futuro de atalhos tomados hoje. Exemplos: emaranhamento (CACE), dependências de dados instáveis, ciclos de realimentação escondidos, consumidores não declarados, código de cola, dívida de configuração e mudanças no mundo externo.
4. Como um aluno que só estuda o que já sabe porque o professor lhe dá mais disso. Em um recomendador, só o popular recebe cliques. Mitiga-se com exploração (mostrar itens variados), experimentos controlados e monitoramento da concentração.
5. Cada teste vale 0 (não feito), 0,5 (manual e documentado) ou 1 (automático e recorrente). Somam-se os pontos por grupo e a nota final é o menor total, porque o sistema é tão confiável quanto seu elo mais fraco: investir só onde já se é bom não melhora a prontidão.
6. Armazém de atributos (guarda atributos reutilizáveis); rastreamento de experimentos (registra execuções); registro de modelos (cataloga versões); CI/CD (testa e implanta); orquestração (executa tarefas na ordem); serviço (entrega previsões); monitoramento (vigia qualidade e saúde).
7. Nível 0, manual: ninguém reproduz resultados. Nível 1, treino automatizado: implantação manual e arriscada. Nível 2, entrega automatizada: o modelo piora em produção sem ninguém ver. Nível 3, operação contínua: custos, escala e conformidade.
8. Reprodutibilidade: obter o mesmo resultado com o mesmo código e dados. Rastreabilidade: saber qual código, dados e configuração geraram cada modelo. Linhagem: o caminho completo de um dado ou modelo, da origem ao destino, passando por todas as transformações.
9. Qual decisão será apoiada, quem usa a previsão e quando, como saberemos que deu certo (métrica de negócio) e quanto custa errar, inclusive qual tipo de erro custa mais.
10. Porque a acurácia pode ser alta apenas por prever sempre a classe majoritária, sem encontrar os casos que importam, ou porque o erro mais caro (falso negativo) não está sendo reduzido. O valor depende do efeito na decisão e do custo dos erros.
11. São exigências além da qualidade das previsões: latência, vazão, custo, explicabilidade, privacidade e conformidade, disponibilidade e segurança.
12. Porque o outro sistema pode mudar formato, unidade ou significado sem aviso (dependência de dados instável), e o modelo passa a receber entradas diferentes das de treino, degradando em silêncio. Contratos de dados e validação reduzem o risco.
13. Porque cada passo deve resolver uma dor real, com custo proporcional. Plataformas grandes exigem pessoas e manutenção; muitas vezes uma solução simples e bem automatizada vale mais. A evolução gradual reduz risco e aumenta o aprendizado da equipe.
14. Se outros sistemas usam a saída do modelo sem registro, mudanças de versão os quebram. É preciso catalogar os consumidores, versionar os contratos de saída, comunicar mudanças e manter compatibilidade ou prazos de transição.
15. Dados pessoais exigem minimização, pseudonimização, controle de acesso, retenção limitada e, possivelmente, treino federado ou privacidade diferencial. Isso afeta o armazenamento, os logs, o pipeline de dados e a forma de servir previsões.
16. Os testes verificam o sistema antes da implantação e o monitoramento o verifica depois. Juntos, reduzem o risco de falhas silenciosas; o ML Test Score inclui ambos como grupos de itens.
17. Métricas offline usam dados históricos antes da implantação; mostram se o modelo é promissor. A avaliação em produção usa dados reais e o efeito no negócio; mostra se continua válido com o mundo mudando. Uma não substitui a outra.

18. Centralizar a configuração em um arquivo versionado reduz erros de parâmetros espalhados; usar o mesmo pipeline para treino e serviço reduz o desvio treino–serviço; escrever testes automáticos reduz o custo de mudanças futuras.

19. Porque o nível 1 (treino automatizado e experimentos rastreados) já resolve reprodutibilidade, e o custo de uma plataforma maior pode não compensar. Sobe-se quando a dor muda: implantações arriscadas pedem CI/CD; quedas de qualidade pedem monitoramento e retreino.

20. Estar no ar é disponibilidade; estar certo é qualidade das previsões. O ML Test Score inclui itens de monitoramento da qualidade e dos dados, além de testes de modelo e de infraestrutura, para cobrir os dois aspectos.

Capítulo 3 — Dados

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	A qualidade do modelo depende da qualidade dos dados de treino e dos que chegam em produção.
2	a)	O estudo mostrou que a maioria dos profissionais relatou pelo menos uma cascata.
3	a)	A unicidade responde se o mesmo registro aparece mais de uma vez.
4	d)	O valor está fora do intervalo permitido.
5	c)	Os dados não estão em dia.
6	c)	Os dados discordam entre si.
7	d)	Os dados não cobrem os casos reais em que o modelo será usado.
8	a)	É a base para detectar anomalias nos lotes de dados.
9	c)	Pode vir de dados diferentes ou de código de pré-processamento diferente.
10	a)	O portão evita que lixo contamine o modelo.
11	d)	O desempenho em teste fica bom demais e desaba em produção.
12	b)	Se a informação só existe depois do evento, é vazamento.
13	b)	Informação do teste participa do cálculo da média e do desvio.
14	c)	Em dados com tempo, o treino deve vir do passado e o teste, do futuro.
15	c)	No experimento do capítulo, a acurácia foi a quase 100% com um atributo vazado.
16	c)	O ruído aleatório moderado é bem tolerado, mas erros sistemáticos são mais danosos.
17	c)	Um rotulador que sempre confunde duas classes ensina o erro ao modelo.
18	c)	Rótulos devem ser tratados como dados de primeira classe.
19	a)	Usar o teste repetidamente para escolher modelos o contamina.
20	c)	Assim nenhum cliente aparece no treino e no teste ao mesmo tempo.
21	c)	Evita o vazamento temporal.
22	b)	Serve como impressão digital para identificar versões de dados.
23	d)	Os dados pesados ficam em armazenamento remoto.
24	c)	Já o dvc checkout traz de volta a versão dos dados correspondente ao Git.
25	d)	É simples, mas ocupa mais espaço.
26	c)	São a "bula" do conjunto de dados.
27	b)	Mantém a capacidade de juntar tabelas sem expor o identificador direto.
28	a)	Menos dados guardados significam menos risco.
29	a)	Os conjuntos Iris, Wine, Breast Cancer e Digits do scikit-learn vêm do UCI.
30	d)	Um aumento súbito de ausentes costuma indicar falha em um sistema de origem.

Discursivas (respostas-modelo)

1. Porque o modelo aprende o que os dados mostram; erros nos dados se propagam. Cascatas de dados são problemas que nascem cedo, passam despercebidos e causam danos graves depois, relatados pela maioria dos profissionais estudados por Sambasivan et al. (2021).
2. Completude (renda vazia), validade (idade −5), unicidade (cliente duplicado), consistência (saída antes da entrada), atualidade (tabela de seis meses atrás). Também exatidão e representatividade.

3. O esquema define tipo, intervalo, categorias e fração máxima de ausentes por coluna. O validador compara cada lote com o esquema e lista as violações. Ele é um portão antes do treino ou da previsão: lotes reprovados vão para quarentena e geram alerta.
4. É quando o modelo usa informação indisponível na hora de prever. Exemplos: vazamento de alvo (atributo derivado do rótulo) e contaminação treino–teste (normalizar tudo antes de dividir). Evita-se pensando na disponibilidade na hora do uso e ajustando as transformações só no treino, dentro de um pipeline.
5. Calculando a AUC de cada atributo sozinho e vendo se algum se destaca de modo suspeito; verificando de onde vem e se existiria na hora da previsão; comparando o desempenho com e sem ele; e pedindo à área de negócio a lógica do atributo.
6. Rótulos errados ensinam informação falsa. Ruído aleatório moderado é diluído por modelos como florestas. Erros sistemáticos, como um rotulador que sempre confunde duas classes, criam um padrão falso que o modelo aprende. Por isso rótulos devem ser revisados e amostrados.
7. Aleatória: dados independentes e sem ordem. Estratificada: classes desbalanceadas, mantém proporções. Temporal: dados com tempo, treina no passado e testa no futuro. Por grupo: várias linhas por indivíduo, evitando o mesmo indivíduo nos dois lados.
8. Porque, se for usado repetidamente para escolher modelos ou ajustar hiperparâmetros, passa a participar do treino de forma indireta e a estimativa fica otimista. O teste serve para o relatório final honesto.
9. É uma impressão digital do conteúdo: muda totalmente se o conteúdo mudar. Registrando o hash dos dados de treino junto ao modelo, é possível provar com qual versão dos dados ele foi treinado e detectar se algo foi alterado.
10. Os dados ficam em armazenamento remoto; no Git ficam arquivos pequenos de ponteiro com o hash e o caminho. Ao trocar de versão no Git, o DVC traz os dados correspondentes. Assim, código e dados têm histórico coerente.
11. Motivação, composição (o que há, quantos exemplos, dados pessoais), processo de coleta e consentimento, pré-processamento, usos indicados e não indicados, distribuição e manutenção, licença.
12. Minimização é coletar e guardar apenas o necessário; pseudonimização troca identificadores diretos por códigos que não identificam sem informação adicional guardada à parte. Ambas reduzem riscos e apoiam as obrigações da LGPD sobre tratamento de dados pessoais.
13. Porque medições do mesmo paciente são parecidas. Em uma divisão aleatória, o modelo pode "reconhecer" o paciente no teste e inflar o desempenho, sem aprender o fenômeno geral. Dividir por paciente evita esse vazamento.
14. Porque os dados de produção mudam e podem ter defeitos novos. Validar cada lote que chega evita previsões baseadas em dados corrompidos e dá alerta precoce sobre problemas nas fontes.
15. Uma junção que usa o valor mais recente do atributo para eventos passados traz informação do futuro para o treino, caracterizando vazamento temporal. A junção deve usar o valor disponível na data do evento.
16. É uma área onde lotes reprovados ficam isolados, sem entrar no treino ou na previsão. A equipe investiga a origem, corrige na fonte, reprocessa ou descarta, e registra o motivo.
17. Para evitar o desvio treino–serviço: se o pré-processamento difere, o modelo recebe na produção entradas diferentes das que viu no treino. O ideal é empacotá-lo no mesmo pipeline.
18. Base legal e finalidade claras; minimização e pseudonimização; controle de acesso e retenção limitada; transparência e possibilidade de revisão de decisões automatizadas; cuidado com atributos sensíveis e vieses.
19. Porque o modelo só aprende o que os dados cobrem. Se faltam regiões, perfis ou condições que aparecem em produção, ele erra justamente para esses casos, e a média esconde o problema.
20. Idade entre 18 e 100 evita valores impossíveis que distorcem o modelo; categorias permitidas de cidade evitam entradas inesperadas ou digitadas de modo diferente; fração máxima de ausentes em renda evita lotes com falha na origem.

Capítulo 4 — Pipelines Reprodutíveis com scikit-learn

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	O mesmo "molde" vale para o treino e para a produção.
2	b)	Transformadores e preditores são tipos de estimadores.
3	b)	Por exemplo, o StandardScaler aplica a média e o desvio aprendidos.
4	c)	No teste e na produção usam-se apenas transform e predict.
5	d)	Uma única chamada treina o pré-processamento e o modelo.
6	d)	Os nomes ajudam também na leitura e na depuração.
7	c)	A seleção olhou os rótulos de todos os dados, inclusive dos dobras de teste.
8	b)	Assim a estimativa de desempenho é honesta.
9	a)	Cada categoria vira uma coluna "é esta categoria?".
10	b)	Por exemplo, mediana e padronização nas numéricas, moda e one-hot nas categóricas.
11	c)	O encoder gera zeros para a categoria nunca vista.
12	c)	Isso evita vazamento e garante consistência.
13	d)	Receber a linha crua e usar o mesmo pipeline elimina boa parte desse problema.
14	a)	Assim ele entra em Pipelines e buscas de hiperparâmetros.
15	b)	É a convenção do scikit-learn para parâmetros aprendidos.
16	a)	Também facilita revisão e rastreabilidade.
17	c)	Qualquer fonte de acaso não controlada pode mudar o resultado.
18	a)	Se as assinaturas forem iguais, o procedimento é determinístico.
19	a)	Ele grava objetos Python, inclusive arrays grandes, de forma eficiente.
20	d)	Só se devem carregar artefatos de fontes confiáveis.
21	b)	A verificação evita erros silenciosos por incompatibilidade.
22	d)	Isso permite automatizar e repetir o treino.
23	c)	Ela permite passar, por exemplo, --semente 7 --n-arvores 300.
24	a)	É frequentemente organizado como um DAG, com tarefas e dependências.
25	b)	DAG é um grafo acíclico dirigido: não há ciclos.
26	d)	Útil para aplicar transformações simples, como logaritmo, dentro do pipeline.
27	a)	A validação cruzada recria o pré-processamento em cada dobra.
28	b)	Algoritmos baseados em árvores são pouco sensíveis à escala.
29	b)	O imputador deve ser ajustado apenas com o treino, dentro do pipeline.
30	b)	Cada etapa usa apenas o que aprendeu no treino.

Discursivas (respostas-modelo)

1. Parâmetros aprendidos (média, mediana, seleção de atributos) devem vir apenas do treino; no teste e na produção só se aplica transform. Violá-la, por exemplo padronizando antes de dividir ou selecionando atributos com todos os rótulos, leva a vazamento e a estimativas otimistas.

2. Com 100 exemplos e 2.000 atributos aleatórios, selecionar os 20 melhores com todos os dados e depois validar deu acurácia falsamente alta; fazê-lo dentro do pipeline deu cerca de 50%. Conclui-se que a seleção fora da validação vaza informação do rótulo.
3. Ele encapsula, em um único objeto, os tratamentos por tipo de coluna (imputação, escala, codificação). Em produção, a linha crua passa pelo mesmo objeto treinado, sem reimplementação manual do pré-processamento.
4. Evita que o serviço quebre quando aparece uma categoria nova. A limitação é que o modelo não sabe usar a categoria nova (ela vira zeros), então é preciso monitorar o surgimento de categorias novas e retreinar se forem frequentes.
5. É uma classe que implementa fit e transform para inserir uma lógica própria no pipeline, por exemplo, criar a razão renda/idade com proteção contra divisão por zero ou cortar valores extremos com limites aprendidos no treino.
6. Calcular os parâmetros em fit apenas com o treino e guardá-los em atributos com sufixo _; não depender de dados externos ao pipeline; testar com tabelas pequenas e valores extremos.
7. Divisão treino/teste, algoritmos aleatórios (florestas, K-means), validação cruzada embaralhada, geradores do NumPy, ordem dos dados e paralelismo; além das versões das bibliotecas e do hardware.
8. Porque desserializar pode executar código malicioso. Alternativas: carregar apenas artefatos de fontes confiáveis, usar formatos mais seguros e portáveis, como skops ou ONNX, e verificar integridade e versão.
9. Permite mudar parâmetros sem alterar código, revisar mudanças, versionar a configuração junto do modelo e registrar exatamente com que valores cada experimento foi rodado.
10. Lê a configuração e parâmetros da linha de comando; carrega os dados; divide; monta e treina o pipeline; avalia; salva o pipeline, a versão do scikit-learn, as métricas, a configuração e o hash dos dados.
11. O primeiro é a sequência dentro do código (pré-processamento e modelo). O segundo é o fluxo maior que liga ingestão, validação, treino, avaliação, registro e implantação, em geral organizado como DAG.
12. Cada etapa depende da saída da anterior. Por exemplo, imputar ausentes antes de padronizar evita que valores faltantes distorçam a média; selecionar atributos antes de escalar pode mudar a seleção.
13. Que ele aceita os dados como vêm da origem, com ausentes, categorias e escalas originais, e cuida de todo o tratamento. Isso elimina código duplicado de pré-processamento na API e reduz erros.
14. Verificar que os limites vêm só do treino; que valores extremos do teste são cortados pelos limites do treino; que valores normais não mudam; que não altera a tabela de entrada; e casos de borda como colunas constantes.
15. Porque dados, versões de bibliotecas, ordem das linhas, paralelismo e hardware também influenciam os resultados. É preciso versionar dados, fixar dependências e, idealmente, usar contêineres.
16. Ao calcular um hash das probabilidades previstas, compara-se rapidamente duas execuções: hashes iguais indicam resultados idênticos; diferentes indicam alguma fonte de variação a investigar.
17. Essencial em KNN, SVC e regressão regularizada, que dependem de distâncias ou penalidades sensíveis à escala. Dispensável em florestas e árvores de decisão, que usam cortes por atributo e não dependem da escala.
18. Porque a CI não tem interface gráfica nem cliques: precisa executar o treino de forma automática, com parâmetros, e usar o código de saída para decidir se segue ou falha.
19. Porque é o objeto que vai para a produção e deve ser rastreável: associado ao código, aos dados, à configuração e às métricas, com versão do scikit-learn registrada. Isso permite auditar, reproduzir e reverter.
20. Ajustar o pré-processamento fora do pipeline (use Pipeline), reimplementar o tratamento na API (use o mesmo artefato) e esquecer categorias novas (use handle_unknown). Também: não fixar sementes, não registrar versões.

Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Ele guarda parâmetros, métricas, artefatos e a origem do código e dos dados.
2	b)	Um experimento agrupa várias execuções.
3	d)	Parâmetros são entradas fixas durante a execução; métricas medem o resultado.
4	b)	Por exemplo, a matriz de confusão em PNG.
5	a)	É o jeito mais simples de começar sem servidor.
6	d)	Ao sair do bloco, a execução é encerrada.
7	a)	Isso permite registrar curvas de aprendizado.
8	c)	Dá rastreabilidade entre dados e modelos.
9	c)	É um contrato: entradas no formato errado geram erro cedo.
10	b)	Depois acessa-se o endereço indicado, normalmente http://127.0.0.1:5000.
11	a)	Isso reduz o risco de execução de código arbitrário ao carregar o modelo.
12	c)	É prático, mas não substitui o registro consciente das decisões importantes.
13	c)	É o catálogo oficial dos modelos aprovados.
14	a)	O título é o modelo registrado; o apelido é "a edição vigente".
15	d)	O challenger é a candidata.
16	d)	O código de produção permanece o mesmo.
17	b)	São mais flexíveis e a ideia de marcar a versão em cada fase se mantém.
18	b)	Evita trocas por ruído e cria rastro de auditoria.
19	b)	Diferenças muito pequenas podem ser sorte estatística.
20	c)	Ideal: conjunto fixo e versionado, e comparação de incertezas.
21	c)	Segredos nunca devem ficar em registros de modelos.
22	a)	Os componentes principais citados são Tracking, Models, Model Registry e Projects.
23	b)	Foi motivado por dificuldades de acompanhar experimentos e reproduzir e implantar modelos.
24	d)	Isso permite rastrear de onde veio cada artefato.
25	b)	A média pode esconder desigualdades.
26	a)	Esse é o valor central do caderno de laboratório digital.
27	d)	Gráficos ficam associados à execução que os gerou.
28	a)	Dá controle de versões e reversão rápida.
29	d)	Só assim a reprodução é confiável.
30	c)	O princípio é registrar de forma sistemática, comparável e reprodutível, com qualquer ferramenta.

Discursivas (respostas-modelo)

1. Sem registro, ninguém sabe quais parâmetros, dados e código geraram cada resultado, e o melhor modelo não pode ser reproduzido. O rastreamento guarda, para cada execução, parâmetros, métricas, artefatos, tags e versões, permitindo comparar e reproduzir.

2. Experimento: grupo de execuções com um mesmo objetivo. Execução: uma tentativa de treino. Parâmetro: valor de entrada fixo. Métrica: número que mede o resultado, que pode variar com o passo. Artefato: arquivo produzido. Tag: rótulo livre para organizar.
3. É a descrição dos nomes e tipos de entrada e saída esperados pelo modelo. Funciona como contrato e permite detectar cedo entradas em formato errado, antes de gerarem resultados sem sentido.
4. Para 5, 50 e 300 árvores, abriu-se uma execução e registraram-se parâmetros, F1 e acurácia, tags com hash dos dados, a matriz de confusão como figura e o modelo com assinatura. Depois, search_runs ordenou as execuções por F1.
5. Porque, com poucos exemplos de teste, a métrica tem incerteza; diferenças pequenas podem ser ruído. É preciso usar validação cruzada, intervalos de confiança e testes estatísticos para decidir.
6. Apelidos são nomes móveis que apontam para versões, como champion e challenger. O código de produção carrega sempre o apelido; para trocar ou reverter, move-se o apelido e recarrega-se o serviço, sem editar o código. Mover o apelido só muda a referência no registro; o objeto já carregado continua sendo a versão antiga até a recarga.
7. Função executada ao final do treino que decide se o candidato vira champion. Critérios: piso absoluto de qualidade, ganho mínimo sobre o campeão (margem) e registro da decisão com data, motivo e responsável. Opcionalmente, intervalo de confiança e testes em fatias.
8. Porque exige declarar quais tipos de objeto são confiáveis ao carregar, em vez de executar código arbitrário embutido no arquivo. Isso reduz o risco de abrir artefatos maliciosos.
9. Nome e versão, objetivo e uso indicado, dados de treino (hash, período, fonte), código (commit), parâmetros, métricas (inclusive por grupo), limitações conhecidas, responsável e data e aprovação.
10. O autologging registra automaticamente parâmetros e métricas padrão, útil para começar. O registro explícito guarda decisões importantes (hash dos dados, semente, métricas de negócio, subgrupos). O ideal é combinar os dois.
11. Porque cada modelo fica ligado a execuções com código, dados, parâmetros e métricas conhecidos. Diante de uma pergunta sobre uma decisão, é possível localizar o modelo e reproduzir seu treino.
12. O piso exige qualidade mínima absoluta (por exemplo, F1 ≥ 0,95); a margem sobre a linha de base exige que o modelo compense o custo de complexidade (por exemplo, +0,01). Só quando ambas são cumpridas o modelo é registrado como champion.
13. Commit do código, requirements.txt ou lista de pacotes, hash dos dados, duração e consumo de memória do treino, métricas por subgrupo, comentários sobre a motivação do experimento.
14. Usando o mesmo conjunto de avaliação fixo e versionado, não usado no treino de nenhum dos dois, com as mesmas métricas, e comparando com intervalos de confiança e margem mínima.
15. Porque trocar a versão exigiria alterar e reimplantar o código. Com o apelido, a mudança de modelo é uma operação de registro, auditável e reversível, seguida da recarga do serviço.
16. Tracking (registra e consulta execuções), Models (formato padrão de empacotamento), Model Registry (catálogo com versões e apelidos) e Projects (empacotamento de código reprodutível).
17. Define-se o endereço do servidor de rastreamento e, se preciso, de armazenamento de artefatos, e a equipe passa a ver os mesmos experimentos e modelos. Muda o controle de acesso, a persistência e a necessidade de segurança.
18. Para dar responsabilização e permitir auditoria: saber quem aprovou, quando e com base em qual evidência. É parte da governança do ciclo de vida do modelo.
19. Observando métricas em produção, alertas de monitoramento, resultados por fatias e feedback de negócio; se a versão atual piorou além de limites definidos, move-se o apelido champion para a versão anterior validada e investiga-se a causa.
20. Porque dois modelos só são comparáveis se treinados e avaliados nos mesmos dados. Hashes diferentes denunciam que os dados mudaram entre as execuções, alertando para comparações injustas.

Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	A nota do teste, sozinha, não garante o comportamento no mundo real.
2	a)	É VP/(VP+FP): mede a confiabilidade do alarme.
3	d)	É VP/(VP+FN): mede o quanto o modelo "enxerga".
4	a)	Só é alto quando precisão e revocação são altas.
5	d)	A acurácia engana em classes desbalanceadas.
6	c)	A ROC pode parecer ótima mesmo com muitos falsos alarmes.
7	d)	O piso da AP é a fração de positivos.
8	a)	O limiar deve refletir o custo dos falsos negativos e positivos.
9	d)	Aceita-se mais falsos alarmes para perder menos casos reais.
10	d)	O teste é reservado para o relatório final.
11	d)	Isso importa quando a probabilidade entra na decisão.
12	d)	Quanto menor, melhor.
13	a)	A calibração pode ser melhorada por métodos como a regressão isotônica.
14	a)	Resume-se por média e desvio dos k resultados.
15	a)	Uma média sem dispersão esconde a variabilidade.
16	c)	A classe TimeSeriesSplit faz isso.
17	c)	Permite construir intervalos de confiança.
18	c)	Nesse caso, desempata-se por simplicidade, custo e explicabilidade.
19	d)	Parte do bom resultado é sorte de ter escolhido o melhor entre muitos.
20	d)	O teste do laço externo não participa da escolha.
21	d)	Parâmetros são aprendidos; hiperparâmetros são configurados.
22	c)	Poucos hiperparâmetros importam muito, e a grade repete valores dos que não importam.
23	a)	É uma abordagem de multi-fidelidade.
24	d)	É eficiente quando cada treino é caro.
25	c)	Um modelo com 95% de acerto geral pode ter 80% para um grupo.
26	a)	Porque eleva os erros ao quadrado antes de tirar a média.
27	b)	A divisão por valores próximos de zero amplifica o erro percentual.
28	d)	Por isso são essenciais experimentos online.
29	a)	Mantém o teste como estimativa honesta.
30	a)	Costuma reduzir o erro de calibração.

Discursivas (respostas-modelo)

1. Com 5% de fraudes, prever sempre "não fraude" dá 95% de acurácia sem encontrar nenhuma fraude. Usam-se precisão, revocação, F1 e AP, que olham a classe rara, e se escolhem de acordo com o custo dos erros.

2. Detector de fumaça sensível: apita com torrada queimada (baixa precisão, alta revocação). Detector frouxo: nunca falha em alarme falso, mas pode deixar passar um incêndio (alta precisão, baixa revocação). A escolha depende do custo de cada erro.
3. Porque a taxa de falsos positivos da ROC usa os negativos, que são enormes, no denominador, e mesmo muitos falsos alarmes mudam pouco essa taxa. A precisão sofre diretamente com esses falsos alarmes, e a curva PR revela a dificuldade real.
4. Obter as probabilidades no conjunto de validação; definir os custos de falso negativo e falso positivo; calcular o custo total para vários limiares; escolher o que minimiza o custo; aplicar no teste e relatar o custo; versionar e monitorar o limiar.
5. Um modelo é calibrado se as probabilidades correspondem às frequências reais. Importa para preço de seguro, risco de crédito e priorização, em que o valor numérico da probabilidade (e não só o ranking) é usado.
6. Divide-se o conjunto em k partes; cada uma serve de teste uma vez, treinando nas outras. Resumem-se os k resultados pela média e pelo desvio. A dispersão mostra a variabilidade; diferenças pequenas diante dela merecem desconfiança e devem ser confirmadas por uma comparação pareada (por dobra ou por bootstrap).
7. Porque o modelo veria o futuro durante o treino, o que é impossível em produção e infla o desempenho. Deve-se treinar no passado e testar no futuro, com divisões que respeitem a ordem.
8. Sortear com reposição índices do conjunto de teste, calcular a diferença de acurácia em cada reamostra, repetir milhares de vezes e tomar os percentis 2,5% e 97,5%. Se o intervalo inclui zero, não há evidência clara de diferença.
9. É o otimismo da melhor pontuação entre muitas configurações avaliadas pelos mesmos dados. A validação aninhada separa o laço que escolhe (interno) do que avalia (externo), estimando o desempenho do procedimento em dados que a escolha não viu.
10. A grade testa todas as combinações, é simples mas explode com muitos hiperparâmetros. A aleatória sorteia combinações e explora melhor o que importa. A bayesiana usa resultados anteriores para escolher a próxima tentativa, sendo eficiente quando o treino é caro.
11. Porque a média pode esconder grupos prejudicados. Define-se, com o negócio, fatias críticas (região, faixa de idade etc.), calculam-se as métricas por fatia, compara-se com pisos e acompanha-se em produção.
12. Procedimentos de treino que produzem modelos com mesmo desempenho no teste, mas comportamentos diferentes em produção. Consequência: é preciso avaliar de várias formas (fatias, estresse, invariância) e monitorar em produção.
13. Offline usam dados históricos antes da implantação; online medem o efeito real em usuários, por experimentos. Melhorar a métrica offline não garante ganho de negócio, então é preciso confirmar online.
14. MAE, fácil de explicar mas trata erros grandes e pequenos igualmente; RMSE, pune erros grandes mas é sensível a atípicos; R^2, pode enganar fora do treino; MAPE, instável perto de zero.
15. Porque a escolha se ajusta ao teste e a estimativa final fica otimista. A busca deve usar apenas treino e validação; o teste serve para um único relatório final.
16. Divisão treino/validação/teste; comparação por validação cruzada repetida; calibração; escolha do limiar na validação com custos; métricas finais no teste com intervalo de confiança; análise por fatias; limitações e recomendação.
17. Se o desempenho no treino é muito maior que na validação, o modelo memorizou o treino. Investiga-se com curvas de aprendizado, regularização, validação cruzada, mais dados e simplificação do modelo.
18. Quando uma classe é muito mais rara que outra. Ajudam métricas adequadas (AP, F1), pesos de classe, reamostragem, ajuste do limiar por custo e validação estratificada.
19. A média dos votos das árvores tende a evitar valores extremos, deixando as probabilidades "espremidas" para o meio. Métodos como a regressão isotônica ou de Platt corrigem isso.
20. Que "melhor" na amostra pode ser sorte. Mostra-se a diferença com um intervalo de confiança: se ele exclui zero, há evidência; se inclui, a decisão deve considerar custo, simplicidade e mais dados.

Capítulo 7 — Empacotamento e Serviço de Modelos

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	Sem serviço, o modelo não gera valor, por melhor que seja.
2	c)	Exemplo: lista diária de clientes com risco de cancelar.
3	d)	A decisão não pode esperar.
4	c)	Exemplo: reconhecimento de voz no celular.
5	a)	Tempo real deve ser usado quando a decisão realmente não pode esperar.
6	a)	Dados fora das regras são rejeitados antes de chegarem ao modelo.
7	b)	Ela rejeita, por exemplo, idade negativa ou plano desconhecido.
8	a)	Outros programas enviam pedidos e recebem respostas.
9	c)	É o formato mais comum de troca de dados em serviços web.
10	c)	No FastAPI usa-se o ciclo de vida (lifespan) para isso.
11	c)	Permite saber qual modelo gerou cada resposta.
12	b)	O modelo nem chega a ser chamado.
13	a)	Assim a API é testada automaticamente a cada mudança.
14	d)	A vazão (throughput) é a quantidade por segundo.
15	b)	A cauda da distribuição (p99) mostra a pior experiência.
16	b)	Metade dos pedidos é mais rápida que o p50.
17	b)	Por isso se usa micro-batching em serviços de alto volume.
18	c)	O contêiner é a instância em uso.
19	b)	Contém instruções como FROM, COPY e RUN.
20	d)	Camadas que mudam pouco ficam embaixo.
21	a)	Além de usar usuário comum e imagem base pequena.
22	c)	É essencial quando o trabalho é reexecutado após falhas.
23	d)	A reversão é rápida.
24	c)	Se tudo estiver bem, aumenta-se gradualmente.
25	b)	Permite comparar respostas sem risco para o usuário.
26	c)	Também proteger segredos e atualizar dependências.
27	b)	Devolver probabilidades muito detalhadas pode facilitar esses ataques.
28	d)	O pipeline cuida de imputação, escala e codificação.
29	d)	É útil para orquestradores e balanceadores verificarem o serviço.
30	b)	Latência e vazão são requisitos não funcionais complementares.

Discursivas (respostas-modelo)

1. Lote: lista diária de risco de cancelamento. Tempo real: aprovar compra no cartão. Fluxo contínuo: detectar fraude em sequência de eventos. Embarcado: reconhecimento de voz no celular. Variam em latência, complexidade e dependência de rede.
2. Porque rejeita antes de chegar ao modelo dados com tipos errados, campos faltando ou valores impossíveis, evitando previsões sem sentido e registrando erros claros. Serve também de documentação viva da API.

3. Permite executar código na partida e no encerramento do serviço. Nele se carrega o modelo uma vez e o mantém em memória para todas as requisições, evitando o custo de carregar o artefato a cada pedido.
4. Instanciar o TestClient com a aplicação e chamar os endpoints: caminho feliz, campos inválidos (422), campos faltando, valores fora da faixa, lote vazio ou acima do limite, e consistência entre previsão individual e em lote.
5. A média esconde a cauda: poucos pedidos muito lentos prejudicam usuários reais. Percentis como p95 e p99 mostram a experiência dos piores casos e permitem definir objetivos de nível de serviço.
6. Porque custos fixos de chamada e de inicialização são amortizados e operações vetorizadas aproveitam melhor o hardware. Em serviços de alto volume, agrupam-se pedidos (micro-batching).
7. Contêiner empacota o programa e tudo de que precisa, compartilhando o núcleo do sistema operacional do hospedeiro; é mais leve que uma máquina virtual completa. Resolve o "funciona na minha máquina" ao reproduzir o ambiente.
8. Imagem base pequena; versões fixas; camadas ordenadas do que muda menos ao que muda mais; sem segredos na imagem; usuário comum; etiquetar com versões de código e modelo; verificar vulnerabilidades.
9. Propriedade de uma operação poder ser repetida sem mudar o resultado nem duplicar efeitos. Garante-se gravando a saída em arquivo/partição nomeada pela data de referência e sobrescrevendo, registrando um manifesto com hashes.
10. Substituição direta: risco alto. Azul-verde: baixo, com reversão rápida. Canário: baixo, limita o dano a uma fração. Sombra: muito baixo, pois as respostas do novo modelo não são usadas.
11. Validação de entrada (dados maliciosos ou absurdos); autenticação e autorização (acesso indevido); limite de taxa (abuso); cofre de segredos (vazamento de chaves); atualização de dependências (vulnerabilidades); logs de auditoria; limitação de detalhes devolvidos (inferência de associação).
12. O atacante usa as saídas do modelo para descobrir se um registro estava no treino. Reduz-se limitando detalhes das probabilidades, regularizando o modelo, limitando consultas e usando privacidade diferencial.
13. Se o pré-processamento em produção for reimplementado de forma diferente do treino, as entradas do modelo diferem. Evita-se empacotando o pré-processamento no pipeline e fazendo a API receber a linha crua.
14. Versão do modelo, data de referência, número de linhas, hash da entrada e da saída, horário de início e fim, e eventuais erros.
15. Informa versão, colunas esperadas e data de treino, ajudando clientes a conferir compatibilidade e a depurar problemas, além de apoiar a rastreabilidade.
16. Autenticação e autorização, limite de taxa, logs estruturados com identificador de requisição, múltiplos processos, monitoramento de dados e desempenho, plano de reversão e empacotamento em contêiner.
17. Para evitar consumo excessivo de memória e tempo, ataques de negação de serviço e latências imprevisíveis. Lotes muito grandes devem ir pelo fluxo de previsão em lote assíncrono.
18. É mais simples, barato e seguro de operar, não exige baixa latência nem infraestrutura online complexa; muitas decisões de negócio não precisam de resposta imediata. Quando surgir necessidade real de tempo real, migra-se.
19. O contrato define o que é esperado e rejeita o inválido; o monitoramento acompanha o que passa (distribuições, ausentes, categorias novas). Juntos, protegem o modelo de dados ruins imediatamente e ao longo do tempo.
20. Para saber exatamente o que roda em produção, reproduzir ambientes e permitir reversão rápida a uma imagem anterior conhecida.

Capítulo 8 — Testes e CI/CD para Machine Learning

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Por isso testam-se código, dados, modelo e infraestrutura.
2	a)	O ML Test Score organiza os itens nesses grupos.
3	d)	No topo ficam menos testes, mais lentos e mais caros.
4	a)	O pytest descobre automaticamente esses testes.
5	d)	É a base da verificação automática.
6	b)	Evita duplicar testes quase iguais.
7	b)	Útil para testar casos de entrada inválida.
8	c)	É onde costumam aparecer erros em produção.
9	b)	Mudanças bruscas indicam problemas na origem.
10	b)	É um teste rápido de sanidade.
11	a)	Foi sugerido pela metodologia CheckList.
12	d)	Exemplo: mais tempo de cliente não deve aumentar o risco de cancelar.
13	a)	Por exemplo, cliente antigo e premium deve ter risco menor que cliente novo e básico.
14	a)	Garante que a mudança não piorou o desempenho.
15	b)	Fixar sementes e usar tolerâncias razoáveis evita o problema.
16	b)	Se reprovar, o fluxo para.
17	d)	Qualquer outro valor indica falha.
18	c)	Juntar as mudanças com frequência e verificar cada uma automaticamente.
19	d)	As verificações baratas vêm antes das caras.
20	a)	Em áreas de alto impacto, muitas equipes preferem aprovação humana.
21	d)	O desenvolvimento é para experimentar à vontade.
22	a)	Entregar pequeno e sempre é mais seguro que entregar grande e raramente.
23	c)	Outros: agendamento, chegada de dados e detecção de deriva.
24	d)	Evita surpresas do tipo "passa aqui, falha lá".
25	b)	Segredos vazados em repositórios são um risco comum.
26	a)	Também o desempenho por fatias e o cartão do modelo.
27	d)	Limites devem ser combinados com o negócio e revisados.
28	d)	Costuma rodar antes dos testes por ser rápida.
29	a)	Dois treinos com a mesma semente devem dar o mesmo resultado.
30	d)	O piso é uma decisão de projeto, acordada com o negócio.

Discursivas (respostas-modelo)

1. Porque o comportamento depende dos dados e do modelo; dados mudados ou modelo piorado não geram exceção. Ajudam testes de dados (esquema, faixas, proporções), de modelo (desempenho, invariância, direção, fatias) e monitoramento em produção.

2. Base: testes unitários rápidos de funções e transformadores. Meio: testes de dados e de modelo (desempenho mínimo, invariância). Topo: testes de integração do pipeline e do sistema (API, treino de ponta a ponta). Quanto mais alto, menos testes, mais lentos e caros.
3. Funcionalidade mínima (cliente premium antigo tem risco menor que novo básico); invariância (trocar a cidade, sem efeito, quase não altera a previsão); expectativa direcional (mais tempo de cliente não aumenta o risco).
4. É um passo que decide se o modelo segue adiante, comparando métricas com limites. Sai com código 0 se aprova e diferente de zero se reprova; a ferramenta de CI usa esse código para interromper o fluxo.
5. Fixar sementes, usar conjuntos de teste fixos e versionados, definir tolerâncias razoáveis, repetir com várias sementes quando necessário e exigir desempenho médio acima do limite.
6. Baixar o código; instalar dependências em ambiente limpo; estilo e tipos; testes de código e de dados; treino rápido em amostra pequena; portão de qualidade; construção da imagem; relatório do resultado.
7. Entrega: mudança aprovada fica pronta e um humano aprova a ida à produção. Implantação: vai sozinha. Em áreas de alto impacto (crédito, saúde), prefere-se a entrega com aprovação humana; em contextos de baixo risco, a implantação contínua.
8. Desenvolvimento: experimentar com dados simulados ou amostras. Homologação: ensaio geral com dados parecidos com produção. Produção: atender usuários reais. A progressão reduz o risco de problemas chegarem aos usuários.
9. É o retreinamento automático por gatilhos (agenda, dados novos, queda de qualidade, deriva). Exige portão de qualidade, comparação com o campeão, registro do candidato, investigação da causa e capacidade de reverter.
10. Porque testes cobrem só o que foi previsto; revisores humanos avaliam a lógica, o risco de vazamento, o desempenho por fatias, a documentação e questões éticas que escapam dos testes.
11. Com parametrize, o mesmo teste roda com vários valores (vazio, zero, negativo, muito grande), aumentando a cobertura com pouco código e facilitando adicionar casos novos.
12. Criar um DataFrame, copiar seu estado, chamar fit_transform e comparar colunas e valores da entrada original; o transformador deve trabalhar sobre uma cópia.
13. Para verificar se a rede de proteção detecta falhas reais, como rótulos embaralhados ou ausência de padronização. Se os testes continuam passando com um modelo quebrado, eles são inúteis ou frouxos.
14. Apertados demais fazem os testes falhar à toa (frágeis); frouxos demais não protegem nada. Os limites devem ser combinados com o negócio e baseados em evidências.
15. Para verificar que o pipeline executa de ponta a ponta, que não há erros de código e que as métricas mínimas básicas são atingidas, de forma rápida. O treino completo ocorre em outra etapa.
16. Estilo (ruff), tipos (mypy), cobertura de testes, varredura de vulnerabilidades, construção e publicação de imagem, implantação em homologação e testes de integração.
17. Mudanças pequenas são mais fáceis de revisar, testar e reverter, geram menos conflitos e diminuem o risco de falhas, conforme a evidência sobre equipes de alto desempenho.
18. Em desenvolvimento, testam-se o código e a lógica sobre amostras fixas; em produção, valida-se cada lote real que chega. São complementares: o primeiro garante que o código funciona; o segundo, que os dados reais atendem ao esperado.
19. O treino rápido gera métricas piores; os testes de desempenho mínimo ou o portão de qualidade falham, a CI sai com código diferente de zero e o fluxo é interrompido, impedindo que o modelo ruim avance.
20. Porque definem o que é aceitável para o projeto e evoluem com ele; versioná-los permite saber quais critérios valiam para cada versão do modelo e reproduzir a decisão de aprovação.

Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	Como o pão que endurece sem ninguém tocar nele.
2	b)	O mesmo perfil passa a ter respostas diferentes.
3	b)	A relação entre perfil e resposta pode continuar a mesma.
4	b)	Só P(y) muda; dentro de cada classe, os atributos se distribuem como antes. Mudar a frequência dos positivos por mudar P(y | X) é outro fenômeno.
5	c)	Outras formas: abrupta, gradual e incremental.
6	b)	Por isso são sinais rápidos, embora indiretos.
7	a)	Útil para atributos numéricos.
8	b)	Por isso se deve olhar também o tamanho da diferença.
9	a)	Abaixo de 0,10 é estável; entre 0,10 e 0,25, moderada.
10	a)	Quanto mais diferentes as distribuições, maior o PSI.
11	b)	Vai de 0 (iguais) a 1 (muito diferentes, na base 2).
12	b)	Também é essencial vigiar o aparecimento de categorias novas.
13	a)	AUC próxima de 0,5 indica ausência de diferença detectável por esse classificador e amostra; não prova igualdade das distribuições.
14	a)	Os testes univariados não enxergam mudanças na dependência entre atributos.
15	c)	Um salto na probabilidade média sugere algo mudou no mundo ou no pipeline.
16	b)	Exemplo: um cliente só cancela meses depois.
17	d)	Alerta com mínimo + 2 desvios; deriva com mínimo + 3 desvios.
18	a)	A janela se ajusta conforme a estabilidade dos dados.
19	a)	Esse é o valor do alerta antecipado sem rótulos.
20	d)	Deve-se investigar a causa antes de retreinar.
21	b)	Bom para deriva de conceito; a janela curta dá pouco dado, a longa reage devagar.
22	a)	É sensível a ruído e a dados ruins.
23	d)	Reduz o risco de decisões baseadas no candidato, mas ainda consome recursos e pode ter efeitos operacionais.
24	b)	A aleatoriedade permite atribuir diferenças ao modelo.
25	b)	Olhar o resultado toda hora e parar cedo distorce a estatística.
26	c)	Sempre olhe o tamanho do efeito e o intervalo de confiança.
27	c)	Depois, quando disponível, o rótulo real.
28	b)	Prefira poucos alertas, bem escolhidos e acionáveis.
29	c)	Planeja a resposta antes que o incidente aconteça.
30	d)	Mostra que algo mudou, mas não diz quanto o desempenho piorou.

Discursivas (respostas-modelo)

1. Porque o modelo continua respondendo mesmo piorando. Monitoram-se: serviço (latência, erros), dados de entrada (ausentes, distribuição), previsões (distribuição das saídas), desempenho (com rótulos, muitas vezes atrasados) e negócio.
2. Covariáveis: muda P(X), p. ex., campanha atrai público mais jovem. Rótulos: muda P(y) com P(X | y) estável, p. ex., uma doença fica mais frequente mas os sintomas de quem a tem não mudam. Conceito: muda a relação X→y, p. ex., o mesmo perfil passa a se comportar de modo diferente.
3. Abrupta: uma lei nova de um dia para outro. Gradual: o conceito antigo e o novo convivem por um tempo. Incremental: deslizamento lento e contínuo. Recorrente: volta periodicamente, como a sazonalidade de fim de ano.
4. Divide o atributo em faixas (por exemplo, decis da referência) e soma $(q-p)\cdot\ln(q/p)$ sobre as faixas. Valores < 0,10 sugerem estabilidade, entre 0,10 e 0,25 mudança moderada e > 0,25 importante. Os limiares são convenções práticas e devem ser ajustados ao contexto.
5. Com muitos dados, qualquer diferença minúscula fica "significativa". É preciso avaliar o tamanho da diferença (KS, PSI) e o impacto no desempenho, definindo limiares de relevância.
6. Treina-se um modelo para distinguir dados de referência de dados atuais; AUC próxima de 0,5 indica ausência de diferença detectável pelo classificador e pela amostra usados, e AUC alta indica diferença. As importâncias dos atributos apontam quais contribuem mais para a distinção, ajudando a investigar a causa.
7. Esperar os rótulos e calcular com atraso; amostrar casos para rotulagem rápida; usar indicadores adiantados (cliques, devoluções); acompanhar sinais sem rótulo (dados e previsões); e usar detectores sobre o fluxo de erros quando os rótulos chegam.
8. Acompanha a taxa de erro p e o desvio s, guardando o menor p+s. Se p+s passar de mínimo + 2 desvios, emite alerta; passar de mínimo + 3 desvios, deriva. Indica aumento significativo da taxa de erro.
9. Estático: nunca retreina e envelhece. Periódico: ritmo fixo, com todos os dados. Janela: só dados recentes, bom para deriva. Incremental: atualiza por lote, para fluxos contínuos. Gatilho: retreina quando o monitoramento acusa, economizando recursos.
10. Porque o alerta pode vir de falha de dados (sensor quebrado, unidade trocada); retreinar com esses dados ensinaria o erro. Deve-se corrigir a origem antes e passar o candidato pelos portões de qualidade.
11. Em sombra, o modelo novo prevê sem afetar usuários e compara-se com o atual. Em canário, atende pequena fração do tráfego e amplia-se se estiver bem. Se métricas pioram, reverte-se movendo o apelido para o modelo anterior.
12. Define-se a taxa base, o efeito mínimo relevante, o nível de significância e o poder, e calcula-se n por grupo. Olhar toda hora e parar quando "fica bonito" infla falsos positivos.
13. Identificador, horário, versão do modelo, entradas ou resumo, saída, latência e rótulo posterior. Cuidados: minimização, pseudonimização, retenção limitada e controle de acesso.
14. Poucos alertas bem escolhidos; níveis de gravidade com destinatários distintos; limiares baseados em dados e revisados; painéis simples; plano de resposta; revisão pós-incidente sem culpados.
15. Porque o novo modelo foi treinado em um mundo diferente; comparar os dados atuais com a referência antiga manteria o alerta disparado indefinidamente. A nova referência reflete o regime atual.
16. Que o PSI sobe muito antes de a acurácia cair de forma clara, mostrando o valor dos sinais sem rótulos como alerta antecipado, enquanto a acurácia só confirma com atraso.
17. Se a relação X→y permanece, a ordenação das previsões pode continuar boa (AUC estável), mas probabilidades podem ficar descalibradas e o modelo extrapola em regiões pouco vistas. Exige monitoramento e, eventualmente, recalibração.
18. O de dados vê as entradas e saídas sem rótulos e é rápido; o de desempenho usa rótulos reais e é decisivo, mas atrasado. Juntos, dão alerta precoce e confirmação.

19. Porque, na deriva de conceito, a distribuição dos atributos pode permanecer a mesma e só muda a relação com o alvo, visível apenas com rótulos. Por isso é preciso monitorar o desempenho com rótulos e fazer retreinos de segurança.
20. Como confirmar o alerta; quem acionar; verificação de dados e de versão; quando e como reverter para o modelo anterior; critérios para retreinar; comunicação com o negócio e registro do incidente.

Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Responde a quem é responsável, para que serve, com quais dados e como se fiscaliza.
2	c)	É a "bula" do modelo.
3	d)	Pode ser exigido no portão de promoção.
4	b)	A local explica uma previsão individual.
5	d)	Foi introduzida por Breiman (2001) no contexto de florestas.
6	c)	Explicações devem ser acompanhadas da análise de correlações.
7	d)	Foi descrita por Friedman (2001) e pode esconder interações.
8	b)	É uma explicação local sensível à vizinhança escolhida.
9	b)	Oferece boa fundamentação teórica, com custo computacional maior.
10	a)	Servem para investigar e levantar hipóteses.
11	d)	A regra dos quatro quintos usa 0,8 como referência de alerta, não como veredito.
12	a)	Entre os casos positivos segundo o critério definido, a fração aprovada é parecida.
13	a)	Atributos correlacionados, como CEP, podem funcionar como proxies.
14	d)	A escolha do critério é uma decisão social e jurídica.
15	d)	Lei n.º 13.709/2018.
16	b)	Regulamento (UE) 2024/1689.
17	c)	Isso é obtido com ruído calibrado.
18	c)	Há um equilíbrio entre privacidade e precisão.
19	d)	Pois uma pessoa altera a contagem em no máximo 1.
20	a)	Os orçamentos de ε se somam.
21	b)	Não elimina todos os riscos e costuma ser combinado a outras proteções.
22	c)	A offline serve ao treino e à previsão em lote.
23	c)	Errar isso é um vazamento comum.
24	a)	Evita usar valores do futuro.
25	a)	O orquestrador executa na ordem certa e repete falhas.
26	c)	Exemplos de ferramentas: Airflow, Prefect, Dagster, Kubeflow.
27	a)	Reduz invenções e permite atualizar o conhecimento sem retreinar.
28	d)	Por isso prompts são versionados como código.
29	d)	Combinam-se testes automáticos, revisão humana e métricas de negócio.
30	a)	Diminui custo e tempo de ajuste.

Discursivas (respostas-modelo)

1. Documentação (cartão do modelo); explicabilidade (importância por permutação); justiça (métricas por grupo); privacidade (pseudonimização, privacidade diferencial); segurança (validação de entradas, controle de artefatos); auditoria (registro e rastreabilidade).

2. Detalhes do modelo, uso pretendido e fora do escopo, fatores, métricas, dados, análises desagregadas, considerações éticas e recomendações. O desempenho por grupo revela desigualdades que a média esconde.
3. Global: o que importa em geral, p. ex., importância por permutação e PDP. Local: por que uma previsão específica saiu assim, p. ex., LIME e SHAP.
4. Mostram o que o modelo usa. Se um atributo é proxy de outro ou correlacionado com a causa real, ele aparece como importante sem ser causa. Por exemplo, o CEP pode importar por refletir renda, não por causar inadimplência.
5. Outros atributos podem carregar a informação (proxies) ou ser medidos com viés diferente entre grupos. Deve-se medir desempenho e seleção por grupo, investigar a origem do viés e aplicar remédios em dados, treino ou saída.
6. Paridade demográfica: mesma taxa de seleção. Igualdade de oportunidade: mesma revocação entre os casos positivos segundo o critério definido. Paridade de precisão: mesma precisão quando aprova. Em geral não se satisfazem todas ao mesmo tempo.
7. Escolhe-se, para cada grupo, o limiar que captura a mesma fração de positivos reais. Isso melhora a igualdade de oportunidade, mas altera taxas de seleção e precisão, podendo gerar dilemas éticos e legais; a decisão é social, não só técnica.
8. Garantia matemática de que as probabilidades dos resultados de uma análise mudam pouco, de forma limitada por ε, entre bases vizinhas (com e sem uma pessoa). Obtém-se com ruído. ε controla o equilíbrio: menor ε, mais privacidade e mais ruído; maior ε, menos privacidade e mais precisão.
9. O atacante tenta descobrir se um registro estava no treino a partir das saídas do modelo. Mitiga-se limitando detalhes das saídas, regularizando, limitando consultas e usando privacidade diferencial.
10. Os dados ficam nos dispositivos, que treinam localmente e enviam atualizações combinadas por um servidor central. Limitação: não elimina todos os riscos (atualizações podem vazar informação), exigindo proteções adicionais.
11. Exemplos adversariais (testes de robustez); envenenamento de dados (validação de dados e controle de fontes); roubo de modelo (limitar consultas); cadeia de suprimentos (assinar e verificar artefatos, evitar pickle desconhecido).
12. É preciso ligar hash dos dados, pipeline e configuração versionados, execução e métricas registradas, modelo registrado e aprovado, registro de cada previsão e cartão do modelo para reconstruir qual modelo, dados e código produziram uma decisão.
13. Evita recriar atributos de maneiras diferentes (retrabalho e desvio treino–serviço). O offline guarda o histórico para treino e lote; o online serve valores recentes em milissegundos para tempo real, ambos alimentados pelo mesmo cálculo.
14. Usar o valor mais recente do atributo para um evento passado vaza o futuro. merge_asof junta cada evento ao último valor disponível até a data do evento, garantindo a correção temporal.
15. Executa as tarefas na ordem das dependências, repete falhas, agenda e guarda o histórico. Depender significa só poder iniciar após a conclusão da outra; tarefas independentes podem rodar em paralelo.
16. Construir dá controle, mas exige equipe e manutenção; comprar economiza tempo, mas gera dependência e custo. Evitar reinventar o que a comunidade mantém, começar pequeno e crescer conforme a necessidade.
17. O "programa" inclui prompts e contexto; a saída é texto livre, difícil de avaliar; o custo é por token e há riscos como alucinação e injeção de prompt. Também: RAG e ajuste leve em vez de treinar do zero.
18. Registrar cada prompt com identificador (hash) e histórico, como código; manter um conjunto de casos de teste com critérios (deve conter, não pode conter), rodar regressão a cada mudança e comparar versões; monitorar em produção.
19. O juiz também erra, tem vieses e pode favorecer respostas parecidas com as suas; por isso combina-se com testes automáticos, revisão humana por amostragem e métricas de negócio.

20. Documentar com cartão do modelo, avaliar por grupo e corrigir vieses de dados, explicar decisões, proteger dados pessoais, manter rastreabilidade, ter supervisão humana e plano de reversão, monitorar desempenho e razão de impacto continuamente.

Capítulo 11 — Projeto Final

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	A pergunta fala de confiança ao longo do tempo, e não de algoritmos.
2	c)	A produção e a deriva são simuladas.
3	d)	O mundo entra em um novo regime.
4	b)	Sem dados validados e versionados, nada mais é confiável.
5	c)	Esses dados alimentam as 8 semanas de produção simulada.
6	a)	O F1 macro trata as dez classes com igual importância.
7	b)	Todos são registrados no MLflow.
8	b)	A decisão deve se basear no problema, e não no hábito.
9	c)	Aprovado, o modelo é registrado como champion.
10	c)	Promover é mover o apelido; o serviço precisa recarregar o modelo.
11	a)	Retorna 422 para dados inválidos.
12	a)	Rodam sobre o artefato exportado do champion.
13	a)	Isso limita quais lotes podem ser usados no retreino.
14	c)	O desfoque reduz o contraste e muda essa distribuição.
15	a)	Não depende de rótulos, por isso é rápida.
16	b)	Evita avaliar no que treinou e vazamento temporal.
17	a)	Evita trocar o modelo por ruído.
18	d)	Assim os alertas passam a comparar com o mundo novo.
19	c)	O resultado é uma simulação, mas ilustra o valor do CT bem controlado.
20	a)	Os rótulos da semana s só estão disponíveis na semana seguinte.
21	b)	Desempenho por classe evita que a média esconda dígitos mal reconhecidos.
22	b)	A média pode esconder desigualdades.
23	a)	Olhar as imagens antes de retreinar é parte do processo.
24	a)	Preserva a estimativa honesta.
25	a)	A simulação permite experimentar sem risco, mas exige cuidado ao generalizar.
26	c)	Escreve-se para públicos diferentes.
27	d)	Também experimentar outros repositórios (UCI, Kaggle).
28	b)	MLOps é uma cultura de responsabilidade sobre o que colocamos no mundo.
29	d)	Cada passagem é protegida por um portão.
30	d)	Execuções guardam parâmetros, métricas, tags e o modelo.

Discursivas (respostas-modelo)

1. Dados (Cap. 3); candidatos (4 e 6); rastreamento e registro (5); serviço (7); testes (8); produção simulada e monitoramento (9); treinamento contínuo (5, 8 e 9); governança e relatório (10).
2. Portão de dados: evita treinar com lotes ruins. Portão de qualidade: impede registrar modelo abaixo do piso ou sem ganho sobre a linha de base. Portão de promoção: evita trocar o campeão por ruído. Juntos, protegem o sistema em cada passagem.

3. O "futuro" simula a produção com dados nunca vistos, e o teste original dá uma estimativa honesta no fim. Usá-los para escolhas contaminaria a avaliação.
4. A entrada tem 64 números entre 0 e 16. Válida: 200 com dígito previsto e versão do modelo. Inválida (tamanho errado ou valores fora da faixa): 422, sem chamar o modelo.
5. Desempenho mínimo (impede modelo ruim); invariância a ruído (robustez); cobertura de classes (nenhuma classe esquecida); determinismo (reprodutibilidade).
6. O desfoque e o desbotamento crescem nas quatro primeiras semanas e depois se estabilizam. A estabilização cria um novo regime, no qual o retreino com rótulos recentes pode, de fato, recuperar o desempenho.
7. Porque a mudança nas entradas é perceptível imediatamente, enquanto o impacto no desempenho depende da sensibilidade do modelo e só é confirmado quando os rótulos chegam. Os sinais sem rótulo são alerta antecipado.
8. Ao haver alerta e rótulos de semanas anteriores, treina-se um candidato com os dados originais e os lotes rotulados; compara-se com o campeão no lote rotulado mais recente; se ganhar por margem, promove-se o apelido; atualiza-se a referência do monitoramento.
9. O lote recente representa o mundo atual, que é o que importa; o teste original representa o passado e deve ser reservado. Além disso, o candidato não treinou nesse lote, evitando avaliação otimista.
10. Versão e dados de treino (hash), métricas gerais e por classe, desempenho na última semana de produção, limitações (imagens 8×8; câmera desfocada exige retreino) e uso indicado (triagem com revisão humana).
11. Porque dígitos específicos podem ser mal reconhecidos, e a média esconde. Identificar a pior classe orienta correções, coleta de dados e supervisão humana.
12. Vazamento temporal (usar só rótulos já chegados); avaliar no que treinou (comparar em dados inéditos para os dois modelos); retreinar sem investigar (olhar a causa do alerta antes); usar o teste para escolher (reservá-lo ao relatório).
13. O sistema lê dígitos de formulários com alta qualidade hoje; foi construído com dados validados, testes e registro; monitora a qualidade das imagens e o desempenho; reage a mudanças da câmera retreinando com portões; recomenda-se manter revisão humana por amostragem e plano de reversão.
14. Na simulação, a deriva é controlada e conhecida, os rótulos têm atraso fixo e o ambiente é simples. Na realidade, as causas são variadas e desconhecidas, os rótulos são ruidosos e incompletos, e há restrições de custo, equipe e conformidade.
15. O código carrega o modelo pelo apelido champion; para reverter, move-se o apelido para uma versão anterior validada e recarrega-se o serviço, sem editar o código; confere-se a versão servida.
16. Trocar o conjunto de dados; usar ColumnTransformer com imputação e codificação; escolher métricas de negócio e limiar por custo; criar atributos temporais; definir o tipo de deriva esperado (covariáveis e rótulos); adaptar os testes e as fatias (regiões, planos).
17. Porque um modelo robusto também falha com dados corrompidos ou muito diferentes do treino; o monitoramento dá alerta precoce, ajuda a distinguir problema de dados de mudança do mundo e evita retreinar com lixo.
18. Escrever um fluxo de CI que instale dependências, rode estilo e testes, treine em amostra, aplique o portão e construa a imagem; criar um Dockerfile com imagem base pequena, versões fixas, código e modelo, usuário comum e comando para iniciar a API.
19. Resposta pessoal. Exemplos: versionar dados e fixar o ambiente; validar dados; usar pipelines; rastrear experimentos; testar código, dados e modelo; monitorar em camadas; retreinar com portões; documentar com cartão do modelo e considerar justiça e privacidade.
20. O PSI olha um atributo (ou uma estatística) por vez; o classificador de domínio detecta diferenças multivariadas entre referência e lote, inclusive mudanças na relação entre atributos, e indica a existência de deriva mesmo quando cada marginal parece normal.

Apêndice B — Glossário

Termos técnicos usados no livro, explicados de forma simples.

A/B (teste) — experimento em que o tráfego é dividido aleatoriamente entre duas versões para comparar o efeito no negócio.

Acurácia — fração de previsões corretas; engana em classes desbalanceadas.

ADWIN — método de detecção de mudança com janela adaptativa.

Alerta (monitoramento) — aviso gerado quando uma medida passa de um limite definido.

AP (precisão média) — resumo da curva de precisão-revocação por soma ponderada das precisões; não é necessariamente igual à área trapezoidal. Para um classificador aleatório, é próxima da fração de positivos.

Apelido (alias) — nome móvel que aponta para uma versão de modelo no registro, como champion.

API — porta de entrada padronizada pela qual programas se comunicam, enviando pedidos e recebendo respostas.

Armazém de atributos (feature store) — sistema que guarda atributos calculados uma só vez e reutilizados no treino e no serviço.

Artefato — arquivo produzido por uma execução, como um modelo, um gráfico ou um relatório.

Atributo (feature) — característica usada como entrada de um modelo.

AUC-ROC — área sob a curva ROC; mede a capacidade de ordenar positivos acima de negativos.

Azul-verde (implantação) — dois ambientes idênticos; o tráfego passa de um para o outro, com reversão rápida.

Baseline (linha de base) — solução simples contra a qual o modelo precisa ganhar.

Batch (lote) — previsão feita em horários marcados sobre muitos registros.

Bootstrap — reamostragem com reposição para estimar a incerteza de uma métrica.

Brier (escore de) — erro quadrático médio das probabilidades previstas; menor é melhor.

CACE — "Changing Anything Changes Everything"; mudar um componente pode mudar todo o sistema.

Calibração — concordância entre probabilidades previstas e frequências observadas.

Canário (implantação) — o modelo novo atende uma pequena fração do tráfego antes de ser liberado a todos.

Cartão do modelo (model card) — documento curto com uso pretendido, dados, métricas, limitações e riscos de um modelo.

CD (entrega ou implantação contínua) — prática de levar mudanças aprovadas à produção de forma automática.

Champion / challenger — modelo em produção / modelo candidato.

CI (integração contínua) — prática de verificar automaticamente cada mudança de código.

Classificador de domínio — modelo treinado para distinguir dados de referência de dados atuais; sua AUC mede a diferença.

Contêiner — pacote leve que leva o programa e tudo de que precisa para rodar.

Contrato de dados — descrição formal do que um serviço recebe e devolve (campos, tipos e intervalos).

Covariate shift (mudança de covariáveis) — mudança na distribuição dos atributos de entrada.

CT (treinamento contínuo) — retreinamento automático do modelo por gatilhos.

DAG — grafo acíclico dirigido; tarefas com dependências e sem ciclos.

Data leakage (vazamento de dados) — uso, no treino, de informação indisponível na hora de prever.

Datasheet — ficha técnica que documenta um conjunto de dados.

DDM — método de detecção de deriva baseado na taxa de erro e seu desvio.

Deriva de conceito (concept drift) — mudança na relação entre atributos e alvo.

DevOps — conjunto de práticas que aproxima desenvolvimento e operação de software.

Dívida técnica — custo futuro de atalhos tomados hoje.

Docker — ferramenta popular de contêineres.

Dockerfile — receita que descreve como montar uma imagem de contêiner.

DVC — ferramenta que versiona dados guardando ponteiros no Git e os arquivos em armazenamento remoto.

Embarcado (edge) — modelo executado no próprio dispositivo.

Endpoint — endereço específico de uma API, como /previsao.

Execução (run) — uma tentativa de treino registrada no rastreamento de experimentos.

Experimento (MLflow) — grupo de execuções com um mesmo objetivo.

Explicabilidade — conjunto de técnicas para entender por que um modelo decide como decide.

F1 — média harmônica entre precisão e revocação.

Falha silenciosa — piora das respostas sem erro de execução.

FastAPI — biblioteca Python para criar APIs.

Fatia (slice) — subgrupo dos dados usado para avaliar o desempenho em separado.

Federado (aprendizado) — treino em que os dados ficam nos dispositivos e só atualizações do modelo são combinadas.

Gate (portão de qualidade) — etapa que decide se o fluxo pode seguir adiante.

Hash — impressão digital de um conteúdo; muda por completo se o conteúdo mudar.

Hiperparâmetro — escolha feita antes do treino, como o número de árvores.

Idempotência — propriedade de repetir uma operação sem mudar o resultado nem duplicar efeitos.

Invariância (teste de) — verifica que uma mudança irrelevante altera pouco a previsão.

Jensen-Shannon (distância) — medida simétrica e limitada de diferença entre distribuições.

JSON — formato de texto estruturado em chaves e valores.

KS (teste de Kolmogorov-Smirnov) — compara as distribuições acumuladas de duas amostras.

Label shift (mudança de rótulos) — mudança na proporção das classes, P(y), com P(X | y) estável.

Latência — tempo entre o pedido e a resposta.

LGPD — Lei Geral de Proteção de Dados Pessoais (Lei n.º 13.709/2018).

Linhagem (lineage) — caminho completo de um dado ou modelo, da origem ao destino.

LLMOps — operação de sistemas baseados em modelos de linguagem de grande porte.

LoRA — técnica de ajuste leve que treina poucos parâmetros adicionais.

Métrica — número que mede o resultado de uma execução.

ML Test Score — rubrica de 28 itens para avaliar a prontidão de um sistema de ML para produção.

MLflow — plataforma de código aberto para rastrear experimentos e registrar modelos.

MLOps — práticas, processos e ferramentas para desenvolver, entregar, operar e melhorar sistemas de ML.

Model registry (registro de modelos) — catálogo de modelos com versões, apelidos e metadados.

Monitoramento — acompanhamento contínuo da saúde, dos dados e da qualidade de um sistema em produção.

One-hot encoding — transformação de uma coluna categórica em colunas de 0 e 1.

Orquestrador — sistema que executa tarefas na ordem das dependências, repete falhas e guarda o histórico.

Parâmetro — no MLflow, valor de entrada fixo durante a execução; em ML, valor aprendido pelo modelo.

Percentil (p50, p95, p99) — valor abaixo do qual está a fração indicada dos casos.

Pipeline (MLOps) — fluxo que une ingestão, validação, treino, avaliação, registro e implantação.

Pipeline (scikit-learn) — objeto que encadeia pré-processamento e modelo.

Point-in-time (correção temporal) — uso dos valores dos atributos como eram na data do evento.

Precisão — fração de previsões positivas que estavam certas.

Privacidade diferencial — garantia de que o resultado muda pouco com a presença de uma pessoa nos dados.

Pseudonimização — troca de identificadores diretos por códigos que só identificam com informação adicional guardada à parte.

PSI (Population Stability Index) — medida de mudança de distribuição entre referência e dados atuais.

Pydantic — biblioteca Python para validar dados por meio de classes.

pytest — biblioteca Python para testes automáticos.

Quarentena (de dados) — área onde lotes reprovados ficam isolados.

RAG — geração com recuperação; busca documentos relevantes e os insere no comando.

Rastreabilidade — capacidade de saber qual código, dados e configuração geraram cada modelo.

Reprodutibilidade — obter o mesmo resultado com o mesmo código e os mesmos dados.

Revocação (recall) — fração dos casos positivos reais que o modelo encontrou.

Runbook — plano de resposta com os passos a seguir diante de um alerta.

Signature (assinatura) — descrição dos tipos e nomes de entrada e saída de um modelo.

Skew treino–serviço — diferença entre os dados ou o processamento do treino e os da produção.

Sombra (modo) — o modelo novo recebe cópia dos pedidos, mas suas respostas não são usadas.

Tag — rótulo livre usado para organizar execuções e modelos.

Throughput (vazão) — quantidade de previsões atendidas por unidade de tempo.

Validação aninhada — validação cruzada com laço interno para escolher e externo para avaliar.

Validação cruzada — repetição de treino e teste em divisões diferentes dos dados.

Versionamento — manutenção do histórico de versões de código, dados e modelos.

Apêndice C — Guia rápido de decisões, ferramentas e códigos

C.1 Mapa de decisão

Se você quer...	Considere	Capítulo
Reproduzir um treino em outra máquina	Semente fixa, `requirements.txt`, ambiente virtual, contêiner	1, 4, 7
Saber se o projeto está pronto para produção	ML Test Score (28 itens, nota = mínimo dos grupos)	2
Garantir a qualidade dos dados de entrada	Esquema, validação com portão, quarentena	3
Identificar a versão dos dados	Hash SHA-256, manifesto, DVC	3
Evitar vazamento no pré-processamento	`Pipeline`, `ColumnTransformer`, `fit` só no treino	4
Registrar experimentos	MLflow Tracking	5
Trocar ou reverter o modelo sem editar o código	Registro de modelos com apelido `champion` + recarga do serviço	5, 7
Avaliar com classes raras	Precisão, revocação, F1, AP	6
Escolher o limiar de decisão	Custo dos erros, escolha na validação	6
Comparar modelos com rigor	Validação cruzada repetida, bootstrap, validação aninhada	6
Ajustar hiperparâmetros	Busca aleatória, Optuna, Hyperband	6
Servir previsões	Lote (padrão), tempo real (FastAPI), fluxo contínuo, embarcado	7
Rejeitar entradas inválidas	Contrato com Pydantic (HTTP 422)	7
Empacotar o ambiente	Contêiner (Docker)	7
Testar o sistema	`pytest`: código, dados e modelo; portão de qualidade	8
Automatizar a entrega	CI/CD; treinamento contínuo (CT)	8
Detectar mudança em um atributo	KS, PSI, Jensen-Shannon, qui-quadrado	9
Detectar mudança multivariada	Classificador de domínio (AUC)	9
Detectar piora com rótulos	DDM, ADWIN, métricas móveis	9
Lançar um modelo novo com segurança	Sombra, canário, teste A/B, reversão	9
Documentar um modelo	Cartão do modelo (*model card*)	10
Explicar um modelo	Permutação, PDP, LIME, SHAP	10
Auditar justiça	Taxa de seleção, revocação e precisão por grupo	10
Proteger privacidade	Minimização, pseudonimização, privacidade diferencial	3, 10
Evitar vazamento do futuro em atributos	Junção temporal (`merge_asof`), armazém de atributos	10
Executar tarefas em ordem	Orquestrador (DAG)	10

C.2 Classes, funções e comandos principais

Tarefa	Ferramenta	Pontos de atenção
Pipeline	`sklearn.pipeline.Pipeline`, `make_pipeline`	Nomeie as etapas; use `fit` só no treino
Colunas mistas	`sklearn.compose.ColumnTransformer`	`OneHotEncoder(handle_unknown="ignore")`
Divisão por grupo	`GroupShuffleSplit`, `GroupKFold`	Evita o mesmo indivíduo nos dois lados
Série temporal	`TimeSeriesSplit`	Treino no passado, teste no futuro
Busca de hiperparâmetros	`GridSearchCV`, `RandomizedSearchCV`	Nunca use o teste
Calibração	`CalibratedClassifierCV`, `calibration_curve`	Compare Brier antes e depois
Explicação	`sklearn.inspection.permutation_importance`, `partial_dependence`	Importância não é causa
Salvar modelo	`joblib.dump` / `joblib.load`	Registre a versão do scikit-learn; só abra arquivos confiáveis
Rastrear	`mlflow.start_run`, `log_param`, `log_metric`, `log_model`	Registre hash dos dados
Registrar	`mlflow.register_model`, `set_registered_model_alias`	Carregue por `models:/nome@champion`
Contrato	`pydantic.BaseModel`, `Field(ge=..., le=...)`	Rejeita dados inválidos antes do modelo
API	`fastapi.FastAPI`, `TestClient`	Carregue o modelo uma vez (`lifespan`)
Testes	`pytest`, `parametrize`, `raises`	Fixe sementes; use tolerâncias
DAG	`graphlib.TopologicalSorter`	Dependências sem ciclos

C.3 Lista de verificação de prontidão para produção

- [ ] Dados validados por esquema; hash/versão registrados.
- [ ] Linha de base definida; métrica de negócio e custo dos erros claros.
- [ ] Pré-processamento dentro do pipeline; sem vazamento.
- [ ] Sementes e versões de bibliotecas fixadas; treino reproduzível.
- [ ] Execuções registradas; modelo no registro, com assinatura e apelido.
- [ ] Avaliação rigorosa: validação cruzada, incerteza, calibração, limiar e fatias.
- [ ] Contrato de entrada e saída; testes de código, dados e modelo.
- [ ] CI com portão de qualidade; imagem versionada; plano de reversão.
- [ ] Monitoramento de dados, previsões e desempenho; alertas e runbook.
- [ ] Cartão do modelo; avaliação de justiça e privacidade; responsáveis definidos.

C.4 Valores de referência usados no livro

Item	Valor	Observação
PSI	< 0,10 estável; 0,10 a 0,25 moderada; > 0,25 importante	Regra prática, não lei
Razão de impacto	0,8	"Regra dos quatro quintos": sinal de alerta, não veredito
Intervalo de confiança	95% (percentis 2,5 e 97,5)	Bootstrap
DDM	alerta em mínimo + 2 desvios; deriva em mínimo + 3	Gama et al. (2004)
ML Test Score	0 / 0,5 / 1 ponto por item; nota = mínimo dos grupos	Breck et al. (2017)

> **Atenção**
>
> Todos os valores acima são **convenções**. Ajuste-os ao seu contexto, ao risco do problema e ao histórico de variação normal dos seus dados.

Apêndice D — Soluções dos exercícios práticos

Cada solução traz o código, a **saída obtida** na elaboração do livro e um comentário sobre o que observar. Há várias maneiras corretas de resolver um exercício: o importante é que o seu resultado seja coerente e que você consiga explicá-lo. Pequenas diferenças numéricas podem ocorrer com outras versões das bibliotecas. Os gráficos aparecem quando você executa o código.

Capítulo 1 — Introdução ao MLOps

Exercício 1 — O custo de não fixar a semente

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

def acuracia(semente):
    return RandomForestClassifier(n_estimators=20, random_state=semente).fit(Xtr, ytr).score(Xte, yte)

sem = [acuracia(None) for _ in range(15)]
com = [acuracia(7) for _ in range(15)]
print(f"Sem semente: média = {np.mean(sem):.4f} | desvio = {np.std(sem):.4f} | valores distintos =
{len(set(sem))}")
print(f"Com semente: média = {np.mean(com):.4f} | desvio = {np.std(com):.4f} | valores distintos =
{len(set(com))}")
```

SAÍDA

```
Sem semente: média = 0.9439 | desvio = 0.0104 | valores distintos = 7
Com semente: média = 0.9591 | desvio = 0.0000 | valores distintos = 1
```

EXEMPLO **O que observar**

Com a semente fixa, o desvio é zero e todos os resultados são iguais. Sem semente, o resultado oscila de uma execução para outra. Em um relatório, uma diferença de 0,5 ponto entre dois modelos pode ser só essa oscilação.

Exercício 2 — Salvando o modelo com seus metadados

PYTHON

```python
import hashlib
import joblib
import numpy as np
import sklearn
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=1)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(Xtr, ytr)

artefato = {
    "pipeline": pipe,
    "versao_sklearn": sklearn.__version__,
```

```
    "semente": 1,
    "acuracia_teste": round(pipe.score(Xte, yte), 4),
    "hash_dados": hashlib.sha256(np.c_[X, y].tobytes()).hexdigest()[:16],
}
joblib.dump(artefato, "wine_v1.joblib")

lido = joblib.load("wine_v1.joblib")
print({k: v for k, v in lido.items() if k != "pipeline"})
print("Previsões idênticas:", np.array_equal(pipe.predict(Xte), lido["pipeline"].predict(Xte)))
```

SAÍDA

```
{'versao_sklearn': '1.9.1', 'semente': 1, 'acuracia_teste': 0.9815, 'hash_dados': 'b6ff7c6847be41ee'}
Previsões idênticas: True
```

EXEMPLO O que observar

O hash dos dados permite provar depois com qual versão do conjunto o modelo foi treinado. A versão do scikit-learn avisa quando o arquivo é aberto em um ambiente diferente.

Exercício 3 — A ordem dos dados também importa

PYTHON

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X / 16.0, y, test_size=0.3, stratify=y, random_state=0)

def treinar(modelo_fn, semente_ordem):
    idx = np.arange(len(Xtr))
    if semente_ordem is not None:
        idx = np.random.default_rng(semente_ordem).permutation(len(Xtr))
    m = modelo_fn().fit(Xtr[idx], ytr[idx])
    return m.predict(Xte), m.score(Xte, yte)

for nome, fn in [("SGDClassifier", lambda: SGDClassifier(random_state=0)),
                 ("LogisticRegression", lambda: LogisticRegression(max_iter=2000))]:
    resultados = [treinar(fn, s) for s in (None, 1, 2)]
    prevs = [r[0] for r in resultados]
    print(f"{nome:19s} acurácias = {[round(r[1], 4) for r in resultados]} | "
          f"previsões diferentes (orig x emb.1): {(prevs[0] != prevs[1]).sum()}, (emb.1 x emb.2):
{(prevs[1] != prevs[2]).sum()}")
```

SAÍDA

```
SGDClassifier       acurácias = [0.9519, 0.9611, 0.9407] | previsões diferentes (orig x emb.1): 20,
(emb.1 x emb.2): …
LogisticRegression  acurácias = [0.9704, 0.9704, 0.9704] | previsões diferentes (orig x emb.1): 0,
(emb.1 x emb.2): 0
```

EXEMPLO O que observar

O `SGDClassifier` processa os exemplos em sequência, então a ordem muda o resultado. A regressão logística (que resolve o problema todo de uma vez) é praticamente imune a isso. Moral: reprodutibilidade inclui fixar ou documentar a ordem dos dados.

Exercício 4 — As quatro medidas do DevOps aplicadas a um modelo

PYTHON

```python
import pandas as pd

hist = pd.DataFrame({
    "commit": pd.to_datetime(["2026-01-03", "2026-01-10", "2026-01-20", "2026-02-02", "2026-02-09",
"2026-02-18",
                              "2026-03-01", "2026-03-08", "2026-03-15", "2026-03-22", "2026-04-02",
"2026-04-09"]),
    "implantacao": pd.to_datetime(["2026-01-08", "2026-01-12", "2026-01-27", "2026-02-04", "2026-02-
16", "2026-02-20",
                                   "2026-03-03", "2026-03-12", "2026-03-16", "2026-03-30", "2026-04-
04", "2026-04-15"]),
    "falhou": [False, False, True, False, True, False, False, False, False, True, False, False],
    "horas_recuperacao": [0, 0, 6, 0, 2, 0, 0, 0, 0, 12, 0, 0],
})
meses = hist["implantacao"].dt.to_period("M").nunique()
print("Implantações por mês       :", round(len(hist) / meses, 1))
print("Tempo de entrega (dias)    :", round((hist["implantacao"] - hist["commit"]).dt.days.mean(), 1))
print("Taxa de falha de mudança   :", f"{hist['falhou'].mean():.0%}")
print("Tempo de recuperação (h)   :", round(hist.loc[hist["falhou"], "horas_recuperacao"].mean(), 1))
```

SAÍDA

```
Implantações por mês       : 3.0
Tempo de entrega (dias)    : 4.0
Taxa de falha de mudança   : 25%
Tempo de recuperação (h)   : 6.7
```

EXEMPLO O que observar

São as quatro medidas de Forsgren, Humble e Kim (2018). Acompanhe-as ao longo do tempo: o objetivo é entregar com frequência, rapidez, poucas falhas e recuperação rápida.

Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade

Exercício 1 — Sempre comece pela linha de base

PYTHON

```python
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, average_precision_score, recall_score
from sklearn.model_selection import train_test_split

X, y = make_classification(8000, n_features=10, n_informative=4, weights=[0.97], class_sep=1.0,
random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

modelos = {"sempre a maioria": DummyClassifier(strategy="most_frequent"),
           "sorteio estratificado": DummyClassifier(strategy="stratified", random_state=0),
           "regressão logística": LogisticRegression(max_iter=1000)}
print(f"Positivos no teste: {yte.mean():.1%}")
for nome, m in modelos.items():
    m.fit(Xtr, ytr)
    prev, p = m.predict(Xte), m.predict_proba(Xte)[:, 1]
    print(f"{nome:22s} acurácia = {accuracy_score(yte, prev):.3f} | revocação = {recall_score(yte,
prev):.3f} | AP = {average_precision_score(yte, p):.3f}")
```

SAÍDA

```
Positivos no teste: 3.4%
sempre a maioria       acurácia = 0.966 | revocação = 0.000 | AP = 0.034
sorteio estratificado  acurácia = 0.928 | revocação = 0.037 | AP = 0.034
regressão logística    acurácia = 0.967 | revocação = 0.037 | AP = 0.245
```

EXEMPLO **O que observar**

A linha de base "sempre a maioria" tem acurácia de cerca de 97% e revocação zero. Por isso, a acurácia sozinha não serve aqui. A AP da linha de base é igual à fração de positivos (cerca de 0,03); a da regressão logística é muito maior, o que mostra que ela ordena bem os clientes, embora, com o limiar padrão de 0,5, encontre poucos positivos (veja o ajuste de limiar no Capítulo 6).

Exercício 2 — Calculadora de ML Test Score

PYTHON

```python
def ml_test_score(projeto):
    totais = {g: sum(pontos) for g, pontos in projeto.items()}
    mais_fraco = min(totais, key=totais.get)
    return totais[mais_fraco], mais_fraco, totais

notebook = {"dados": [0, 0.5, 0, 0], "modelo": [0.5, 0, 0, 0], "infraestrutura": [0, 0, 0, 0],
"monitoramento": [0, 0, 0, 0]}
pipeline = {"dados": [1, 1, 0.5, 1], "modelo": [1, 0.5, 1, 1], "infraestrutura": [1, 1, 0.5, 1],
"monitoramento": [0.5, 0, 0.5, 0]}
```

```
for nome, proj in [("notebook solto", notebook), ("pipeline automatizado", pipeline)]:
    nota, fraco, totais = ml_test_score(proj)
    print(f"{nome:22s} nota = {nota:.1f} | elo mais fraco = {fraco} | totais = {totais}")
```

SAÍDA

```
notebook solto         nota = 0.0 | elo mais fraco = infraestrutura | totais = {'dados': 0.5, 'modelo':
0.5, 'infraes…
pipeline automatizado  nota = 1.0 | elo mais fraco = monitoramento | totais = {'dados': 3.5, 'modelo':
3.5, 'infraest…
```

EXEMPLO O que observar

Mesmo o pipeline automatizado tem nota baixa porque o monitoramento é o elo mais fraco. Como a nota é o mínimo, não adianta investir só onde já se é bom.

Exercício 3 — Quebrando o ciclo de realimentação

PYTHON

```python
import numpy as np

def simular(epsilon, semente=0, n_itens=10, n_visitas=5000):
    rng = np.random.default_rng(semente)
    cliques = np.ones(n_itens)
    for _ in range(n_visitas):
        exibidos = list(np.argsort(-cliques)[:3])
        for k in range(3):
            if rng.random() < epsilon:
                exibidos[k] = int(rng.integers(n_itens))        # exploração
        for item in set(exibidos):
            if rng.random() < 0.10:
                cliques[item] += 1
    return cliques

for eps in [0.0, 0.1, 0.3]:
    c = simular(eps)
    print(f"epsilon = {eps:.1f} | participação do mais clicado = {c.max() / c.sum():.2f} | itens com ≥
50 cliques = {(c >= 50).sum()}")
```

SAÍDA

```
epsilon = 0.0 | participação do mais clicado = 0.34 | itens com ≥ 50 cliques = 3
epsilon = 0.1 | participação do mais clicado = 0.32 | itens com ≥ 50 cliques = 3
epsilon = 0.3 | participação do mais clicado = 0.27 | itens com ≥ 50 cliques = 5
```

EXEMPLO O que observar

Sem exploração, só 3 itens recebem cliques. Com exploração pequena (0,1) o efeito é discreto; com 0,3, mais itens passam de 50 cliques e a concentração cai. Este é o princípio das estratégias de exploração usadas em recomendadores: pagar um pequeno custo agora para continuar aprendendo sobre todos os itens.

Exercício 4 — "Mudar qualquer coisa muda tudo" (CACE)

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

d = load_breast_cancer(as_frame=True)
X, y = d.data, d.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
esc = StandardScaler().fit(Xtr)
Z = pd.DataFrame(esc.transform(Xtr), columns=X.columns)
Zt = pd.DataFrame(esc.transform(Xte), columns=X.columns)

completo = LogisticRegression(max_iter=5000).fit(Z, ytr)
sem = LogisticRegression(max_iter=5000).fit(Z.drop(columns="mean radius"), ytr)

coef_c = pd.Series(completo.coef_[0], index=X.columns)
coef_s = pd.Series(sem.coef_[0], index=Z.drop(columns="mean radius").columns)
tab = pd.DataFrame({"com mean radius": coef_c, "sem mean radius": coef_s}).loc[["mean perimeter", "mean
area", "mean texture"]]
print(tab.round(3))
dif = (completo.predict(Zt) != sem.predict(Zt.drop(columns="mean radius"))).sum()
print("Previsões de teste que mudaram:", int(dif), "de", len(yte))
print("Correlação entre mean radius e mean perimeter:", round(X["mean radius"].corr(X["mean
perimeter"]), 3))
```

SAÍDA

```
                com mean radius  sem mean radius
mean perimeter           -0.535           -0.661
mean area                -0.597           -0.709
mean texture             -0.409           -0.410
Previsões de teste que mudaram: 1 de 171
Correlação entre mean radius e mean perimeter: 0.998
```

EXEMPLO **O que observar**

Retirar um único atributo altera os coeficientes de vários outros, principalmente dos correlacionados. É o princípio CACE: os componentes de um modelo estão emaranhados. As previsões podem mudar pouco, mas a interpretação dos coeficientes muda bastante.

Capítulo 3 — Dados

Exercício 1 — Relatório de qualidade de dados

PYTHON

```python
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 500
df = pd.DataFrame({"idade": rng.integers(18, 90, n).astype(float),
                   "peso_kg": rng.normal(75, 12, n).round(1),
                   "pressao": rng.normal(120, 15, n).round(0),
                   "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n)})
df.loc[rng.choice(n, 50, replace=False), "peso_kg"] = np.nan
df.loc[rng.choice(n, 5, replace=False), "idade"] = -3
df.loc[rng.choice(n, 25, replace=False), "cidade"] = "natal "
df = pd.concat([df, df.sample(20, random_state=0)], ignore_index=True)

def relatorio(df, intervalos):
    rel = {"ausentes (%)": (df.isna().mean() * 100).round(1).to_dict(),
           "linhas duplicadas": int(df.duplicated().sum())}
    rel["fora do intervalo"] = {c: int((~df[c].dropna().between(lo, hi)).sum()) for c, (lo, hi) in
intervalos.items()}
    norm = df["cidade"].str.strip().str.lower()
    rel["categorias de cidade (bruto)"] = sorted(df["cidade"].unique())
    rel["categorias de cidade (normalizado)"] = sorted(norm.unique())
    return rel

for k, v in relatorio(df, {"idade": (0, 120), "peso_kg": (30, 250), "pressao": (50, 250)}).items():
    print(f"{k}: {v}")
```

SAÍDA

```
ausentes (%): {'idade': 0.0, 'peso_kg': 10.2, 'pressao': 0.0, 'cidade': 0.0}
linhas duplicadas: 20
fora do intervalo: {'idade': 5, 'peso_kg': 2, 'pressao': 0}
categorias de cidade (bruto): ['Fortaleza', 'Natal', 'Recife', 'natal ']
categorias de cidade (normalizado): ['fortaleza', 'natal', 'recife']
```

EXEMPLO **O que observar**

O relatório revela os quatro problemas. Repare que "Natal" e "natal " aparecem como categorias diferentes; normalizar (tirar espaços e caixa) é uma correção simples, mas deve ser feita **dentro do pipeline**, para valer também em produção.

Exercício 2 — Um validador de esquema que para o pipeline

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
```

```
X = load_wine(as_frame=True).data
esquema = {c: {"min": X[c].min() - 0.2 * (X[c].max() - X[c].min()),
               "max": X[c].max() + 0.2 * (X[c].max() - X[c].min()),
               "max_ausentes": 0.05} for c in X.columns}

class LoteInvalido(Exception):
    pass

def validar(lote, esquema):
    erros = []
    for c, r in esquema.items():
        if c not in lote:
            erros.append(f"coluna ausente: {c}")
            continue
        if lote[c].isna().mean() > r["max_ausentes"]:
            erros.append(f"{c}: {lote[c].isna().mean():.0%} de ausentes")
        v = lote[c].dropna()
        fora = int(((v < r["min"]) | (v > r["max"])).sum())
        if fora:
            erros.append(f"{c}: {fora} valores fora de [{r['min']:.2f}, {r['max']:.2f}]")
    if erros:
        raise LoteInvalido("; ".join(erros))
    return True

saudavel = X.sample(60, random_state=0)
ruim = X.sample(60, random_state=1).copy()
ruim["alcohol"] *= 10
ruim.loc[ruim.sample(frac=0.3, random_state=0).index, "proline"] = np.nan

for nome, lote in [("saudável", saudavel), ("corrompido", ruim)]:
    try:
        print(nome, "->", validar(lote, esquema))
    except LoteInvalido as e:
        print(nome, "-> REPROVADO:", e)
```

SAÍDA

```
saudável -> True
corrompido -> REPROVADO: alcohol: 60 valores fora de [10.27, 15.59]; proline: 30% de ausentes
```

EXEMPLO O que observar

Ao lançar uma exceção, o pipeline para antes do treino ou da previsão. Em produção, o lote reprovado iria para uma área de quarentena e geraria um alerta.

Exercício 3 — Caçando um atributo vazado

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

d = load_breast_cancer(as_frame=True)
```

```
X, y = d.data.copy(), d.target
rng = np.random.default_rng(0)
X["vazado"] = y + rng.normal(0, 0.1, len(y))

auc = {c: roc_auc_score(y, X[c]) for c in X.columns}
auc = pd.Series({c: max(a, 1 - a) for c, a in auc.items()}).sort_values(ascending=False)
print("AUC de cada atributo sozinho (cinco maiores):")
print(auc.head(5).round(3).to_string())

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
com = RandomForestClassifier(100, random_state=0).fit(Xtr, ytr).score(Xte, yte)
sem = RandomForestClassifier(100, random_state=0).fit(Xtr.drop(columns="vazado"),
ytr).score(Xte.drop(columns="vazado"), yte)
print(f"Acurácia com o atributo vazado: {com:.3f} | sem ele: {sem:.3f}")
```

SAÍDA

```
AUC de cada atributo sozinho (cinco maiores):
vazado                  1.000
worst perimeter         0.975
worst radius            0.970
worst area              0.970
worst concave points    0.967
Acurácia com o atributo vazado: 1.000 | sem ele: 0.953
```

EXEMPLO **O que observar**

O atributo vazado tem AUC individual de quase 1,0, acima de todos os demais. Um atributo que sozinho "prevê quase tudo" é suspeito e deve ser investigado: de onde ele vem? Estará disponível na hora da previsão? Este é um bom teste automático para rodar no pipeline.

Exercício 4 — Divisão aleatória versus divisão por grupo

PYTHON

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupShuffleSplit, train_test_split

rng = np.random.default_rng(0)
n_pac, n_med = 100, 10
efeito = rng.normal(0, 3, (n_pac, 5))                                    # "impressão digital" de cada
paciente
alvo_pac = rng.integers(0, 2, n_pac)                                     # o alvo é fixo por paciente e
independe das medições
X = np.repeat(efeito, n_med, axis=0) + rng.normal(0, 0.3, (n_pac * n_med, 5))
y = np.repeat(alvo_pac, n_med)
grupos = np.repeat(np.arange(n_pac), n_med)

def acc(tr, te):
    return RandomForestClassifier(100, random_state=0).fit(X[tr], y[tr]).score(X[te], y[te])

tr, te = train_test_split(np.arange(len(y)), test_size=0.3, random_state=0)
print(f"(a) divisão aleatória das linhas: acurácia = {acc(tr, te):.3f}")
tr, te = next(GroupShuffleSplit(1, test_size=0.3, random_state=0).split(X, y, grupos))
print(f"(b) divisão por paciente      : acurácia = {acc(tr, te):.3f}")
```

SAÍDA

```
(a) divisão aleatória das linhas: acurácia = 0.957
(b) divisão por paciente      : acurácia = 0.417
```

EXEMPLO O que observar

Na divisão aleatória, o modelo "reconhece" o paciente pelas medições parecidas e acerta quase tudo, embora o alvo não dependa das medições. Na divisão por paciente, a acurácia cai para a faixa de 40% a 50% (o acaso), que é a verdade. É um vazamento causado por dependência entre linhas.

Capítulo 4 — Pipelines Reprodutíveis com scikit-learn

Exercício 1 — Padronizar importa para uns, não para outros

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_wine(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
modelos = {"KNN": KNeighborsClassifier(), "SVC": SVC(), "Floresta": RandomForestClassifier(100,
random_state=0)}
for nome, m in modelos.items():
    sem = cross_val_score(m, X, y, cv=cv).mean()
    com = cross_val_score(make_pipeline(StandardScaler(), m), X, y, cv=cv).mean()
    print(f"{nome:9s} sem padronizar = {sem:.3f} | padronizado = {com:.3f}")
```

SAÍDA

```
KNN       sem padronizar = 0.663 | padronizado = 0.961
SVC       sem padronizar = 0.657 | padronizado = 0.983
Floresta  sem padronizar = 0.972 | padronizado = 0.972
```

EXEMPLO **O que observar**

KNN e SVC usam distâncias e sofrem com atributos de escalas muito diferentes (como `proline`, na casa das centenas). A floresta usa cortes em um atributo por vez e quase não é afetada.

Exercício 2 — Dados mistos com ColumnTransformer

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

rng = np.random.default_rng(0)
n = 1500
df = pd.DataFrame({"idade": rng.integers(18, 70, n).astype(float),
                   "renda": rng.lognormal(8, 0.5, n),
                   "plano": rng.choice(["basico", "padrao", "premium"], n),
                   "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n)})
y = ((df["idade"] < 40) * 1.0 + (df["plano"] == "premium") * 1.5 + rng.normal(0, 1, n) >
1.2).astype(int)
```

```
df.loc[rng.choice(n, 120, replace=False), "renda"] = np.nan
df.loc[rng.choice(n, 75, replace=False), "plano"] = np.nan

num, cat = ["idade", "renda"], ["plano", "cidade"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("esc", StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("ohe", OneHotEncoder(handle_unknown="ignore"))]), cat)])
pipe = Pipeline([("prep", prep), ("modelo", LogisticRegression(max_iter=1000))])
Xtr, Xte, ytr, yte = train_test_split(df, y, test_size=0.3, random_state=0, stratify=y)
pipe.fit(Xtr, ytr)
print("Acurácia no teste:", round(pipe.score(Xte, yte), 3))
print("Colunas após o pré-processamento:", pipe.named_steps["prep"].get_feature_names_out().shape[0])

novo = pd.DataFrame([{"idade": 30, "renda": np.nan, "plano": "ultra", "cidade": "Salvador"}])
print("Previsão para categorias novas e renda ausente:", round(pipe.predict_proba(novo)[0, 1], 3))
```

SAÍDA

```
Acurácia no teste: 0.718
Colunas após o pré-processamento: 8
Previsão para categorias novas e renda ausente: 0.598
```

EXEMPLO **O que observar**

O pipeline aceitou a linha crua com ausente e categorias inéditas: os imputadores e o `handle_unknown="ignore"` cuidaram de tudo. Sem eles, a API de produção quebraria na primeira categoria nova.

Exercício 3 — Um transformador que aprende limites

PYTHON

```
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin

class Winsorizador(BaseEstimator, TransformerMixin):
    def __init__(self, inferior=1, superior=99):
        self.inferior, self.superior = inferior, superior

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.min_ = np.percentile(X, self.inferior, axis=0)
        self.max_ = np.percentile(X, self.superior, axis=0)
        return self

    def transform(self, X):
        return np.clip(np.asarray(X, dtype=float), self.min_, self.max_)

rng = np.random.default_rng(0)
treino = rng.normal(100, 10, (1000, 2))
treino[:5, 0] = 10_000                                  # alguns valores absurdos no treino
teste = np.array([[100.0, 100.0], [50_000.0, 100.0], [100.0, -500.0]])

w = Winsorizador().fit(treino)
print("Limites aprendidos no treino (mín):", w.min_.round(1), "| (máx):", w.max_.round(1))
print("Teste original:\n", teste)
print("Teste após o transformador:\n", w.transform(teste).round(1))
```

SAÍDA

```
Limites aprendidos no treino (mín): [74.8 78. ] | (máx): [123.9 122.8]
Teste original:
 [[  100.   100.]
 [50000.   100.]
 [  100.  -500.]]
Teste após o transformador:
 [[100.  100. ]
 [123.9 100. ]
 [100.   78. ]]
```

EXEMPLO **O que observar**

Os limites foram calculados só com o treino e aplicados ao teste: o valor 50.000 foi cortado para o limite superior aprendido. Atributos terminados em `_` guardam o que foi aprendido em `fit`.

Exercício 4 — O vazamento da seleção de atributos, em muitas sementes

PYTHON

```python
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline

fora, dentro = [], []
for semente in range(20):
    rng = np.random.default_rng(semente)
    X, y = rng.normal(size=(100, 1000)), rng.integers(0, 2, 100)
    cv = StratifiedKFold(5, shuffle=True, random_state=semente)
    Xs = SelectKBest(f_classif, k=10).fit_transform(X, y)                # errado: vê todos os
rótulos
    fora.append(cross_val_score(LogisticRegression(max_iter=500), Xs, y, cv=cv).mean())
    pipe = make_pipeline(SelectKBest(f_classif, k=10), LogisticRegression(max_iter=500))
    dentro.append(cross_val_score(pipe, X, y, cv=cv).mean())

print(f"Seleção fora do pipeline : média = {np.mean(fora):.3f} ± {np.std(fora):.3f}")
print(f"Seleção dentro (correto) : média = {np.mean(dentro):.3f} ± {np.std(dentro):.3f}")
print("Sementes em que a abordagem errada passou de 60%:", int(np.sum(np.array(fora) > 0.60)), "de 20")
```

SAÍDA

```
Seleção fora do pipeline : média = 0.778 ± 0.030
Seleção dentro (correto) : média = 0.517 ± 0.048
Sementes em que a abordagem errada passou de 60%: 20 de 20
```

EXEMPLO **O que observar**

Em dados sem nenhum sinal, a abordagem errada "descobre" um desempenho bem acima de 50% em todas as sementes, enquanto a correta fica em torno de 50%. A diferença não é sorte: é vazamento.

Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos

Exercício 1 — Três modelos, três execuções, uma consulta

PYTHON

```python
import os
import tempfile
import warnings
import mlflow
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

warnings.filterwarnings("ignore")
os.chdir(tempfile.mkdtemp())
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("wine")

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
modelos = {"logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
           "floresta": RandomForestClassifier(100, random_state=0),
           "svc": make_pipeline(StandardScaler(), SVC())}
for nome, m in modelos.items():
    with mlflow.start_run(run_name=nome):
        m.fit(Xtr, ytr)
        mlflow.log_param("modelo", nome)
        mlflow.log_metric("acuracia", m.score(Xte, yte))
        mlflow.set_tag("autor", "turma-mlops")

res = mlflow.search_runs(order_by=["metrics.acuracia DESC", "tags.mlflow.runName ASC"])
print(res[["tags.mlflow.runName", "params.modelo", "metrics.acuracia",
"tags.autor"]].round(4).to_string(index=False))
```

SAÍDA

```
tags.mlflow.runName params.modelo  metrics.acuracia  tags.autor
           floresta      floresta              1.0 turma-mlops
             logreg        logreg              1.0 turma-mlops
                svc           svc              1.0 turma-mlops
```

EXEMPLO **O que observar**

Neste teste pequeno (54 exemplos), os três modelos acertam tudo, então a ordem do desempate é apenas alfabética. O ponto do exercício é que o resultado fica salvo em `mlflow.db` e pode ser consultado depois, por qualquer pessoa com acesso, em vez de anotado num papel. Diferenças tão pequenas não devem ser interpretadas sem estatística (Capítulo 6).

Exercício 2 — Curva de aprendizado como métrica por passo

PYTHON

```python
import os
import tempfile
import warnings
import mlflow
import numpy as np
from mlflow import MlflowClient
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import log_loss
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
os.chdir(tempfile.mkdtemp())
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("curva_de_aprendizado")

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
gb = GradientBoostingClassifier(n_estimators=150, learning_rate=0.2, max_depth=3,
random_state=0).fit(Xtr, ytr)

with mlflow.start_run() as run:
    mlflow.log_params({"n_estimators": 150, "learning_rate": 0.2})
    for passo, p in enumerate(gb.staged_predict_proba(Xva), start=1):
        mlflow.log_metric("perda_validacao", log_loss(yva, p), step=passo)

hist = MlflowClient().get_metric_history(run.info.run_id, "perda_validacao")
perdas = np.array([h.value for h in sorted(hist, key=lambda h: h.step)])
print("Pontos registrados:", len(perdas))
print(f"Menor perda: {perdas.min():.4f} com {int(perdas.argmin()) + 1} árvores | perda com 150 árvores:
{perdas[-1]:.4f}")
```

SAÍDA

```
Pontos registrados: 150
Menor perda: 0.1504 com 20 árvores | perda com 150 árvores: 0.2544
```

EXEMPLO **O que observar**

Registrar métricas por passo permite ver o ponto em que o modelo começa a piorar na validação (superajuste) e escolher o número de árvores. É a base das curvas que as interfaces de rastreamento desenham.

Exercício 3 — Versões, apelidos e reversão

PYTHON

```python
import os
import tempfile
import warnings
import mlflow
import mlflow.sklearn
from mlflow import MlflowClient
from mlflow.models import infer_signature
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
os.chdir(tempfile.mkdtemp())
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("reversao")
NOME = "diagnostico"

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

def registrar(C):
    with mlflow.start_run(run_name=f"C={C}"):
        m = make_pipeline(StandardScaler(), LogisticRegression(C=C, max_iter=2000)).fit(Xtr, ytr)
        info = mlflow.sklearn.log_model(m, name="modelo", signature=infer_signature(Xte,
m.predict(Xte)))
    return mlflow.register_model(info.model_uri, NOME).version

cliente = MlflowClient()
v1, v2 = registrar(0.001), registrar(10)
cliente.set_registered_model_alias(NOME, "champion", v2)

def acuracia_do_champion():
    modelo = mlflow.sklearn.load_model(f"models:/{NOME}@champion")            # o código de carga nunca
muda
    return cliente.get_model_version_by_alias(NOME, "champion").version, modelo.score(Xte, yte)

em_memoria = mlflow.sklearn.load_model(f"models:/{NOME}@champion")            # carregado quando o
champion era a v2
print("Antes:  versão", *[f"{v} | acurácia = {a:.4f}" for v, a in [acuracia_do_champion()]])
cliente.set_registered_model_alias(NOME, "champion", v1)                          # reversão
print("Depois: versão", *[f"{v} | acurácia = {a:.4f}" for v, a in [acuracia_do_champion()]])
print(f"Objeto carregado antes da reversão ainda tem acurácia {em_memoria.score(Xte, yte):.4f} (é a
v2); "
      "só um novo load_model pelo apelido traz a v1.")
```

SAÍDA

```
Antes:  versão 2 | acurácia = 0.9532
Depois: versão 1 | acurácia = 0.8772
Objeto carregado antes da reversão ainda tem acurácia 0.9532 (é a v2); só um novo load_model pelo
apelido traz a v1.
```

EXEMPLO **O que observar**

O código de produção carrega sempre `models:/diagnostico@champion`. Promover ou reverter é mover o apelido: nenhuma edição de código é necessária e a troca é auditável. Mas o objeto já carregado em memória continua sendo a versão antiga até que o serviço recarregue o modelo, como mostra o `load_model` repetido no exercício.

Exercício 4 — Promoção só com evidência estatística

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X / 16, y, test_size=0.4, stratify=y, random_state=0)

campeao = LogisticRegression(C=1, max_iter=3000).fit(Xtr, ytr)
candidatos = {"SVC (muito diferente)": SVC(C=10).fit(Xtr, ytr),
              "logreg C=0.7 (quase igual)": LogisticRegression(C=0.7, max_iter=3000).fit(Xtr, ytr)}
acerto_c = (campeao.predict(Xte) == yte).astype(float)

def decidir(candidato, n_boot=2000, semente=0):
    acerto_n = (candidato.predict(Xte) == yte).astype(float)
    rng = np.random.default_rng(semente)
    dif = [(acerto_n[i] - acerto_c[i]).mean() for i in (rng.integers(0, len(yte), len(yte)) for _ in
range(n_boot))]
    lo, hi = np.percentile(dif, [2.5, 97.5])
    return acerto_n.mean() - acerto_c.mean(), lo, hi, lo > 0

print(f"Campeão: acurácia = {acerto_c.mean():.4f}")
for nome, cand in candidatos.items():
    d, lo, hi, promover = decidir(cand)
    print(f"{nome:30s} diferença = {d:+.4f} | IC95% = [{lo:+.4f}, {hi:+.4f}] | promover? {promover}")
```

SAÍDA

```
Campeão: acurácia = 0.9722
SVC (muito diferente)          diferença = +0.0195 | IC95% = [+0.0070, +0.0306] | promover? True
logreg C=0.7 (quase igual)     diferença = -0.0042 | IC95% = [-0.0111, +0.0014] | promover? False
```

EXEMPLO **O que observar**

O SVC tem ganho pequeno, mas o intervalo fica todo acima de zero, e ele é promovido. O candidato quase idêntico tem intervalo que inclui zero, e a regra o mantém de fora, evitando trocas por ruído. Esta é a lógica de um portão de promoção.

Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros

Exercício 1 — Métricas para classes raras

PYTHON

```python
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, average_precision_score, f1_score,
                             precision_score, recall_score, roc_auc_score)
from sklearn.model_selection import train_test_split

X, y = make_classification(10000, n_features=12, n_informative=5, weights=[0.96], class_sep=0.9,
random_state=1)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
for nome, m in [("sempre negativo", DummyClassifier(strategy="most_frequent")),
                ("floresta", RandomForestClassifier(200, random_state=0))]:
    m.fit(Xtr, ytr)
    prev, p = m.predict(Xte), m.predict_proba(Xte)[:, 1]
    print(f"{nome:16s} acc={accuracy_score(yte, prev):.3f} prec={precision_score(yte, prev,
zero_division=0):.3f} "
          f"rev={recall_score(yte, prev):.3f} F1={f1_score(yte, prev):.3f} "
          f"AUC-ROC={roc_auc_score(yte, p):.3f} AP={average_precision_score(yte, p):.3f}")
print(f"Fração de positivos no teste (piso da AP): {yte.mean():.3f}")
```

SAÍDA

```
sempre negativo  acc=0.956 prec=0.000 rev=0.000 F1=0.000 AUC-ROC=0.500 AP=0.044
floresta         acc=0.963 prec=0.958 rev=0.173 F1=0.293 AUC-ROC=0.902 AP=0.575
Fração de positivos no teste (piso da AP): 0.044
```

EXEMPLO O que observar

A AUC-ROC da floresta parece alta (acima de 0,8), mas a AP é bem menor, porque, com poucos positivos, muitos falsos positivos pouco alteram a taxa de falsos positivos (que usa um denominador enorme). A curva PR expõe essa dificuldade.

Exercício 2 — Escolhendo o limiar pelo custo

PYTHON

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(10000, n_features=12, n_informative=5, weights=[0.96], class_sep=0.9,
random_state=1)
Xtr, Xtmp, ytr, ytmp = train_test_split(X, y, test_size=0.5, stratify=y, random_state=0)
Xva, Xte, yva, yte = train_test_split(Xtmp, ytmp, test_size=0.5, stratify=ytmp, random_state=0)
m = RandomForestClassifier(200, random_state=0).fit(Xtr, ytr)
pva, pte = m.predict_proba(Xva)[:, 1], m.predict_proba(Xte)[:, 1]
```

```
def custo(p, y, t, c_fn=400, c_fp=8):
    prev = p >= t
    return c_fn * ((~prev) & (y == 1)).sum() + c_fp * (prev & (y == 0)).sum()

limiares = np.linspace(0.01, 0.99, 99)
t_otimo = limiares[int(np.argmin([custo(pva, yva, t) for t in limiares]))]
print(f"Limiar escolhido na validação: {t_otimo:.2f}")
print(f"Custo no teste com limiar 0,50: R$ {custo(pte, yte, 0.5):,.0f}")
print(f"Custo no teste com limiar {t_otimo:.2f}: R$ {custo(pte, yte, t_otimo):,.0f}")
```

SAÍDA

```
Limiar escolhido na validação: 0.07
Custo no teste com limiar 0,50: R$ 36,000
Custo no teste com limiar 0.07: R$ 12,232
```

EXEMPLO O que observar

O limiar escolhido é bem menor que 0,5, porque o falso negativo é 50 vezes mais caro que o falso positivo. Ele foi escolhido na validação e o custo foi medido no teste, evitando um resultado otimista.

Exercício 3 — Calibração antes e depois

PYTHON

```
import numpy as np
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import brier_score_loss
from sklearn.model_selection import train_test_split

X, y = make_classification(8000, n_features=15, n_informative=6, flip_y=0.05, class_sep=0.7,
random_state=1)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=1)
modelos = {"floresta bruta": RandomForestClassifier(200, random_state=0),
           "floresta calibrada": CalibratedClassifierCV(RandomForestClassifier(200, random_state=0),
method="isotonic", cv=5)}
for nome, m in modelos.items():
    p = m.fit(Xtr, ytr).predict_proba(Xte)[:, 1]
    obs, prev = calibration_curve(yte, p, n_bins=8, strategy="quantile")
    print(f"{nome:19s} Brier = {brier_score_loss(yte, p):.4f} | erro médio de calibração = {np.abs(obs
- prev).mean():.3f}")
```

SAÍDA

```
floresta bruta      Brier = 0.1007 | erro médio de calibração = 0.095
floresta calibrada  Brier = 0.0910 | erro médio de calibração = 0.017
```

EXEMPLO O que observar

A calibração reduz bastante o erro entre probabilidade prevista e frequência observada, e melhora o Brier. Se a probabilidade for usada para calcular preço, risco ou prioridade, calibrar é essencial.

Exercício 4 — O viés da busca de hiperparâmetros

PYTHON

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.svm import SVC

grade = {"C": [0.01, 0.1, 1, 10, 100], "gamma": [1e-4, 1e-3, 1e-2, 1e-1, 1]}
dif = []
for s in range(5):
    X, y = make_classification(80, n_features=60, n_informative=3, n_redundant=0, class_sep=0.4,
flip_y=0.2, random_state=s)
    interno = StratifiedKFold(5, shuffle=True, random_state=s)
    externo = StratifiedKFold(5, shuffle=True, random_state=100 + s)
    otimista = GridSearchCV(SVC(), grade, cv=interno).fit(X, y).best_score_
    honesta = cross_val_score(GridSearchCV(SVC(), grade, cv=interno), X, y, cv=externo).mean()
    dif.append(otimista - honesta)
    print(f"semente {s}: busca = {otimista:.3f} | aninhada = {honesta:.3f} | diferença = {otimista -
honesta:+.3f}")
print(f"Viés médio de otimismo: {np.mean(dif):+.3f}")
```

SAÍDA

```
semente 0: busca = 0.512 | aninhada = 0.487 | diferença = +0.025
semente 1: busca = 0.588 | aninhada = 0.512 | diferença = +0.075
semente 2: busca = 0.637 | aninhada = 0.588 | diferença = +0.050
semente 3: busca = 0.613 | aninhada = 0.512 | diferença = +0.100
semente 4: busca = 0.637 | aninhada = 0.562 | diferença = +0.075
Viés médio de otimismo: +0.065
```

EXEMPLO **O que observar**

Na média, a melhor pontuação da busca é maior que a estimativa aninhada: parte do bom resultado é sorte de ter escolhido a melhor entre 25 combinações. O viés cresce com poucos dados, muitos atributos e muitas configurações testadas.

Capítulo 7 — Empacotamento e Serviço de Modelos

Exercício 1 — Um contrato gerado a partir dos dados de treino

PYTHON

```python
from pydantic import ValidationError, create_model, Field
from sklearn.datasets import load_wine

X = load_wine(as_frame=True).data
campos = {}
for c in X.columns:
    folga = 0.25 * (X[c].max() - X[c].min())
    campos[c] = (float, Field(ge=float(X[c].min() - folga), le=float(X[c].max() + folga)))
Vinho = create_model("Vinho", **campos)

valida = X.iloc[0].to_dict()
fora = dict(valida, alcohol=99.0)
faltando = {k: v for k, v in valida.items() if k != "proline"}

for nome, dados in [("válida", valida), ("álcool fora da faixa", fora), ("campo faltando", faltando)]:
    try:
        Vinho(**dados)
        print(f"{nome:22s} -> aceita")
    except ValidationError as e:
        erro = e.errors()[0]
        print(f"{nome:22s} -> rejeitada ({erro['loc'][0]}: {erro['type']})")
```

SAÍDA

```
válida                 -> aceita
álcool fora da faixa   -> rejeitada (alcohol: less_than_equal)
campo faltando         -> rejeitada (proline: missing)
```

EXEMPLO **O que observar**

O contrato nasceu dos próprios dados de treino, então cada campo recebe limites realistas. Pedidos fora da faixa são rejeitados antes de chegar ao modelo, em vez de gerar previsões sem sentido.

Exercício 2 — Uma API mínima com saúde e previsão

PYTHON

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

iris = load_iris()
modelo = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(iris.data, iris.target)

class Flor(BaseModel):
```

```
    sepala_comp: float = Field(gt=0, lt=20)
    sepala_larg: float = Field(gt=0, lt=20)
    petala_comp: float = Field(gt=0, lt=20)
    petala_larg: float = Field(gt=0, lt=20)

app = FastAPI()

@app.get("/saude")
def saude():
    return {"status": "ok"}

@app.post("/previsao")
def prever(f: Flor):
    x = [[f.sepala_comp, f.sepala_larg, f.petala_comp, f.petala_larg]]
    p = modelo.predict_proba(x)[0]
    return {"especie": str(iris.target_names[p.argmax()]), "probabilidade": round(float(p.max()), 3)}

cli = TestClient(app)
print("saúde      :", cli.get("/saude").json())
ok = cli.post("/previsao", json={"sepala_comp": 5.1, "sepala_larg": 3.5, "petala_comp": 1.4,
"petala_larg": 0.2})
print("válida     :", ok.status_code, ok.json())
neg = cli.post("/previsao", json={"sepala_comp": -1, "sepala_larg": 3.5, "petala_comp": 1.4,
"petala_larg": 0.2})
print("negativa   :", neg.status_code)
vazio = cli.post("/previsao", json={})
print("sem campos :", vazio.status_code, f"({len(vazio.json()['detail'])} erros)")
```

SAÍDA

```
saúde      : {'status': 'ok'}
válida     : 200 {'especie': 'setosa', 'probabilidade': 0.985}
negativa   : 422
sem campos : 422 (4 erros)
```

EXEMPLO O que observar

O `TestClient` permite testar a API sem subir servidor, o que a torna testável na integração contínua. O código 422 indica que o contrato foi violado.

Exercício 3 — Percentis de latência e o ganho do lote

PYTHON

```
import time
import numpy as np
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier

X, y = load_digits(return_X_y=True)
m = RandomForestClassifier(200, random_state=0, n_jobs=1).fit(X, y)
m.predict(X[:5])                                              # aquecimento

tempos = []
for i in range(300):
    t0 = time.perf_counter()
    m.predict(X[i:i + 1])
```

```
    tempos.append((time.perf_counter() - t0) * 1000)
p50, p95, p99 = np.percentile(tempos, [50, 95, 99])
print(f"individuais: p50 = {p50:.1f} ms | p95 = {p95:.1f} ms | p99 = {p99:.1f} ms | média =
{np.mean(tempos):.1f} ms")

t0 = time.perf_counter(); m.predict(X[:300]); lote = (time.perf_counter() - t0) * 1000
print(f"lote de 300: {lote:.1f} ms no total = {lote / 300:.3f} ms por previsão")
print(f"Economia: cerca de {np.mean(tempos) / (lote / 300):.0f} vezes mais rápido por previsão")
```

SAÍDA

```
individuais: p50 = 10.6 ms | p95 = 11.9 ms | p99 = 14.4 ms | média = 10.8 ms
lote de 300: 17.5 ms no total = 0.058 ms por previsão
Economia: cerca de 185 vezes mais rápido por previsão
```

EXEMPLO O que observar

Os valores variam com o computador, mas o padrão se repete: p99 maior que p50 e o lote muito mais eficiente por previsão. Isso justifica agrupar pedidos (*micro-batching*) em serviços de alto volume.

Exercício 4 — Previsão em lote idempotente com manifesto

PYTHON

```
import hashlib
import json
import os
import tempfile
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression

os.chdir(tempfile.mkdtemp())
X, y = load_digits(return_X_y=True)
modelo = LogisticRegression(max_iter=3000).fit(X[:1200] / 16, y[:1200])
VERSAO = "1.0.0"

def hash_bytes(b):
    return hashlib.sha256(b).hexdigest()[:12]

def prever_em_lote(X, data_ref, tam=200):
    blocos = [pd.DataFrame({"indice": np.arange(i, min(i + tam, len(X))),
                            "classe": modelo.predict(X[i:i + tam] / 16)}) for i in range(0, len(X),
tam)]
    saida = pd.concat(blocos, ignore_index=True)
    caminho = f"previsoes_{data_ref}.csv"
    saida.to_csv(caminho, index=False)                      # sobrescreve: idempotente
    manifesto = {"data_ref": data_ref, "versao_modelo": VERSAO, "linhas": len(saida),
                 "hash_entrada": hash_bytes(X.tobytes()), "hash_saida": hash_bytes(open(caminho,
"rb").read())}
    json.dump(manifesto, open("manifesto.json", "w"), indent=1)
    return manifesto

m1 = prever_em_lote(X, "2026-10-17")
m2 = prever_em_lote(X, "2026-10-17")
print("Manifesto:", m1)
```

```
print("Mesmo manifesto na reexecução?", m1 == m2)
print("Linhas no arquivo:", len(pd.read_csv("previsoes_2026-10-17.csv")))
```

SAÍDA

```
Manifesto: {'data_ref': '2026-10-17', 'versao_modelo': '1.0.0', 'linhas': 1797, 'hash_entrada':
'20def7f70a70', 'hash…
Mesmo manifesto na reexecução? True
Linhas no arquivo: 1797
```

EXEMPLO **O que observar**

Reexecutar o trabalho gera exatamente o mesmo arquivo e o mesmo manifesto (o arquivo é sobrescrito, e não acrescentado). Se o hash da entrada ou a versão do modelo mudarem, o manifesto mostra a diferença, o que facilita a auditoria.

Capítulo 8 — Testes e CI/CD para Machine Learning

Exercício 1 — Testes unitários para uma função de limpeza

PYTHON

```
import os
import tempfile
import textwrap
import pytest

os.chdir(tempfile.mkdtemp())
open("limpeza.py", "w").write(textwrap.dedent('''
    import unicodedata

    def normalizar_cidade(texto):
        if texto is None:
            return None
        sem_acento = "".join(c for c in unicodedata.normalize("NFD", str(texto)) if
unicodedata.category(c) != "Mn")
        limpo = sem_acento.strip().lower()
        return limpo or None
'''))
open("test_limpeza.py", "w").write(textwrap.dedent('''
    import pytest
    from limpeza import normalizar_cidade

    @pytest.mark.parametrize("entrada, esperado", [
        ("Natal", "natal"), ("  Recife ", "recife"), ("São Paulo", "sao paulo"),
        ("FORTALEZA", "fortaleza"), ("", None), ("   ", None), (None, None),
    ])
    def test_normalizar(entrada, esperado):
        assert normalizar_cidade(entrada) == esperado

    def test_idempotente():
        assert normalizar_cidade(normalizar_cidade("  São Luís ")) == normalizar_cidade("  São Luís ")
'''))
codigo = pytest.main(["test_limpeza.py", "-q", "-p", "no:cacheprovider"])
print("Código de saída do pytest:", int(codigo))
```

SAÍDA

```
........                                                                 [100%]
8 passed in 0.01s
Código de saída do pytest: 0
```

EXEMPLO **O que observar**

Todos os casos passam (7 parametrizados + 1 de idempotência = 8 testes). Os casos de borda (vazio, só espaços, None) são justamente onde os erros costumam aparecer em produção.

Exercício 2 — Testes de comportamento do modelo

PYTHON

```
import os
import tempfile
import textwrap
import pytest

os.chdir(tempfile.mkdtemp())
open("test_modelo.py", "w").write(textwrap.dedent('''
    import numpy as np, pandas as pd, pytest
    from sklearn.compose import ColumnTransformer
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import f1_score
    from sklearn.model_selection import train_test_split
    from sklearn.pipeline import Pipeline
    from sklearn.preprocessing import OneHotEncoder, StandardScaler

    @pytest.fixture(scope="module")
    def ctx():
        rng = np.random.default_rng(0); n = 4000
        df = pd.DataFrame({"idade": rng.integers(18, 75, n).astype(float),
                           "tempo_cliente": rng.integers(1, 120, n).astype(float),
                           "plano": rng.choice(["basico", "padrao", "premium"], n, p=[.5, .35, .15]),
                           "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n)})
        r = 0.03 * (50 - df["idade"]) - 0.02 * df["tempo_cliente"] + (df["plano"] == "basico") * 0.8
        y = (r + rng.normal(0, 0.8, n) > 0.3).astype(int)
        Xtr, Xte, ytr, yte = train_test_split(df, y, test_size=0.3, stratify=y, random_state=0)
        prep = ColumnTransformer([("n", StandardScaler(), ["idade", "tempo_cliente"]),
                                  ("c", OneHotEncoder(handle_unknown="ignore"), ["plano", "cidade"])])
        return Pipeline([("p", prep), ("m", LogisticRegression(max_iter=1000))]).fit(Xtr, ytr), Xte,
yte

    def test_f1_minimo(ctx):
        m, X, y = ctx
        assert f1_score(y, m.predict(X)) >= 0.60

    def test_invariancia_cidade(ctx):
        m, X, _ = ctx
        a, b = m.predict_proba(X)[:, 1], m.predict_proba(X.assign(cidade="Natal"))[:, 1]
        assert np.abs(a - b).mean() < 0.03

    def test_direcao_tempo(ctx):
        m, X, _ = ctx
        antes = m.predict_proba(X)[:, 1]
        depois = m.predict_proba(X.assign(tempo_cliente=X["tempo_cliente"] + 24))[:, 1]
        assert (depois <= antes + 1e-9).mean() >= 0.95
'''))
print("Código de saída do pytest:", int(pytest.main(["test_modelo.py", "-q", "-p",
"no:cacheprovider"])))
```

```
SAÍDA
...                                                                      [100%]
3 passed in 1.18s
Código de saída do pytest: 0
```

EXEMPLO O que observar

Os três testes passam. Os limites (0,60; 0,03; 95%) são decisões do projeto: apertados demais, os testes falham à toa; frouxos demais, não protegem nada.

Exercício 3 — Portão de qualidade que valida as métricas

PYTHON

```
import json
import os
import subprocess
import sys
import tempfile
import textwrap

os.chdir(tempfile.mkdtemp())
open("gate.py", "w").write(textwrap.dedent('''
    import json, math, sys
    m, lim = json.load(open("metricas.json")), json.load(open("limites.json"))
    erros = [f"ausente: {k}" for k in list(lim) + ["n_avaliados"] if k not in m]
    erros += [f"{k}: não finito ou não numérico ({v!r})" for k, v in m.items()
              if isinstance(v, bool) or not isinstance(v, (int, float)) or not math.isfinite(v)]
    erros += [f"{k}: fora de [0, 1] ({m[k]})" for k in lim if isinstance(m.get(k), (int, float)) and
math.isfinite(m[k]) and not 0 <= m[k] <= 1]
    if isinstance(m.get("n_avaliados"), (int, float)) and m["n_avaliados"] < 100:
        erros.append(f"n_avaliados = {m['n_avaliados']} < 100")
    if erros:
        print("REPROVADO (métricas inválidas) ->", "; ".join(erros)); sys.exit(2)
    falhas = [f"{k}: {m[k]:.3f} < {v}" for k, v in lim.items() if m[k] < v]
    if falhas:
        print("REPROVADO ->", "; ".join(falhas)); sys.exit(1)
    print("APROVADO")
'''))
json.dump({"f1": 0.90, "revocacao": 0.85, "auc_pr": 0.80}, open("limites.json", "w"))

nan = float("nan")
cenarios = {"aprovado": {"f1": 0.93, "revocacao": 0.90, "auc_pr": 0.85, "n_avaliados": 300},
            "reprovado por uma": {"f1": 0.93, "revocacao": 0.80, "auc_pr": 0.85, "n_avaliados": 300},
            "reprovado por duas": {"f1": 0.85, "revocacao": 0.80, "auc_pr": 0.85, "n_avaliados": 300},
            "F1 = NaN": {"f1": nan, "revocacao": 0.90, "auc_pr": 0.85, "n_avaliados": 300},
            "métrica ausente": {"f1": 0.93, "auc_pr": 0.85, "n_avaliados": 300}}
for nome, met in cenarios.items():
    json.dump(met, open("metricas.json", "w"))
    r = subprocess.run([sys.executable, "gate.py"], capture_output=True, text=True)
    print(f"{nome:20s} -> código {r.returncode} | {r.stdout.strip()}")

# o portão ingênuo, sem validação, aprovaria o NaN:
m, lim = cenarios["F1 = NaN"], {"f1": 0.90, "revocacao": 0.85, "auc_pr": 0.80}
print("Portão ingênuo aprova o caso do NaN?", not [k for k, v in lim.items() if m[k] < v])
```

SAÍDA

```
aprovado             -> código 0 | APROVADO
reprovado por uma    -> código 1 | REPROVADO -> revocacao: 0.800 < 0.85
reprovado por duas   -> código 1 | REPROVADO -> f1: 0.850 < 0.9; revocacao: 0.800 < 0.85
F1 = NaN             -> código 2 | REPROVADO (métricas inválidas) -> f1: não finito ou não numérico
(nan)
```

```
métrica ausente      -> código 2 | REPROVADO (métricas inválidas) -> ausente: revocacao
Portão ingênuo aprova o caso do NaN? True
```

EXEMPLO O que observar

Em CI, o código de saída diferente de zero interrompe o fluxo automaticamente. O portão deve mostrar **todos** os motivos de uma vez e distinguir "modelo ruim" (1) de "avaliação inválida" (2). O portão ingênuo aprova o NaN, porque qualquer comparação com NaN é falsa: por isso, valide antes de comparar.

Exercício 4 — Os testes realmente protegem? Quebrando o modelo de propósito

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

def bateria(modelo, nome):
    prev = modelo.predict(Xte)
    testes = {"F1 >= 0.90": f1_score(yte, prev) >= 0.90,
              "revocação da classe maligna >= 0.90": recall_score(yte, prev, pos_label=0) >= 0.90,
              "determinismo": np.array_equal(prev, modelo.predict(Xte))}
    falhas = [t for t, ok in testes.items() if not ok]
    print(f"{nome:34s} -> {'passou em tudo' if not falhas else 'FALHOU em: ' + '; '.join(falhas)}")

bom = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
bateria(bom, "modelo correto")

rng = np.random.default_rng(0)
bateria(make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr,
rng.permutation(ytr)), "(a) rótulos embaralhados")
bateria(LogisticRegression(max_iter=2000).fit(StandardScaler().fit_transform(Xtr), ytr), "(b) sem
padronizar em produção")
bateria(make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr[:10], ytr[:10]),
"(c) treino com 10 exemplos")
```

SAÍDA

```
modelo correto                    -> passou em tudo
(a) rótulos embaralhados          -> FALHOU em: F1 >= 0.90; revocação da classe maligna >= 0.90
(b) sem padronizar em produção    -> FALHOU em: F1 >= 0.90
(c) treino com 10 exemplos        -> FALHOU em: revocação da classe maligna >= 0.90
```

EXEMPLO O que observar

Os testes de desempenho detectam as falhas (a) e (b); a falha (c) pode ou não ser detectada, dependendo dos limites. Um bom exercício é pensar em testes que cobrem o que passou despercebido. Testar os testes, "quebrando" o sistema de propósito, é uma forma de verificar se a rede de proteção realmente funciona.

Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento

Exercício 1 — Calculando o PSI à mão

PYTHON

```python
import numpy as np

def psi(ref, atual, faixas=10):
    cortes = np.quantile(ref, np.linspace(0, 1, faixas + 1))
    cortes[0], cortes[-1] = -np.inf, np.inf
    p = np.clip(np.histogram(ref, cortes)[0] / len(ref), 1e-6, None)
    q = np.clip(np.histogram(atual, cortes)[0] / len(atual), 1e-6, None)
    return float(np.sum((q - p) * np.log(q / p)))

def nivel(v):
    return "estável" if v < 0.10 else "moderada" if v < 0.25 else "importante"

rng = np.random.default_rng(0)
ref = rng.normal(0, 1, 5000)
for nome, atual in {"sem mudança": rng.normal(0, 1, 2000), "média +0,3": rng.normal(0.3, 1, 2000),
                    "média +1": rng.normal(1, 1, 2000), "desvio padrão ×2": rng.normal(0, 2,
2000)}.items():
    v = psi(ref, atual)
    print(f"{nome:12s} PSI = {v:.3f} -> {nivel(v)}")
```

SAÍDA

```
sem mudança  PSI = 0.007 -> estável
média +0,3   PSI = 0.120 -> moderada
média +1     PSI = 0.939 -> importante
desvio padrão ×2 PSI = 0.494 -> importante
```

EXEMPLO **O que observar**

O PSI cresce com a intensidade da mudança. Os limiares 0,10 e 0,25 são convenções práticas; em cada problema vale calibrá-los observando o histórico de variações normais.

Exercício 2 — Deriva em atributo categórico

PYTHON

```python
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

rng = np.random.default_rng(0)
ref = pd.Series(rng.choice(["basico", "padrao", "premium"], 5000, p=[0.5, 0.35, 0.15]))
lotes = {"(a) igual": pd.Series(rng.choice(["basico", "padrao", "premium"], 1500, p=[0.5, 0.35,
0.15])),
         "(b) mudou": pd.Series(rng.choice(["basico", "padrao", "premium"], 1500, p=[0.35, 0.40,
0.25])),
         "(c) categoria nova": pd.Series(rng.choice(["basico", "padrao", "premium", "familia"], 1500,
```

```
p=[0.46, 0.32, 0.14, 0.08]))}

for nome, atual in lotes.items():
    novas = sorted(set(atual) - set(ref))
    cats = sorted(set(ref) | set(atual))
    tabela = np.array([[ (ref == c).sum() for c in cats], [(atual == c).sum() for c in cats]])
    chi2, p, _, _ = chi2_contingency(tabela)
    print(f"{nome:20s} qui² = {chi2:7.1f} | p = {p:.3g} | categorias novas: {novas or 'nenhuma'}")
```

SAÍDA

```
(a) igual            qui² =     0.5 | p = 0.786 | categorias novas: nenhuma
(b) mudou            qui² =   134.0 | p = 8.18e-30 | categorias novas: nenhuma
(c) categoria nova   qui² =   399.4 | p = 3.05e-86 | categorias novas: ['familia']
```

EXEMPLO O que observar

O teste detecta a mudança nas proporções e também a categoria nova. Em produção, o aparecimento de categorias que o modelo nunca viu deve gerar alerta próprio, porque o encoder pode ignorá-las e o modelo "ficar cego" para elas.

Exercício 3 — O classificador de domínio acha o que os testes simples perdem

PYTHON

```
import numpy as np
from scipy.stats import ks_2samp
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(0)
ref = rng.normal(size=(3000, 5))
atual = rng.normal(size=(3000, 5))
atual[:, 1] = 0.8 * atual[:, 0] + 0.6 * atual[:, 1]           # correlação 0,8; variância preservada

print("p-valores do KS por atributo:", [round(float(ks_2samp(ref[:, j], atual[:, j]).pvalue), 2) for j
in range(5)])
X = np.vstack([ref, atual]); d = np.r_[np.zeros(3000), np.ones(3000)]
clf = RandomForestClassifier(150, min_samples_leaf=5, random_state=0)
print("AUC do classificador de domínio:", round(cross_val_score(clf, X, d, cv=4,
scoring="roc_auc").mean(), 3))
clf.fit(X, d)
print("Importâncias:", {j: round(float(v), 2) for j, v in enumerate(clf.feature_importances_)})
```

SAÍDA

```
p-valores do KS por atributo: [0.9, 0.25, 0.67, 0.93, 0.32]
AUC do classificador de domínio: 0.734
Importâncias: {0: 0.32, 1: 0.27, 2: 0.13, 3: 0.14, 4: 0.14}
```

EXEMPLO O que observar

Nenhum atributo, olhado sozinho, acusa mudança, mas o classificador distingue os dois conjuntos e aponta os atributos 0 e 1 como os mais importantes, pois foi a relação entre eles que mudou.

Exercício 4 — Quanto o DDM demora a perceber?

PYTHON

```python
import numpy as np

class DDM:
    def __init__(self, min_amostras=500):
        self.n, self.p, self.pmin, self.smin, self.min_amostras = 0, 1.0, np.inf, np.inf, min_amostras
    def deriva(self, erro):
        self.n += 1
        self.p += (erro - self.p) / self.n
        s = np.sqrt(self.p * (1 - self.p) / self.n)
        if self.n < self.min_amostras:
            return False
        if self.p + s < self.pmin + self.smin:
            self.pmin, self.smin = self.p, s
        return self.p + s >= self.pmin + 3 * self.smin

def avaliar(taxa_depois, semente):
    rng = np.random.default_rng(semente)
    erros = np.r_[rng.random(3000) < 0.10, rng.random(3000) < taxa_depois].astype(int)
    d = DDM()
    for i, e in enumerate(erros):
        if d.deriva(e):
            return ("falso alarme", None) if i < 3000 else ("detectou", i - 3000)
    return ("nada", None)

print("40 sementes por cenário (mudança na posição 3.000):")
for depois in [0.10, 0.15, 0.20, 0.30]:
    r = [avaliar(depois, s) for s in range(40)]
    falsos = sum(x[0] == "falso alarme" for x in r)
    atrasos = [x[1] for x in r if x[0] == "detectou"]
    med = f"{int(np.median(atrasos))}" if atrasos else "-"
    print(f"taxa depois = {depois:.2f} | falsos alarmes = {falsos:2d} | detectou = {len(atrasos):2d} |
mediana do atraso = {med}")
```

SAÍDA

```
40 sementes por cenário (mudança na posição 3.000):
taxa depois = 0.10 | falsos alarmes =  7 | detectou =  2 | mediana do atraso = 1944
taxa depois = 0.15 | falsos alarmes =  7 | detectou = 29 | mediana do atraso = 631
taxa depois = 0.20 | falsos alarmes =  7 | detectou = 33 | mediana do atraso = 317
taxa depois = 0.30 | falsos alarmes =  7 | detectou = 33 | mediana do atraso = 149
```

EXEMPLO **O que observar**

Quanto maior a piora, mais sementes detectam e menor o atraso. Sem mudança real (0,10), não deve haver detecção depois de 3.000, mas pode haver falsos alarmes: nenhum detector é perfeito. Mudanças pequenas demoram ou nem são percebidas: por isso o monitoramento combina vários sinais.

Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas

Exercício 1 — Cartão do modelo gerado automaticamente

PYTHON

```python
import os
import tempfile
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

os.chdir(tempfile.mkdtemp())
d = load_breast_cancer(as_frame=True)
X, y = d.data, d.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
prev = pd.Series(m.predict(Xte), index=Xte.index)

def metricas(idx):
    return (accuracy_score(yte[idx], prev[idx]), f1_score(yte[idx], prev[idx]), recall_score(yte[idx],
prev[idx], pos_label=0))

mediana = Xtr["mean radius"].median()
grupos = {"todos": Xte.index, "tumor menor (<= mediana)": Xte.index[Xte["mean radius"] <= mediana],
          "tumor maior (> mediana)": Xte.index[Xte["mean radius"] > mediana]}

linhas = ["# Cartão do modelo: triagem_wdbc (v1.0.0)", "",
          "## Uso pretendido", "Estudo e ensino de classificação binária. Não é um dispositivo
médico.", "",
          "## Fora do escopo", "Apoio a diagnóstico clínico real sem validação médica e regulatória.",
"",
          "## Dados", f"UCI Breast Cancer Wisconsin (Diagnostic): {len(X)} exemplos, {X.shape[1]}
atributos; teste com {len(Xte)} exemplos.", "",
          "## Métricas (acurácia | F1 | revocação da classe maligna)"]
for nome, idx in grupos.items():
    a, f, r = metricas(idx)
    linhas.append(f"- {nome} (n = {len(idx)}): {a:.3f} | {f:.3f} | {r:.3f}")
linhas += ["", "## Limitações", "- Amostra pequena e de uma única instituição.", "- Sem avaliação por
subgrupos demográficos (não disponíveis no conjunto)."]
texto = "\n".join(linhas)
open("cartao.md", "w", encoding="utf-8").write(texto)
print("\n".join(open("cartao.md", encoding="utf-8").read().splitlines()[:16]))
```

SAÍDA

```
# Cartão do modelo: triagem_wdbc (v1.0.0)

## Uso pretendido
Estudo e ensino de classificação binária. Não é um dispositivo médico.
```

```
## Fora do escopo
Apoio a diagnóstico clínico real sem validação médica e regulatória.

## Dados
UCI Breast Cancer Wisconsin (Diagnostic): 569 exemplos, 30 atributos; teste com 171 exemplos.

## Métricas (acurácia | F1 | revocação da classe maligna)
- todos (n = 171): 0.959 | 0.967 | 0.953
- tumor menor (<= mediana) (n = 78): 0.987 | 0.993 | 0.833
- tumor maior (> mediana) (n = 93): 0.935 | 0.912 | 0.966
```

EXEMPLO **O que observar**

O cartão mostra o desempenho por faixa de tamanho, e não só o geral. Se uma faixa tiver revocação da classe maligna bem menor, isso precisa aparecer nas limitações. Gerar o cartão por código evita que ele fique desatualizado.

Exercício 2 — Explicando o modelo: permutação, PDP e a armadilha da correlação

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

d = load_breast_cancer(as_frame=True)
X, y = d.data, d.target

def top(X, n=5):
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
    m = RandomForestClassifier(300, random_state=0).fit(Xtr, ytr)
    r = permutation_importance(m, Xte, yte, n_repeats=20, random_state=0, scoring="f1")
    return pd.Series(r.importances_mean, index=X.columns).sort_values(ascending=False).head(n)

original = top(X)
print("Original:\n", original.round(4).to_string())
mais_importante = original.index[0]
X2 = X.copy()
X2[mais_importante + " (cópia)"] = X[mais_importante] + np.random.default_rng(0).normal(0, 1e-3 *
X[mais_importante].std(), len(X))
com_copia = top(X2)
print("\nCom uma cópia quase idêntica:\n", com_copia.round(4).to_string())
```

SAÍDA

```
Original:
 worst radius            0.0170
worst area              0.0143
worst concave points    0.0132
worst perimeter         0.0119
worst texture           0.0111
```

```
Com uma cópia quase idêntica:
 worst concave points    0.0102
worst area              0.0081
worst radius            0.0076
worst texture           0.0066
worst radius (cópia)    0.0062
```

EXEMPLO **O que observar**

Com atributos quase duplicados, a importância se divide entre eles, e cada um pode parecer menos relevante do que realmente é. Por isso, explicações devem ser lidas com cuidado e acompanhadas da análise de correlação entre atributos.

Exercício 3 — Auditoria de justiça e ajuste de limiares

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
n = 20000
grupo = rng.choice(["A", "B"], n, p=[0.7, 0.3])
merito = rng.normal(size=n)
y = (merito + rng.normal(0, 0.8, n) > 0).astype(int)                # resultado positivo segundo o
critério definido
x1 = merito + np.where(grupo == "B", -0.6, 0.0) + rng.normal(0, 0.5, n)       # medido com viés no
grupo B
x2 = merito + rng.normal(0, 0.9, n)
d = pd.DataFrame({"x1": x1, "x2": x2, "grupo": grupo, "y": y})
tr, resto = train_test_split(d, test_size=0.5, random_state=0, stratify=d["grupo"])
va, te = train_test_split(resto, test_size=0.5, random_state=0, stratify=resto["grupo"])
m = LogisticRegression().fit(tr[["x1", "x2"]], tr["y"])
va = va.assign(p=m.predict_proba(va[["x1", "x2"]])[:, 1])
te = te.assign(p=m.predict_proba(te[["x1", "x2"]])[:, 1])

def relatorio(te, sel):
    linhas = {}
    for g, dg in te.assign(sel=sel).groupby("grupo"):
        linhas[g] = {"taxa_selecao": dg["sel"].mean(), "revocacao": dg.loc[dg["y"] == 1, "sel"].mean(),
                     "precisao": dg.loc[dg["sel"], "y"].mean()}
    return pd.DataFrame(linhas).T.round(3)

t1 = relatorio(te, te["p"] >= 0.5)
print("Limiar único 0,5 (teste):\n", t1)
print("Razão de impacto B/A:", round(t1.loc["B", "taxa_selecao"] / t1.loc["A", "taxa_selecao"], 3))

limiares = {g: float(np.quantile(dg.loc[dg["y"] == 1, "p"], 1 - 0.80)) for g, dg in
va.groupby("grupo")}   # ajuste na validação
sel2 = np.array([p >= limiares[g] for p, g in zip(te["p"], te["grupo"])])
t2 = relatorio(te, sel2)
print("\nLimiares por grupo (da validação)", {g: round(v, 3) for g, v in limiares.items()}, ":\n", t2)
print("Razão de impacto B/A:", round(t2.loc["B", "taxa_selecao"] / t2.loc["A", "taxa_selecao"], 3))
```

SAÍDA

```
Limiar único 0,5 (teste):
    taxa_selecao  revocacao  precisao
A         0.542      0.792     0.736
B         0.381      0.647     0.830
Razão de impacto B/A: 0.703

Limiares por grupo (da validação) {'A': 0.52, 'B': 0.333} :
    taxa_selecao  revocacao  precisao
A         0.524      0.772     0.743
B         0.541      0.805     0.728
Razão de impacto B/A: 1.032
```

EXEMPLO **O que observar**

Sem usar o grupo, o modelo ainda seleciona menos pessoas do grupo B e encontra menos dos casos positivos desse grupo: o viés entrou pelo atributo medido com erro. Com limiares ajustados na validação, as revocações no teste ficam próximas (não exatamente iguais, pois os limiares foram escolhidos em outra amostra), e a precisão e a taxa de seleção também mudam. Escolher o critério de justiça é uma decisão social e jurídica, e não apenas técnica.

Exercício 4 — Privacidade diferencial: o custo do ruído

PYTHON

```python
import numpy as np

rng = np.random.default_rng(0)
verdade = 300
print("Erro absoluto médio de uma resposta (2.000 repetições):")
for eps in [0.05, 0.5, 5.0]:
    respostas = verdade + rng.laplace(0, 1 / eps, 2000)
    print(f"  ε = {eps:4.2f} -> erro médio = {np.abs(respostas - verdade).mean():6.2f} | ex.: resposta
divulgada = {respostas[0]:.1f}")

print("\nMédia de 100 consultas repetidas com ε = 0,05 cada:")
medias = [np.mean(verdade + rng.laplace(0, 1 / 0.05, 100)) for _ in range(2000)]
print(f"  erro médio da média = {np.abs(np.array(medias) - verdade).mean():.2f}  (privacidade total
gasta ≈ 100 × 0,05 = 5,0)")
```

SAÍDA

```
Erro absoluto médio de uma resposta (2.000 repetições):
  ε = 0.05 -> erro médio =  20.41 | ex.: resposta divulgada = 306.4
  ε = 0.50 -> erro médio =   1.96 | ex.: resposta divulgada = 306.2
  ε = 5.00 -> erro médio =   0.20 | ex.: resposta divulgada = 300.0

Média de 100 consultas repetidas com ε = 0,05 cada:
  erro médio da média = 2.22  (privacidade total gasta ≈ 100 × 0,05 = 5,0)
```

EXEMPLO **O que observar**

Quanto menor o ε, maior o ruído e mais privacidade. Repetir a consulta e tirar a média **recupera a precisão**, mas "gasta" privacidade: os orçamentos de ε somam. Por isso sistemas reais controlam o orçamento total de privacidade.

Apêndice E — Soluções de referência dos projetos integradores

Estas são **soluções de referência**: caminhos possíveis, e não os únicos. Compare com o seu trabalho, observe as decisões e discuta as diferenças. A avaliação de um projeto deve considerar também a qualidade da argumentação, e não apenas o código. Os critérios de avaliação de cada projeto aparecem antes do código.

Capítulo 1 — Introdução ao MLOps

Esqueleto de um projeto reprodutível

Critérios de avaliação

- Estrutura de pastas criada corretamente.
- `requirements.txt` com versões exatas lidas do ambiente (e não escritas à mão).
- Script parametrizado por linha de comando, com semente fixa.
- Execuções em processos separados, comparando hashes.
- Conclusão que cita semente, versões das bibliotecas, dados e ambiente.

```python
import os
import subprocess
import sys
import tempfile
import textwrap
from importlib.metadata import version

raiz = tempfile.mkdtemp()
for pasta in ["data", "src", "tests", "models"]:
    os.makedirs(os.path.join(raiz, pasta))
open(os.path.join(raiz, "README.md"), "w").write("# Projeto reprodutível\nExecute: python src/treino.py
--semente 42\n")

with open(os.path.join(raiz, "requirements.txt"), "w") as f:
    for pacote in ["numpy", "pandas", "scikit-learn", "joblib"]:
        f.write(f"{pacote}=={version(pacote)}\n")
print(open(os.path.join(raiz, "requirements.txt")).read().strip())

codigo = textwrap.dedent('''
    import argparse, hashlib
    from sklearn.datasets import load_breast_cancer
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.pipeline import make_pipeline
    from sklearn.preprocessing import StandardScaler

    ap = argparse.ArgumentParser()
    ap.add_argument("--semente", type=int, default=42)
    args = ap.parse_args()
    X, y = load_breast_cancer(return_X_y=True)
    pipe = make_pipeline(StandardScaler(), RandomForestClassifier(n_estimators=100,
random_state=args.semente))
    p = pipe.fit(X, y).predict_proba(X)
    print(hashlib.sha256(p.tobytes()).hexdigest()[:12])
''')
open(os.path.join(raiz, "src", "treino.py"), "w").write(codigo)

def executar(semente):
    r = subprocess.run([sys.executable, "src/treino.py", "--semente", str(semente)],
                       cwd=raiz, capture_output=True, text=True)
    return r.stdout.strip()
```

```
h1, h2, h3 = executar(42), executar(42), executar(7)
print("Execução 1 (semente 42):", h1)
print("Execução 2 (semente 42):", h2)
print("Execução 3 (semente  7):", h3)
print("Reprodutível com a mesma semente?", h1 == h2, "| muda com outra semente?", h1 != h3)
```

SAÍDA

```
numpy==2.5.4
pandas==3.0.5
scikit-learn==1.9.1
joblib==1.6.0
Execução 1 (semente 42): 785604881206
Execução 2 (semente 42): 785604881206
Execução 3 (semente  7): 6fee6da85e69
Reprodutível com a mesma semente? True | muda com outra semente? True
```

EXEMPLO Discussão dos resultados

Hashes iguais com a mesma semente mostram que o código é determinístico. Mas reprodutibilidade completa também exige os mesmos dados e as mesmas versões das bibliotecas, por isso o `requirements.txt`. Em outro computador, diferenças de hardware ou de versões ainda podem alterar os números.

Capítulo 2 — O Ciclo de Vida do ML, a Dívida Técnica e a Maturidade

Planejando um projeto de retenção de clientes

Critérios de avaliação

- Métrica de negócio definida e calculada corretamente (receita preservada menos custo de contato).
- Limiar escolhido na validação, nunca no teste.
- Comparação com "não contatar", "contatar todos" e "ao acaso" com o mesmo volume.
- Recomendação clara, mencionando incerteza do efeito da campanha e o risco de depender de um único cenário.
- Discussão de que o modelo só vale se o lucro superar as linhas de base de forma consistente.

```python
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

CUSTO, EFEITO, VALOR = 20.0, 0.40, 300.0
X, y = make_classification(10000, n_features=8, n_informative=5, weights=[0.92], class_sep=0.9,
random_state=3)
Xtr, Xtmp, ytr, ytmp = train_test_split(X, y, test_size=0.5, stratify=y, random_state=0)
Xva, Xte, yva, yte = train_test_split(Xtmp, ytmp, test_size=0.5, stratify=ytmp, random_state=0)

def lucro(contatar, y_real):
    evitados = EFEITO * (contatar & (y_real == 1)).sum()          # valor esperado de cancelamentos
evitados
    return evitados * VALOR - contatar.sum() * CUSTO

modelo = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
p_va, p_te = modelo.predict_proba(Xva)[:, 1], modelo.predict_proba(Xte)[:, 1]

limiares = np.linspace(0.02, 0.8, 40)
lucros_va = [lucro(p_va >= t, yva) for t in limiares]
t_otimo = limiares[int(np.argmax(lucros_va))]

rng = np.random.default_rng(0)
contatar_modelo = p_te >= t_otimo
aleatorio = np.zeros(len(yte), bool)
aleatorio[rng.choice(len(yte), contatar_modelo.sum(), replace=False)] = True

tab = pd.DataFrame({
    "política": ["não contatar", "contatar todos", f"modelo (limiar {t_otimo:.2f})", "ao acaso (mesmo
volume)"],
    "contatos": [0, len(yte), contatar_modelo.sum(), aleatorio.sum()],
    "lucro (R$)": [0.0, lucro(np.ones(len(yte), bool), yte), lucro(contatar_modelo, yte),
lucro(aleatorio, yte)],
})
print(tab.round(0).to_string(index=False))
print(f"Cancelamento no teste: {yte.mean():.1%}")
```

SAÍDA

```
               política  contatos  lucro (R$)
          não contatar         0         0.0
         contatar todos      2500    -24680.0
   modelo (limiar 0.16)       383      9260.0
ao acaso (mesmo volume)       383     -3940.0
Cancelamento no teste: 8.4%
```

EXEMPLO Discussão dos resultados

O resultado exato varia, mas o padrão esperado é: contatar todos gera prejuízo (o custo de contato supera o ganho), não contatar dá zero, e contatar os clientes de maior risco gera lucro positivo, muito acima do contato ao acaso com o mesmo volume. Como o efeito de 40% é uma suposição, convém refazer a conta com 20% e 60% (análise de sensibilidade) antes de decidir.

Capítulo 3 — Dados

Portão de dados com quarentena e versionamento

Critérios de avaliação

- Esquema derivado de um lote saudável, com tipos, intervalos, ausentes e categorias.
- Lotes reprovados isolados em quarentena com motivos legíveis; nenhum entra no conjunto acumulado.
- Manifesto de versões com hash do lote e do acumulado.
- Verificação de que os hashes são determinísticos.
- Proposta realista para a quarentena (investigar a origem, corrigir na fonte, reprocessar).

```python
import hashlib
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
CIDADES = ["Natal", "Recife", "Fortaleza"]

def gerar_lote(i):
    r = np.random.default_rng(100 + i)
    df = pd.DataFrame({"idade": r.integers(18, 80, 300).astype(float),
                       "renda": r.lognormal(8, 0.5, 300).round(2),
                       "cidade": r.choice(CIDADES, 300),
                       "compras_30d": r.poisson(3, 300).astype(float)})
    if i == 2: df.loc[df.sample(frac=0.2, random_state=0).index, "renda"] = np.nan
    if i == 4: df.loc[df.index[:10], "idade"] = -5
    if i == 5: df.loc[df.index[:15], "cidade"] = "Salvador"
    if i == 6: df["renda"] *= 100
    return df

lotes = {i: gerar_lote(i) for i in range(1, 7)}
base = lotes[1]
esquema = {c: {"min": base[c].min() * 0.5, "max": base[c].max() * 2, "max_ausentes": 0.02}
           for c in ["idade", "renda", "compras_30d"]}
esquema["idade"].update(min=0, max=120)
permitidas = set(base["cidade"].unique())

def validar(df):
    erros = []
    for c, r in esquema.items():
        if df[c].isna().mean() > r["max_ausentes"]:
            erros.append(f"{c}: {df[c].isna().mean():.0%} ausentes")
        v = df[c].dropna()
        if ((v < r["min"]) | (v > r["max"])).any():
            erros.append(f"{c}: {int(((v < r['min']) | (v > r['max'])).sum())} fora do intervalo")
    novas = set(df["cidade"]) - permitidas
    if novas:
        erros.append(f"cidades novas: {sorted(novas)}")
    return erros

def hash_df(df):
    return hashlib.sha256(df.to_csv(index=False).encode()).hexdigest()[:10]
```

```
acumulado, manifesto, quarentena, linhas = None, [], {}, []
for i, lote in lotes.items():
    erros = validar(lote)
    if erros:
        quarentena[i] = erros
        linhas.append((i, "QUARENTENA", "; ".join(erros)))
        continue
    acumulado = lote if acumulado is None else pd.concat([acumulado, lote], ignore_index=True)
    manifesto.append({"lote": i, "hash_lote": hash_df(lote), "linhas": len(lote), "hash_acumulado":
hash_df(acumulado)})
    linhas.append((i, "aprovado", ""))

print(pd.DataFrame(linhas, columns=["lote", "situação", "motivos"]).to_string(index=False))
print(pd.DataFrame(manifesto).to_string(index=False))
print("Mesmo lote, mesmo hash?", hash_df(lotes[3]) == hash_df(gerar_lote(3)))
```

SAÍDA

```
 lote   situação                     motivos
    1   aprovado
    2 QUARENTENA          renda: 20% ausentes
    3   aprovado
    4 QUARENTENA  idade: 10 fora do intervalo
    5 QUARENTENA  cidades novas: ['Salvador']
    6 QUARENTENA renda: 300 fora do intervalo
 lote  hash_lote  linhas hash_acumulado
    1 860b596a68     300     860b596a68
    3 e8c540c17b     300     3a3e70ae36
Mesmo lote, mesmo hash? True
```

EXEMPLO Discussão dos resultados

Os lotes 2, 4, 5 e 6 vão para a quarentena, cada um por um motivo diferente, e os lotes 1 e 3 são aprovados. O hash acumulado muda quando um lote é aprovado. Corrigir a fonte (por exemplo, a unidade da renda) e reprocessar é melhor do que "consertar" os dados à mão no treino.

Capítulo 4 — Pipelines Reprodutíveis com scikit-learn

Treino como script parametrizado e reprodutível

Critérios de avaliação

- Configuração externa ao código, lida pelo script.
- Artefato salvo com pipeline, configuração, métricas, versão do scikit-learn e hash dos dados.
- Script de previsão que verifica a versão antes de usar o modelo.
- Teste de reprodutibilidade em processos separados (métricas e hash iguais).
- Demonstração de que mudar a configuração muda o artefato.
- Conclusão que cita dados versionados, ambiente fixo (contêiner ou lockfile) e testes.

```python
import hashlib
import json
import os
import subprocess
import sys
import tempfile
import textwrap

pasta = tempfile.mkdtemp()
cfg = {"semente": 42, "tamanho_teste": 0.25, "n_estimators": 100, "max_depth": None}
json.dump(cfg, open(os.path.join(pasta, "config.json"), "w"))

open(os.path.join(pasta, "treino.py"), "w").write(textwrap.dedent('''
    import hashlib, json, sys
    import joblib, numpy as np, sklearn
    from sklearn.datasets import load_breast_cancer
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.metrics import accuracy_score, f1_score
    from sklearn.model_selection import train_test_split
    from sklearn.pipeline import make_pipeline
    from sklearn.preprocessing import StandardScaler

    cfg = json.load(open(sys.argv[1]))
    X, y = load_breast_cancer(return_X_y=True)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=cfg["tamanho_teste"], stratify=y,
random_state=cfg["semente"])
    pipe = make_pipeline(StandardScaler(), RandomForestClassifier(
        n_estimators=cfg["n_estimators"], max_depth=cfg["max_depth"],
random_state=cfg["semente"])).fit(Xtr, ytr)
    prev = pipe.predict(Xte)
    met = {"f1": round(f1_score(yte, prev), 4), "acuracia": round(accuracy_score(yte, prev), 4)}
    hash_prev = hashlib.sha256(pipe.predict_proba(X).tobytes()).hexdigest()[:12]
    joblib.dump({"pipeline": pipe, "config": cfg, "metricas": met, "versao_sklearn":
sklearn.__version__,
                 "hash_dados": hashlib.sha256(np.c_[X, y].tobytes()).hexdigest()[:12]},
"modelo.joblib")
    print(json.dumps({**met, "hash_previsoes": hash_prev}))
'''))

open(os.path.join(pasta, "prever.py"), "w").write(textwrap.dedent('''
```

```
    import sys
    import joblib, pandas as pd, sklearn
    art = joblib.load("modelo.joblib")
    assert art["versao_sklearn"] == sklearn.__version__, "versão do scikit-learn diferente da do
treino!"
    entrada = pd.read_csv(sys.argv[1])
    entrada.assign(previsao=art["pipeline"].predict(entrada.values)).to_csv(sys.argv[2], index=False)
    print("previsões gravadas:", len(entrada))
'''))

def treinar():
    r = subprocess.run([sys.executable, "treino.py", "config.json"], cwd=pasta, capture_output=True,
text=True)
    return json.loads(r.stdout)

a, b = treinar(), treinar()
print("Execução 1:", a)
print("Execução 2:", b)
print("Idênticas?", a == b)

cfg["n_estimators"] = 300
json.dump(cfg, open(os.path.join(pasta, "config.json"), "w"))
c = treinar()
print("Com 300 árvores:", c, "| hash mudou?", c["hash_previsoes"] != a["hash_previsoes"])

from sklearn.datasets import load_breast_cancer
import pandas as pd
pd.DataFrame(load_breast_cancer().data[:5]).to_csv(os.path.join(pasta, "entrada.csv"), index=False)
r = subprocess.run([sys.executable, "prever.py", "entrada.csv", "saida.csv"], cwd=pasta,
capture_output=True, text=True)
print(r.stdout.strip() or r.stderr[-300:])
print(open(os.path.join(pasta, "saida.csv")).read().splitlines()[0][-20:], "...")
```

SAÍDA

```
Execução 1: {'f1': 0.967, 'acuracia': 0.958, 'hash_previsoes': 'd973e01c0b9e'}
Execução 2: {'f1': 0.967, 'acuracia': 0.958, 'hash_previsoes': 'd973e01c0b9e'}
Idênticas? True
Com 300 árvores: {'f1': 0.967, 'acuracia': 0.958, 'hash_previsoes': '09d4b7a56d44'} | hash mudou? True
previsões gravadas: 5
26,27,28,29,previsao ...
```

EXEMPLO Discussão dos resultados

As duas execuções com a mesma configuração dão métricas e hash iguais; com 300 árvores, o hash muda. Ainda faltariam: versionar os dados de forma externa (aqui o conjunto é embutido), fixar todas as dependências (lockfile) e o ambiente do sistema (contêiner), além de testes automáticos do código.

Capítulo 5 — Rastreamento de Experimentos e Registro de Modelos

Central de experimentos com portão e registro

Critérios de avaliação

- Pelo menos 12 execuções registradas, com parâmetros, métricas, tags e modelo com assinatura.
- Linha de base registrada e usada no portão.
- Escolha pela validação cruzada, nunca pelo teste; teste usado uma única vez.
- Portão com piso e margem; registro com apelido `champion`.
- Reprodução do modelo a partir dos parâmetros registrados, com a mesma acurácia.

```python
import hashlib
import itertools
import os
import tempfile
import time
import warnings
import mlflow
import mlflow.sklearn
import numpy as np
from mlflow import MlflowClient
from mlflow.models import infer_signature
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_split
from sklearn.svm import SVC

warnings.filterwarnings("ignore")
os.chdir(tempfile.mkdtemp())
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("central")

X, y = load_digits(return_X_y=True)
hash_dados = hashlib.sha256(np.c_[X, y].tobytes()).hexdigest()[:12]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
cv = StratifiedKFold(5, shuffle=True, random_state=0)

configs = [("svc", dict(C=C, gamma=g)) for C, g in itertools.product([1, 10, 100], [0.0005, 0.002])]
configs += [("floresta", dict(n_estimators=n, max_depth=d)) for n, d in itertools.product([50, 200],
[None, 8, 4])]
fabrica = {"svc": lambda **p: SVC(**p),
           "floresta": lambda **p: RandomForestClassifier(random_state=0, **p),
           "baseline": lambda **p: LogisticRegression(max_iter=3000, **p)}

def executar(familia, params):
    with mlflow.start_run(run_name=f"{familia}_{params}") as run:
        t0 = time.perf_counter()
        modelo = fabrica[familia](**params)
        f1 = cross_val_score(modelo, Xtr, ytr, cv=cv, scoring="f1_macro").mean()
```

```
        modelo.fit(Xtr, ytr)
        mlflow.log_params({"familia": familia, **{k: str(v) for k, v in params.items()}})
        mlflow.log_metrics({"f1_cv": f1, "tempo_s": time.perf_counter() - t0})
        mlflow.set_tag("hash_dados", hash_dados)
        info = mlflow.sklearn.log_model(modelo, name="modelo", signature=infer_signature(Xte,
modelo.predict(Xte)),
                                        skops_trusted_types=["sklearn.tree._tree.Tree"])
    return f1, modelo, info.model_uri, run.info.run_id

base_f1 = executar("baseline", {})[0]
for fam, p in configs:
    executar(fam, p)

res = mlflow.search_runs(order_by=["metrics.f1_cv DESC"])
res = res[res["params.familia"] != "baseline"]
print(res[["params.familia", "params.C", "params.gamma", "params.n_estimators", "params.max_depth",
"metrics.f1_cv"]].head(5).round(4).to_string(index=False))

melhor = res.iloc[0]
f1_melhor = melhor["metrics.f1_cv"]
aprovado = f1_melhor >= 0.95 and f1_melhor >= base_f1 + 0.01
print(f"Linha de base F1 = {base_f1:.4f} | melhor F1 = {f1_melhor:.4f} | portão {'APROVADO' if aprovado
else 'REPROVADO'}")

if aprovado:
    uri = f"runs:/{melhor['run_id']}/modelo"
    versao = mlflow.register_model(uri, "digitos").version
    MlflowClient().set_registered_model_alias("digitos", "champion", versao)
    champion = mlflow.sklearn.load_model("models:/digitos@champion")
    print(f"Registrado como versão {versao}; acurácia de teste (uso único) = {champion.score(Xte,
yte):.4f}")

    # reprodução a partir dos parâmetros registrados
    familia = melhor["params.familia"]
    params = {k.split(".")[1]: v for k, v in melhor.items()
              if k.startswith("params.") and k != "params.familia" and v is not None and v == v}
    conv = {"C": float, "gamma": float, "n_estimators": int, "max_depth": lambda s: None if s == "None"
else int(s)}
    params = {k: conv[k](v) for k, v in params.items()}
    reproduzido = fabrica[familia](**params).fit(Xtr, ytr)
    print("Modelo reconstruído dos parâmetros registrados:", familia, params,
          "| mesma acurácia?", reproduzido.score(Xte, yte) == champion.score(Xte, yte))
```

SAÍDA

```
params.familia params.C params.gamma params.n_estimators params.max_depth  metrics.f1_cv
           svc        1       0.0005                  NaN              NaN         0.9857
           svc        1        0.002                  NaN              NaN         0.9848
           svc      100       0.0005                  NaN              NaN         0.9833
           svc       10       0.0005                  NaN              NaN         0.9833
           svc      100        0.002                  NaN              NaN         0.9833
Linha de base F1 = 0.9625 | melhor F1 = 0.9857 | portão APROVADO
Registrado como versão 1; acurácia de teste (uso único) = 0.9870
Modelo reconstruído dos parâmetros registrados: svc {'gamma': 0.0005, 'C': 1.0} | mesma acurácia? True
```

EXEMPLO Discussão dos resultados

O resultado exato depende do computador e das versões. Nesta execução, o SVC com `C` alto e `gamma` pequeno ficou no topo, a linha de base de regressão logística ficou abaixo e o portão foi aprovado. A reconstrução do modelo a partir dos parâmetros registrados deu a mesma acurácia, o que comprova a rastreabilidade do processo.

Capítulo 6 — Avaliação Rigorosa e Otimização de Hiperparâmetros

Relatório de avaliação rigorosa de um modelo de risco

Critérios de avaliação

- Divisão estratificada em treino, validação e teste, com o teste usado uma única vez.
- Comparação por validação cruzada repetida, com média e desvio.
- Calibração verificada com Brier; limiar escolhido na validação com custos explícitos.
- Intervalo de confiança por bootstrap para a métrica final.
- Análise por região com discussão de desigualdade e limites (amostras pequenas na fatia).
- Recomendação objetiva, com ressalvas sobre incerteza, custo e monitoramento.

```python
import numpy as np
import pandas as pd
from sklearn.calibration import CalibratedClassifierCV
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, precision_score, recall_score
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(12000, n_features=15, n_informative=6, weights=[0.94], class_sep=0.8,
random_state=5)
rng = np.random.default_rng(5)
regiao = rng.choice(["A", "B"], len(y), p=[0.7, 0.3])
X[regiao == "B", 0] += 0.8

idx = np.arange(len(y))
itr, itmp = train_test_split(idx, test_size=0.4, stratify=y, random_state=0)
iva, ite = train_test_split(itmp, test_size=0.5, stratify=y[itmp], random_state=0)
Xtr, ytr, Xva, yva, Xte, yte = X[itr], y[itr], X[iva], y[iva], X[ite], y[ite]

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=0)
candidatos = {"logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
              "floresta": RandomForestClassifier(200, min_samples_leaf=3, random_state=0)}
medias = {}
for nome, m in candidatos.items():
    s = cross_val_score(m, Xtr, ytr, cv=cv, scoring="average_precision")
    medias[nome] = s.mean()
    print(f"{nome:9s} AP (CV) = {s.mean():.3f} ± {s.std():.3f}")
melhor = max(medias, key=medias.get)

bruto = candidatos[melhor].fit(Xtr, ytr)
calibrado = CalibratedClassifierCV(candidatos[melhor], method="isotonic", cv=5).fit(Xtr, ytr)
pva_b, pva_c = bruto.predict_proba(Xva)[:, 1], calibrado.predict_proba(Xva)[:, 1]
print(f"Melhor: {melhor} | Brier na validação: bruto = {brier_score_loss(yva, pva_b):.4f}, calibrado =
{brier_score_loss(yva, pva_c):.4f}")
```

```
C_FN, C_FP = 1000, 30
custo = lambda p, y, t: C_FN * ((p < t) & (y == 1)).sum() + C_FP * ((p >= t) & (y == 0)).sum()
limiares = np.linspace(0.01, 0.9, 90)
t = limiares[int(np.argmin([custo(pva_c, yva, t_) for t_ in limiares]))]
print(f"Limiar escolhido na validação: {t:.2f}")

pte = calibrado.predict_proba(Xte)[:, 1]
rng = np.random.default_rng(0)
boot = [average_precision_score(yte[i], pte[i]) for i in (rng.integers(0, len(yte), len(yte)) for _ in
range(500))]
lo, hi = np.percentile(boot, [2.5, 97.5])
prev = pte >= t
print(f"Teste: AP = {average_precision_score(yte, pte):.3f} (IC95% {lo:.3f}-{hi:.3f}) | revocação =
{recall_score(yte, prev):.3f} | "
      f"precisão = {precision_score(yte, prev):.3f} | custo = R$ {custo(pte, yte, t):,.0f}")

reg_te = regiao[ite]
linhas = []
for r in ["A", "B"]:
    m_ = reg_te == r
    linhas.append({"região": r, "n": int(m_.sum()), "positivos (%)": round(yte[m_].mean() * 100, 1),
                   "AP": round(average_precision_score(yte[m_], pte[m_]), 3),
                   "revocação": round(recall_score(yte[m_], prev[m_]), 3),
                   "precisão": round(precision_score(yte[m_], prev[m_], zero_division=0), 3)})
print(pd.DataFrame(linhas).to_string(index=False))
```

SAÍDA

```
logreg    AP (CV) = 0.402 ± 0.039
floresta  AP (CV) = 0.580 ± 0.043
Melhor: floresta | Brier na validação: bruto = 0.0368, calibrado = 0.0351
Limiar escolhido na validação: 0.03
Teste: AP = 0.538 (IC95% 0.460-0.616) | revocação = 0.845 | precisão = 0.154 | custo = R$ 45,660
região    n  positivos (%)    AP  revocação  precisão
     A 1659            6.3 0.593      0.865     0.147
     B  741            6.9 0.441      0.804     0.170
```

EXEMPLO Discussão dos resultados

O resultado exato depende do computador e das versões das bibliotecas. O padrão esperado: a floresta supera a regressão logística em AP; a calibração reduz o Brier; o limiar escolhido é muito menor que 0,5 por causa do custo do falso negativo; o intervalo de confiança da AP é razoavelmente largo, pois há poucos positivos; e as métricas diferem entre as regiões, apontando que a fatia B merece atenção e monitoramento próprio.

Capítulo 7 — Empacotamento e Serviço de Modelos

API completa com tempo real, lote, limites e testes

Critérios de avaliação

- Endpoints de saúde, metadados, previsão individual e em lote implementados.
- Contrato com faixas e categorias; limites de tamanho do lote.
- Modelo carregado uma vez; versão presente nas respostas.
- Testes de caminho feliz, erros de contrato, limites e consistência entre individual e lote.
- Medição de latência com percentis e comparação entre individual e lote.
- Lista realista de pendências: autenticação, limite de taxa, logs estruturados, contêiner e monitoramento.

```python
import os
import tempfile
import time
from contextlib import asynccontextmanager
from datetime import date
from typing import Literal
import joblib
import numpy as np
import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

os.chdir(tempfile.mkdtemp())
rng = np.random.default_rng(0)
n = 3000
df = pd.DataFrame({"idade": rng.integers(18, 75, n).astype(float), "renda": rng.lognormal(8, 0.5, n),
                   "tempo_cliente": rng.integers(1, 120, n).astype(float),
                   "plano": rng.choice(["basico", "padrao", "premium"], n, p=[0.5, 0.35, 0.15]),
                   "cidade": rng.choice(["Natal", "Recife", "Fortaleza"], n)})
risco = 0.03 * (50 - df["idade"]) - 0.02 * df["tempo_cliente"] + (df["plano"] == "basico") * 0.8
y = (risco + rng.normal(0, 0.8, n) > 0.3).astype(int)
num, cat = ["idade", "renda", "tempo_cliente"], ["plano", "cidade"]
prep = ColumnTransformer([("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("esc",
StandardScaler())]), num),
                          ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
pipe = Pipeline([("prep", prep), ("m", RandomForestClassifier(100, random_state=0))]).fit(df[num +
cat], y)
joblib.dump({"pipeline": pipe, "versao": "1.2.0", "colunas": num + cat, "data": date(2026, 10,
17).isoformat()}, "modelo.joblib")

class Cliente(BaseModel):
    idade: float = Field(ge=18, le=100)
    renda: float = Field(ge=0, le=1_000_000)
    tempo_cliente: float = Field(ge=0, le=600)
```

```
    plano: Literal["basico", "padrao", "premium"]
    cidade: Literal["Natal", "Recife", "Fortaleza"]

class Lote(BaseModel):
    clientes: list[Cliente]

def criar_app():
    estado = {}
    @asynccontextmanager
    async def ciclo(app):
        estado["a"] = joblib.load("modelo.joblib")
        yield
    app = FastAPI(lifespan=ciclo)

    def prever_df(lista):
        a = estado["a"]
        X = pd.DataFrame([c.model_dump() for c in lista])[a["colunas"]]
        return [round(float(p), 4) for p in a["pipeline"].predict_proba(X)[:, 1]]

    @app.get("/saude")
    def saude():
        return {"status": "ok", "versao": estado["a"]["versao"]}

    @app.get("/metadados")
    def metadados():
        a = estado["a"]
        return {"versao": a["versao"], "colunas": a["colunas"], "data_treino": a["data"]}

    @app.post("/previsao")
    def previsao(c: Cliente):
        return {"probabilidade": prever_df([c])[0], "versao": estado["a"]["versao"]}

    @app.post("/previsao_lote")
    def previsao_lote(l: Lote):
        if not 1 <= len(l.clientes) <= 100:
            raise HTTPException(status_code=422, detail="o lote deve ter de 1 a 100 clientes")
        return {"probabilidades": prever_df(l.clientes), "versao": estado["a"]["versao"]}
    return app

ok = {"idade": 33, "renda": 4500, "tempo_cliente": 8, "plano": "basico", "cidade": "Natal"}
resultados = {}
with TestClient(criar_app()) as cli:
    resultados["saúde"] = cli.get("/saude").status_code == 200
    resultados["caminho feliz"] = cli.post("/previsao", json=ok).status_code == 200
    resultados["campo inválido"] = cli.post("/previsao", json=dict(ok, plano="ultra")).status_code ==
422
    resultados["lote vazio"] = cli.post("/previsao_lote", json={"clientes": []}).status_code == 422
    resultados["lote acima do limite"] = cli.post("/previsao_lote", json={"clientes": [ok] *
101}).status_code == 422
    outro = dict(ok, idade=60, plano="premium")
    ind = cli.post("/previsao", json=outro).json()["probabilidade"]
    lot = cli.post("/previsao_lote", json={"clientes": [ok, outro]}).json()["probabilidades"][1]
    resultados["individual == lote"] = ind == lot
    for nome, passou in resultados.items():
        print(f"teste {nome:22s}: {'OK' if passou else 'FALHOU'}")

    ts = []
    for _ in range(200):
```

```
        t0 = time.perf_counter(); cli.post("/previsao", json=ok); ts.append((time.perf_counter() - t0)
* 1000)
    t0 = time.perf_counter(); cli.post("/previsao_lote", json={"clientes": [ok] * 100}); lote_ms =
(time.perf_counter() - t0) * 1000
    print(f"individual: p50 = {np.percentile(ts, 50):.1f} ms | p95 = {np.percentile(ts, 95):.1f} ms")
    print(f"lote de 100 em uma requisição: {lote_ms:.1f} ms ({lote_ms / 100:.2f} ms por cliente)")
```

SAÍDA

```
teste saúde                 : OK
teste caminho feliz         : OK
teste campo inválido        : OK
teste lote vazio            : OK
teste lote acima do limite  : OK
teste individual == lote    : OK
individual: p50 = 12.3 ms | p95 = 15.2 ms
lote de 100 em uma requisição: 14.1 ms (0.14 ms por cliente)
```

EXEMPLO Discussão dos resultados

Todos os testes passam e o lote de 100 clientes custa muito menos por cliente do que 100 chamadas separadas. Os valores de latência dependem do computador e incluem a camada simulada do `TestClient`. Pendências típicas: autenticação, limite de taxa, logs estruturados com identificador de requisição, empacotamento em contêiner, monitoramento de dados e de desempenho e plano de reversão.

Capítulo 8 — Testes e CI/CD para Machine Learning

Suíte de testes e CI local de ponta a ponta

Critérios de avaliação

- Estrutura `src/` e `tests/` criada e funcional; imports corretos.
- Testes de função, de dados e de modelo (desempenho, determinismo e invariância).
- `ci.py` executa etapas em ordem, imprime resultado e retorna código de saída coerente.
- Cenário com defeito mostra a falha na etapa correta, interrompendo o fluxo.
- Lista de etapas adicionais: estilo (ruff), tipos (mypy), cobertura, construção de imagem, varredura de vulnerabilidades, implantação em homologação.

```python
import os
import subprocess
import sys
import tempfile
import textwrap

raiz = tempfile.mkdtemp()
os.chdir(raiz)
for p in ["src", "tests"]:
    os.makedirs(p)
open("src/__init__.py", "w").write("")

def escrever(caminho, texto):
    open(caminho, "w").write(textwrap.dedent(texto))

escrever("src/preproc.py", '''
    def limpar(df):
        df = df.drop_duplicates()
        if (df["alcohol"] < 5).any() or (df["alcohol"] > 20).any():
            raise ValueError("álcool fora da faixa esperada")
        return df
''')
escrever("src/modelo.py", '''
    from sklearn.datasets import load_wine
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import f1_score
    from sklearn.model_selection import train_test_split
    from sklearn.pipeline import make_pipeline
    from sklearn.preprocessing import StandardScaler

    SEMENTE = 0

    def treinar():
        X, y = load_wine(return_X_y=True, as_frame=True)
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=SEMENTE)
        pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
        return pipe, Xte, yte, f1_score(yte, pipe.predict(Xte), average="macro")
''')
escrever("tests/conftest.py", '''
    import sys, os
```

```
    sys.path.insert(0, os.getcwd())
''')
escrever("tests/test_preproc.py", '''
    import pandas as pd, pytest
    from src.preproc import limpar

    def test_remove_duplicados():
        df = pd.DataFrame({"alcohol": [12.0, 12.0, 13.0]})
        assert len(limpar(df)) == 2

    def test_rejeita_alcool_absurdo():
        with pytest.raises(ValueError):
            limpar(pd.DataFrame({"alcohol": [120.0]}))

    def test_nao_altera_entrada():
        df = pd.DataFrame({"alcohol": [12.0, 12.0]})
        limpar(df)
        assert len(df) == 2
''')
escrever("tests/test_dados.py", '''
    from sklearn.datasets import load_wine

    def test_esquema_e_faixas():
        X, y = load_wine(return_X_y=True, as_frame=True)
        assert X.shape[1] == 13 and not X.isna().any().any()
        assert X["alcohol"].between(5, 20).all()
        assert set(y.unique()) == {0, 1, 2}
''')
escrever("tests/test_modelo.py", '''
    import numpy as np
    from src.modelo import treinar

    def test_f1_minimo():
        assert treinar()[3] >= 0.93

    def test_determinismo():
        a, b = treinar(), treinar()
        assert np.array_equal(a[0].predict_proba(a[1]), b[0].predict_proba(b[1]))

    def test_invariancia_a_ruido_minimo():
        pipe, Xte, _, _ = treinar()
        ruido = np.random.default_rng(0).normal(0, 1e-3, Xte.shape) * Xte.std().values
        assert (pipe.predict(Xte) != pipe.predict(Xte + ruido)).mean() <= 0.03
''')
escrever("ci.py", '''
    import subprocess, sys
    ETAPAS = [("sintaxe", [sys.executable, "-m", "compileall", "-q", "src", "tests"]),
              ("testes", [sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests"]),
              ("portão de qualidade", [sys.executable, "-c",
                  "from src.modelo import treinar; f = treinar()[3]; print(f'F1 macro = {f:.3f}');
raise SystemExit(0 if f >= 0.93 else 1)"])]
    for nome, cmd in ETAPAS:
        r = subprocess.run(cmd, capture_output=True, text=True)
        ultima = (r.stdout.strip().splitlines() or r.stderr.strip().splitlines() or ["sem mensagens"])
[-1]
        print(f"[{'OK' if r.returncode == 0 else 'FALHOU'}] {nome}: {ultima}")
        if r.returncode:
            sys.exit(1)
```

```
    print("CI APROVADA")
''')

def rodar_ci():
    r = subprocess.run([sys.executable, "ci.py"], capture_output=True, text=True)
    print(r.stdout.strip()); print("código de saída:", r.returncode)

print("--- cenário normal ---")
rodar_ci()
print("--- cenário com defeito (regularização exagerada no modelo) ---")
texto = open("src/modelo.py").read()
open("src/modelo.py", "w").write(texto.replace("LogisticRegression(max_iter=2000)",
"LogisticRegression(C=1e-6, max_iter=2000)"))
rodar_ci()
```

SAÍDA

```
--- cenário normal ---
[OK] sintaxe: sem mensagens
[OK] testes: 7 passed in 1.23s
[OK] portão de qualidade: F1 macro = 1.000
CI APROVADA
código de saída: 0
--- cenário com defeito (regularização exagerada no modelo) ---
[OK] sintaxe: sem mensagens
[FALHOU] testes: 1 failed, 6 passed in 1.29s
código de saída: 1
```

EXEMPLO Discussão dos resultados

No cenário normal, as três etapas passam e a CI é aprovada. No cenário com defeito (regularização exagerada, `C=1e-6`), o modelo perde qualidade e a CI falha na etapa de testes (F1 mínimo), com código de saída 1, interrompendo o fluxo antes do portão. Em uma CI real, haveria ainda etapas de estilo, tipos, cobertura, construção do contêiner e implantação em homologação.

Capítulo 9 — Monitoramento, Deriva de Dados e Retreinamento

Painel de monitoramento de 12 semanas com três tipos de deriva

Critérios de avaliação

- Simulação documentada, com três mudanças em semanas distintas.
- Cálculo semanal de PSI, AUC de domínio, proporção real de positivos, probabilidade média e AUC do modelo.
- Regra de alerta sem rótulo definida e comparada com a queda da AUC.
- Comparação das três estratégias com as mesmas métricas e as mesmas semanas.
- Atualização da referência após o retreino por gatilho.
- Recomendação que explica que a mudança de covariáveis e a de rótulos nem sempre derrubam a AUC, mas a deriva de conceito sim.

```python
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

def semana(s, n=1200):
    r = np.random.default_rng(500 + s)
    N = 6 * n                                                    # reserva grande, para reamostrar por classe
    X = r.normal(size=(N, 4))
    if s >= 3:
        X[:, 0] += 0.8                                           # mudança de covariáveis (P(y | X) fixo)
    w = np.array([2.0, 1.6, -1.2, 0.0])
    if s >= 8:
        ang = np.deg2rad(80)                                     # deriva de conceito: a fronteira gira
        w = np.array([2.0 * np.cos(ang), 1.6, -1.2, 2.0 * np.sin(ang)])
    y = (r.random(N) < 1 / (1 + np.exp(-(X @ w - 3.4)))).astype(int)
    # mudança de rótulos (label shift): muda P(y) e preserva P(X | y), sorteando cada classe separadamente
    alvo = 0.15 if s < 6 else 0.35
    n1 = int(round(alvo * n)); i1 = r.choice(np.flatnonzero(y == 1), n1, replace=False); i0 = r.choice(np.flatnonzero(y == 0), n - n1, replace=False)
    idx = r.permutation(np.r_[i1, i0])
    return X[idx], y[idx]

def psi(ref, atual, faixas=10):
    cortes = np.quantile(ref, np.linspace(0, 1, faixas + 1)); cortes[0], cortes[-1] = -np.inf, np.inf
    p = np.clip(np.histogram(ref, cortes)[0] / len(ref), 1e-6, None)
    q = np.clip(np.histogram(atual, cortes)[0] / len(atual), 1e-6, None)
    return float(np.sum((q - p) * np.log(q / p)))
```

```
def auc_dominio(A, B):
    Z = np.vstack([A, B]); d = np.r_[np.zeros(len(A)), np.ones(len(B))]
    return cross_val_score(RandomForestClassifier(60, min_samples_leaf=5, random_state=0, n_jobs=2), Z,
d, cv=3, scoring="roc_auc").mean()

fabrica = lambda: make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
dados = [semana(s) for s in range(13)]
Xref, yref = dados[0]
estatico = fabrica().fit(Xref, yref)

def treinar_com(ini, fim):
    return fabrica().fit(np.vstack([d[0] for d in dados[ini:fim]]), np.concatenate([d[1] for d in
dados[ini:fim]]))

linhas, ref_X = [], Xref
periodico, gatilho, espera = estatico, estatico, None
for s in range(1, 13):
    X, y = dados[s]
    if s >= 4 and (s - 1) % 3 == 0:                                  # (b) a cada 3 semanas, janela de
3 semanas
        periodico = treinar_com(s - 3, s)
    psi0, auc_dom = psi(ref_X[:, 0], X[:, 0]), auc_dominio(ref_X, X)
    alerta = psi0 > 0.25 or auc_dom > 0.80
    acao = ""
    if alerta and espera is None:
        espera = s                                                   # (c) alerta: espera chegarem
rótulos do regime novo
    if espera is not None and s - espera >= 2:
        gatilho, espera = treinar_com(s - 3, s), None
        ref_X = np.vstack([d[0] for d in dados[s - 3:s]])            # nova referência de
monitoramento
        acao = "retreinou"
    p = {"estatico": estatico, "periodico": periodico, "gatilho": gatilho}
    linha = {"semana": s, "PSI_x0": psi0, "AUC_dom": auc_dom, "alerta": int(alerta), "acao": acao,
"pos_real": y.mean()}
    for nome, m in p.items():
        pr = m.predict_proba(X)[:, 1]
        linha[f"AUC_{nome}"] = roc_auc_score(y, pr)
        linha[f"dif_prob_{nome}"] = abs(pr.mean() - y.mean())
    linhas.append(linha)
tab = pd.DataFrame(linhas)
cols = ["semana", "PSI_x0", "AUC_dom", "alerta", "acao", "pos_real", "AUC_estatico", "AUC_periodico",
"AUC_gatilho"]
print(tab[cols].round(3).to_string(index=False))

prim_alerta = int(tab.loc[tab["alerta"] == 1, "semana"].min())
queda = tab.loc[tab["AUC_estatico"] < tab["AUC_estatico"].iloc[0] - 0.10, "semana"]
print(f"Primeiro alerta sem rótulos: semana {prim_alerta} | AUC do estático cai > 0,10: semana
{int(queda.min()) if len(queda) else 'nunca'}")
r = tab[tab["semana"] >= 4]
print("AUC média (semanas 4-12):", {k: round(float(r[f"AUC_{k}"].mean()), 3) for k in ["estatico",
"periodico", "gatilho"]})
print("Erro médio de proporção (semanas 4-12):", {k: round(float(r[f"dif_prob_{k}"].mean()), 3) for k
in ["estatico", "periodico", "gatilho"]})
```

SAÍDA

```
 semana  PSI_x0  AUC_dom  alerta      acao  pos_real  AUC_estatico  AUC_periodico  AUC_gatilho
      1   0.009    0.498       0                0.15         0.921          0.921        0.921
      2   0.016    0.509       0                0.15         0.934          0.934        0.934
      3   0.386    0.634       1                0.15         0.915          0.915        0.915
      4   0.463    0.661       1                0.15         0.908          0.912        0.908
      5   0.384    0.632       1 retreinou      0.15         0.919          0.921        0.921
      6   0.209    0.565       0                0.35         0.928          0.929        0.929
      7   0.145    0.563       0                0.35         0.916          0.915        0.914
      8   0.167    0.573       0                0.35         0.718          0.752        0.764
      9   0.181    0.594       0                0.35         0.727          0.759        0.769
     10   0.197    0.587       0                0.35         0.741          0.892        0.780
     11   0.204    0.611       0                0.35         0.719          0.896        0.760
     12   0.178    0.599       0                0.35         0.716          0.892        0.761
Primeiro alerta sem rótulos: semana 3 | AUC do estático cai > 0,10: semana 8
AUC média (semanas 4-12): {'estatico': 0.81, 'periodico': 0.874, 'gatilho': 0.834}
Erro médio de proporção (semanas 4-12): {'estatico': 0.034, 'periodico': 0.045, 'gatilho': 0.066}
```

EXEMPLO Discussão dos resultados

O resultado exato depende do computador e das versões. Nesta execução: (i) o alerta sem rótulos dispara na semana 3, junto com a mudança de covariáveis, mas a AUC do modelo quase não muda nessa fase (nem toda deriva prejudica o desempenho); (ii) na semana 6, o *label shift* (a fração de positivos sobe de 15% para 35%, com P(X | y) preservado) não derruba a AUC, pois o ordenamento dos casos continua bom; o efeito aparece sobretudo nas probabilidades e no limiar de decisão, e a diferença entre a probabilidade média prevista e a fração real oscila de forma irregular entre as estratégias, de modo que não se deve tirar uma conclusão forte só desse indicador; (iii) o alerta sem rótulos não enxerga a deriva de conceito da semana 8, porque a distribuição dos atributos não muda: só o desempenho com rótulos mostra a queda da AUC do modelo estático (de cerca de 0,92 para cerca de 0,72); (iv) por isso, o retreino periódico recupera a AUC (cerca de 0,89 nas semanas 10 a 12), enquanto o retreino por gatilho baseado apenas em sinais sem rótulos recupera pouco. A lição é combinar monitoramento sem rótulos (rápido) com monitoramento de desempenho com rótulos (mais lento, mas decisivo), além de retreinos periódicos de segurança.

Capítulo 10 — Governança, Explicabilidade, Ética e Arquiteturas Avançadas

Auditoria de um modelo de crédito: documentação, explicação, justiça e privacidade

Critérios de avaliação

- Modelo treinado sem o atributo sensível e avaliado por grupo, com razão de impacto e diferença de revocação.
- Explicação global (permutação) e local simples, com cuidado ao interpretar.
- Divulgação privada com Laplace, comparando valor real e valor divulgado.
- Demonstração clara do erro temporal da junção ingênua.
- Cartão do modelo com uso pretendido, métricas por grupo e limitações.
- Recomendação justificada (por exemplo, aprovar com condições: corrigir a captação da renda e monitorar a razão de impacto).

```python
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import f1_score, recall_score, roc_auc_score
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
n = 15000
grupo = rng.choice(["A", "B"], n, p=[0.65, 0.35])
renda_real = rng.lognormal(8.2, 0.5, n)
idade = rng.integers(21, 70, n).astype(float)
emprego = rng.integers(0, 30, n).astype(float)
dividas = rng.lognormal(6.5, 0.8, n)
escore = 1.2 * np.log(renda_real / 3000) - 0.9 * np.log(dividas / 700) + 0.04 * emprego
y = (escore + rng.normal(0, 0.6, n) > 0.2).astype(int)                                  # 1 = bom
pagador
renda_obs = renda_real * np.where(grupo == "B", rng.lognormal(-0.35, 0.8, n), rng.lognormal(0, 0.1, n))
# erro maior e viés para baixo no grupo B
d = pd.DataFrame({"renda": renda_obs, "idade": idade, "tempo_emprego": emprego, "dividas": dividas,
"grupo": grupo, "y": y})
feats = ["renda", "idade", "tempo_emprego", "dividas"]
tr, te = train_test_split(d, test_size=0.3, random_state=0, stratify=d["y"])
m = RandomForestClassifier(300, min_samples_leaf=5, random_state=0).fit(tr[feats], tr["y"])
te = te.assign(p=m.predict_proba(te[feats])[:, 1]); te["aprov"] = te["p"] >= 0.5

# 1-2. desempenho e justiça
linhas = []
for nome, dg in [("geral", te)] + list(te.groupby("grupo")):
    linhas.append({"grupo": nome, "n": len(dg), "AUC": roc_auc_score(dg["y"], dg["p"]), "F1":
f1_score(dg["y"], dg["aprov"]),
                   "revocacao": recall_score(dg["y"], dg["aprov"]), "taxa_aprov": dg["aprov"].mean()})
tab = pd.DataFrame(linhas).round(3)
print(tab.to_string(index=False))
```

```
ta, tb = tab.set_index("grupo").loc["A", "taxa_aprov"], tab.set_index("grupo").loc["B", "taxa_aprov"]
rev = tab.set_index("grupo")["revocacao"]
print(f"Razão de impacto B/A = {tb / ta:.3f} | diferença de revocação (A − B) = {rev['A'] - rev['B']:
+.3f}")

# 3. explicações
imp = permutation_importance(m, te[feats], te["y"], n_repeats=10, random_state=0, scoring="roc_auc")
print("Importância por permutação (queda da AUC):", {f: round(float(v), 3) for f, v in zip(feats,
imp.importances_mean)})
reprovados_b = te[(te["grupo"] == "B") & (~te["aprov"]) & (te["y"] == 1)]
caso = reprovados_b.iloc[0]
base = m.predict_proba(caso[feats].to_frame().T.astype(float))[0, 1]
efeito = {}
for f in feats:
    alt = caso[feats].copy(); alt[f] = d[f].median()
    efeito[f] = m.predict_proba(alt.to_frame().T.astype(float))[0, 1] - base
print(f"Caso reprovado do grupo B (bom pagador real): prob = {base:.2f} | mudança ao trocar cada
atributo pela mediana:", {k: round(float(v), 2) for k, v in efeito.items()})

# 4. divulgação privada
eps = 0.5
for g in ["A", "B"]:
    real = int(te.loc[te["grupo"] == g, "aprov"].sum())
    print(f"Aprovados no grupo {g}: real = {real} | divulgado com ε = {eps}: {real + rng.laplace(0, 1 /
eps):.1f}")

# 5. correção temporal
eventos = pd.DataFrame({"cliente": [1, 1, 2], "data": pd.to_datetime(["2026-03-10", "2026-05-20",
"2026-04-15"]), "inadimpliu": [0, 1, 0]}).sort_values("data")
atributos = pd.DataFrame({"cliente": [1, 1, 1, 2, 2], "data_calculo": pd.to_datetime(["2026-03-01",
"2026-04-01", "2026-05-01", "2026-03-15", "2026-04-10"]),
                          "dividas_30d": [100.0, 80.0, 20.0, 300.0,
310.0]}).sort_values("data_calculo")
correto = pd.merge_asof(eventos, atributos, left_on="data", right_on="data_calculo", by="cliente")
ingenuo = eventos.merge(atributos.groupby("cliente")["dividas_30d"].last().rename("mais_recente"),
on="cliente")
print("Cliente 1 em 10/03 -> correto:", float(correto.loc[(correto.cliente == 1) & (correto.data ==
"2026-03-10"), "dividas_30d"].iloc[0]),
      "| ingênuo (vaza o futuro):", float(ingenuo.loc[(ingenuo.cliente == 1) & (ingenuo.data == "2026-
03-10"), "mais_recente"].iloc[0]))

# 6. cartão do modelo
print("\n# Cartão do modelo: credito (v1.0.0)")
print(f"- Uso: apoio à decisão de crédito, com revisão humana. Fora do escopo: decisão totalmente
automática.")
print(f"- AUC geral {tab.loc[0, 'AUC']} | revocação A {rev['A']} / B {rev['B']} | razão de impacto B/A
{tb / ta:.2f}")
print("- Limitação: renda registrada com mais erro no grupo B; recomenda-se corrigir a captação antes
de ampliar o uso.")
```

SAÍDA

```
grupo    n   AUC    F1  revocacao  taxa_aprov
geral 4500 0.870 0.872      0.891       0.744
    A 2936 0.889 0.885      0.913       0.766
    B 1564 0.834 0.845      0.848       0.701
Razão de impacto B/A = 0.915 | diferença de revocação (A − B) = +0.065
Importância por permutação (queda da AUC): {'renda': 0.093, 'idade': 0.001, 'tempo_emprego': 0.055,
```

```
'dividas': 0.224}
Caso reprovado do grupo B (bom pagador real): prob = 0.22 | mudança ao trocar cada atributo pela
mediana: {'renda': 0…
Aprovados no grupo A: real = 2250 | divulgado com ε = 0.5: 2248.4
Aprovados no grupo B: real = 1096 | divulgado com ε = 0.5: 1098.6
Cliente 1 em 10/03 -> correto: 100.0 | ingênuo (vaza o futuro): 20.0

# Cartão do modelo: credito (v1.0.0)
- Uso: apoio à decisão de crédito, com revisão humana. Fora do escopo: decisão totalmente automática.
- AUC geral 0.87 | revocação A 0.913 / B 0.848 | razão de impacto B/A 0.92
- Limitação: renda registrada com mais erro no grupo B; recomenda-se corrigir a captação antes de
ampliar o uso.
```

EXEMPLO Discussão dos resultados

O resultado exato depende do computador e das versões. Nesta execução: (i) o desempenho geral é bom (AUC de cerca de 0,87), mas pior no grupo B (AUC de cerca de 0,83, contra cerca de 0,89 no grupo A), e a revocação dos bons pagadores é menor no grupo B (cerca de 0,85, contra cerca de 0,91); (ii) a razão de impacto B/A fica em torno de 0,92, acima da regra prática de 0,8, mas a diferença de revocação merece atenção, pois bons pagadores do grupo B são rejeitados com mais frequência; (iii) as dívidas e a renda são os atributos mais importantes; (iv) a contagem divulgada com ε = 0,5 difere do valor real em poucas unidades; (v) a junção ingênua usa um valor do futuro (20 em vez de 100). A recomendação razoável é aprovar com condições: corrigir a captação da renda do grupo B, monitorar a razão de impacto e a revocação por grupo e revisar o modelo periodicamente.

Capítulo 11 — Projeto Final

Do dado à produção: um sistema de reconhecimento de dígitos com monitoramento e retreino

Critérios de avaliação

- Dados validados, versionados por hash, com teste original reservado para uso único.
- Candidatos comparados com validação cruzada, incluindo linha de base, e tudo registrado.
- Portão de qualidade com piso e margem; apelido `champion` usado pela API.
- API com contrato de entrada que rejeita dados inválidos e informa a versão do modelo.
- Testes de código e de comportamento do modelo que realmente rodam e passam.
- Produção simulada sem vazamento temporal: rótulos só são usados após chegarem.
- Monitoramento sem rótulo comparado com a queda de desempenho; regra de alerta justificada.
- Treinamento contínuo com comparação em dados inéditos para ambos os modelos, margem mínima e atualização da referência.
- Cartão do modelo e relatório claros, com limitações, desempenho por classe e plano de reversão.

```python
import hashlib, os, subprocess, sys, tempfile, textwrap, warnings
import joblib, numpy as np, pandas as pd
import mlflow, mlflow.sklearn
from mlflow import MlflowClient
from mlflow.models import infer_signature
from sklearn.datasets import load_digits
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, recall_score
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold, cross_val_score,
train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from scipy.stats import loguniform

warnings.filterwarnings("ignore")
os.chdir(tempfile.mkdtemp())
SEMENTE = 42
rng = np.random.default_rng(SEMENTE)

# ---------------------------------------------------------------- 1. dados, validação e versão
X, y = load_digits(return_X_y=True)
def validar(X, y):
    erros = []
    if X.shape[1] != 64: erros.append("número de colunas diferente de 64")
    if np.isnan(X).any(): erros.append("há valores ausentes")
    if X.min() < 0 or X.max() > 16: erros.append("pixels fora de [0, 16]")
    if set(np.unique(y)) != set(range(10)): erros.append("classes diferentes de 0..9")
    return erros
print("[1] validação:", validar(X, y) or "aprovada")
hash_dados = hashlib.sha256(np.c_[X, y].tobytes()).hexdigest()[:12]
print("[1] versão dos dados (hash):", hash_dados)
```

```
# 60% desenvolvimento, 40% "futuro" (produção simulada, 8 semanas)
Xdev, Xfut, ydev, yfut = train_test_split(X, y, test_size=0.4, stratify=y, random_state=SEMENTE)
Xtr, Xte, ytr, yte = train_test_split(Xdev, ydev, test_size=0.25, stratify=ydev, random_state=SEMENTE)
print(f"[1] treino={len(Xtr)} teste={len(Xte)} produção simulada={len(Xfut)}")

# ------------------------------------------------------------------ 2/3. candidatos rastreados no
MLflow
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("digitos")
cv = StratifiedKFold(5, shuffle=True, random_state=SEMENTE)
candidatos = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logreg": LogisticRegression(max_iter=5000),
    "floresta": RandomForestClassifier(n_estimators=200, random_state=SEMENTE),
    "svc": RandomizedSearchCV(SVC(),
                              {"C": loguniform(0.1, 100), "gamma": loguniform(1e-4, 1e-1)},
                              n_iter=12, cv=cv, scoring="f1_macro", random_state=SEMENTE),
}
resultados = {}
for nome, modelo in candidatos.items():
    with mlflow.start_run(run_name=nome):
        modelo.fit(Xtr, ytr)
        est = modelo.best_estimator_ if hasattr(modelo, "best_estimator_") else modelo
        f1_cv = cross_val_score(est, Xtr, ytr, cv=cv, scoring="f1_macro").mean()
        mlflow.log_params({"modelo": nome, "semente": SEMENTE})
        mlflow.log_metric("f1_macro_cv", f1_cv)
        mlflow.set_tag("hash_dados", hash_dados)
        info = mlflow.sklearn.log_model(est, name="modelo", signature=infer_signature(Xte,
est.predict(Xte)),
                                        serialization_format="cloudpickle")
        resultados[nome] = (f1_cv, est, info.model_uri)
for nome, (f1_cv, _, _) in resultados.items():
    print(f"[2] {nome:9s} F1-macro (validação cruzada) = {f1_cv:.3f}")

# ------------------------------------------------------------------ 3. portão de qualidade e registro
(apelido champion)
PISO_F1, MARGEM = 0.95, 0.01
melhor = max((k for k in resultados if k != "baseline"), key=lambda k: resultados[k][0])
f1_melhor, modelo_melhor, uri_melhor = resultados[melhor]
f1_base = resultados["baseline"][0]
aprovado = f1_melhor >= PISO_F1 and f1_melhor >= f1_base + MARGEM
print(f"[3] melhor candidato = {melhor} (F1 {f1_melhor:.3f}) | piso {PISO_F1} | portão {'APROVADO' if
aprovado else 'REPROVADO'}")
assert aprovado
cliente = MlflowClient()
NOME = "digitos"
v = mlflow.register_model(uri_melhor, NOME)
cliente.set_registered_model_alias(NOME, "champion", v.version)
champion = mlflow.sklearn.load_model(f"models:/{NOME}@champion")
f1_teste = f1_score(yte, champion.predict(Xte), average="macro")
print(f"[3] versão {v.version} registrada como champion | F1-macro no teste (uso único) =
{f1_teste:.3f}")

# ------------------------------------------------------------------ 4. API com contrato, carregando o
champion pelo apelido
from contextlib import asynccontextmanager
from fastapi import FastAPI
```

```
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field, field_validator

class Imagem(BaseModel):
    pixels: list[float] = Field(min_length=64, max_length=64)
    @field_validator("pixels")
    @classmethod
    def faixa(cls, v):
        if min(v) < 0 or max(v) > 16:
            raise ValueError("pixels devem estar entre 0 e 16")
        return v

def criar_app():
    estado = {}
    @asynccontextmanager
    async def ciclo(app):
        estado["m"] = mlflow.sklearn.load_model(f"models:/{NOME}@champion")
        estado["v"] = cliente.get_model_version_by_alias(NOME, "champion").version
        yield
    app = FastAPI(lifespan=ciclo)
    @app.post("/previsao")
    def prever(img: Imagem):
        x = np.array(img.pixels).reshape(1, -1)
        return {"digito": int(estado["m"].predict(x)[0]), "versao_modelo": estado["v"]}
    return app

with TestClient(criar_app()) as api:
    ok = api.post("/previsao", json={"pixels": Xte[0].tolist()})
    ruim = api.post("/previsao", json={"pixels": [99.0] * 64})
    curta = api.post("/previsao", json={"pixels": [1.0] * 10})
print(f"[4] API: válida -> {ok.status_code} {ok.json()} (real = {yte[0]}) | pixel fora da faixa ->
{ruim.status_code} | tamanho errado -> {curta.status_code}")

# ---------------------------------------------------------------- 5. testes automáticos (pytest)
sobre o artefato exportado
joblib.dump(champion, "champion.joblib")
np.savez("teste.npz", X=Xte, y=yte)
open("test_modelo.py", "w").write(textwrap.dedent('''
    import joblib, numpy as np
    from sklearn.metrics import f1_score
    M = joblib.load("champion.joblib"); D = np.load("teste.npz"); X, y = D["X"], D["y"]

    def test_desempenho_minimo():
        assert f1_score(y, M.predict(X), average="macro") >= 0.95

    def test_invariancia_a_ruido_minimo():
        ruido = np.random.default_rng(0).normal(0, 0.2, X.shape)
        mudou = (M.predict(X) != M.predict(np.clip(X + ruido, 0, 16))).mean()
        assert mudou <= 0.05

    def test_todas_as_classes_sao_reconhecidas():
        rec = [(M.predict(X[y == c]) == c).mean() for c in range(10)]
        assert min(rec) >= 0.85

    def test_determinismo():
        assert np.array_equal(M.predict(X), M.predict(X))
'''))
r = subprocess.run([sys.executable, "-m", "pytest", "test_modelo.py", "-q", "-p", "no:cacheprovider"],
```

```
capture_output=True, text=True)
print("[5] pytest:", r.stdout.strip().splitlines()[-1])

# ---------------------------------------------------------------- 6/7. produção simulada com deriva, monitoramento e CT
def psi(ref, atual, faixas=10):
    cortes = np.quantile(ref, np.linspace(0, 1, faixas + 1)); cortes[0], cortes[-1] = -np.inf, np.inf
    p = np.clip(np.histogram(ref, cortes)[0] / len(ref), 1e-6, None)
    q = np.clip(np.histogram(atual, cortes)[0] / len(atual), 1e-6, None)
    return float(np.sum((q - p) * np.log(q / p)))

def auc_dominio(A, B):
    Z = np.vstack([A, B]); d = np.r_[np.zeros(len(A)), np.ones(len(B))]
    return cross_val_score(RandomForestClassifier(100, min_samples_leaf=3, random_state=0, n_jobs=2), Z, d, cv=4, scoring="roc_auc").mean()

from scipy.ndimage import gaussian_filter
def aplicar_deriva(X, semana, r):          # a câmera sai de foco aos poucos e a tinta desbota
    imgs = X.reshape(-1, 8, 8)
    nivel = min(semana, 4)                       # a degradação cresce por 4 semanas e depois se estabiliza
    emb = np.stack([gaussian_filter(im, sigma=FOCO * nivel, mode="constant") for im in imgs])
    return np.clip(emb.reshape(len(X), -1) * (1 - FADE * nivel), 0, 16)

FADE, FOCO = 0.03, 0.30                      # intensidade da degradação por semana
SEMANAS = 8
lotes = np.array_split(np.arange(len(Xfut)), SEMANAS)
rd = np.random.default_rng(7)
prod = [(aplicar_deriva(Xfut[i], s + 1, rd), yfut[i]) for s, i in enumerate(lotes)]

ref_int = Xtr.std(axis=1)
ref_amostra = Xtr[rng.choice(len(Xtr), 300, replace=False)]
fixo = champion                              # v1 congelado (sem CT), para comparação
atual = champion                             # champion corrente (com CT)
versao_atual = int(v.version)
rotulados_X, rotulados_y = [], []            # lotes já rotulados (chegam com 1 semana de atraso)
linhas = []
for s, (Xs, ys) in enumerate(prod, start=1):
    p_atual = atual.predict(Xs)
    m = {"semana": s,
         "PSI_contraste": psi(ref_int, Xs.std(axis=1)),
         "AUC_dominio": auc_dominio(ref_amostra, Xs),
         "acc_sem_CT": (fixo.predict(Xs) == ys).mean(),
         "acc_com_CT": (p_atual == ys).mean(),
         "versao": versao_atual, "acao": ""}
    alerta = m["PSI_contraste"] > 0.25 or m["AUC_dominio"] > 0.80
    # CT: rótulos da semana s-1 já chegaram; a semana s-2 e anteriores entram no treino
    if alerta and s >= 3:
        Xtreino = np.vstack([Xtr] + [a for a in rotulados_X[:-1]]); ytreino = np.concatenate([ytr] + rotulados_y[:-1])
        pesos = np.r_[np.ones(len(Xtr)), np.concatenate([np.full(len(b), 3.0) for b in rotulados_y[:-1]])] if rotulados_y[:-1] else np.ones(len(Xtreino))
        cand = SVC(C=modelo_melhor.C, gamma=modelo_melhor.gamma)
        cand.fit(Xtreino, ytreino, sample_weight=pesos)
        Xv, yv = rotulados_X[-1], rotulados_y[-1]          # semana s-1: recente e não vista pelo candidato
        f1_c, f1_a = f1_score(yv, cand.predict(Xv), average="macro"), f1_score(yv, atual.predict(Xv), average="macro")
```

```
        if f1_c >= f1_a + 0.02:
            nv = mlflow.register_model(mlflow.sklearn.log_model(cand, name="modelo_ct").model_uri,
NOME)
            cliente.set_registered_model_alias(NOME, "champion", nv.version)
            atual, versao_atual = cand, int(nv.version)
            ref_int = np.concatenate([b.std(axis=1) for b in rotulados_X[-2:]])          # nova
referência: dados recentes
            ref_amostra = np.vstack(rotulados_X[-2:])
            m["acao"] = f"promoveu v{nv.version} (F1 recente {f1_a:.2f}->{f1_c:.2f})"
        else:
            m["acao"] = f"manteve (candidato {f1_c:.2f} vs {f1_a:.2f})"
    elif alerta:
        m["acao"] = "alerta (aguardando rótulos)"
    linhas.append(m)
    rotulados_X.append(Xs); rotulados_y.append(ys)
tab = pd.DataFrame(linhas)
print(tab.round(3).to_string(index=False))
print(f"[7] acurácia média nas semanas 5-8: sem CT = {tab.acc_sem_CT[4:].mean():.3f} | com CT =
{tab.acc_com_CT[4:].mean():.3f}")

# ------------------------------------------------------------------- 8. após a RECARGA do serviço, a
API serve o novo champion (sem editar o código)
with TestClient(criar_app()) as api:
    resp = api.post("/previsao", json={"pixels": Xte[0].tolist()})
print(f"[8] API após recarregar o serviço (novo lifespan): {resp.json()} (apelido champion aponta para
a versão {versao_atual})")

# ------------------------------------------------------------------- 9. cartão do modelo com desempenho
por classe
X_ult, y_ult = prod[-1]
rec_antes = recall_score(yte, atual.predict(Xte), average=None)
rec_agora = recall_score(y_ult, atual.predict(X_ult), average=None)
cartao = [f"# Cartão do modelo: digitos v{versao_atual}",
          f"- Dados de treino original (hash): {hash_dados}",
          f"- F1-macro no teste original: {f1_score(yte, atual.predict(Xte), average='macro'):.3f}",
          f"- F1-macro na última semana de produção: {f1_score(y_ult, atual.predict(X_ult),
average='macro'):.3f}",
          f"- Pior revocação por dígito na última semana: dígito {int(np.argmin(rec_agora))} =
{rec_agora.min():.2f}",
          "- Limitação: treinado com imagens 8x8; câmera desfocada exige novo retreino.",
          "- Uso indicado: triagem de apoio, com amostras revisadas por pessoas."]
print("[9]\n" + "\n".join(cartao))
print(f"[10] RESUMO: sem CT a acurácia média nas semanas 5-8 foi {tab.acc_sem_CT[4:].mean():.2f}; com
CT, {tab.acc_com_CT[4:].mean():.2f}.")
```

SAÍDA

```
[1] validação: aprovada
[1] versão dos dados (hash): 30aa855cd904
[1] treino=808 teste=270 produção simulada=719
[2] baseline  F1-macro (validação cruzada) = 0.018
[2] logreg    F1-macro (validação cruzada) = 0.963
[2] floresta  F1-macro (validação cruzada) = 0.965
[2] svc       F1-macro (validação cruzada) = 0.990
[3] melhor candidato = svc (F1 0.990) | piso 0.95 | portão APROVADO
[3] versão 1 registrada como champion | F1-macro no teste (uso único) = 0.989
[4] API: válida -> 200 {'digito': 4, 'versao_modelo': 1} (real = 4) | pixel fora da faixa -> 422 |
tamanho errado -> …
```

```
[5] pytest: 4 passed in 1.17s
 semana  PSI_contraste  AUC_dominio  acc_sem_CT  acc_com_CT  versao                                acao
      1          0.500        1.000       0.978       0.978       1         alerta (aguardando rótulos)
      2         12.428        1.000       0.978       0.978       1         alerta (aguardando rótulos)
      3         12.428        1.000       0.700       0.700       1    manteve (candidato 0.98 vs 0.98)
      4         12.428        1.000       0.311       0.311       1 promoveu v2 (F1 recente 0.70->0.84)
      5          8.688        0.996       0.422       0.578       2 promoveu v3 (F1 recente 0.61->0.77)
      6          1.901        0.696       0.478       0.822       3 promoveu v4 (F1 recente 0.79->0.91)
      7          0.105        0.433       0.333       0.933       4
      8          0.213        0.587       0.404       0.933       4
[7] acurácia média nas semanas 5-8: sem CT = 0.409 | com CT = 0.816
[8] API após recarregar o serviço (novo lifespan): {'digito': 4, 'versao_modelo': 4} (apelido champion
aponta para a …
[9]
# Cartão do modelo: digitos v4
- Dados de treino original (hash): 30aa855cd904
- F1-macro no teste original: 0.974
- F1-macro na última semana de produção: 0.919
- Pior revocação por dígito na última semana: dígito 4 = 0.67
- Limitação: treinado com imagens 8x8; câmera desfocada exige novo retreino.
- Uso indicado: triagem de apoio, com amostras revisadas por pessoas.
[10] RESUMO: sem CT a acurácia média nas semanas 5-8 foi 0.41; com CT, 0.82.
```

EXEMPLO Discussão dos resultados

O resultado exato depende do computador e das versões das bibliotecas. Nesta execução: (i) o SVC com busca aleatória foi o melhor candidato (F1 macro de cerca de 0,99 em validação cruzada), muito acima da linha de base (cerca de 0,02) e da regressão logística e da floresta (cerca de 0,96); (ii) os testes automáticos passaram e a API rejeitou, com código 422, tanto pixels fora da faixa quanto tamanho errado; (iii) na simulação, o PSI e a AUC de domínio dispararam já na semana 1 ou 2, enquanto a acurácia do modelo fixo só caiu de forma clara na semana 3 (cerca de 0,70) e chegou a cerca de 0,31 na semana 4, ficando em torno de 0,4 nas semanas seguintes; (iv) com treinamento contínuo, foram feitas três promoções (versões 2, 3 e 4), a acurácia voltou a cerca de 0,93 nas semanas 7 e 8, e os alertas se apagaram depois que a referência foi atualizada; (v) o campeão com CT na semana 5 e 6 ainda era imperfeito, porque o mundo estava mudando e os rótulos mais recentes chegam com atraso; (vi) após a recarga do serviço (novo `lifespan`), a API passou a servir a versão 4 sem edição de código; sem a recarga, o processo antigo continuaria na versão 1; (vii) por classe, o dígito 4 teve a pior revocação na última semana. Lembre-se: tudo aqui é simulado e serve para treinar o raciocínio; em produção real, a causa do alerta deve ser investigada antes de retreinar.

Referências

Lista de artigos de periódicos e conferências de referência, além de livros reconhecidos, citados ao longo do texto.

AKIBA, T.; SANO, S.; YANASE, T.; OHTA, T.; KOYAMA, M. Optuna: a next-generation hyperparameter optimization framework. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2019. p. 2623-2631.

AMERSHI, S. et al. Software engineering for machine learning: a case study. In: IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2019. p. 291-300.

ARLOT, S.; CELISSE, A. A survey of cross-validation procedures for model selection. Statistics Surveys, v. 4, p. 40-79, 2010.

ARRIETA, A. B. et al. Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI. Information Fusion, v. 58, p. 82-115, 2020.

BAROCAS, S.; HARDT, M.; NARAYANAN, A. Fairness and Machine Learning: Limitations and Opportunities. Cambridge: MIT Press, 2023.

BAYLOR, D. et al. TFX: a TensorFlow-based production-scale machine learning platform. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2017. p. 1387-1395.

BERGMEIR, C.; BENÍTEZ, J. M. On the use of cross-validation for time series predictor evaluation. Information Sciences, v. 191, p. 192-213, 2012.

BERGSTRA, J.; BENGIO, Y. Random search for hyper-parameter optimization. Journal of Machine Learning Research, v. 13, p. 281-305, 2012.

BERNARDI, L. et al. 150 successful machine learning models: 6 lessons learned at Booking.com. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2019. p. 1743-1751.

BEYER, B.; JONES, C.; PETOFF, J.; MURPHY, N. R. Site Reliability Engineering: How Google Runs Production Systems. Sebastopol: O'Reilly, 2016.

BIFET, A.; GAVALDÀ, R. Learning from time-changing data with adaptive windowing. In: SIAM International Conference on Data Mining (SDM), 2007. p. 443-448.

BRECK, E.; CAI, S.; NIELSEN, E.; SALIB, M.; SCULLEY, D. The ML test score: a rubric for ML production readiness and technical debt reduction. In: IEEE International Conference on Big Data, 2017. p. 1123-1132.

BRECK, E.; POLYZOTIS, N.; ROY, S.; WHANG, S. E.; ZINKEVICH, M. Data validation for machine learning. In: Conference on Systems and Machine Learning (SysML), 2019.

BREIMAN, L. Random forests. Machine Learning, v. 45, n. 1, p. 5-32, 2001.

BROWN, T. B. et al. Language models are few-shot learners. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020. p. 1877-1901.

BUITINCK, L. et al. API design for machine learning software: experiences from the scikit-learn project. In: ECML PKDD Workshop on Languages for Data Mining and Machine Learning, 2013. p. 108-122.

CAWLEY, G. C.; TALBOT, N. L. C. On over-fitting in model selection and subsequent selection bias in performance evaluation. Journal of Machine Learning Research, v. 11, p. 2079-2107, 2010.

D'AMOUR, A. et al. Underspecification presents challenges for credibility in modern machine learning. Journal of Machine Learning Research, v. 23, n. 226, p. 1-61, 2022.

DAVIS, J.; GOADRICH, M. The relationship between precision-recall and ROC curves. In: International Conference on Machine Learning (ICML), 2006. p. 233-240.

DEMŠAR, J. Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research, v. 7, p. 1-30, 2006.

DIETTERICH, T. G. Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation, v. 10, n. 7, p. 1895-1923, 1998.

DUA, D.; GRAFF, C. UCI Machine Learning Repository. Irvine: University of California, School of Information and Computer Science, 2019.

DWORK, C.; MCSHERRY, F.; NISSIM, K.; SMITH, A. Calibrating noise to sensitivity in private data analysis. In: Theory of Cryptography Conference (TCC), 2006. Lecture Notes in Computer Science, v. 3876, p. 265-284.

DWORK, C.; ROTH, A. The algorithmic foundations of differential privacy. Foundations and Trends in Theoretical Computer Science, v. 9, n. 3-4, p. 211-407, 2014.

FEURER, M. et al. Efficient and robust automated machine learning. In: Advances in Neural Information Processing Systems (NeurIPS), v. 28, 2015. p. 2962-2970.

FORSGREN, N.; HUMBLE, J.; KIM, G. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. The Annals of Statistics, v. 29, n. 5, p. 1189-1232, 2001.

GAMA, J. et al. A survey on concept drift adaptation. ACM Computing Surveys, v. 46, n. 4, art. 44, 2014.

GAMA, J.; MEDAS, P.; CASTILLO, G.; RODRIGUES, P. Learning with drift detection. In: Brazilian Symposium on Artificial Intelligence (SBIA), 2004. Lecture Notes in Artificial Intelligence, v. 3171, p. 286-295.

GEBRU, T. et al. Datasheets for datasets. Communications of the ACM, v. 64, n. 12, p. 86-92, 2021.

GRETTON, A. et al. A kernel two-sample test. Journal of Machine Learning Research, v. 13, p. 723-773, 2012.

GUIDOTTI, R. et al. A survey of methods for explaining black box models. ACM Computing Surveys, v. 51, n. 5, art. 93, 2018.

GUO, C.; PLEISS, G.; SUN, Y.; WEINBERGER, K. Q. On calibration of modern neural networks. In: International Conference on Machine Learning (ICML), 2017. PMLR, v. 70, p. 1321-1330.

HARDT, M.; PRICE, E.; SREBRO, N. Equality of opportunity in supervised learning. In: Advances in Neural Information Processing Systems (NeurIPS), v. 29, 2016. p. 3315-3323.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2. ed. New York: Springer, 2009.

HAZELWOOD, K. et al. Applied machine learning at Facebook: a datacenter infrastructure perspective. In: IEEE International Symposium on High Performance Computer Architecture (HPCA), 2018. p. 620-629.

HU, E. J. et al. LoRA: low-rank adaptation of large language models. In: International Conference on Learning Representations (ICLR), 2022.

HUMBLE, J.; FARLEY, D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Upper Saddle River: Addison-Wesley, 2010.

HUTTER, F.; KOTTHOFF, L.; VANSCHOREN, J. (ed.). Automated Machine Learning: Methods, Systems, Challenges. Cham: Springer, 2019.

HUYEN, C. Designing Machine Learning Systems. Sebastopol: O'Reilly, 2022.

JOHN, M. M.; OLSSON, H. H.; BOSCH, J. Towards MLOps: a framework and maturity model. In: Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 2021. p. 334-341.

KAUFMAN, S.; ROSSET, S.; PERLICH, C.; STITELMAN, O. Leakage in data mining: formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, v. 6, n. 4, art. 15, 2012.

KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: International Joint Conference on Artificial Intelligence (IJCAI), 1995. p. 1137-1145.

KOHAVI, R.; DENG, A.; FRASCA, B.; WALKER, T.; XU, Y.; POHLMANN, N. Online controlled experiments at large scale. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2013. p. 1168-1176.

KOHAVI, R.; TANG, D.; XU, Y. Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing. Cambridge: Cambridge University Press, 2020.

KREUZBERGER, D.; KÜHL, N.; HIRSCHL, S. Machine learning operations (MLOps): overview, definition, and architecture. IEEE Access, v. 11, p. 31866-31879, 2023.

KULLBACK, S.; LEIBLER, R. A. On information and sufficiency. The Annals of Mathematical Statistics, v. 22, n. 1, p. 79-86, 1951.

LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020. p. 9459-9474.

LI, L.; JAMIESON, K.; DESALVO, G.; ROSTAMIZADEH, A.; TALWALKAR, A. Hyperband: a novel bandit-based approach to hyperparameter optimization. Journal of Machine Learning Research, v. 18, n. 185, p. 1-52, 2018.

LIN, J. Divergence measures based on the Shannon entropy. IEEE Transactions on Information Theory, v. 37, n. 1, p. 145-151, 1991.

LIPTON, Z.; WANG, Y.-X.; SMOLA, A. Detecting and correcting for label shift with black box predictors. In: International Conference on Machine Learning (ICML), 2018. PMLR, v. 80, p. 3122-3130.

LU, J. et al. Learning under concept drift: a review. IEEE Transactions on Knowledge and Data Engineering, v. 31, n. 12, p. 2346-2363, 2019.

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems (NeurIPS), v. 30, 2017. p. 4765-4774.

MASSEY, F. J. The Kolmogorov-Smirnov test for goodness of fit. Journal of the American Statistical Association, v. 46, n. 253, p. 68-78, 1951.

MCMAHAN, H. B. et al. Communication-efficient learning of deep networks from decentralized data. In: International Conference on Artificial Intelligence and Statistics (AISTATS), 2017. PMLR, v. 54, p. 1273-1282.

MEHRABI, N.; MORSTATTER, F.; SAXENA, N.; LERMAN, K.; GALSTYAN, A. A survey on bias and fairness in machine learning. ACM Computing Surveys, v. 54, n. 6, art. 115, 2021.

MERKEL, D. Docker: lightweight Linux containers for consistent development and deployment. Linux Journal, n. 239, art. 2, 2014.

MITCHELL, M. et al. Model cards for model reporting. In: ACM Conference on Fairness, Accountability, and Transparency (FAT*), 2019. p. 220-229.

MORENO-TORRES, J. G.; RAEDER, T.; ALAIZ-RODRÍGUEZ, R.; CHAWLA, N. V.; HERRERA, F. A unifying view on dataset shift in classification. Pattern Recognition, v. 45, n. 1, p. 521-530, 2012.

NICULESCU-MIZIL, A.; CARUANA, R. Predicting good probabilities with supervised learning. In: International Conference on Machine Learning (ICML), 2005. p. 625-632.

NORTHCUTT, C. G.; ATHALYE, A.; MUELLER, J. Pervasive label errors in test sets destabilize machine learning benchmarks. In: Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS), 2021.

PALEYES, A.; URMA, R.-G.; LAWRENCE, N. D. Challenges in deploying machine learning: a survey of case studies. ACM Computing Surveys, v. 55, n. 6, art. 114, 2022.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research, v. 12, p. 2825-2830, 2011.

PINEAU, J. et al. Improving reproducibility in machine learning research. Journal of Machine Learning Research, v. 22, n. 164, p. 1-20, 2021.

QUIÑONERO-CANDELA, J.; SUGIYAMA, M.; SCHWAIGHOFER, A.; LAWRENCE, N. D. (ed.). Dataset Shift in Machine Learning. Cambridge: MIT Press, 2009.

RABANSER, S.; GÜNNEMANN, S.; LIPTON, Z. C. Failing loudly: an empirical study of methods for detecting dataset shift. In: Advances in Neural Information Processing Systems (NeurIPS), v. 32, 2019. p. 1396-1408.

RIBEIRO, M. T.; SINGH, S.; GUESTRIN, C. "Why should I trust you?": explaining the predictions of any classifier. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016. p. 1135-1144.

RIBEIRO, M. T.; WU, T.; GUESTRIN, C.; SINGH, S. Beyond accuracy: behavioral testing of NLP models with CheckList. In: Annual Meeting of the Association for Computational Linguistics (ACL), 2020. p. 4902-4912.

SAITO, T.; REHMSMEIER, M. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, v. 10, n. 3, art. e0118432, 2015.

SAMBASIVAN, N. et al. "Everyone wants to do the model work, not the data work": data cascades in high-stakes AI. In: ACM CHI Conference on Human Factors in Computing Systems, 2021. art. 39.

SCHELTER, S.; LANGE, D.; SCHMIDT, P.; CELIKEL, M.; BIESSMANN, F.; GRAFBERGER, A. Automating large-scale data quality verification. Proceedings of the VLDB Endowment, v. 11, n. 12, p. 1781-1794, 2018.

SCULLEY, D. et al. Hidden technical debt in machine learning systems. In: Advances in Neural Information Processing Systems (NeurIPS), v. 28, 2015. p. 2503-2511.

SHOKRI, R.; STRONATI, M.; SONG, C.; SHMATIKOV, V. Membership inference attacks against machine learning models. In: IEEE Symposium on Security and Privacy (S&P), 2017. p. 3-18.

SNOEK, J.; LAROCHELLE, H.; ADAMS, R. P. Practical Bayesian optimization of machine learning algorithms. In: Advances in Neural Information Processing Systems (NeurIPS), v. 25, 2012. p. 2951-2959.

SUGIYAMA, M.; KRAULEDAT, M.; MÜLLER, K.-R. Covariate shift adaptation by importance weighted cross validation. Journal of Machine Learning Research, v. 8, p. 985-1005, 2007.

VANSCHOREN, J.; VAN RIJN, J. N.; BISCHL, B.; TORGO, L. OpenML: networked science in machine learning. ACM SIGKDD Explorations Newsletter, v. 15, n. 2, p. 49-60, 2013.

VARMA, S.; SIMON, R. Bias in error estimation when using cross-validation for model selection. BMC Bioinformatics, v. 7, art. 91, 2006.

VASWANI, A. et al. Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS), v. 30, 2017. p. 5998-6008.

WANG, R. Y.; STRONG, D. M. Beyond accuracy: what data quality means to data consumers. Journal of Management Information Systems, v. 12, n. 4, p. 5-33, 1996.

WHANG, S. E.; ROH, Y.; SONG, H.; LEE, J.-G. Data collection and quality challenges in deep learning: a data-centric AI perspective. The VLDB Journal, v. 32, p. 791-813, 2023.

ZAHARIA, M. et al. Accelerating the machine learning lifecycle with MLflow. IEEE Data Engineering Bulletin, v. 41, n. 4, p. 39-45, 2018.